
STACK SÊNIOR

Código consistente sem perder o controle

Guia prático de arquitetura, fluxo e qualidade
para manter cada contribuição **no seu padrão**.

Next.js 16 • React 19 • TypeScript 7 • Bun

Guia de início e manual de referência



Sumário

1	O que é o Stack Sênior	1
2	Primeiros passos	8
3	Adaptando ao seu projeto.	27
4	A metodologia: skills e comandos	46
5	Code review: por que é o coração do método	65
6	Arquitetura: o DAG de camadas	80
7	Dados e bordas: RSC puro, sempre fresco	96
8	A camada de referência que já vem escrita	115
9	Guardrails: as regras que o lint impõe.	137
10	Os portões: verify, lefthook e CI	144
11	Hooks de agente: o guardrail que roda sozinho	158
12	Integrações de terceiros.	175
13	Decisões e porquês (ADRs)	191
14	Quando algo dá errado.	216

Não precisa ler tudo em ordem. Comece pelas duas primeiras partes. Depois use o capítulo que resolve a dúvida do momento. O PDF mantém todas as subseções nos bookmarks para você chegar rápido ao ponto certo.

PARTE 1

O que é o Stack Sênior

O QUE VOCÊ LEVA DESTE CAPÍTULO

O Stack Sênior é uma base de produto para quem constrói SaaS com Next.js e quer trabalhar com agentes sem entregar a arquitetura ao acaso. Ele junta aplicação de referência, decisões registradas, regras verificáveis e um caminho de trabalho que cabe no dia a dia.

1.1 Em uma frase

Stack Sênior é o ponto de partida entre uma ideia de produto e um repositório pronto para evoluir. Você recebe uma aplicação Next.js 16, uma arquitetura com direção clara, um gate de qualidade e instruções que ajudam Claude Code, Codex, Cursor ou outro agente a trabalhar dentro desse desenho.

Isso importa porque velocidade, sozinha, não entrega um produto sustentável. O que parece um atalho na primeira tarefa costuma voltar depois como review longa, comportamento inconsistente, regra duplicada ou retrabalho. A proposta desta base é simples: deixar o caminho principal visível e fazer o repositório acusar cedo quando alguém sair dele.

REGRA | O que você está comprando

Não é apenas um template visual, um pacote de prompts ou uma coleção de scripts. É uma forma de construir: código de referência para consultar, decisões para não rediscutir sem necessidade e portões para separar o que a máquina verifica do que ainda precisa de experiência humana.

1.2 O problema que a base resolve

Se você já trabalha com Claude Code, Codex ou Cursor, provavelmente conhece este ciclo: você explica o contexto, o agente começa bem e, algumas mudanças depois, aparece uma solução plausível que não combina com o restante do projeto. O problema não é o agente ser rápido. É o contexto importante ficar espalhado pela conversa, pela memória de quem mantém o projeto e por documentos que ninguém consulta no momento da decisão.

O Stack Sênior coloca parte desse contexto em lugares que o próprio projeto consegue usar e verificar. A regra aparece na documentação para a pessoa, nas skills para o agente, no lint ou no teste quando pode ser automatizada e no code review quando depende de julgamento.

O trabalho deixa de depender de memória

Quando tudo fica na cabeça

Quando o repositório participa

Você repete a arquitetura a cada tarefa

O projeto aponta o caminho antes do agente começar

O desvio aparece na review ou em produção

Lint, tipos, testes e hooks barram o que é verificável

Cada ferramenta recebe instruções diferentes

Hosts diferentes compartilham as mesmas regras

Você revisa formato, estrutura e intenção de uma vez

A automação cuida do mecânico e libera sua atenção para produto e domínio

O método não promete eliminar erro. Ele reduz a quantidade de erro barato que chega tarde e deixa mais claro onde uma decisão realmente precisa de uma pessoa.

1.3 Como as peças trabalham juntas

O fluxo também pode ter um orquestrador, como o Orca, quando você divide o trabalho entre agentes, worktrees ou tarefas independentes. O orquestrador organiza o contexto e acompanha o andamento; ele não substitui as regras do repositório. Cada agente recebe a mesma base de instruções, trabalha em um escopo definido e devolve uma mudança que passa pelos mesmos gates.

Um fluxo com orquestrador, sem prender o produto a uma ferramenta

Peça	Responsabilidade
Orquestrador	Divide a tarefa, encaminha contexto, acompanha os agentes e reúne os resultados.
Agente	Explora, implementa e testa dentro do escopo recebido, consultando as regras e skills do projeto.
Repositório	Expõe decisões, limita imports, executa hooks e verifica o estado da entrega.
Pessoa responsável	Define produto e domínio, arbitra trade-offs e aprova o que está pronto para seguir.

Em termos práticos, o ciclo é este:

1. a pessoa define o resultado esperado e o contexto;
2. o orquestrador, se existir, divide o trabalho sem esconder a origem da decisão;
3. o agente lê as instruções canônicas e implementa no seu espaço;
4. hooks e gates verificam formato, tipos, testes, arquitetura e regras do projeto;

5. a pessoa revisa intenção, produto e impacto antes de aceitar a mudança.

Se você trabalha sozinho, os mesmos papéis continuam existindo: você faz o papel de orquestrador e de pessoa responsável, enquanto o agente executa a parte operacional. Se trabalha em equipe, o fluxo dá nomes para responsabilidades que já precisavam estar claras.

1.4 O que muda no trabalho diário

- uma feature começa com contexto e critérios, não apenas com um pedido curto;
- a arquitetura oferece um caminho principal em vez de uma lista de alternativas;
- o agente recebe o padrão no momento em que vai escrever;
- o gate encontra boa parte dos desvios antes do pull request;
- o code review fica concentrado em intenção, domínio, risco e experiência do usuário.

Este é o ganho central: não fazer menos engenharia, e sim gastar a engenharia certa no lugar certo.

1.5 Este produto foi feito para você se

- você constrói ou quer construir um SaaS em Next.js;
- já usa um agente no desenvolvimento ou pretende começar agora;
- sente que passa tempo demais lembrando convenções e corrigindo improvisos;
- quer uma arquitetura opinativa, com um caminho principal;
- prefere descobrir um desvio no seu computador, antes do pull request;
- valoriza uma base que explica as decisões sem fingir que toda regra serve para todo projeto.

Você pode estar começando ou já ter anos de experiência. Quem está começando ganha um trilha mais seguro; quem já domina a stack deixa de gastar energia com o básico. O mesmo mecanismo atende os dois, por motivos diferentes.

POR QUE | Como aproveitar o guia

Leia as primeiras páginas como um percurso acompanhado. Depois use o PDF como manual de bancada: vá para a seção que responde à dúvida, copie o exemplo, rode o comando e volte ao motivo quando precisar adaptar a base. Não há prêmio por ler tudo em ordem.

1.6 Como ler os exemplos

Os exemplos foram escolhidos para mostrar a ligação entre uma decisão e o código que a sustenta. A legenda informa de onde veio o trecho; o texto ao redor explica o que observar; e o bloco mostra o menor recorte capaz de provar a ideia. Quando o exemplo for um comando, ele pode ser executado ou adaptado. Quando for um arquivo, trate-o como referência e confirme o contexto antes de colar.

Convenções usadas nos exemplos

Indicação	Como interpretar
<code>caminho/arquivo.ts</code>	O arquivo real que deve ser aberto para conferir o contexto.
trecho ou linhas	Recorte selecionado para destacar uma decisão, não o arquivo inteiro.
integral ou arquivo completo	O bloco representa todo o conteúdo relevante daquele arquivo.
Terminal	Comando para rodar na raiz do projeto; comentários indicam o que pode ser adaptado.
Saída esperada	Resultado de referência. Mensagens de caminho, versão ou tempo podem variar.

Quando uma caixa continua na página seguinte, o título ganha “(continuação)”. Isso faz parte do mesmo exemplo; não é um novo comando nem uma nova regra.

1.7 Talvez esta não seja a base certa se

Uma base opinativa economiza decisões, mas também exige adaptação quando o seu projeto:

- não usa um backend REST separado;
- depende de cache, ISR ou busca de dados no cliente como padrão principal;
- precisa de npm, pnpm, yarn ou Node como runtime dos scripts;
- quer manter várias alternativas arquiteturais abertas dentro do mesmo repositório;
- trata supressão de lint como saída normal para uma regra inconveniente.

Esses pontos podem ser alterados porque o código é seu. Só não trate a alteração como detalhe: ela muda decisões centrais, testes e portões. Primeiro entenda o motivo; depois escolha conscientemente o que precisa sair do caminho original.

1.8 Seu primeiro resultado em 30 minutos

O primeiro resultado não é terminar o produto. É ver a base rodando e provar que o gate está funcionando. Faça este percurso em dois terminais, porque o servidor de desenvolvimento fica ocupando o primeiro.

PRÓXIMO PASSO: coloque a base de pé

1. extraia o ZIP em uma pasta nova;
2. inicie o Git antes de instalar as dependências;
3. instale com Bun;
4. copie o arquivo de ambiente e informe as URLs do seu backend;
5. rode o servidor de desenvolvimento no primeiro terminal;
6. execute o gate no segundo terminal.

Terminal 1 – preparar e subir a aplicação

```
git init
bun install
cp .env.example .env.local
bun run dev
```

Com a aplicação aberta em `http://localhost:3000`, use outro terminal na mesma pasta:

Terminal 2 – conferir a base

```
bun run verify
```

O capítulo **Primeiros passos** mostra o que preencher e como interpretar cada resultado. Siga-o antes de remover arquivos ou trocar decisões da base.

CUIDADO | Não comece apagando o que ainda não conhece

Algumas pastas parecem excesso quando vistas isoladamente, mas sustentam lint, hooks e gates em mais de um editor. Primeiro faça o projeto passar no `bun run verify`. Depois personalize com um estado verde para comparar.

1.9 Escolha seu caminho de leitura

Este PDF tem duas funções. As primeiras partes ajudam você a começar; as demais formam um manual de referência para consultar durante o trabalho.

Rotas recomendadas

Seu objetivo agora	Leia nesta ordem
Quero rodar o projeto hoje	Primeiros passos , depois Adaptando ao seu projeto
Quero entender como trabalhar com agentes	A metodologia , Code review e Os portões
Quero dominar a arquitetura	Arquitetura , Dados e bordas e A camada de referência
Quero mudar ou criar uma regra	Guardrails , Hooks de agente e Decisões e porquês
Algo falhou e eu preciso destravar	Vá direto para Quando algo dá errado
Quero ligar ferramentas opcionais	Consulte Integrações de terceiros

1.10 O que vem no pacote

Mapa do produto

Parte	O que entrega
Aplicação de referência	Next.js 16 com autenticação, leitura no servidor, mutação e validação de borda em código real
Arquitetura verificável	Direção entre camadas, limites de dependência e regras específicas do projeto
Gate de qualidade	Lint, formatação, tipos, código morto, testes e guards em um comando
Método com agentes	Skills, comandos e hooks que reforçam o padrão durante a sessão
Fluxo de entrega	Pré-review, proteção de branch e abertura de pull request com critérios claros
Documentação de decisão	Arquivos canônicos e ADRs que registram o motivo das escolhas
Integrações	Guias para CodeRabbit, GitNexus e impeccable

As peças importantes não aparecem apenas em prosa. Quando o guia diz que um comportamento é obrigatório, ele aponta para a regra, o teste ou o script que o verifica. Quando algo depende de julgamento humano, o texto também deixa isso claro.

1.11 O que ainda é seu

O pacote não tenta adivinhar decisões que pertencem ao seu produto. Você ainda precisa:

- apontar a aplicação para o seu backend;
- preencher o `PRODUCT.md` e o `DESIGN.md`;
- trocar nome, licença, responsáveis e placeholders;
- escolher sua estratégia de deploy e fazer o smoke test de produção;
- adicionar E2E, páginas de erro e endurecimento de produção conforme a necessidade;
- decidir quando uma regra do boilerplate deve mudar para atender ao seu contexto.

Essa parte não é uma lacuna escondida. O Stack Sênior entrega a base e o método; produto, domínio e operação continuam sendo decisões suas.

PRÓXIMO PASSO: vá para [Primeiros passos](#)

Abra o projeto em outra janela e execute o capítulo seguinte junto com o terminal. O melhor jeito de entender esta base é vê-la interromper um desvio, explicar o motivo e apontar a saída correta.

PARTE 2

Primeiros passos

O QUE VOCÊ LEVA DESTE CAPÍTULO

Você vai sair da pasta recém-extraída para um projeto rodando, com ambiente configurado e `bun run verify` verde. A ordem dos comandos evita os tropeços mais comuns do primeiro dia.

Este capítulo acompanha o caminho da pasta recém-extraída até o primeiro `bun run verify` verde. Há uma ordem simples para seguir: instalar as dependências, configurar o ambiente, subir o servidor e só então conferir a base. Os gates podem barrar commit, push e até o encerramento do turno de um agente; entender esse percurso evita descobrir isso no pior momento.

A sequência curta, para quem já sabe o que está fazendo:

Terminal 1 – preparar e subir o servidor

```
git init                                # ANTES do bun install (ver 1.2)
bun install                             # instala deps + hooks do lefthook
cp .env.example .env.local              # preencha as 3 variáveis
bun run dev                             # http://localhost:3000
```

Com o servidor aberto, use outro terminal na mesma pasta:

Terminal 2 – conferir a base

```
bun run verify                          # gate canônico: precisa ficar verde
```

O resto do capítulo explica por que cada linha existe e o que quebra se você inverter a ordem.

2.1 Pré-requisitos

2.1.1 Bun 1.4.0-canary.1, e é essa versão mesmo

O `package.json` declara o gerenciador:

package.json (trecho)

```
"packageManager": "bun@1.4.0-canary.1",  
"ignoreScripts": ["sharp", "unrs-resolver"],  
"trustedDependencies": ["sharp", "unrs-resolver", "lefthook"]
```

Esse campo não é decoração de metadado. O CI lê exatamente ele: [.github/workflows/ci.yml](https://github.com/vercel/next.js/blob/main/.github/workflows/ci.yml) usa `bun-version-file: package.json`, com o comentário "Alinhado ao 'packageManager' do package.json (quem gerou o bun.lock)". Versão local diferente da do CI significa lockfile resolvido diferente e o clássico "na minha máquina passa".

CUIDADO | Bun mais novo não é automaticamente melhor

O [MEMORY.md](#) registra em [ADR-028](#) a mudança de baseline: o Bun 1.4.0-canary.1 é o runtime que acompanha o Next 16.3 e o build foi validado com as flags novas. Subir a versão do Bun continua sendo uma mudança de infraestrutura com ADR, não um bun upgrade distraído. Se subir, suba junto o [package.json](#), regenere o [bun.lock](#) e rode `bun run verify` inteiro.

2.1.2 E o Node?

O repositório roda Bun ponta a ponta: dev, build, start e test todos passam por `bun --bun`. Ainda assim existe uma exceção consciente e documentada: o binário do `oxlint` em [node_modules/oxlint/bin/oxlint](#) começa com `#!/usr/bin/env node`. O [VERCEL-OXLINT-MAPPING.md](#) já trata isso: *"o binário do oxlint é um CLI Node (#!/usr/bin/env node): é ferramenta de dev (como git/ripgrep), não o runtime da aplicação"*.

Ou seja: a aplicação não depende de Node, mas a caixa de ferramentas pode encostar nele. Tenha Node no PATH e você não descobre isso do jeito difícil.

POR QUE | Por que não existe .nvmrc nem engines

Não existem no repo, nem [.nvmrc](#), nem [.tool-versions](#), nem campo `engines` no [package.json](#). A versão que importa está travada num lugar só, o `packageManager`, e é o mesmo lugar que o CI consome. Duas fontes de verdade para versão de runtime é como uma delas fica desatualizada em silêncio.

2.2 Ordem das operações: git init vem primeiro

Esta é a pegadinha número um do onboarding, e ela não está documentada em nenhum [.md](#) do produto.

2.2.1 O produto chega sem .git

O boilerplate é entregue como pasta/zip. A cópia que você acabou de extrair **não é um repositório git**; não há diretório `.git`. Isso é esperado; o próprio código antecipa o cenário. Em [tooling/no-lint-suppression.ts](#) (linhas 9–12):

tooling/no-lint-suppression.ts (docblock)

```
Usa `git grep` quando há repositório (rápido e já respeita o .gitignore) e cai pro `grep` recursivo quando ainda não há – o comprador extrai o zip e roda `bun run verify` antes do `git init`.
```

O mesmo raciocínio está registrado em [ADR-020](#) no [MEMORY.md](#). Então sim: `bun run verify` funciona antes do `git init`. O problema é outro.

2.2.2 Os git hooks se instalam sozinhos: se houver onde instalar

Não existe script prepare nem postinstall no `package.json` do projeto, e nenhum documento do repo ensina a rodar `lefthook install`. A instalação acontece implicitamente, pelo `postinstall` do próprio pacote `lefthook`, que executa:

node_modules/lefthook/postinstall.js (trecho)

```
spawnSync(getExePath(), ["install", "-f"], {
  cwd: process.env.INIT_CWD || process.cwd(),
  stdio: "inherit",
});
```

Esse `postinstall` só roda porque `lefthook` está listado em `trustedDependencies` no `package.json` (o Bun não executa scripts de instalação de dependências não confiáveis por padrão).

CUIDADO | `bun install` num diretório sem `.git` não instala hook nenhum

O `lefthook install` precisa de um diretório `.git` para escrever os hooks. Se você rodar `bun install` antes do `git init`, o `postinstall` roda, não encontra repositório, e você fica com **pre-commit e pre-push desligados**; sem nenhum erro visível. O gate parece existir (o `lefthook.yml` está lá) e não roda nunca.

Correção: rode `git init` **antes** do `bun install`. Se já instalou antes, rode `bunx lefthook install` depois do `git init`. O binário está em `node_modules/.bin/lefthook` (versão 2.1.10).

2.2.3 O que os hooks fazem

O `lefthook.yml` tem 22 linhas e vale ler inteiro:

lefthook.yml

```
# Pré-commit: barato e focado nos arquivos staged, pra não travar o
# loop de
# commit. O gate completo é o pré-push.
pre-commit:
  parallel: true
  commands:
    oxlint:
      glob: "{*.ts,tsx,js,jsx}"
      # --no-error-on-unmatched-pattern: não falhar quando os staged
      # files
      # forem todos ignorados (ex.: só .claude/** ou docs).
      run: bunx oxlint --disable-nested-config --no-error-on-unmatched-
      pattern {staged_files}
    oxfmt:
      glob: "{*.ts,tsx,js,jsx,mjs,cjs,json}"
      run: bunx oxfmt --check {staged_files}

pre-push:
  commands:
    verify:
      run: bun run verify
```

REGRA | Dois níveis, propósitos diferentes

Pre-commit é barato e só olha os arquivos staged: oxlint e oxfmt. Serve para não travar o loop de commit.

Pre-push é o gate completo, no projeto inteiro: `bun run verify`. É a última barreira antes do código sair da máquina.

2.3 bun install e a política de supply-chain

O `bunfig.toml` tem seis linhas e uma decisão relevante:

bunfig.toml (arquivo completo)

```
# Só instala versão publicada há pelo menos 3 dias (259200s). Versão
# mais
# recente fica filtrada no `bun install` / `bun add` / `bun update` –
# defesa
# contra supply-chain que ainda não teve tempo de ser detectada/yankada.

# Exceção pontual: `minimumReleaseAgeExcludes = ["pacote"]` no mesmo
# bloco.
[install]
minimumReleaseAge = 259200
```

POR QUE | 259200 segundos = 3 dias

Ataque de supply-chain em registro público tem uma janela: entre a publicação da versão comprometida e o momento em que alguém detecta e faz o yank. Esperar três dias antes de aceitar uma versão nova cobre a maior parte dessa janela sem custo prático; você raramente precisa de um pacote publicado há 40 minutos.

Se precisar de uma versão recém-publicada (correção crítica, por exemplo), a saída documentada é pontual e explícita: `minimumReleaseAgeExcludes = ["pacote"]` no mesmo bloco `[install]`. É uma exceção nomeada, não um desligamento global.

Este é o único ponto do inventário cuja justificativa vive só no comentário do arquivo: não há ADR no [MEMORY.md](#) sobre `minimumReleaseAge`.

O `bun.lock` é versionado. [ADR-023](#) no [MEMORY.md](#) explica: *"o CI roda `bun install --frozen-lockfile`; sem `lockfile` versionado ele falhava no primeiro push... Sem `.gitignore`, o primeiro `git add .` do comprador commitava `node_modules/`, `.next/` e `.env.local`".* Ou seja: o `.gitignore` que veio na caixa é parte do produto, não sobra.

2.4 .env.local: as três variáveis

Passo obrigatório

```
cp .env.example .env.local
```

O `.env.example` é curto e é o documento mais denso desta seção:

`.env.example` (arquivo completo)

```
# Backend REST separado: a arquitetura é BFF → backend (ver
  ARCHITECTURE.md).
# Copie para .env.local e preencha. NÃO commite .env.local.

# Base URL do backend para o BROWSER (login/logout: fluxos onde o Set-
  Cookie
# HttpOnly precisa pousar). Duas formas:
#   "/"                               same-origin (dev: ligue o rewrite
  comentado no
#                                   next.config.ts; prod: domínio same-site).
#   É o único
#                                   jeito de o cookie de sessão pousar sem
  SameSite=None.
#   https://api.example.com   outro origin: exige CORS com credentials
  no backend.
NEXT_PUBLIC_API_URL=https://api.example.com

# Base URL do backend para o SERVIDOR (RSC e Server Actions, via
  kyServer).
# Sempre absoluta: no servidor não existe o proxy same-origin. Validada
  por
# src/lib/env.schema.ts, então ausente ou malformada estoura no boot.
INTERNAL_API_URL=https://api.example.com
```

```
.env.example (arquivo completo) (continuação)

# Fonte do contrato OpenAPI/Swagger (usada pela skill api-contract)
API_CONTRACT_PATH=/caminho/para/openapi
```

As três variáveis, resumidas

Variável	Papel
NEXT_PUBLIC_API_URL	Backend visto pelo browser . Usada só nos fluxos browser-direct (login/logout/refresh). Inlinada no bundle pelo Next; não passa por validação de schema.
INTERNAL_API_URL	Backend visto pelo servidor (RSC e Server Actions, via kyServer). Validada por valibot; ausente ou malformada derruba o boot.
API_CONTRACT_PATH	Caminho do spec OpenAPI/Swagger. Não é lida por código de aplicação: é consumida pelas skills de agente.

2.4.1 NEXT_PUBLIC_API_URL: a decisão é sobre onde o cookie pousa

A função que resolve essa URL cabe na tela e explica o desenho todo:

```
src/lib/api/base-url.ts
```

```
export function getClientApiBaseUrl(): string {
  if (typeof window === "undefined") {
    return "/";
  }

  const baseUrl = process.env.NEXT_PUBLIC_API_URL;
  if (!baseUrl) {
    throw new Error(
      "NEXT_PUBLIC_API_URL não configurada. Copie o .env.example para .env.local e preencha."
    );
  }
  return baseUrl;
}
```

Três coisas para notar.

Primeiro, o early-return sem window. Do docblock do arquivo: *"No servidor a instância é criada mas nunca dispara requisição, então sem window devolve "/" e não exige env: build e prerender ficam livres de configuração"*. É por isso que `bun run build` funciona sem essa variável.

Segundo, os dois formatos aceitos, e o que cada um implica:

Os dois valores possíveis de NEXT_PUBLIC_API_URL

Valor	Consequência
"/" (same-origin)	Em dev vem do rewrite /api/* do <code>next.config.ts</code> (comentado por padrão); em produção, de um domínio same-site. Citando o docblock: <i>"É o único jeito de um cookie HttpOnly de login pousar sem depender de SameSite=None"</i> .
URL absoluta	Backend em outro origin. Exige CORS com <code>Access-Control-Allow-Credentials</code> no backend e cookie <code>SameSite=None; Secure</code> .

Terceiro, quem consome. É só o `kyClient`:

src/lib/api/ky.client.ts (trecho)

```
export const kyClient = ky.create({
  baseUrl: getClientApiBaseUrl(),
  credentials: "include",
  throwHttpErrors: true,
  headers: { Accept: "application/json" },
});
```

Do docblock dele: *"Existe só para os fluxos que precisam do Set-Cookie do backend pousando no browser; login, logout e refresh. Leitura de dado NÃO passa por aqui: é RSC via `kyServer`"*.

CUIDADO | O rewrite de dev vem comentado

Se o seu backend de dev sobe em outro host e você quer "/" same-origin, precisa descomentar o bloco em `next.config.ts`:

next.config.ts (comentado por padrão)

```
// async rewrites() {
//   const apiBaseUrl = process.env.NEXT_PUBLIC_API_URL;
//   if (process.env.NODE_ENV === "production" || !apiBaseUrl) {
//     return [];
//   }
//   return [{ source: "/api/:path*", destination: `${apiBaseUrl}/api/:path*` }];
// },
```

O comentário acima do bloco diz o porquê de vir desligado: *"Em produção o domínio já é same-site, então não há rewrite. Descomente quando o backend de dev subir em outro host."*

2.4.2 INTERNAL_API_URL: validada, falha fechado

Esta variável, ao contrário da anterior, passa por schema:

src/lib/env.schema.ts (trecho)

```
const serverEnvSchema = object({
  /** Base URL do backend para chamadas server-to-server (RSC e Server Actions). */
  INTERNAL_API_URL: pipe(string(), url("INTERNAL_API_URL precisa ser uma URL absoluta.")),
});
```

E o consumo é um arquivo de cinco linhas úteis:

src/lib/env.ts (arquivo completo)

```
import "server-only";

import { parseServerEnv, type ServerEnv } from "@lib/env.schema";

export const serverEnv: ServerEnv = parseServerEnv(process.env);
```

POR QUE | Falha fechado, no boot

O docblock de `parseServerEnv` é explícito: *"Falha fechado: env ausente ou malformada é erro de configuração, e estourar no boot é melhor do que descobrir com um fetch para undefined/rota em produção"*.

E o `import "server-only"` no `env.ts` tem função própria: *"garante que importar isto de um Client Component quebra o build, não o runtime"*. Vazamento de env de servidor para o bundle do browser vira erro de compilação, não incidente.

O consumidor é o `kyServer`, que usa `baseUrl: serverEnv.INTERNAL_API_URL` e encaminha o cookie da requisição para o backend:

src/lib/api/ky.server.ts (trecho)

```
hooks: {
  beforeRequest: [
    async ({ request }) => {
      const cookieHeader = (await cookies()).toString();
      if (cookieHeader) {
        request.headers.set("cookie", cookieHeader);
      }
    },
  ],
},
```

REGRA | A diferença entre as duas, em uma frase

NEXT_PUBLIC_API_URL é inlinada no bundle do browser, nunca passa pelo schema, só é exigida quando window existe e aceita "/" para o cookie pousar same-origin. INTERNAL_API_URL só existe no servidor, é validada com `url()` do valibot e derruba o boot se faltar, porque, nas palavras do `.env.example`, "no servidor não existe o proxy same-origin".

2.4.3 API_CONTRACT_PATH: para os agentes, não para o app

Um `grep` em `src/` não encontra nenhuma ocorrência dessa variável: ela não é lida por código de aplicação. Quem lê são as skills e comandos de agente. A skill `api-contract` abre com este bloco, na "Regra 1":

```
.claude/skills/api-contract/SKILL.md (Regra 1)

source <(grep API_CONTRACT_PATH .env.local) 2>/dev/null
test -n "$API_CONTRACT_PATH" || { echo "STOP: API_CONTRACT_PATH não
    definido"; exit 1; }
test -e "$API_CONTRACT_PATH" || { echo "STOP: path inválido"; exit 1; }
```

E logo abaixo: "Se `$API_CONTRACT_PATH` não existe ou aponta para local inválido → STOP, perguntar ao usuário. Nunca prosseguir inferindo."

A mesma variável também é lida pelos fluxos de sync, feature nova, refatoração e auditoria:

- `.claude/commands/sync-contract.md`;
- `.claude/skills/workflow-new-feature/SKILL.md`;
- `.claude/skills/workflow-refactor/SKILL.md`;
- `.claude/skills/feature-audit/SKILL.md`.

CUIDADO | Preencha em `.env.local`, não em `.env.example`

A skill lê literalmente de `.env.local` (`grep API_CONTRACT_PATH .env.local`). Preencher o `.env.example` não tem efeito, e ainda por cima é um arquivo versionado; você estaria commitando um caminho da sua máquina.

2.4.4 O `.env.local` não vaza

O `.gitignore` já cobre:

.gitignore (linhas 20-22)

```
# env files (can opt-in for committing if needed)
.env*
!.env.example
```

Tudo que começa com `.env` é ignorado, com exceção explícita do exemplo. Não mexa nisso.

2.5 bun run dev

Subir o dev server

```
bun run dev          # http://localhost:3000
```

O script real é `bun --bun next dev`. Duas observações honestas:

- O `--bun` força o runtime Bun em vez de Node. O mesmo prefixo aparece em `build` e `start`.
- **Não há flag `--turbo` explícita** no script. O [CLAUDE.md](#) comenta `# dev server` (Turbopack) e o [README.md](#) lista "Next.js 16 (App Router, Turbopack)"; a afirmação é dos documentos, não do script.

Versões fixadas no [package.json](#): `next 16.3.0`, `react` e `react-dom 19.2.8`, `typescript 7.0.2` com `@typescript/native-preview 7.0.0-dev.20260707.2`, `oxlint 1.77.0`, `oxfmt 0.62.0`, `lefthook ^2.1.10` e `packageManager: bun@1.4.0-canary.1`.

2.6 Todos os scripts

Scripts do `package.json`, verbatim

Script	Comando real e o que faz
dev	<code>bun --bun next dev</code> ; dev server sob runtime Bun.
build	<code>bun --bun next typegen && tsc --noEmit && bun --bun next build</code> ; typegen de rotas tipadas, typecheck do app, build.
start	<code>bun --bun next start</code> ; servidor de produção.
lint	<code>oxlint --disable-nested-config --type-aware</code> ; regras built-in mais o plugin custom boilerplate/*, com análise type-aware.
lint:fix	Idem com <code>--fix</code> .
fmt	<code>oxfmt</code> ; formata (Markdown fica de fora via <code>.oxfmttrc.json</code>).
fmt:check	<code>oxfmt --check</code> ; falha se algo estiver fora do formato.
knip	<code>knip --no-progress</code> ; código, export e dependência mortos.
typecheck	<code>next typegen && tsc --noEmit && tsc --noEmit -p tsconfig.test.json</code> ; app e testes.
typecheck:tooling	<code>tsc --noEmit -p tooling/tsconfig.json</code> ; as regras custom de lint e os agent-hooks.
test	<code>bun test --pass-with-no-tests --parallel</code> .
verify:fast	Subconjunto do loop: lint, fmt:check, knip, test.
verify	O gate canônico. Detalhado a seguir.

Sobre as flags que aparecem repetidas:

- `--disable-nested-config`: decisão registrada no [MEMORY.md](#); “rodar `oxlint` com `--disable-nested-config` (lint/lefthook/CI/Stop hook)”. Uma configuração de lint, sem sobrescrita por diretório.
- `--type-aware`: [ADR-019](#). Liga no-unnecessary-condition, no-unsafe-type-assertion, consistent-return e no-redundant-type-constituents, casando com `oxlint-tsgolint` do `package.json`. Depende da devDependency `oxlint-tsgolint`.
- `knip`: [ADR-017](#). `oxfmt` com Markdown fora: [ADR-018](#) mais `"ignorePatterns": ["**/*.md"]` no `.oxfmttrc.json`.

2.6.1 Por que o typecheck são três projetos

O `verify` roda `tsc` sobre três configurações distintas, e isso não é redundância:

Os três `tsconfig`

Arquivo	Escopo
<code>tsconfig.json</code>	O app. <code>strict: true</code> , <code>paths @/*</code> e <code>@test/*</code> , <code>plugin next</code> . Exclui <code>tooling</code> , <code>test</code> e <code>*.test.ts(x)</code> . A política de acesso indexado acompanha o baseline do Avante (ADR-028).
<code>tsconfig.test.json</code>	Os testes. Estende o raiz, <code>composite: true</code> , <code>types: ["bun"]</code> . Comentário do arquivo: <i>"Separado do tsconfig do app para os globais do Bun não vazarem pro código de produção"</i> .
<code>tooling/tsconfig.json</code>	Plugin <code>oxlint</code> e <code>agent-hooks</code> . <code>module: "Preserve"</code> , <code>allowImportingTsExtensions</code> , <code>types: ["bun"]</code> , e a única concessão do repo: <code>"noImplicitAny": false</code> .

POR QUE | A única exceção de tipagem do repo, e por quê

O `tooling/tsconfig.json` traz `"noImplicitAny": false` com justificativa em comentário: *"A API de plugin do oxlint é alpha e entrega context/node sem tipos estáveis; relaxamos só o implicit-any neste contexto de tooling."*

Relaxado **só ali, só esse flag**, e com o motivo escrito ao lado. É o padrão que o repo espera de qualquer exceção que você venha a precisar: nomeada, escopada e explicada.

2.7 bun run verify: o gate canônico

A string literal, do `package.json`:

O que `verify` realmente executa

```
bun run lint && bun run fmt:check && bun run typecheck && bun run
typecheck:tooling \
  && bun run knip && bun run test \
  && bun tooling/no-lint-suppression.ts && bun tooling/no-heavy-public-
assets.ts
```

Oito etapas encadeadas por `&&`: para na primeira que falhar.

As oito etapas de `bun run verify`

Etapa	O que checa
1. lint	oxlint built-in + plugin boilerplate/*, type-aware.
2. fmt:check	Formatação (oxfmt), sem escrever.
3. typecheck	typegen, depois tsc do app e tsc dos testes.
4. typecheck:tooling	tsc das regras custom de lint e dos agent-hooks.
5. knip	Código, export e dependência não usados.
6. test	bun test em paralelo.
7. no-lint-suppression.ts	Grep, fora do linter, por diretivas de supressão.
8. no-heavy-public-assets.ts	Budget de peso de public/ .

2.7.1 Etapa 7: a regra que não podia ser uma regra de lint

O docblock de [tooling/no-lint-suppression.ts](#) explica o desenho:

tooling/no-lint-suppression.ts (docblock)

```
proíbe diretivas de supressão de lint do ESLint e do oxlint no código-fonte...
não há escape hatch porque uma supressão permitiria burlar os guardrails com o
gate verde. Não é uma regra do plugin oxlint de propósito: uma regra de lint
pode ser desligada pela própria diretiva que ela proíbe. Um grep fora do
linter, não.
```

O detalhe de implementação é bonito e resolve um problema real: o arquivo montaria auto-match se escrevesse as strings literalmente, então elas são construídas em runtime.

tooling/no-lint-suppression.ts (trecho)

```
const NEEDLES = [
  ["eslint", "disable"].join("-"),
  ["oxlint", "disable"].join("-")
];
```

REGRA | Não existe supressão de lint neste repo

Se uma regra te bloqueou, o caminho é seguir o padrão que ela impõe; não desligá-la. O `CODE-PATTERN.md` é literal: "Se uma regra te bloquear, siga o padrão: não suprima a regra." A etapa 7 do `verify` existe exatamente para que a tentativa de contornar isso não passe verde.

2.7.2 Etapa 8: budget de assets

tooling/no-heavy-public-assets.ts (trecho)

```
const BUDGET_BYTES = 200 * 1024;
const glob = new Bun.Glob("public/**/*");
```

Do docblock: "Tudo em `public/` é baixado pelo cliente. Asset pesado na origem é o gargalo nº 1 de LCP... Não há allowlist: exceção por formato ou diretório vira buraco no gate." A mensagem de falha é Asset acima do budget de 200 KB em `public/`.

2.7.3 `verify` contra `verify:fast`

O que cada gate cobre

Etapa	verify	verify:fast
lint	sim	sim
fmt:check	sim	sim
typecheck	sim	não
typecheck:tooling	sim	não
knip	sim	sim
test	sim	sim
no-lint-suppression	sim	não
no-heavy-public-assets	sim	não

Onde cada um está efetivamente conectado:

Wiring dos gates

Gatilho	Comando
Pre-push (lefthook)	<code>bun run verify</code>
CI(.github/workflows/ci.yml)	<code>bun run verify</code> , no step "Gate canônico"
Hook Stop de agente	<code>bun run verify:fast</code> , via tooling/agent-hooks/verify-before-stop.ts

O hook de Stop bloqueia o encerramento do turno do agente com esta mensagem: "Stop bloqueado: 'bun run verify:fast' falhou. Corrija os erros do gate e só então encerre o turno." O timeout dele está em [.claude/settings.json](#): 300000 ms.

REGRA | O gate canônico é um só

O [AGENTS.md](#) e o [CLAUDE.md](#) convergem: *"bun run verify: o gate canônico. É o que roda no pré-push e no CI"* e *"É o único gate que importa antes de concluir"*. O `verify:fast` é conveniência de loop; use durante a iteração, nunca como prova de que terminou.

2.8 Tornar o boilerplate seu

Não existe checklist consolidado no repo: os pontos de personalização estão espalhados em comentários de arquivo. Esta é a lista, levantada arquivo a arquivo.

Checklist de apropriação

Arquivo	O que trocar
package.json	"name": "boilerplate"; troque pelo nome do seu projeto. Também version e license (o LICENSE.md existe).
CLAUDE.md	Seção "Branch base (PRs)".
.github/CODEOWNERS	26 ocorrências de @owner.
.github/workflows/ci.yml	example\ com no grep-gate de backend.
PRODUCT.md	Template inteiro, com placeholders.
DESIGN.md	Frontmatter YAML: nome, fontes, paleta.
.env.local	API_CONTRACT_PATH apontando para o seu spec.
src/proxy.ts	Nome do cookie de sessão e o matcher das rotas protegidas.
knowledge.md	Não existe; só o knowledge.template.md . É você quem gera.

2.8.1 Branch base (CLAUDE.md)

O bloco em disco, verbatim:

CLAUDE.md, seção "Branch base (PRs)"

```
`main` (default trunk-based). **Edite esta linha** se o time usa outro
fluxo
(ex.: `develop`/gitflow). As skills `pr-ready`/`pre-review`/`pr-
creation` e os
hooks `guard-shell`/`pre-pr-review-gate` leem daqui; se estiver no
default e
ainda não confirmado, o agente pergunta **uma vez** antes de abrir PR e
grava a
escolha aqui: depois não pergunta mais.
```

O [ARCHITECTURE.md](#) reforça: "Modelo de branch é escolha do time: o boilerplate não impõe um fluxo. Default: trunk-based em main". Se o seu time usa gitflow, edite essa linha antes do primeiro PR; ela é lida por skills e por hooks que bloqueiam push em branch protegida.

2.8.2 CODEOWNERS

O cabeçalho do [.github/CODEOWNERS](#) diz o que está em jogo:

`.github/CODEOWNERS` (cabeçalho)

```
# Ownership dos guardrails. O enforcement de verdade depende de branch
# protection
# / ruleset no GitHub: este arquivo só declara quem revisa.
#
# Troque `@owner` pelo seu usuário/time. As superfícies abaixo são as
# que, se
# mudarem sem revisão, desligam o método em silêncio – hooks de agente,
# regras de
# lint, docs canônicos e a config dos gates.
```

As 26 linhas cobrem três blocos: superfícies de agente (`/.github/**`, `/.agents/**`, `/.claude/**`, `/.codex/**`, `/.cursor/**`, `/tooling/**`), docs canônicos (`/AGENTS.md`, `/CLAUDE.md`, `/ARCHITECTURE.md`, `/CODE-PATTERN.md`, `/MEMORY.md`, `/VERCEL-OXLINT-MAPPING.md`) e config de gates (`/package.json`, `/bun.lock`, `/bunfig.toml`, `/lefthook.yml`, `/.oxlintrc.json`, `/.oxfmttrc.json`, `/knip.json`, `/next.config.ts`, `/tsconfig*.json`, `/.env.example`, `/.gitignore`).

CUIDADO | CODEOWNERS sozinho não impõe nada

O próprio arquivo avisa: o enforcement depende de branch protection ou ruleset configurado no GitHub. Trocar `@owner` sem ligar a proteção de branch deixa você com um arquivo declarativo bonito e zero garantia.

2.8.3 O domínio no grep-gate do CI

O CI tem dois grep-gates que servem de backstop caso o plugin custom de lint (API alpha) falhe em carregar. Um deles precisa da sua edição:

`.github/workflows/ci.yml` (grep-gate de backend)

```
- name: Grep-gate, backend só em src/lib/api
  run: |
    # Backend = URL http(s) do domínio do backend fora de src/lib
    /api.
    # `-o` extrai uma URL por linha (não a linha inteira): sem
    isso, uma linha
    # com uma URL pública E uma de backend juntas seria
    descartada pelo
    # allowlist, escondendo a violação. Troque `example\.com`
    pelo seu domínio.
    if grep -rEon "https?://[^\"]*example\.com" src --include='
    *.ts' --include='*.tsx' --include='*.js' --include='*.jsx' 2>/dev/
    null | grep -vE '^src/lib/api/'; then
      echo "::error::URL do backend fora de src/lib/api"; exit 1
    fi
```

O outro, o de "sempre fresco", não precisa de edição; ele barra `'use cache'` e `unstable_cache` em `src/` e o padrão já é genérico.

CUIDADO | example.com é placeholder puro

Verificado: `example.com` não aparece em nenhum arquivo de `src/`. Enquanto você não trocar pelo seu domínio, esse gate está **sempre verde**, e verde por não estar procurando nada. Os paths do backend ficam em `src/lib/api/endpoints.ts` (`api/v1/login`, `api/v1/perfil`), sem host.

Nota útil sobre build: o CI injeta env placeholder no step de build, o que confirma que valores fictícios bastam para compilar.

`.github/workflows/ci.yml` (step de build)

```
- name: Build (typegen + typecheck + next build sob Bun)
  run: bun run build
  env:
    NEXT_PUBLIC_API_URL: http://localhost:3333
    INTERNAL_API_URL: http://localhost:3333
```

2.8.4 PRODUCT.md e DESIGN.md

Ambos são templates com placeholders entre `<>`, e ambos alimentam decisões dos agentes.

O `PRODUCT.md` abre com: *"Template. O Claude lê este arquivo para entender para quem e por quê: alimenta decisões de produto, UX e priorização. Escreva curto e concreto; nada de marketing vazio."* Seções a preencher: Register, Users, Product Purpose, Brand Personality, Não-objetivos (opcional).

O `DESIGN.md` é mais estrutural: o frontmatter YAML **é a fonte da verdade de tokens**; *"código consome daqui, nunca valores mágicos"*. O default é deliberadamente neutro: `primary: '#000000'`, `surface: '#FFFFFF'`, `danger: '#EF4444'`, `spacing.base: 4`, raios `sm 6 / md 10 / lg 16`, mais um bloco `dm-*` opcional de dark mode. As fontes vêm como `'<ex.: Inter_700Bold>'`. Seções a escrever: Princípios visuais, Acessibilidade, Componentes-chave (opcional).

2.8.5 Cookie de sessão e rotas protegidas

O `src/proxy.ts` traz um ajuste que quase todo mundo vai precisar fazer. Do docblock: *"Trocar o nome aqui é o único ajuste se o seu backend usar outro."* A constante é `const SESSION_COOKIE = "session_token";`

O matcher das áreas autenticadas vem como `matcher: ["/perfil/:path*"]`, com a nota: *"Entre por prefixo, não rota a rota: listar rota a rota deixa toda rota nova sem gate por padrão, que é o erro caro."* Siga a orientação ao adicionar suas áreas: prefixo, não enumeração.

2.8.6 Dois slots que ficam vazios de propósito

- O `CLAUDE.md` tem um slot de integração: `<!-- Cole aqui o bloco de integração do GitNexus, se usar: ver integracoes/gitnexus.md -->`.

- O `knowledge.md` é citado no `CLAUDE.md` como artefato de memória do projeto, mas **não existe** no repo; só o `knowledge.template.md`. É você (ou seu agente) quem gera, por varredura do código.

2.9 Divergências conhecidas entre docs e código

Levantadas na leitura dos arquivos reais, listadas aqui para que você confie no código e não no texto quando os dois discordarem.

Documentação contra realidade

Fonte	Divergência encontrada
<code>ARCHITECTURE.md</code>	Diz que o pre-commit roda <code>oxlint</code> , <code>typecheck</code> e testes. O <code>lefthook.yml</code> roda <code>oxlint</code> e <code>oxfmt</code> .
<code>VERCEL-OXLINT-MAPPING.md</code>	Repete a divergência do pre-commit e diz que o hook <code>Stop</code> inclui <code>typecheck</code> . Na prática, ele roda <code>verify:fast</code> , sem <code>typecheck</code> .
<code>no-heavy-public-assets.ts</code>	Diz que o pre-push chama o script diretamente. O pre-push chama <code>bun run verify</code> , que então executa o script.

REGRA | Na dúvida, o arquivo executável vence

`lefthook.yml`, `package.json` e `.github/workflows/ci.yml` são a verdade operacional: são eles que rodam. Os `.md` descrevem intenção. Quando divergirem, corrija o `.md`, e, já que os docs canônicos estão sob CODEOWNERS, a correção passa por review.

PARTE 3

Adaptando ao seu projeto

O QUE VOCÊ LEVA DESTE CAPÍTULO

Você vai transformar o boilerplate em uma base do seu produto sem desligar o método pela metade. O capítulo organiza nome, backend, branch, identidade visual e exemplos na ordem mais segura.

O boilerplate chega com nome provisório, backend apontando para `api.example.com`, EULA com placeholders, uma feature de exemplo (perfil) e um design system em formato de template. Esses pontos ficaram abertos porque pertencem ao seu produto; não são detalhes para esconder dentro de um script de renomeação.

Esta seção é o roteiro do primeiro dia: o que trocar, em que ordem e o que precisa ser conferido quando uma mudança atravessa mais de um arquivo. Todos os caminhos são relativos à raiz do repo.

REGRA | O gate é o seu critério de done

Depois de cada bloco de mudanças desta seção, rode `bun run verify`. Ele é a composição canônica definida no `package.json`: `lint` → `fmt:check` → `typecheck` → `typecheck:tooling` → `knip` → `test` → `no-lint-suppression.ts` → `no-heavy-public-assets.ts`. O mesmo comando roda no `pre-push` (`lefthook.yml`) e no CI. Se você adaptar o repo sem rodar o `verify`, vai descobrir o estrago no push.

3.1 Ordem sugerida

A ordem importa porque alguns passos dependem de outros (os tokens antes do plugin de Tailwind; o backend antes de qualquer feature real).

Roteiro de adaptação, do primeiro ao último passo

#	Passo	Por que nessa posição
1	Nome e licença	Barato, e evita commitar "name": "boilerplate" para sempre
2	Backend (<code>.env.local</code> , <code>envelope</code> , <code>endpoints</code> , <code>cookie</code>)	Sem isso nada roda de verdade
3	Branch base	Antes do primeiro PR, senão o hook te barra no meio do fluxo
4	<code>PRODUCT.md</code> e <code>DESIGN.md</code>	O agente lê os dois antes de decidir qualquer coisa de UI
5	<code>knowledge.md</code>	Gerado por varredura: melhor depois de o código já ser seu
6	Grep-gate de domínio no CI	Depende de você já saber o domínio do backend
7	Remover a feature de exemplo	Só depois de existir uma rota real para substituí-la
8	Afrouxar/apertar regras de lint	Último: só afrouxe o que você provou que atrapalha
9	Ligar o <code>oxlint-tailwindcss</code>	Depende do passo 4 estar concluído no <code>@theme</code>

3.2 Nome do projeto e licença

3.2.1 Onde a string “boilerplate” realmente aparece

Não existe script de rename no repo. São poucos lugares e todos são edição manual:

Ocorrências do nome que você deve trocar

Arquivo	O que trocar
<code>package.json</code>	"name": "boilerplate" (linha 2); confira também "version": "0.1.0" e "license"
<code>src/app/layout.tsx</code>	title e description do metadata
<code>src/app/page.tsx</code>	A home de scaffold (“Boilerplate pronto. Edite ...”)
<code>ARCHITECTURE.md</code>	Título e a frase que descreve o app como frontend / BFF
<code>README.md</code>	Título

```
src/app/layout.tsx, metadata a trocar
```

```
export const metadata: Metadata = {
  title: "Next.js Boilerplate",
  description:
    "Boilerplate Next.js premium: arquitetura imposta por lint.",
};
```

CUIDADO | Não troque a palavra “boilerplate” com find and replace global

As demais ocorrências dentro de `src/` são referências ao *prefixo do plugin de lint* (boilerplate/no-forbidden-imports, boilerplate/enforce-boundaries) dentro de JSDoc (ex.: `src/lib/env.schema.ts`, `src/lib/api/__tests__/client.test.ts`). Esse prefixo vem do `meta.name` declarado em `tooling/oxlint-rules/index.ts` e é referenciado no `.oxlintrc.json`. Renomear em massa desliga regras silenciosamente ou quebra o registro do plugin.

3.2.2 A licença

O `LICENSE.md` é um EULA comercial em PT-BR com placeholders literais: <NOME DO PRODUTO>, <LICENCIANTE> e <CONTATO/EMAIL>. O próprio arquivo avisa, na linha 5, que é um modelo e que deve ser revisado por um advogado antes de comercializar. A seção 4 registra que o scaffold inicial deriva do `create-next-app` (MIT) e que cada dependência mantém a própria licença.

POR QUE | Esse EULA é a licença do produto, não da sua app

O `LICENSE.md` entregue licencia o *boilerplate* para quem o comprou. Se você comprou para construir a sua aplicação, o caminho normal é substituir o arquivo inteiro pela licença da sua empresa (ou removê-lo, se o repo é privado) e ajustar o campo `"license"` do `package.json`, hoje `"SEE LICENSE IN LICENSE.md"`. Manter o EULA do vendedor no seu repo é herdar termos que não são seus.

3.2.3 CODEOWNERS

O `.github/CODEOWNERS` vem com o placeholder `@owner` em mais de vinte linhas e explica no topo por que essas superfícies estão listadas: são as que, se mudarem sem revisão, “desligam o método em silêncio”. Ele cobre `/.github/**`, `/.agents/**`, `/.claude/**`, `/.codex/**`, `/.cursor/**`, `/tooling/**`, os seis documentos canônicos e todas as configs de gate (`package.json`, `bun.lock`, `bunfig.toml`, `lefthook.yml`, `.oxlintrc.json`, `.oxfmttrc.json`, `knip.json`, `next.config.ts`, `tsconfig*.json`, `.env.example`, `.gitignore`).

Troque `@owner` pelo seu usuário ou time. Sem isso o arquivo é inerte: o GitHub não exige review de ninguém.

3.3 Apontar para o seu backend

A arquitetura é BFF → backend REST separado. Adaptar o backend é o passo com mais superfície, mas cada peça tem um único lugar.

3.3.1 As variáveis de ambiente

Primeiro comando depois do clone

```
cp .env.example .env.local
```

As três variáveis do .env.example

Variável	O que é
NEXT_PUBLIC_API_URL	Base URL do backend <i>para o browser</i> : login/logout, os fluxos onde o Set-Cookie HttpOnly precisa pousar
INTERNAL_API_URL	Base URL <i>para o servidor</i> (RSC e Server Actions, via kyServer). Sempre absoluta
API_CONTRACT_PATH	Caminho da fonte do contrato OpenAPI/Swagger, lido pela skill api-contract

REGRA | INTERNAL_API_URL falha fechado no boot

Ela é validada por `src/lib/env.schema.ts` com `pipe(string(), url(...))`, e `src/lib/env.ts` importa "server-only" e roda `parseServerEnv(process.env)` uma vez por processo. Ausente ou malformada = erro no boot, não erro em runtime no meio de um request.

O `NEXT_PUBLIC_API_URL` *não* passa pelo schema; o Next inlina essa variável no bundle do cliente. Quem valida é `src/lib/api/base-url.ts`:

src/lib/api/base-url.ts, a validação do lado do browser

```
export function getClientApiBaseUrl(): string {
  if (typeof window === "undefined") {
    return "/";
  }

  const baseUrl = process.env.NEXT_PUBLIC_API_URL;
  if (!baseUrl) {
    throw new Error(
      "NEXT_PUBLIC_API_URL não configurada. Copie o .env.example para .env.local e preencha.",
    );
  }
  return baseUrl;
}
```

O retorno "/" sem window é deliberado: build e prerender não exigem configuração de env do cliente.

3.3.2 Same-origin ou cross-origin: a decisão do cookie

O `.env.example` documenta os dois formatos aceitos por `NEXT_PUBLIC_API_URL`, e a escolha é sobre onde o cookie de sessão pousa:

Os dois formatos e o que cada um exige

Valor	O que exige
/	Same-origin. Em dev, ligar o rewrite comentado no <code>next.config.ts</code> ; em produção, domínio same-site. É o único jeito de o cookie pousar sem <code>SameSite=None</code>
<code>https://api.example.com</code>	Outro origin. Exige CORS com <code>Access-Control-Allow-Credentials</code> no backend e cookie <code>SameSite=None; Secure</code>

Se o seu backend de dev sobe em outro host, descomente o bloco de rewrites do `next.config.ts` (linhas 12–23). Ele já está escrito, só desativado:

next.config.ts, o rewrite de dev a descomentar

```
// async rewrites() {
//   const apiBaseUrl = process.env.NEXT_PUBLIC_API_URL;
//   if (process.env.NODE_ENV === "production" || !apiBaseUrl) {
//     return [];
//   }
//   return [{ source: "/api/:path*", destination: `${apiBaseUrl}/api/:path*` }];
// },
```

Note que ele desliga sozinho em produção: o comentário no arquivo explica que em produção o domínio já é same-site, então não há rewrite.

3.3.3 O formato do envelope

O boilerplate assume um envelope fixo em toda resposta do backend, sucesso e erro. Se o seu backend responde outro formato, existe exatamente um lugar para mudar.

src/lib/api/client.ts, o type do envelope

```
type ApiEnvelope = {
  success: boolean;
  message: string;
  data?: Json;
  pagination?: Json;
};
```

O JSDoc acima dele diz textualmente: “Se o seu backend responde outro formato, este type e o requestEnvelope são o ÚNICO lugar a mudar; o resto do app não conhece o envelope.”

A função requestEnvelope é a borda de erro. Quando recebe um `HTTPError` com uma message válida, ela preserva a mensagem e o erro original no cause. Nos demais casos, usa `CONNECTION_ERROR_MESSAGE` e mantém a causa para diagnóstico.

POR QUE | Por que a validação usa Standard Schema e não a API do valibot

Em `src/lib/api/client.ts` a validação roda por `await dataSchema["~standard"].validate(envelope.data)`. O lint confina `import "valibot"` aos `*.schema.ts`; usar a interface Standard Schema deixa `lib/api` validar sem acoplar a um validador específico (ADR-022). Se você trocar o validador do projeto, o gateway não muda.

Dois detalhes que valem customização imediata:

- `CONNECTION_ERROR_MESSAGE` é texto de UI em PT-BR (“Não foi possível concluir a solicitação. Tente novamente em instantes.”). Se o seu produto é multilíngue, esse é o primeiro ponto de i18n.

- `apiClient` tem duas sobrecargas: com `dataSchema` (retorna o `InferOutput`) e com `dataSchema?: undefined` (retorna `undefined`, para endpoints sem data; o login, cuja identidade vem do cookie).

O rodapé do arquivo é o gancho oficial para paginação:

`src/lib/api/client.ts`, o comentário final

```
// Sem variante paginada aqui de propósito: o repo proíbe abstração sem caller
// real (YAGNI, ver CODE-PATTERN §9). Quando o seu primeiro endpoint devolver
// `pagination` como irmã de `data` no envelope, extraia uma `apiClientPaginated`
// reaproveitando o `requestEnvelope` acima – a borda já está pronta pra isso.
```

3.3.4 Os paths

`src/lib/api/endpoints.ts` tem 23 linhas e é a fonte única dos paths REST, agrupados por domínio:

`src/lib/api/endpoints.ts`

```
export const API_ENDPOINTS = {
  /** Sessão do usuário. Chamado browser-direct: o backend emite o cookie. */
  auth: {
    login: "api/v1/login",
  },

  /** Perfil do usuário autenticado – a feature de exemplo do boilerplate. */
  perfil: {
    me: "api/v1/perfil",
  },
} as const;
```

REGRA | Path relativo, sem barra inicial

Cada valor é relativo ao `baseUrl` da instância `ky`. A base vive em `src/lib/api/ky.server.ts` (`baseUrl: serverEnv.INTERNAL_API_URL`) e `src/lib/api/ky.client.ts` (`baseUrl: getClientApiBaseUrl()`, com `credentials: "include"`). Endpoint novo entra aqui no domínio correspondente, nunca como `const` local dentro de uma feature.

O `kyServer` tem um hook `beforeRequest` que faz `(await cookies()).toString()` e repassa o header `cookie` para o backend; é assim que o RSC autentica sem o front nunca ler o token. O `kyClient` existe só para login, logout e refresh.

3.3.5 O nome do cookie de sessão

src/proxy.ts, a constante e o matcher

```
const SESSION_COOKIE = "session_token";

export const config = {
  matcher: ["/perfil/:path*"],
};
```

O JSDoc é explícito: o cookie é a fonte de verdade de “tem sessão”, o front nunca decodifica, valida ou assina o JWT que ele carrega (ADR-007), e “trocar o nome aqui é o único ajuste se o seu backend usar outro”. O [ARCHITECTURE.md](#) prevê também um `refresh_token` (HttpOnly, só com `rememberMe`), mas só o `session_token` está no código.

CUIDADO | O matcher entra por prefixo: e é gate de roteamento, não de autorização

O JSDoc do [src/proxy.ts](#) avisa: “Entre por prefixo, não rota a rota: listar rota a rota deixa toda rota nova sem gate por padrão, que é o erro caro.” Ao adaptar, troque `"/perfil/:path*"` pelo prefixo da sua área autenticada (ex.: `"/app/:path*"`), não pela lista das telas que existem hoje. E lembre: a autorização real é sempre do backend; o proxy só evita largar o usuário numa tela que responderia 403.

A função inteira tem seis linhas. Quando você precisar de refresh proativo (relayando o `Set-Cookie` da resposta), é aqui que ele entra; o próprio JSDoc e o [ARCHITECTURE.md](#) apontam esse ponto de extensão.

3.4 Trocar a branch base

O [CLAUDE.md](#) tem uma seção `## Branch base` (PRs) cujo conteúdo é literalmente uma linha a editar: `main` (default trunk-based), com a instrução de trocar se o time usa outro fluxo.

CUIDADO | Editar o CLAUDE.md muda as skills, não os hooks

Verificação feita nos scripts: nenhum dos hooks abre ou parseia o [CLAUDE.md](#). Eles têm a base *hardcoded* e apenas *citam* o documento na mensagem de erro. Se você editar só a linha do [CLAUDE.md](#), o agente vai passar a mirar `develop`, e o hook vai vetar o `gh pr create` mesmo assim.

Todos os pontos a editar para trocar a base de verdade

Arquivo	O que tem lá
CLAUDE.md	A seção <i>Branch base (PRs)</i> ; lida pelas skills
pre-pr-review-gate.ts	<code>const BASE_BRANCH = "main"</code> , usada inclusive no veto
guard-shell.ts	Lista de branches protegidas
inject-patterns.ts	Texto com <code>main</code> injetado em toda sessão
pre-pr-review-gate.test.ts	Testes do comportamento com <code>main</code> e <code>develop</code>
guard-shell.test.ts	Repete a lista de branches protegidas

O veto do gate é este:

`tooling/agent-hooks/pre-pr-review-gate.ts`, o veto de base

```
if (base !== BASE_BRANCH && !base.startsWith("feature/"))
```

CUIDADO | Mudar a constante sem mudar os testes quebra o `bun test`

`tooling/agent-hooks/pre-pr-review-gate.test.ts` tem o caso *"denies a base that is neither main nor a feature branch"*, que roda `gh pr create --base develop` e espera uma razão contendo "base informada foi develop" e `main`. Se `BASE_BRANCH` virar "develop", esse teste falha, e `bun test` está dentro do `verify`. Trocar a base é uma mudança de 6 arquivos, não de 1.

3.4.1 Quem consome a seção

Vale saber o que muda quando você edita a linha do `CLAUDE.md`: `.claude/skills/pr-creation/SKILL.md` ("Base: lida da seção *Branch base* do `CLAUDE.md` (default `main`). Não assuma: confirme na Fase 0"), `.claude/skills/pr-ready/SKILL.md` (os checks #5/#6 rodam `git diff main --numstat`, com instrução de substituir pela base configurada), `.claude/skills/pre-review/SKILL.md` e `.claude/commands/pre-review.md`. O `ARCHITECTURE.md` e o ADR-013 no `MEMORY.md` registram que o modelo de branch é escolha do time.

POR QUE | O que o gate de PR está protegendo

O `pre-pr-review-gate` bloqueia `gh pr create` enquanto não existir o selo `.agents/tmp/pre-review-<HEAD>.ok`, e trava PR acima de `MAX_PR_LOC = 3000` linhas (adições + remoções contra `origin/<base>`) sem um `ack` explícito. A cadeia oficial (ADR-021) é `/pr-ready → /pre-review → /create-pr`. Os três hosts (`.claude/settings.json`, `.codex/hooks.json`, `.cursor/hooks.json`) apontam para os

mesmos scripts, resolvidos via `git rev-parse --show-toplevel || pwd`; o fallback existe porque o comprador costuma abrir o agente antes do `git init` (ADR-016).

3.5 Preencher o **PRODUCT.md** e o **DESIGN.md**

Os dois são templates, e os dois são lidos por agente. O **PRODUCT.md** abre com a instrução: “O Claude lê este arquivo para entender **para quem e por quê**: alimenta decisões de produto, UX e priorização. Escreva curto e concreto; nada de marketing vazio.” As seções são Register, Users, Product Purpose, Brand Personality e Não-objetivos (opcional).

O **DESIGN.md** tem 72 linhas: um frontmatter YAML que é a fonte da verdade de tokens, mais um corpo com princípios visuais, uso dos tokens, acessibilidade e componentes-chave. O frontmatter traz 17 chaves de cor (incluindo quatro `dm-*` de dark mode), quatro papéis tipográficos (headline, title, body, caption), `spacing.base: 4` (escala 4pt) e radius em `sm/md/lg`; tudo com valores placeholder.

REGRA | Tokens definidos uma vez; telas não inventam valores

A skill design-system é categórica: os tokens vivem no **DESIGN.md** e nas CSS vars / `@theme` do Tailwind v4, definidos uma única vez. “Valor novo → entra no **DESIGN.md**/tema primeiro.” Os anti-padrões que reprovam review são: cor, spacing ou tipografia hardcoded; CSS global solto; misturar dois sistemas de estilo; remover outline de foco.

3.5.1 O trabalho que sobra para você: levar os tokens ao `@theme`

O `src/app/globals.css` ainda é o scaffold do create-next-app; 27 linhas, com apenas quatro entradas no `@theme inline`:

`src/app/globals.css`, o estado atual

```
@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
}
```

Ou seja: os tokens do frontmatter do **DESIGN.md** *ainda não estão* no tema. Levá-los para lá é trabalho do adotante, e é pré-condição de vários outros passos (ver § do `oxlint-tailwindcss`). As fontes atuais são Geist e Geist_Mono via `next/font/google` em `src/app/layout.tsx`; exatamente o tipo de default genérico que a skill design-system manda evitar.

3.5.2 Com o impeccable

O [integracoes/impeccable.md](#) documenta a integração opcional com o impeccable (projeto de terceiros, de Paul Bakaus, Apache-2.0): uma skill com 23 comandos mais um CLI detector com 41 regras determinísticas de “AI slop”, sem LLM e sem API key.

Instalação e init, conforme [integracoes/impeccable.md](#)

```
npx impeccable skills install
# ou, via plugin:
# /plugin marketplace add pbakaus/impeccable
# /plugin install impeccable

# depois, uma vez:
/impeccable init
```

O `init` pergunta se a superfície é **brand** (marketing/landing) ou **product** (app/dashboard) e escreve o [PRODUCT.md](#) e o [DESIGN.md](#). Como o boilerplate já traz os dois como template, a recomendação do arquivo é deixar o `init` preenchê-los; ou usar `/impeccable document` para gerar o [DESIGN.md](#) a partir do código existente. O terceiro passo é registrar no [CLAUDE.md](#) o bloco delimitado por `<!-- impeccable:start -->` e `<!-- impeccable:end -->`, que está pronto no próprio [integracoes/impeccable.md](#). Há ainda um gate opcional: `npx impeccable detect src/` (com `--fast --json` para saída programática).

POR QUE | O método não depende do impeccable

A seção `## Sem impeccable` do arquivo é explícita: “O [DESIGN.md](#) sozinho já dá disciplina visual... o método não depende dele para funcionar.” Se você não quer uma dependência externa no fluxo de design, preencha o [DESIGN.md](#) à mão e siga a skill `design-system`. O que não é opcional é ter os tokens definidos em *algum* lugar canônico.

3.6 Gerar o [knowledge.md](#)

O [knowledge.md](#) não existe no repo entregue; só o [knowledge.template.md](#). Isso é proposital: ele é gerado por varredura do *seu* código, não do código de exemplo.

knowledge.template.md, a instrução de geração

```
> Template. A `knowledge.md` é a base de conhecimento do código,
gerada por
> varredura sistemática do código-fonte e mantida ao longo do tempo.
Ela compõe:
> cada vez que o agente entende algo novo do sistema, registra aqui, e
a próxima
> sessão não precisa redescobrir.
>
> Como gerar: peça ao Claude Code para varrer o repositório (mó-
dulos, fluxos,
> integrações) e escrever esta página baseada exclusivamente no có-
digo
> existente: pontos em aberto viram perguntas ao time, não suposições.

> Reindexe quando a arquitetura mudar. Potencializa com GitNexus
> (ver `integracoes/gitnexus.md`).
```

A estrutura a preencher é: Visão geral, Arquitetura & stack (com um bloco de diagrama em texto), Módulos / domínios, Fronteiras e integrações, Convenções não-óbvias e gotchas e Como manter.

REGRA | Divisão de responsabilidade entre knowledge.md e MEMORY.md

O template é explícito: “As **decisões arquiteturais e o porquê** vão no **MEMORY.md** (log de ADRs): aqui fica só o **mapa do sistema** e os gotchas.” As regras de manutenção: atualizar ao criar ou alterar módulo; basear no código, nunca na memória; citar caminhos no formato `src/...:linha`.

3.7 Ajustar o grep-gate de domínio no CI

O `.github/workflows/ci.yml` tem um job único (gate) com dois grep-gates que rodam *antes* do `bun run verify`. Um deles precisa da sua edição.

POR QUE | Por que existe grep-gate se já existe lint

ADR-009: `jsPlugins` do `oxlint` é alpha, sem semver. Por isso o `oxlint` está pinado (hoje `1.77.0` no `package.json`) e o CI mantém grep-gates como backstop. O comentário no workflow diz: “se o plugin um dia falhar em carregar, estes gates ainda barram as violações.”

O primeiro gate barra cache (`'use cache'` e `unstable_cache`) em `src/` e não precisa de customização. O segundo é o que você edita:

.github/workflows/ci.yml, o gate a customizar (linha do grep quebrada aqui para caber)

```
- name: Grep-gate, backend só em src/lib/api
  run: |
    # Backend = URL http(s) do domínio do backend fora de src/lib
    /api.
    # `-o` extrai uma URL por linha (não a linha inteira): sem
    isso, uma linha
    # com uma URL pública E uma de backend juntas seria
    descartada pelo
    # allowlist, escondendo a violação. Troque `example\.com`
    pelo seu domínio.
    if grep -rEon "https?://[^\"] ]*example\.com" src
      --include='*.ts' --include='*.tsx' --include='*.js' --
      include='*.jsx'
      2>/dev/null | grep -vE '^src/lib/api/'; then
        echo "::error::URL do backend fora de src/lib/api"; exit 1
      fi
```

O que trocar é o literal `example\.com` pelo domínio real do seu backend. É regex ERE; o ponto vai escapado.

Vale também conferir os placeholders do passo de build, que hoje são locais:

.github/workflows/ci.yml, env do build

```
env:
  NEXT_PUBLIC_API_URL: http://localhost:3333
  INTERNAL_API_URL: http://localhost:3333
```

O comentário justifica: “A borda falha fechada sem a base URL; no build nenhuma request sai de verdade (o prerender vira dinâmico no `cookies()`), então um placeholder local basta.”

3.8 Remover a feature de exemplo (perfil)

O perfil existe para mostrar o padrão completo funcionando: RSC lendo via `apiClient`, schema na borda, Server Action, hook de formulário e componente client. Quando você tiver a sua primeira feature real, remova-o.

Os cinco arquivos da feature

Arquivo	LOC
src/app/perfil/page.tsx	48
src/features/perfil/perfil.schema.ts	63
src/features/perfil/perfil.actions.ts	45
src/features/perfil/use-perfil-form.ts	55
src/features/perfil/perfil-form.tsx	77

Apagar os cinco não basta: há três referências fora da feature.

Referências a resolver depois de apagar

Local	O que acontece se esquecer
src/lib/api/endpoints.ts	O bloco perfil: { me: ... } fica órfão
src/proxy.ts	O matcher: ["/perfil/:path*"] aponta para rota inexistente: o gate deixa de proteger qualquer coisa
src/app/login/page.tsx	Quebra o typecheck

CUIDADO | typedRoutes transforma a remoção de rota em erro de compilação

[src/app/login/page.tsx](#) declara `const DESTINO_POS_LOGIN: Route = "/perfil"`. Como o [next.config.ts](#) tem `typedRoutes: true`, o next typegen gera um union com as rotas existentes. Removida a pasta [src/app/perfil](#), `"/perfil"` some do union e `bun run typecheck` falha. O tipo é consumido por `useLoginForm(destino: Route)` em [src/features/auth/use-login-form.ts](#). Isso é o sistema funcionando: você não consegue deixar um destino de login apontando para o nada.

Sequência segura de remoção

```
rm -rf src/app/perfil src/features/perfil
# depois, à mão:
# 1. remova o bloco `perfil` de src/lib/api/endpoints.ts
# 2. troque DESTINO_POS_LOGIN em src/app/login/page.tsx por uma rota
   que exista
# 3. ajuste o matcher em src/proxy.ts para o prefixo da sua área
   autenticada
bun run verify
```

POR QUE | O knip é a rede de segurança do meio-caminho

O `knip.json` declara "entry" com `src/app/**/*.{ts,tsx}` (e como `src/features/**/*.{ts,tsx}` só é alcançável por import a partir de `src/app/**/*.{ts,tsx}`, apagar a página e esquecer os arquivos da feature faz o knip reportar os quatro órfãos. O knip está dentro do `verify`, que roda no pré-push e no CI. O ADR-017 registra que o knip já pegou um server-only órfão no `package.json` e forçou o `src/lib/api/ky.client.ts` a ter caller real) foi ele que exigiu a tela de login existir. No repo como entregue, `bun run knip` roda limpo.

O que *não* quebra: `src/lib/api/__tests__/client.test.ts` menciona "perfil", mas com fixture local própria (type `Perfil` e `perfilSchema` definidos no próprio arquivo de teste); ele não importa a feature. E `tooling/oxlint-rules/rules/no-schema-output-converter.ts` cita `PerfilFormInput` apenas num comentário.

Opcionalmente, atualize os documentos que usam a feature como exemplo: [README.md](#), [ARCHITECTURE.md](#), [CODE-PATTERN.md](#) ("Exemplo real: `src/app/perfil/page.tsx`"), [MEMORY.md](#) (ADR-022 e ADR-025, este último apontando a página como o exemplo de `<Suspense>` sob `cacheComponents`) e o comentário do `next.config.ts`.

3.9 Afrouxar ou apertar regras

Antes de mexer: o repo tem 421 linhas de `.oxlinttrc.json` comentadas, com o motivo de cada bloco escrito ali. Leia o comentário antes de desligar a regra.

3.9.1 As quatro alavancas do `.oxlinttrc.json`

O que cada seção controla

Seção	Efeito
plugins/jsPlugins	Quais conjuntos de regras carregam. Hoje 8 plugins built-in mais ./tooling/oxlint-rules/index.ts
categories	Buckets inteiros. Hoje correctness, perf e suspicious como error
rules	Regra a regra, incluindo as 15 boilerplate/* e as listas restritivas
overrides	Exceções por zona de arquivos (5 zonas já existentes)

O comentário do próprio arquivo explica a escolha de categorias: “Buckets ‘errado / lento / provavelmente errado’ sobem para erro. style/pedantic/restriction ficam fora das categorias e são escolhidos a dedo em rules para não gerar ruído.”

Dentro de rules, quatro pontos concentram as mudanças mais comuns:

- no-restricted-imports, com os pacotes banidos e a mensagem de cada um;
- no-restricted-properties, que cobre document.cookie;
- no-restricted-exports, que cobre fetchCache;
- no-console, hoje liberando apenas warn e error.

Duas regras custom são deliberadamente warn, com a justificativa escrita no arquivo:

`.oxlintrc.json`, as duas advisory

```
// — Advisory (`warn`): levantam candidatos pra review, não reprovam o gate.
// Ambas veem só a AST do arquivo, sem saber quantos callers a função tem —
// e como supressão de lint é proibida no repo, `error` aqui seria um bloqueio
// sem saída pra um padrão às vezes legítimo. Ver VERCEL-OXLINT-MAPPING.md.
"boilerplate/no-passthrough-mapper": "warn",
"boilerplate/no-literal-dispatch": "warn"
```

Os overrides são o lugar certo para uma exceção nova: as cinco zonas existentes são `*.config.{js,mjs,ts}`, `src/app/**` (libera default export), `**/*.schema.ts` (libera só o valibot), `src/lib/api/**` (libera só o ky), `tooling/**` e `**/*.test.ts(x)`. Cada uma traz o motivo escrito. Prefira uma zona nova a desligar a regra globalmente.

CUIDADO | Desligar uma regra boilerplate/* quebra o bun test

`tooling/oxlint-rules/rules/__tests__/custom-rules.test.ts`

é teste de *integração*: ele roda o oxlint real com o `.oxlintrc.json` do pro-

```
jeto      (Bun.spawnSync(["bunx", "oxlint@1.77.0", "-f", "json",
"-c", OXLINTRC, fixturePath])) e afirma presença ou ausência do código
boilerplate(<rule-id>) nas fixtures. São 11 blocos describe. Pôr uma dessas
regras em "off" faz os testes daquela regra falharem, e bun test está dentro do
verify. Afrouxar de verdade significa mexer também no teste, o que aparece no diff.
```

Para desligar uma regra custom por completo, além do `.oxlintrc.json` existe o registro em `tooling/oxlint-rules/index.ts` (meta.name: "boilerplate" mais o mapa rules) e listas internas editáveis, como ALWAYS_FORBIDDEN em `tooling/oxlint-rules/rules/no-forbidden-imports.ts`.

POR QUE | Por que existe uma regra custom se no-restricted-imports já existe

no-restricted-imports não pega subpath. `swr/mutation`, `zod/v4-mini` e `jose/jwt/verify` passariam batido. Por isso a lista ALWAYS_FORBIDDEN da regra custom duplica os pacotes: `swr`, `@tanstack/react-query` (e `-devtools` e `query-core`), `react-query`, `axios`, `zod`, `jsonwebtoken`, `jose`.

REGRA | Supressão de lint continua proibida, e não há como contornar

`tooling/no-lint-suppression.ts` roda dentro do `verify` e faz `git grep` pelas needles montadas em runtime (`eslint-disable` / `oxlint-disable`) em `.ts`, `.tsx`, `.js`, `.jsx`, `.mjs`, `.cjs`, `.mts`, `.cts`. Mensagem de falha: "Proibido: diretiva de supressão de lint encontrada no código-fonte... Corrija a causa do lint; supressões não são permitidas." O ADR-020 explica por que o `guard` vive fora do linter: "uma regra de lint pode ser desligada pela própria diretiva que ela proíbe; o `guard` se anulava."

A consequência prática dessa combinação é o ponto central: **a única forma legítima de afrouxar uma regra é editar a config**. Isso aparece no diff, e o `.github/CODEOWNERS` cobre `/.oxlintrc.json`, `/tooling/**` e `/MEMORY.md`; ou seja, passa por review nominal.

3.9.2 Registre a decisão

O `MEMORY.md` é "registro **append-only** das decisões de arquitetura/toolchain e o **porquê**, incl. alternativas descartadas", com formato fixo por entrada: **Decisão • Contexto • Por quê • Alternativas descartadas • Status**. Decisões revertidas viram uma *nova* entrada, nunca reescrita da antiga.

Há precedente direto para decisões sobre regras: ADR-026 (remoção do plugin `react-perf`), ADR-027 (`oxlint-tailwindcss` fora), ADR-019 (ligar `type-aware`), ADR-020 (`guard` fora do linter), ADR-012 (`--disable-nested-config`). Se você afrouxar uma regra, o lugar do "por quê" é ali; senão o próximo dev (ou o próximo agente) vai reapertar sem saber que houve motivo.

3.10 Ligar o `oxlint-tailwindcss` depois dos tokens

Este é o único passo que o boilerplate deixa explicitamente para depois, com data marcada. O ADR-027 do `MEMORY.md` está inteiro aqui:

MEMORY.md, ADR-027

```
## ADR-027: `oxlint-tailwindcss` fora, por enquanto
- **Decisão:** não adotar o plugin `oxlint-tailwindcss` (que barra
  paleta default e valor arbitrary).
- **Por quê:** ele só tem valor com um `@theme` real preenchido em `
  globals.css`, e aqui o design system é template – o `DESIGN.md` é para o adotante
  preencher.
  Ligar agora reprovava qualquer classe do comprador antes de ele
  definir os tokens.
- **Revisitar quando:** o `DESIGN.md` do projeto adotante estiver
  preenchido e os tokens vivos no `@theme`.
- **Status:** não adotado (candidato).
```

A pré-condição é objetiva e verificável: enquanto o `@theme` inline do `src/app/globals.css` tiver só as quatro entradas de scaffold, o plugin não tem contra o que validar. Depois de levar os tokens do `DESIGN.md` para lá, ele passa a fazer sentido, e o caminho é o mesmo dos outros oito plugins: adicionar ao array `plugins` do `.oxlintrc.json`.

CUIDADO | O boilerplate não documenta a config desse plugin

Ele não está no `package.json` nem no `.oxlintrc.json`, e não há no repo nenhuma instrução de instalação, nome de regra ou opções. Consulte a documentação do plugin na hora de adotar; não presuma nomes de regra. E lembre do ADR-009: o `oxlint` está pinado em `1.77.0`; “subir a versão exige rodar `bun test` das regras custom: elas são o canário da API do plugin.”

3.11 Detalhes que aparecem no caminho

3.11.1 A camada `shared` não existe fisicamente

`ls -R src` mostra só `app/`, `features/`, `lib/` e `proxy.ts`. O `ARCHITECTURE.md` explica: `shared/{ui,utils,schemas,types}/` é “folha pura; vazia até o 1º primitivo cross-feature”. A regra `boilerplate/enforce-boundaries` já reconhece a camada no seu `LAYER_RANK` (`app: 4, features: 3, lib: 2, shared: 1`), então basta criar a pasta no dia em que o primeiro primitivo compartilhado aparecer; nada mais a configurar.

3.11.2 Onde os gates te encontram

Momentos em que a verificação roda sozinha

Momento	O que roda
pre-commit (lefthook.yml)	oxlint e oxfmt, só nos arquivos staged
pre-push (lefthook.yml)	<code>bun run verify</code> no projeto inteiro
Fim de turno do agente	tooling/agent-hooks/verify-before-stop.ts , nos três hosts
CI (.github/workflows/ci.yml)	Os dois grep-gates, <code>bun run verify</code> e <code>bun run build</code>

Existe também `verify:fast` (`lint + fmt:check + knip + test`, sem `typecheck`), usado no fluxo do agente. Para validar uma adaptação estrutural (remoção de rota, troca de envelope, mexida em tipos) use o `verify` completo: é o `typecheck` que pega o `typedRoutes` quebrado.

REGRA | Checklist final da adaptação

`.env.local` criado e preenchido; name/license do `package.json` trocados; metadata do `layout.tsx` trocado; @owner do CODEOWNERS resolvido; SESSION_COOKIE e matcher do `proxy.ts` alinhados ao seu backend; `example\.` do CI trocado pelo seu domínio; `PRODUCT.md` e `DESIGN.md` preenchidos e tokens no @theme; `knowledge.md` gerado; branch base decidida (nos 6 pontos); feature perfil removida quando houver rota real. E `bun run verify` verde.

PARTE 4

A metodologia: skills e comandos

O QUE VOCÊ LEVA DESTE CAPÍTULO

Você vai entender qual ferramenta usar em cada momento, como os fluxos se conectam e onde o método depende de uma decisão sua. Use este capítulo para trabalhar com a base, não apenas dentro dela.

O boilerplate não entrega só código e regras de lint. Ele traz um jeito de trabalhar quando parte da execução fica nas mãos de um agente: 21 skills, 13 comandos e hooks que transformam algumas decisões em verificações executáveis. Esta seção mostra o papel de cada peça, quando usá-la e o que acontece quando uma etapa é pulada.

Os números, medidos nos arquivos reais do repositório:

Inventário da metodologia (contagem por WC -l)

Item	Quantidade
Skills em <code>.claude/skills/</code>	21 pastas, cada uma com um <code>SKILL.md</code>
Skills em <code>.agents/skills/</code>	2 (pre-review e orquestrar, canônicas compartilhadas)
Comandos em <code>.claude/commands/</code>	13 arquivos <code>.md</code> soltos
Linhas somadas dos 13 comandos	184
Linhas somadas das 21 skills de <code>.claude/skills/</code>	2470
Linhas somadas das 2 skills canônicas de <code>.agents/skills/</code>	221

A proporção já conta a história: cada comando tem entre 9 e 19 linhas; as skills vão de 9 linhas (o ponteiro de orquestrar) a 229 (api-contract). São aproximadamente 13 linhas de skill para cada linha de comando. Isso não é acidente, é o desenho.

4.1 O desenho: comando fino, skill grossa

Todos os 13 comandos seguem a mesma estrutura: frontmatter com `description` e `argument-hint`, um título, a linha `Entrada: $ARGUMENTS`, uma linha `Ativa:` listando as skills, uma linha dizendo qual fluxo executar, e a frase *“Não duplicar regras aqui”*.

O exemplo abaixo é o arquivo `.claude/commands/new-feature.md` *inteiro*; 14 linhas, nada foi cortado:

```
.claude/commands/new-feature.md, integral

---
description: Inicia workflow de nova feature com gate de viabilidade, 3
  agentes paralelos e arquitetura
argument-hint: <KEY> ou descrição da feature
---

# Nova Feature

Entrada: $ARGUMENTS

Ativa: `workflow-new-feature`, `layer-boundaries`, `api-contract`, `no-
  barrel-policy`,
`nextjs-react19-architecture`, `naming-conventions`, `core-principles`,
`typescript-strict`,
e as skills `forms`, `design-system`.

Se input ambíguo, ativar `deep-interview` antes da Phase 1.

Executa o workflow definido em `workflow-new-feature` (Phase 0
  Feasibility Gate → Phase 1
  Architecture → Phase 2 Implementation → Phase 3 Audit). Não duplicar
  regras aqui.
```

O comando não ensina nada. Ele resolve a entrada, escolhe skills e delega. Todo o conteúdo; as fases, os gates, os entregáveis, as racionalizações proibidas; vive na skill.

REGRA | Comando não carrega regra

A frase “Não duplicar regras aqui” (ou variação) aparece literalmente em `new-feature.md`, `expand.md`, `refactor.md`, `create-pr.md`, `create-task.md`, `pr-ready.md`, `review-pr.md` e `pre-review.md`. Se você for customizar o boilerplate, mantenha a disciplina: regra nova entra na skill, não no comando.

POR QUE | Por que separar assim

Uma regra escrita em dois lugares vira duas regras assim que uma das cópias for editada. Com o comando fino, existe exatamente uma fonte para cada regra; a skill, e o comando é só o roteamento. Além disso, skills são ativadas também por descrição (*model invocation*): o agente carrega `layer-boundaries` ao mexer em `src/features/**` mesmo sem ninguém digitar comando algum. Se a regra morasse no comando, ela sumiria justamente no caso mais comum: o agente trabalhando sem slash command.

Detalhe verificado: `/pr-ready` é o único comando sem `argument-hint` no frontmatter; é o único que não recebe argumento.

4.2 Onde as skills moram: `.claude/skills/` e `.agents/skills/`

Há dois diretórios de skills, e a diferença é sobre *host*, não sobre importância.

`.claude/skills/` é o registro do Claude Code. `.agents/skills/` é o lugar das skills compartilhadas entre os três hosts que o boilerplate suporta: Claude Code, Codex e Cursor. Hoje `pre-review` e `orquestrar` estão lá. A cópia em `.claude/skills/pre-review/SKILL.md` tem 17 linhas e é um ponteiro:

```
.claude/skills/pre-review/SKILL.md (corpo, sem frontmatter)

# Pré-review de guardrails

Leia completamente e siga a skill canônica em
`.agents/skills/pre-review/SKILL.md`. Use o argumento recebido como
base; quando
estiver vazio, use a base definida na seção *Branch base* do `CLAUDE.md
(default `main`).
```

Esse ponteiro é também a única skill do repositório com frontmatter estendido: `disable-model-invocation: true`, `argument-hint` e `allowed-tools` restrito a `Bash(git *)`, `Bash(mkdir *)`, `Read`, `Grep`, `Glob`, `Agent` e `AskUserQuestion`. As outras 19 skills declaram apenas `name` e `description`.

O mesmo padrão multi-host vale para os hooks: os scripts vivem em `tooling/agent-hooks/` e cada host só aponta para eles, via `.claude/settings.json`, `.codex/hooks.json` e `.cursor/hooks.json`. Todos invocam o script com a mesma linha, resolvendo a raiz do repo por `git rev-parse --show-toplevel`.

CUIDADO | Duas skills são canônicas hoje

`.agents/skills/` contém `pre-review` e `orquestrar`. As outras 19 vivem apenas em `.claude/skills/`; ou seja, hoje só o Claude Code as enxerga. Se você usa Codex ou Cursor como host principal, saiba que apenas `pre-review` e os hooks estão realmente compartilhados. Mover mais skills para `.agents/` é trabalho seu, não algo que já venha feito.

Vale registrar: o diretório `.agents/tmp/`; onde o selo da `pre-review` é gravado; não existe no repositório em repouso. Ele é criado em runtime pela própria skill, com `mkdir -p`.

4.3 Mapa: comando → skill → quando usar

Os 13 comandos e a skill que cada um ativa

Comando	Skill principal	Quando usar
<code>/interview</code>	<code>deep-interview</code>	Requisito ambíguo; descoberta em 5 fases
<code>/create-task</code>	<code>task-creation</code>	Demanda informal vira issue rastreável
<code>/new-feature</code>	<code>workflow-new-feature</code>	Feature inédita (a pasta não existe)
<code>/expand</code>	<code>workflow-feature-expansion</code>	Aba, seção, modal ou fluxo novo em feature existente
<code>/refactor</code>	<code>workflow-refactor</code>	Reestruturar feature; migrar data-fetching; matar barrels
<code>/sync-contract</code>	<code>api-contract</code>	Revalidar schemas contra o spec do backend
<code>/audit</code>	<code>feature-audit</code>	Auditoria arquitetural de um path
<code>/review</code>	<code>feature-audit</code>	Code review semântica por issue ou path
<code>/pr-ready</code>	<code>pr-ready</code>	Gate mecânico com veredito READY ou NOT READY
<code>/pre-review</code>	<code>pre-review</code>	Gate de julgamento; grava o selo do PR
<code>/create-pr</code>	<code>pr-creation</code>	Cria branch e abre PR via gh
<code>/review-pr</code>	<code>pr-review-response</code>	Responde comentários de um PR aberto
<code>/orquestrar</code>	<code>orquestrar</code>	Coordena worktrees, workers, review independente e PRs encadeadas

Os quatro comandos de fluxo pesado são `/new-feature`, `/expand`, `/refactor` e `/review`. Eles carregam a skill principal junto com o núcleo arquitetural:

- limites de camada, contrato de API e política sem barrel;
- arquitetura de Next.js e React 19;
- naming, princípios centrais e TypeScript estrito;
- forms e design system nos fluxos de feature.

Algumas skills não têm comando dedicado. Elas entram pelas descrições e pelas linhas Ativa: dos comandos. A `vercel-react-best-practices` aparece explicitamente no `refactor` apenas quando há upgrade de framework ou migração da camada de dados.

CUIDADO | `gitnexus-impact-analysis` não é skill deste repositório

`/audit`, `/review` e `/refactor` citam `gitnexus-impact-analysis` como *opcional*. Ela pertence ao plugin externo GitNexus e a integração está documentada em

[integracoes/gitnexus.md](#). Sem GitNexus instalado, as skills mandam mapear os call-sites na mão; `workflow-refactor` é explícita: “Sem GitNexus: mapeie manualmente os call-sites (grep dos símbolos exportados) e registre o risco; NÃO infira o blast radius sem evidência”.

4.4 A anatomia de uma skill

Antes de percorrer as 20, vale entender a forma. As skills do repositório repetem um esqueleto:

- **Frontmatter** com name e uma description escrita para ser *gatilho*: ela lista as palavras que o usuário digitaria e os “Sintomas” observáveis no código. Exemplo, de `layer-boundaries`: “Sintomas: useQuery dentro de .tsx, regra de negócio dentro de componente, schema com transform de UI [...] util importando React, prop opcional sem default ou justificativa”.
- **Visão geral**; o que a skill cobre e, quase sempre, o que ela *não* cobre.
- **Quando usar / Quando NÃO usar**; a seção que evita sobreposição entre skills.
- **Regras centrais**, quase sempre em tabela ou matriz.
- **Sinais de alerta: PARE**; a lista de gatilhos de parada.
- **Racionalizações comuns**; uma tabela de duas colunas, “Desculpa” contra “Realidade”.
- **Verificação**; um comando ou um grep que prova que a skill foi seguida.

A tabela de racionalizações é o traço mais característico do produto. Ela existe porque a falha típica de um agente não é ignorar a regra: é justificá-la para fora. Dois exemplos reais de `workflow-new-feature`:

Racionalizações comuns (`workflow-new-feature`)

Desculpa	Realidade
“Vou pular o reuse scan, sei que não tem nada parecido”	“Sei” é palavra perigosa. Scan é grep, custa 30 segundos.
“Crio só um <code>index.ts</code> pequenininho na feature”	Zero barrel é zero. Não existe “pequeninho”.
“Esse endpoint nem mudou recente, não precisa abrir spec”	O spec é a fonte. Mudou ou não, abre.

4.5 As 21 skills, por grupo

4.5.1 Descoberta (2)

deep-interview (175 linhas). Entrevista estruturada para descobrir requisitos escondidos *antes* de implementar. São 5 fases (Contexto & Escopo, Fluxo & Estados, Dados & Integração, UX & Comportamento, Restrições & Riscos) precedidas de uma Fase 0 de exploração do codebase, cujo gate é “Pular perguntas cuja resposta está no código”. Regra de condução: uma pergunta por vez, no máximo duas se diretamente relacionadas. Em *plan mode*, **toda** pergunta vai por `AskUserQuestion`.

A parte que define a skill é a distinção entre pergunta óbvia e pergunta útil:

```
deep-interview/SKILL.md, seção ``Perguntas não-óbvias``

**Proibidas (já estão no CLAUDE.md / skills):**
- "Qual framework?" / "Qual linguagem?" / "Vai ter testes?" / "Vai usar TypeScript?"

**Esperadas (revelam complexidade escondida):**
- "O que acontece se o usuário tentar X enquanto Y está em progresso?"
- "Esse dado pode mudar entre o usuário ver e clicar enviar?"
- "Se a API leva 30s, o que o usuário vê?"
- "Esse estado precisa sobreviver reload da página ou navegação?"
```

task-creation (114 linhas). Converte demanda informal em issue rastreável. Template com títulos de seção marcados por emoji, critérios de aceite verificáveis e um bloco final obrigatório de *Instruções para agente*. Regra dura: **sem código de implementação** no texto da task; payload, JSON ou comando de terminal só entram se ajudarem a execução. Se houver dúvida relevante, a skill manda ativar `deep-interview` antes de criar.

4.5.2 Arquitetura e bordas (5)

Skills de arquitetura

Skill (linhas)	Núcleo
layer-boundaries (110)	Matriz do DAG app -> features -> lib -> shared e matriz de responsabilidade por artefato dentro da feature
no-barrel-policy (92)	Zero barrel em todo <code>src/**</code> , sem exceção nem para <code>shared/ui/</code>
naming-conventions (127)	Tabelas de arquivo e símbolo, organização de <code>types/</code> , sub-organização de componentes
nextjs-react19-architecture (159)	Árvore canônica de feature, RSC por padrão, Server Actions, estado na URL, contrato de entrega
api-contract (229)	8 regras para manter schemas e tipos 1:1 com o spec OpenAPI

A árvore canônica que `nextjs-react19-architecture` impõe:

Árvore de feature padrão

```
src/features/<feature>/
  <feature>.schema.ts      # valibot: validação de borda + tipos (
    InferOutput)
  <feature>.actions.ts      # Server Actions (mutação) →
    revalidatePath / router.refresh
  components/
    shell/ listing/ form/ details/ sections/ shared/
    hooks/ helpers/ constants/ types/
```

E o que *não* existe: não há `api/queries`, `api/mutations` nem `adapters`. Leitura de dados é RSC em `src/app/**` chamando `apiClient(kyServer, "rota", schema)`. Rotas vivem em `src/app/**` como `page.tsx`, `layout.tsx` e `proxy.ts`; `*.screen.tsx` é proibido.

`no-barrel-policy` merece destaque porque é a skill que declara abertamente o buraco do lint:

POR QUE | Onde a skill existe porque o lint não alcança

"O lint cobre só parte disto: `boilerplate/no-index-barrel` mata `barrel` em `index.*` e `oxc/no-barrel-file` pega `export *`. Mas `named re-export` (`export { x } from`) em arquivo que **não é index** é GAP do lint: fica por disciplina humana. É justamente pra esse buraco que esta skill existe."

E, coerente com o produto, a skill entrega a verificação junto:

Verificação embutida em `no-barrel-policy`

```
# GAP do lint: named re-export fora de index é disciplina humana. Cace
na mão:
rg "^export \{.*\} from" src/ -g '!*/index.*'
# Saída esperada: vazia
```

`api-contract` é a maior skill do repositório e a mais contra-intuitiva. Sua Regra 4 diz que `api.types.ts` é a **exceção**, não o caminho padrão: o `*.schema.ts` é a fonte única do tipo, e `InferOutput<typeof schema>` é o único tipo que nomeia o contrato. `types/api.types.ts` só existe quando o `shape cru` (`snake_case`, antes da validação) difere genuinamente do output validado, e aí recebe nome próprio, como `UserApiRaw`, nunca o mesmo nome do `InferOutput`.

REGRA | Ordem de atualização quando o contrato muda

A skill fixa a sequência: `schema valibot` → `api.types.ts` (só se o cru diferir) → `mapper` em `utils/` → chamada `apiClient` (JSDoc e retorno) → **só então** RSC, Server Actions e componentes.

4.5.3 Implementação (4)

core-principles (153 linhas) fica acima de qualquer skill específica: YAGNI, KISS, sem fallback silencioso, console restrito a warn/error, JSDoc obrigatório em exports e testes de comportamento em PT-BR. O critério de YAGNI é uma pergunta única: “Isso é necessário AGORA ou estou antecipando um requisito que talvez nunca venha?”. Se a resposta for “talvez no futuro”, não implementa. E generalização só com três casos concretos. Os limites de tamanho são declarados como guia, com a honestidade explícita de que ninguém os impõe: “Arquivo $\leq$ 300 linhas, função $\leq$ 120 linhas (guia KISS: não há regra de lint pra isso)”.

typescript-strict (45 linhas) é a menor skill do conjunto, e é pequena de propósito: “A maioria das regras é **imposta por oxlint**: esta skill carrega só o que o lint NÃO expressa: o racional do ‘por que não’, a escolha type vs interface, e o fluxo de tipos.” Ela lista o que o oxlint já barra; any, as exceto as const, non-null !, default export fora de `src/app/**`, import relativo de pai, console.log, e não repete nada disso.

forms (126 linhas). React Hook Form com valibotResolver. As regras que importam: RHF é instanciado **dentro do hook** da feature, e o componente recebe form pronto mais onSubmit; o tipo do form vem de InferOutput, nunca escrito à mão; mensagens em PT-BR com acentuação; defaultValues literais e completos. E a distinção de submit que quase todo mundo erra: mutação de dado vai por Server Action com revalidatePath/router.refresh, mas *submit de auth que seta cookie* (login, refresh) vai por chamada browser-direct, porque o Set-Cookie precisa pousar no browser.

design-system (64 linhas). Tailwind v4 fixo, tokens em [DESIGN.md](#) como fonte da verdade, e um bloco anti-slop:

CUIDADO | “Sem cara de IA”

“**Paleta própria e contida**: não o roxo/índigo default genérico. [...] **Proibido slop**: gradiente decorativo, glassmorphism, sombra genérica em tudo, ‘cara de template’.” A skill também cobre o que `jsx-ally/*` não pega: contraste AA, foco visível e ordem de foco. Ela parecia com o impeccable (ver [integracoes/impeccable.md](#)).

4.5.4 Performance (1)

vercel-react-best-practices (89 linhas) resume as 70 regras de performance da Vercel Engineering em 8 categorias: Eliminating Waterfalls, Bundle Size, Server-Side, Client-Side, Re-render, Rendering, JavaScript Perf e Advanced. O recorte é declarado no topo: “**Metade é estaticamente verificável** e já vira error no oxlint [...] A outra metade é **arquitetural** [...] Esta skill cobre só a parte arquitetural”. O mapeamento regra $\longleftrightarrow$ lint está em [VERCEL-OXLINT-MAPPING.md](#).

O item que melhor mostra a doutrina do boilerplate é a regra `client-swr-dedup`, marcada como **N/A**: não se usa SWR nem React Query. Leitura é RSC; interatividade é React 19.

4.5.5 Qualidade e review (2)

feature-audit (124 linhas) faz auditoria *semântica* em 6 fases: Fase 0 impact pre-flight (opcional, GitNexus), Fase 1 alinhamento com requisitos, Fase 2 estrutura da feature, Fase

3 arquitetura e layer compliance, Fase 4 sincronização de contrato de API, Fase 5 qualidade de código com `file:line`. Saída: relatório por fase com veredito READY / NEEDS FIXES e issues ordenadas Critical → Major → Minor.

REGRA | Escopo declarado, para não duplicar gate

“**feature-audit = auditoria SEMÂNTICA** [...] O mecânico (lint, dead code, JSDoc, naming, contract-invoke, diff scope, secrets) vive no `pr-ready`: não duplicar aqui. **Pré-requisito: `pr-ready` verde.**”

`pre-review` (106 linhas, canônica) é tratada em detalhe na seção da cadeia de entrega.

4.5.6 Coordenação (1)

orquestrar (115 linhas, canônica) é o coordenador do trabalho no Orca. Ele decide se uma demanda cabe em uma PR ou precisa de uma cascata de até três fatias, abre worktrees, despacha workers, aguarda o sinal de conclusão e chama reviewers independentes antes de liberar cada PR. A skill é genérica: não assume Jira, nomes de projeto ou um modelo específico; o projeto que adaptar a base fornece essas integrações.

4.5.7 Entrega e PR (3)

pr-ready (103 linhas): 9 checks numerados, tabela de saída, veredito binário. **pr-creation** (191 linhas): 4 fases; coleta (título no tracker, tipo de branch, confirmar base), criação da branch, análise de diff, template de descrição em PT-BR e `gh pr create`. **pr-review-response** (151 linhas): fases 0 a 4 para responder comentários de review, com classificação por comentário e resposta via `gh api .../comments/{id}/replies`.

A classificação do `pr-review-response` é simples e útil:

Classificação de comentário de review

Classe	Significado
FIX	Problema real. Corrige.
JUSTIFY	Comentário válido, mas a implementação atual está certa por um motivo específico.
PARTIAL	Parte aplica, parte se justifica.

E três regras obrigatórias, literais: “Nunca responder sem aprovação do usuário / Nunca push automático: usuário controla / Sempre ler o código antes de classificar (nunca só pelo texto do comment)”.

REGRA | `pr-creation` não re-roda checks

A skill declara: se `pr-ready` não retornou READY, volta para `pr-ready` primeiro; `pr-creation` **confia** no veredito. Ela também nunca abre PR contra uma base não con-

firmada: lê a seção *Branch base* do [CLAUDE.md](#), e se ainda estiver no default main sem confirmação, pergunta **uma vez** e grava a escolha no próprio [CLAUDE.md](#).

4.6 Os três workflows

`workflow-new-feature`, `workflow-feature-expansion` e `workflow-refactor` são os fluxos principais. Os três declaram no topo o mesmo par de blocos; “Referência cruzada (núcleo)” e “Específico da stack”, e os três têm seções “Quando usar” e “Quando NÃO usar” que apontam um para o outro, para que o agente escolha o fluxo certo em vez de improvisar.

A simetria entre eles é a parte que vale memorizar:

O que os três workflows têm em comum

Elemento	Como aparece
Fase 0 com gate	Sempre. Aprovação explícita do usuário antes de qualquer código.
Análise paralela	3 frentes em <code>/new-feature</code> e <code>/refactor</code> ; 2 frentes em <code>/expand</code> .
Plano aprovado	Fase de plano com gate próprio antes da implementação.
Ordem de implementação	<code>reuse scan</code> → <code>types</code> → <code>schemas</code> → <code>lib/api</code> → RSC e Server Actions → <code>components</code> → <code>rotas</code> → <code>tests</code> .
Verificação final	<code>bun run verify</code> && <code>bun run build</code> .

REGRA | Nenhum dos três permite código antes do plano aprovado
`workflow-new-feature` lista textualmente sob “NUNCA”: “Assumir regra de negócio sem confirmar / Pular Phase 0 (Feasibility Gate) / Escrever código antes de Phase 1 aprovada”.

4.6.1 `/new-feature`: greenfield

Fases de workflow-new-feature (134 linhas)

Fase	Conteúdo
0; Gate de viabilidade	0.1 Descoberta (issue → acceptance criteria; descrição livre → deep-interview se ambíguo; sempre localizar \$API_CONTRACT_PATH). 0.2 Três frentes em paralelo: infra e viabilidade; padrões existentes e baseline de qualidade; naming e contrato. 0.3 Checklist de 5 itens. Gate: aguardar aprovação.
1; Arquitetura antes do código	10 entregáveis: árvore completa, hierarquia de rotas, lista de componentes, mapa de estado, leitura e mutação, hooks com estado, helpers/constants/types, boundaries de loading e erro, plano de acessibilidade, plano de testes. Gate.
2; Implementação (ordem estrita)	0 reuse scan, 1 types, 2 schemas valibot, 3 constants, 4 lib/api , 5 RSC e Server Actions, 6 components ('use client' só nas folhas), 7 rotas, 8 tests.
3; Audit	8 checkboxes: re-renders, listas, sem efeito de fetch, acessibilidade, sem cache proibido, JSDoc em todos os exports, KISS/YAGNI/SOLID, coverage de comportamento. Issues → fix → re-audit até clean.
Entrega	<code>bun run verify</code> && <code>bun run build</code> mais 5 critérios. “Pronto pra /pr-ready”.

Note o item 3 da Fase 2: **constants sem query keys**. E o item 4 da Fase 1: “estado de URL/local, params de navegação (sem query keys; servidor é fresco via RSC)”. O workflow não descreve a doutrina RSC-pura; ele a assume e organiza o trabalho em volta dela.

4.6.2 /refactor: o código atual não é a verdade

POR QUE | O princípio central do workflow-refactor

“Em refactor, o código atual existe **justamente porque** carrega decisões que não devem persistir.; **NUNCA** infira ‘intenção’ do código atual para preservar ‘como está’.; **NUNCA** crie fallback ou modo de compatibilidade pra padrão identificado como antipattern na Fase 0.” Decisões de produto ou UX que estejam no código antigo só permanecem se o usuário confirmar explicitamente.

Fases de workflow-refactor (157 linhas)

Fase	Conteúdo
0; Gate de infraestrutura	0.1 Descoberta e resolução de \$API_CONTRACT_PATH. 0.2 Pre-flight opcional de blast radius (GitNexus impact upstream e detect_changes; símbolo CRITICAL exige aprovação explícita; sem GitNexus, grep manual). 0.3 Três frentes: infra audit, qualidade e padrões, naming e contrato. 0.4 Gate: inventário mais blast radius; CRITICAL exige um "sim, pode tocar".
1; Mapeamento e diagnóstico	6 entregáveis: inventário de antipadrões com file:line, dead code, boundary errada, mapa de dependências e ciclos, risk assessment, drift report.
2; Plano de refactor	9 entregáveis, incluindo a ordem de migração: types → schemas → lib/api → RSC e Server Actions → components → rotas → tests → remoção de dead code. Gate.
3; Implementação	Executa exatamente na ordem do plano, com commit pequeno por etapa. Re-ativação obrigatória: api-contract antes de schema/types/API; naming-conventions e layer-boundaries antes de cada novo arquivo; no-barrel-policy ao remover barrel.
4; Verificação	bun run verify && bun run build; todo antipattern da Fase 1 resolvido sem fallback; tests faltantes viram débito declarado no PR.

A tabela de referência rápida da skill é, na prática, a doutrina inteira do boilerplate em sete linhas:

Sintoma → ação, em workflow-refactor

Sintoma encontrado	Ação no refactor
Efeito que faz fetch acoplado à UI	Leitura → RSC (apiClient); mutação → Server Action
React Query/SWR ou cache no client	Remover; servidor fresco via RSC, revalidar via revalidatePath/router.refresh
*.screen.tsx (tela fora de rota)	Mover conteúdo para a rota em src/app/**
Loading/error state manual	Eliminar; usar Suspense e streaming no RSC
Barrel (index.ts) em feature	Deletar e ajustar todos os callers para path completo
Component declarado dentro de outro component	Extrair para arquivo próprio
try { ... } catch { showToast('Erro') }	Propagar; o hook decide; mensagem em constante

4.6.3 /expand: adição cirúrgica

A skill se define pela negativa: “Não é greenfield (a feature já existe) nem refactor (a estrutura geral fica). É **adição cirúrgica**: novo arquivo cai no lugar certo, sem vazar responsabilidade.”

É o único dos três com análise paralela em 2 frentes, não 3: *feature audit* (árvore atual, organização, leitura e mutação em uso, coverage) e *integration points* (endpoints novos → *api-contract*; navegação muda?; que componentes do design system reutilizar). A Fase 1 produz 7 entregáveis, com destaque para a árvore atualizada mostrando as adições *no contexto da árvore existente*. A Fase 3 verifica 5 itens, sendo os dois primeiros os mais característicos: a árvore ainda está reconhecível, e não houve vazamento de responsabilidade para outra feature.

CUIDADO | Os desvios que a expansão convida

Sinais de alerta literais da skill: “Criar [helpers/](#) no root da feature ‘pra essa coisa específica’: usar [utils/](#) (existente)” e “Tocar em outra feature pra ‘deixar consistente’: fora do escopo, faz separado”. Na tabela de racionalizações: “Já que estou aqui, refatoro essa parte velha também” → “Não. Fora do escopo. Vira outra task.”

4.7 A cadeia de entrega: /pr-ready → /pre-review → /create-pr

Esta cadeia é declarada em quatro lugares independentes do repositório: *pr-ready/SKILL.md*, *.agents/skills/pre-review/SKILL.md*, *README.md* e o ADR-021 em *MEMORY.md*. A divisão de trabalho é explícita: “*pr-ready* é o gate **mecânico** (lint, ty-pecheck, testes, tamanho de diff, secrets, JSDoc, naming, contrato). Esta skill é o **julgamento** que nenhum gate automático pega. Não se sobrepõem.”

4.7.1 /pr-ready: o que uma máquina prova

As 9 verificações do `pr-ready`

#	Check e comando	Passa se
1	Lint; <code>bun run lint</code>	<code>exit 0</code>
2	Type check; <code>bun run typecheck</code>	<code>exit 0</code>
3	Testes; <code>bun test</code>	<code>exit 0</code>
4	Unused exports cross-module; manual (lint não pega cross-module)	sem export que nenhum outro módulo consome
5	Diff scope; <code>git diff main --numstat</code>	≤ 1500 linhas; acima disso é HARD FAIL
6	Secrets; <code>git diff main</code>	sem <code>.env</code> , sem token hardcoded, sem <code>credentials.json</code>
7	JSDoc coverage; manual	100% das funções e hooks exportados
8	Naming compliance; invocar <code>naming-conventions</code>	sem violações
9	Contract alignment; invocar <code>api-contract</code> se schemas ou contratos foram tocados	spec espelhado 1:1, sem drift

A saída é uma tabela com status por check e um veredito binário: `READY` ou `NOT READY`. A própria skill fecha com a nota que evita trabalho duplicado: “Os checks #1–#3 desta skill são um subconjunto de `bun run verify`”. Se você já roda o gate canônico, use a tabela apenas para os checks #4–#9, que o `verify` não cobre.

CUIDADO | O teto de 1500 linhas é hard fail, não sugestão

Na tabela de racionalizações do `pr-ready`: “Diff grande porque é refactor” → “Refactor grande = PRs menores em ordem. Reviewer humano não consegue revisar 3000 linhas.” Note que este teto (1500, texto da skill) é diferente e mais rígido que o teto de 3000 LOC do hook. O segundo está em código e bloqueia de verdade.

4.7.2 /pre-review: o julgamento

A skill canônica tem 106 linhas e roda em seis passos numerados. Ela roda no contexto principal para poder confirmar decisões com o usuário, e delega o julgamento a subagentes.

Os passos da pre-review

Passo	O que faz
0; Escopo	<code>git fetch origin <base> --quiet</code> <code>git diff --stat origin/<base>...HEAD</code> ; agrupa arquivos por domínio coeso (schemas, hooks, components, lib, app e <code>error.tsx</code> , <code>shared/ui</code> , <code>auth</code>).
1; Revisar por domínio	Um subagente reviewer <i>novo</i> por domínio, em paralelo quando possível, sem reutilizar reviewer entre domínios. Cada achado exige arquivo: linha, categoria, cenário concreto e evidência.
2; Revalidar	No contexto principal, relendo os arquivos. “Feedback de reviewer é claim, não veredito.”
3; Relatório	Placar no topo, achados por categoria e severidade, correção concreta, lista dos descartados e ordem de ataque.
4; Selo	Só após confirmação do usuário. Vinculado ao HEAD.
5; Portão de 3000 LOC	Conta add mais del contra a base; acima do teto, pergunta se divide ou segue e exige ack explícito.

O catálogo de julgamento tem 7 categorias: **A** over-engineering e abstração especulativa; **B** fallback defensivo e tratamento de erro; **C** regra de negócio e authz inventadas no frontend; **D** slop, DRY e naming; **E** acessibilidade e estados; **F** mídia e responsividade; **G** testes. Acompanham duas listas de exclusão; “Não flagrar” e “Não duplicar gates automáticos”, esta última com a justificativa: “bun run verify cobre essas famílias”.

Duas passagens definem o rigor da skill. A primeira, sobre leitura:

REGRA | Ler o arquivo inteiro, não o diff

“Leia os arquivos reais, não apenas as linhas do diff. Um ?? @ ou uma abstração prematura só pode ser julgado com o arquivo inteiro.”

A segunda é a régua numérica de abstração; o número concreto que substitui a discussão sobre “isso é over-engineering?”:

REGRA | A régua de abstração (categoria A)

“Um caller fica inline; extração local comum começa no segundo caller real. Abstração genérica reutilizável, como framework, provider, factory ou hook genérico, exige pelo menos três callers/ usos concretos.”

Exceção declarada: primitivo concreto cross-feature (calendar, DateRangePicker, formatters) pode ir para `shared` no primeiro write; isso é colocação, não abstração especulativa.

O selo é gravado assim, e só depois da confirmação:

Gravação do selo (passo 4)

```
mkdir -p .agents/tmp
git rev-parse HEAD > .agents/tmp/pre-review-$(git rev-parse HEAD).ok
```

E a contagem do portão de 3000 LOC:

Portão de 3000 LOC (passo 5)

```
BASE=main
git diff --numstat "origin/$BASE...HEAD" | awk -F'\t' '$1!="-" {s+=$1+$2} END {print s}'
```

4.7.3 O gate que é código, não texto

Aqui está a diferença entre esta metodologia e um conjunto de documentos: o selo é verificado por um hook. [tooling/agent-hooks/pre-pr-review-gate.ts](#) roda como PreToolUse no Claude Code e como beforeShellExecution nos outros hosts, registrado nos três.

O fluxo do script, na ordem em que ele executa:

1. Se o comando não casa com `/\bgh\s+pr\s+create\b/`, libera.
2. Valida a base: precisa ser `main` ou `feature/*`. Qualquer outra é negada com mensagem explicando como reexecutar.
3. Resolve `git rev-parse HEAD`.
4. Verifica `.agents/tmp/pre-review-<HEAD>.ok`. Sem o arquivo, nega.
5. Conta `add` mais del contra `origin/<base>` e, sem `.agents/tmp/pr-size-ack-<HEAD>.ok`, nega acima de 3000 linhas.

As duas constantes do arquivo:

tooling/agent-hooks/pre-pr-review-gate.ts

```
const MAX_PR_LOC = 3000;

/** Base default de PR. Alinhada à seção "Branch base" do CLAUDE.md. */
const BASE_BRANCH = "main";
```

E a negação, com a mensagem que o agente vê:

O bloqueio propriamente dito

```
if (!existsSync(`.agents/tmp/pre-review-${head}.ok`)) {
  emitDeny(
    host,
    `PR travada: a pré-review de guardrails não rodou pro commit atual
    (${head.slice(0, 8)}).\n` +
    `Rode a skill \pre-review\` (\pre-review\`) primeiro. Ela
    grava o selo que libera o gh pr create.` +
    `Qualquer commit novo depois disso exige nova pré-review.`,
  );
}
```

POR QUE | Por que virar código (ADR-021)

“pr-ready é mecânico (lint/type/test/diff/secrets) e não pega o que custa caro na review humana: abstração especulativa, fallback defensivo, regra de negócio inventada no front, slop. Sem gate, a etapa vira opcional, e regra que dá para pular não é guardrail.”

CUIDADO | O selo morre a cada commit

O selo é `pre-review-<HEAD>.ok`: ele vale para *aquela* commit. Se você rodar `/pre-review`, depois corrigir um achado e commitar, o selo antigo não serve mais e o `gh pr create` volta a ser bloqueado. A skill diz isso na cara: “O selo vale apenas para esse HEAD. Commit novo exige outra pré-review.” O ack de tamanho (`.agents/tmp/pr-size-ack-<HEAD>.ok`) tem a mesma vinculação.

Na prática: rode `/pre-review` **depois** de aplicar as correções, não antes.

O `/create-pr` documenta essa dependência dentro do próprio comando, para que o agente não descubra o bloqueio só quando bater nele:

`.claude/commands/create-pr.md`, pré-requisitos

Pré-requisitos, nesta ordem:

1. `/pr-ready` – só prosseguir se o veredito foi READY.
2. `/pre-review` – grava o selo `.agents/tmp/pre-review-<HEAD>.ok`. **
Sem ele o hook `pre-pr-review-gate.ts` bloqueia a abertura do PR**, e commit novo invalida o selo.

4.8 Como as skills entram sozinhas: os hooks

Nem tudo depende de o usuário digitar um comando. O boilerplate registra cinco pontos de hook, com os mesmos scripts nos três hosts:

Hooks registrados em [.claude/settings.json](#)

Evento	Script
SessionStart	inject-patterns.ts ; injeta as regras no início da sessão
UserPromptSubmit	reinforce-patterns.ts ; reforça a cada turno
PreToolUse (Bash)	pre-pr-review-gate.ts e guard-shell.ts
PostToolUse (Edit Write)	reinforce-after-write.ts e format-after-write.ts
Stop	verify-before-stop.ts (timeout de 300000 ms)

O Stop é o que fecha o ciclo: ele impede o agente de declarar “concluído” com o gate vermelho. Ou seja, a metodologia não termina quando o agente acha que terminou; termina quando `bun run verify` passa.

4.9 O que a metodologia não cobre

Regra que mente é pior que ausente, e isso vale também para esta documentação. As lacunas reais:

- **Não existe skill nem comando de debugging ou bugfix.** Os workflows dizem literalmente “Bug fix → corrigir direto” (`workflow-new-feature`) e “Hotfix de bug pontual → corrigir direto, sem workflow” (`workflow-refactor`). O `deep-interview` remete a uma skill externa: “Bug óbvio (corrija direto ou use sua skill de debugging)”.
- **gitnexus-impact-analysis não existe no repositório.** É skill do plugin externo GitNexus, citada como opcional por `/audit`, `/review` e `/refactor`, com integração documentada em [integracoes/gitnexus.md](#).
- **docs/contract-drift.md** é citado por `api-contract` com a instrução “criar se não existir”, e de fato não existe hoje.
- **.agents/skills/ tem só pre-review.** As outras 19 skills vivem apenas em [.claude/skills/](#).
- **Vários checks são manuais por decisão.** Os checks #4 (`unused exports cross-module`) e #7 (`JSDoc coverage`) do `pr-ready` são explicitamente marcados como manuais, e o limite KISS de 300 linhas por arquivo do `core-principles` vem com a ressalva “não há regra de lint pra isso”.

POR QUE | Por que essa transparência importa

O produto todo se apoia numa premissa: o agente confia no que os documentos dizem. Uma skill que promettesse cobertura que não tem faria o agente pular a verificação humana justamente onde ela é necessária. Por isso cada skill declara o seu buraco; `no-barrel-policy` sobre o `named re-export`, `typescript-strict` sobre `type vs`

interface, `vercel-react-best-practices` sobre a metade arquitetural. O gap declarado é gap gerenciável.

PARTE 5

Code review: por que é o coração do método

O QUE VOCÊ LEVA DESTE CAPÍTULO

Você vai separar o que a máquina consegue provar do que ainda exige julgamento. No fim, saberá por que a pré-review é um portão e como manter achados úteis sem transformar o processo em ruído.

Há uma promessa central nesta base: a arquitetura não fica apenas descrita, ela é **imposta** onde isso pode ser verificado. Camada errada, barrel, any, cache proibido e fetch fora do `apiClient` quebram o build antes de virar discussão de PR. Mas o lint tem um limite, e esta seção começa justamente por ele.

REGRA | O lint cobre uma parte; o restante é julgamento

As regras encontram padrões verificáveis, como mapper pass-through, parse defensivo, comparação com literal de catálogo, conversor de output de schema e condição desnecessária. Intenção, necessidade da abstração e regra de negócio ainda precisam de contexto.

“Parcial” e “o restante é julgamento” resumem a seção. O linter prova que o código é *válido*; não prova que ele é *bom para o produto*. Abstração especulativa, fallback defensivo, regra de negócio inventada no frontend e slop podem ser sintaticamente impecáveis e ainda assim custar caro numa review humana. Por isso, o code review é parte do método, não uma cerimônia no fim do fluxo.

POR QUE | ADR-021: por que o gate existe

Registro literal em `MEMORY.md`:

Decisão: a skill `pre-review` (canônica em `.agents/skills/`, compartilhada pelos três hosts) revisa a branch com subagentes por domínio e grava `.agents/tmp/pre-review-<HEAD>.ok`. O hook `pre-pr-review-gate.ts` bloqueia a abertura de PR sem o selo do HEAD atual, e acima de 3000 LOC sem ack.

Por quê: `pr-ready` é mecânico (lint/type/test/diff/secrets) e não pega o que custa caro na review humana: abstração especulativa, fallback defensivo, regra de negócio inventada no front, slop. Sem gate, a etapa vira opcional, e **regra que dá para pular não é guardrail**.

Guarde essa última frase. Ela é a tese do produto aplicada ao review: se a etapa depende da boa vontade de quem (ou do que) está no teclado, ela não vai acontecer na sexta-feira às 19h, nem no décimo turno de um agente que já “sabe” que o código está bom.

5.1 A cadeia de três portões

O fluxo até o PR tem três etapas que *não se sobrepõem*. Cada uma cobre o que as outras não cobrem, e isso é declarado explicitamente em cinco arquivos independentes ([MEMORY.md](#), [AGENTS.md](#), [README.md](#), [.claude/skills/pr-ready/SKILL.md](#) e [.agents/skills/pre-review/SKILL.md](#)); redundância proposital, para que qualquer host que leia qualquer um dos docs chegue na mesma ordem.

A cadeia até o PR

Portão	Comando	O que prova
1	/pr-ready	O mecânico: o que uma máquina prova (lint, typecheck, testes, tamanho do diff, secrets, JSDoc, naming, contrato)
2	/pre-review	O julgamento: over-engineering, fallback defensivo, regra de negócio inventada no front, slop. Grava o selo
3	/create-pr	A abertura: branch, descrição estruturada, gh pr create. Confia no veredito dos anteriores

REGRA | Não duplicar checks entre as três

Texto literal de [.claude/skills/pr-ready/SKILL.md](#): “Não duplicar checks entre as três: cada uma cobre o que as outras não cobrem.” Se você for customizar o fluxo, essa é a linha que segura tudo em pé. No momento em que a `pre-review` começa a re-rodar lint, ela vira ruído caro, o agente aprende que o relatório é redundante, e o portão perde autoridade.

5.2 Portão 1: /pr-ready, o que a máquina prova

A skill se apresenta em uma linha: “Gate antes de abrir PR. 9 verificações concretas: passa todas ou não abre o PR.”

As 9 verificações do pr-ready

#	Check	Pass se
1	Lint(bun run lint)	exit 0
2	Type check (bun run typecheck)	exit 0
3	Testes (bun test)	exit 0
4	Unused exports cross-module	sem export que nenhum outro módulo consome (manual: lint não pega cross-module)
5	Diff scope (git diff main --numstat)	até 1500 linhas; acima disso é HARD FAIL (NOT READY)
6	Secrets (git diff main)	sem .env, sem token hardcoded, sem credentials.json
7	JSDoc coverage	100% das funções/hooks exportados
8	Naming compliance	invocar naming-conventions; sem violações
9	Contract alignment	invocar api-contract se schemas/contratos tocados; spec espelhado 1:1, sem drift

A skill fecha com um bloco de auto-verificação executável; ela se cobra de ter sido seguida:

Auto-verificação do pr-ready

```
bun run lint && \
bun run typecheck && \
bun test && \
git diff main --numstat | awk '{add += $1; del += $2} END {if (add + del > 1500) exit 1}' && \
! git diff main | grep -qE "(\\.env\\.local|password.*=.*['\"]\\w+|api[_-]?key.*=.*['\"]\\w+)" && \
echo "READY"
```

Regra da skill logo abaixo do bloco: “Se o output não termina com READY → não é ready.”

Os checks 1 a 3 são um subconjunto de bun run verify; o gate canônico do repo (lint, fmt:check, typecheck, typecheck:tooling, knip, test, guard de supressão de lint e budget de peso de assets). A própria skill diz que você pode rodar bun run verify de uma vez e usar a tabela só para os checks 4 a 9, que o verify não cobre.

CUIDADO | Dois tetos de LOC diferentes, e isso é intencional

O pr-ready reprova acima de **1500** linhas (check #5). O hook pre-pr-review-gate.ts só bloqueia acima de **3000** (MAX_PR_LOC = 3000). Não é inconsistência: 1500 é *conselho de skill*, o número a partir do qual um reviewer

humano começa a ler na diagonal; 3000 é *lei de hook*, o ponto em que a máquina se recusa a deixar passar sem um “sim, siga” explícito. A racionalização que a skill responde é literal: “Refactor grande = PRs menores em ordem. Reviewer humano não consegue revisar 3000 linhas.”

A tabela de racionalizações do pr-ready tem a frase que resume a economia do fluxo inteiro: “Reviewer humano custa caro. CI é grátis. Resolve antes.”

5.3 Portão 2: /pre-review, o julgamento

Esta é a peça central. A skill canônica é um arquivo único, `.agents/skills/pre-review/SKILL.md`, com 193 linhas. Ela abre definindo a régua:

Revise a branch atual contra a base informada pelo usuário (padrão: `main`) procurando violações de julgamento que `lint`, `knip` e testes não detectam. Esta skill é o portão do `gh pr create: pre-pr-review-gate.ts` exige o selo do HEAD atual.

O objetivo é seniorização, simplificação e cleanup. A régua não é apenas “funciona?”, mas se um sênior deixaria a solução assim. Corte camadas, indireções e abstrações que não se pagam.

Leia os arquivos reais, não apenas as linhas do diff. Um `?? 0` ou uma abstração prematura só pode ser julgado com o arquivo inteiro.

Aquele último parágrafo é o que separa esta skill de um review de diff comum. Um `?? 0` isolado no diff é indecível: só o arquivo inteiro diz se o tipo já garantia o valor.

5.3.1 Por que ela mora em `.agents/`

`.agents/skills/` contém *apenas* pre-review. É a única skill promovida a compartilhada, e o registro no Claude Code (`.claude/skills/pre-review/SKILL.md`) é um stub de 18 linhas que só aponta para a canônica, explicando: “A skill canônica mora em `.agents/` porque é compartilhada entre Claude Code, Codex e Cursor; este arquivo é só o registro dela no Claude Code.”

O frontmatter desse stub carrega dois detalhes que não são óbvios e que definem o comportamento:

Frontmatter do registro no Claude Code

```
disable-model-invocation: true
allowed-tools: Bash(git *), Bash(mkdir *), Read, Grep, Glob, Agent,
  AskUserQuestion
```

POR QUE | A pre-review não se auto-invoca e não escreve código

`disable-model-invocation: true` significa que o modelo *não* pode disparar a skill sozinho: é chamada humana, via `/pre-review`. E a `allowlist` de ferramentas é cirúrgica; `git`, `mkdir`, leitura, subagentes e pergunta ao usuário. Não há `Write` nem `Edit`. A skill *julga e relata*; ela não pode aproveitar a passagem para “já arrumar”. Quem corrige é você, ou outro turno, com o relatório na mão.

5.3.2 Arquitetura da execução: orquestrador mais subagentes

Esta skill roda no contexto principal para poder confirmar decisões com o usuário. Para isolar o julgamento, lance um subagente novo para cada domínio independente. A skill principal orquestra os reviewers, relê o código, revalida cada achado, pede confirmação e só então grava selo ou `ack`.

Passo 0; escopo. Delimita a base e agrupa o diff por domínio coeso (por exemplo `schemas`, `hooks`, `components`, `lib`, `app` mais `error.tsx`, `shared/ui` e `auth`):

Passo 0 da pré-review – ajuste BASE se necessário

```
BASE=main
git fetch origin "$BASE" --quiet
git diff --stat "origin/$BASE...HEAD"
```

Passo 1; reviewers isolados. Um subagente *novo* por domínio, sem reutilizar reviewer entre domínios, em paralelo quando possível. Cada reviewer recebe o catálogo de julgamento, o foco do domínio e a ordem de ler os arquivos completos. E uma instrução que evita o erro clássico de review por diff: “Se a mudança altera assinatura, retorno ou contrato, procure todos os callers, mesmo fora do diff, e cheque irmãos que possam ter ficado inconsistentes.”

Formato exigido de cada achado: `arquivo:linha`, categoria, cenário concreto e evidência.

Passo 2; revalidação no contexto principal. É o contrapeso ao ruído inevitável de LLM lançado em modo caça-erro:

REGRA | Feedback de reviewer é *claim*, não veredito

“Para cada achado, releia o arquivo real e rejeite se for nitpick, cair nas permissões acima, se o fallback for necessário ou se a abstração tiver o número de usos exigido. Só mantenha o que você consegue defender com o código aberto.”

Sem esse passo, subagente em modo review produz lista infinita de nitpick e o relatório vira teatro. Com ele, o orquestrador é obrigado a assinar embaixo de cada item.

Passo 3; relatório. Placar no topo, achados agrupados por categoria e severidade, cada um com `arquivo:linha`, o que cortar ou mudar e a correção concreta. E dois detalhes que fazem diferença na prática: o relatório lista *também* os achados descartados na revalidação (você vê o que foi rejeitado e por quê) e termina com a *ordem de ataque*.

5.4 O catálogo de julgamento: A até G

Esta é a régua concreta que cada subagente recebe. Não é uma lista de boas práticas genéricas: é o inverso exato dos vícios que um agente de IA produz quando escreve código sem supervisão.

A. Over-engineering e abstração especulativa

- Um caller fica inline; extração local comum começa *no segundo caller real*.
- Abstração genérica reutilizável (framework, provider, factory, hook genérico) exige *pelo menos três callers/ usos concretos*. Exceção nomeada: primitivo concreto cross-feature (calendar, DateRangePicker, formatters) pode ir em shared no primeiro write; colocação, não abstração especulativa.
- Função ou branch de uso único (narrowing, map pequeno, parse de searchParam que só o próprio app escreveu) não vira módulo mais teste só para organizar. URL inválida que nós mesmos emitimos → redirect canônico, não segunda camada de parse.
- Wrapper que apenas repassa para outra função, *inclusive* apiClient, deve ser substituído pela chamada direta.
- Helper cerimonial (normalizeApiError, isRecord, isDefined, ensureArray) não envolve valor já garantido pelo schema. Não normalize dado que o app já possui tipado: props de RSC, catálogo const, estado interno.
- Ids de catálogo definidos em registry const: comparar via registry ou binding .id, nunca literal solto no if/switch.

B. Fallback defensivo e tratamento de erro

- Flagre ?? '', || [], || {}, ?? 0 e optional chaining redundante quando o tipo garante o valor.
- Flagre catch que engole o erro ou fabrica null, array ou shape vazio. Falha visível recebe feedback seguro; falha server-side recebe contexto útil sem vaziar token, cookie ou segredo.
- Não aceite narrowing ou fallback para shapes que o schema já validou; o apiClient devolve dado validado, não unknown.
- Caminho antigo mantido ao lado do novo, flag de compatibilidade ou ponte para código inexistente, quando a mudança não pede compatibilidade real. Contrato externo ou migração de estado vivo: pergunte antes de assumir.

REGRA | Erro do backend nunca vira workaround no frontend

Literal da skill: “Erro do backend (HTTP não-2xx ou envelope de falha) nunca vira workaround no frontend. O fluxo correto é reportar, perguntar ao usuário e recomendar corrigir o backend; contorno no frontend exige aval explícito.”

É a regra mais forte do catálogo inteiro, e a que mais economiza dívida a longo prazo. O agente tem viés brutal para “consertar” um 500 com um ?? [] e seguir a vida. Isso transforma um bug de backend rastreável em um comportamento silencioso do frontend que ninguém mais encontra.

C. Regra de negócio e authz inventadas no frontend

- Flagre preço, desconto, elegibilidade, limite ou limiar mágico reafirmando regra do backend. Um range apenas exibido pela UI não é invariante do frontend.
- Flagre espelho no front de período/data/métrica que o backend já tipou ou deveria agregar; presets e resolução de intervalo são do backend quando o contrato existir; não re-derive domínio.
- Cookies públicos de papel/permissão são dicas de UI, *nunca* autorização real.
- Flagre decode manual de JWT.
- Mutação exige `revalidatePath` ou `router.refresh`; login, refresh e logout devem chamar o backend diretamente do browser para o Set-Cookie chegar.
- Em regra ambígua, pergunte em vez de adivinhar.

D. Slop, DRY e naming

- Schema, tipo, util, máscara ou formatter que já existe no repositório sob nome sinônimo. “Reusar código existente não é abstrair cedo.”
- Nome genérico (`data`, `item`, `helper`, `result`), comentário que narra a linha, ou JSDoc sobre a sessão/PR em vez do porquê duradouro.
- Valor visual hardcoded em `.tsx`, componente visual copiado em vez de variar por prop/variant, lógica pesada em `.tsx`, `form.watch()` no corpo do componente e texto visível fora do PT-BR natural.

E. Acessibilidade e estados. `<div onClick>` no papel de controle, botão-ícone sem nome acessível, input sem label e fluxo assíncrono sem loading/empty/error.

F. Mídia e responsividade. `<img>` cru, raster pesado sem WebP/AVIF, layout que quebra no mobile e media query acima de 1366.

G. Testes. Bugfix sem teste de regressão, e teste que acopla a detalhe interno em vez de comportamento.

5.5 As duas listas anti-ruído

Um catálogo de coisas para flagrar, sozinho, produz um reviewer paranoico. A skill carrega duas listas explícitas de *o que não é achado*, e essa é a parte que a maioria dos processos de review caseiros esquece.

5.5.1 “Não flagrar”

- `React.cache()`, `Promise.all`, `useOptimistic` e `cacheComponents` são permitidos.
- `instanceof Error` que não engole falha e validação `valibot` no client são corretos quando há revalidação server-side.
- E-mail, telefone, URL de marketing e copy são conteúdo de UI em `shared`, não backend em `lib/api`.

- Imports node: em tooling/hooks executados pelo Bun são permitidos.
- 'use client' em folha realmente interativa e as const são permitidos.
- Extração local comum a partir do segundo caller real é permitida; abstração genérica reutilizável exige pelo menos três callers/ usos concretos.
- Regras de commit, PR e base são processo, não defeito do diff.

5.5.2 “Não duplicar gates automáticos”

Não re-revise cache proibido, fetch/watch em effect, localização de valibot e ky, acesso ao backend fora do apiClient, barrels, direção de camadas, document.cookie, bibliotecas JWT, middleware.ts, assertions/tipos proibidos, código/dependência morta, return type explícito, no-await-in-loop, runtime Node ou TypeScript clássico. bun run verify cobre essas famílias.

POR QUE | A pre-review é complemento disjuncto do lint, não uma segunda passada

Esse parágrafo é o que fecha o argumento da seção. bun run verify é, nas palavras do README.md, “o único gate que importa antes de concluir”: lint type-aware, oxfmt, typecheck de app/testes/tooling, knip, testes, guard de supressão de lint e budget de peso de assets. Tudo que ele prova está *proibido* de reaparecer no relatório da pre-review. O efeito prático: o relatório que você lê contém só o que a máquina não sabia dizer. Um relatório de 6 itens onde todos os 6 exigem um humano decidir vale mais que um de 40 onde 34 são eco do linter, e, principalmente, é um relatório que você ainda vai ler no décimo PR da semana.

5.6 O selo: como o portão realmente fecha

Aqui o método deixa de ser documentação e vira código executável.

5.6.1 Como a skill grava

Depois do relatório, a skill pede confirmação ao usuário. Mesmo sem achado alto, ela precisa informar claramente que vai gravar o selo. Só depois do OK:

Gravação do selo, vinculada ao HEAD

```
mkdir -p .agents/tmp  
git rev-parse HEAD > .agents/tmp/pre-review-$(git rev-parse HEAD).ok
```

“O selo vale apenas para esse HEAD. Commit novo exige outra pré-review. Com violação alta em aberto, não grave sem liberação explícita.”

Note o detalhe elegante: o SHA está no *nome* do arquivo e no conteúdo. Renomear um selo antigo à toa não produz um selo válido em espírito, e “renovar o selo” é operacionalmente indistinguível de rodar a pré-review de novo.

5.6.2 Como o hook exige

O arquivo é [tooling/agent-hooks/pre-pr-review-gate.ts](#). Ele roda como PreToolUse de Bash (ou beforeShellExecution no Cursor) e executa nesta ordem exata:

Ordem de decisao do pre-pr-review-gate.ts

#	Situação	Decisão
1	JSON de entrada inválido	deny (fail-closed)
2	Não é evento de shell	allow
3	Comando não é string	deny
4	Comando não casa gh pr create	allow ; tudo o mais passa
5	--base/-B passado mas malformado	deny
6	Base não é main nem feature/*	deny (PR empilhado é permitido)
7	git rev-parse HEAD falha	deny
8	Selo do HEAD ausente	deny
9	Sem ack e diff acima de 3000 LOC	deny
10	Tudo ok	allow

O coração são estas linhas:

tooling/agent-hooks/pre-pr-review-gate.ts – verificação do selo

```
if (!existsSync(`.agents/tmp/pre-review-${head}.ok`)) {
  emitDeny(
    host,
    `PR travada: a pré-review de guardrails não rodou pro commit atual
    (${head.slice(0, 8)}).\n` +
    `Rode a skill \pre-review\` (\pre-review\`) primeiro. Ela
    grava o selo que libera o gh pr create. ` +
    `Qualquer commit novo depois disso exige nova pré-review.`
  );
}
```

E o teto de tamanho, avaliado apenas quando o ack não existe:

Contagem de LOC no hook

```
const loc = numstat
  .split("\n")
  .filter(Boolean)
  .reduce((sum, line) => {
    const [added, deleted] = line.split("\t");
    return sum + (added === "-" ? 0 : Number(added) + Number(deleted));
  }, 0);
```

O `added === "-"` é o caso de arquivo binário no `--numstat`: conta zero em vez de virar NaN e envenenar a soma. Detalhe pequeno, mas é a diferença entre um gate confiável e um gate que trava aleatoriamente quando alguém commita um PNG.

Acima de 3000 LOC, a skill *não* grava o ack automaticamente. Ela mede e pergunta:

Medida de LOC do lado da skill

```
BASE=main
git diff --numstat "origin/$BASE...HEAD" | awk -F'\t' '$1!="-" {s+=$1+$2} END {print s}'
```

“Pergunte se o usuário prefere dividir por domínio (recomendado) ou seguir por ser uma unidade coesa indivisível.” Só após confirmação, o ack; também vinculado ao HEAD:

Ack de tamanho, também vinculado ao HEAD

```
git rev-parse HEAD > .agents/tmp/pr-size-ack-$(git rev-parse HEAD).ok
```

5.6.3 Fatos operacionais que mudam seu dia a dia

- `.gitignore` contém `.agents/tmp/`. O selo **nunca é commitado**: não viaja entre máquinas, não entra no PR, não é auditável no GitHub. É estado local, por commit, na estação de quem está abrindo o PR.
- Qualquer commit novo invalida selo e ack ao mesmo tempo; inclusive `--amend` e rebase, que mudam o HEAD sem mudar “o trabalho”. Rebaseou para resolver conflito? Rode a pré-review de novo.
- O único “sim, siga” do sistema é o ack de tamanho, e ele cobre *apenas* o teto de 3000 LOC. Não existe ack para a ausência do selo.

CAUIDADO | Fluxo típico que trava e como sair

Você rodou `/pre-review`, gravou o selo, aí aplicou um fix do relatório e commitou. O selo agora é de um HEAD que não existe mais, e `gh pr create` é negado com a mensagem que cita os 8 primeiros caracteres do SHA atual. Não há bypass: rode `/pre-review` de

novos. Na prática, a ordem econômica é: rodar a pré-review, *corrigir tudo*, commitar, e só então rodar a pré-review final que grava o selo.

5.6.4 O gate é multi-host e é testado

Os três hosts apontam para o *mesmo* script; os hooks vivem em `tooling/agent-hooks/` e cada host só referencia:

Registro do hook nos três hosts

Host	Arquivo e evento	Detalhe
Claude Code	<code>.claude/settings.json</code> , PreToolUse / Bash	timeout de 15 s
Codex	<code>.codex/hooks.json</code> , PreToolUse / Bash	mensagem de status
Cursor	<code>.cursor/hooks.json</code> , beforeShellExecution	failClosed

O gate tem suíte própria em `tooling/agent-hooks/pre-pr-review-gate.test.ts`, com 7 casos: comandos Bash não relacionados passam; JSON malformatado é negado com exit zero; base que não é `main` nem `feature/*` é negada *antes* das checagens de Git; o selo por HEAD é checado com base `main` entre aspas; o selo do diretório compartilhado de skills é aceito; o comando citado dentro de heredoc literal é ignorado; e fora de um repositório Git ele nega com segurança. `tooling/agent-hooks/hook-configs.test.ts` ainda testa que “a skill pre-review é descobrível no diretório compartilhado de skills”.

POR QUE | O caso do heredoc: documentar o fluxo não é executar o fluxo

Comentário literal no teste: “Escrever um doc que EXPLICA `gh pr create` não é abrir uma PR. Sem isto, documentar o próprio fluxo travava na escrita do documento.”

A solução está em `stripQuotedHeredocBodies`, em `tooling/agent-hooks/host.ts`: o corpo de heredocs com delimitador entre aspas é removido antes da checagem, porque o shell trata aquilo como dado literal. E o JSDoc marca o limite com honestidade: “Só o delimitador entre aspas é removido; heredoc sem aspas expande variáveis e substituição de comando, então ali ainda pode haver execução real.” É o tipo de precisão que separa um guardrail utilizável de um que todo mundo desliga na segunda semana.

5.6.5 Um deny que ensina

Detalhe fino de `tooling/agent-hooks/host.ts`: `emitDeny` não sai com código de erro. Ele escreve JSON e faz `process.exit(0)`. Claude e Codex recebem `hookSpecificOutput.permissionDecision: "deny"` com `permissionDecisionReason`; o Cursor recebe `{ permission: "deny", user_message, agent_message }`.

O efeito é que a mensagem de bloqueio volta *para o agente*, em português, dizendo exatamente o que fazer: rode `/pre-review`. O agente lê, entende e executa. O guardrail não só barra; ele ensina o próximo passo. Num boilerplate feito para ser escrito por IA, isso

não é cosmético: é o mecanismo pelo qual o processo se auto-propaga sem você repetir a instrução em cada turno.

5.7 Portão 3: /create-pr e a regra de confiança

A skill `pr-creation` é deliberadamente burra a respeito de qualidade. Ela declara três vezes, em pontos diferentes do arquivo, a mesma coisa: “`pr-creation` **confia no veredito READY do** `pr-ready`: não re-roda `lint/typecheck/test` nem o gate de `diff`.” Se `pr-ready` não deu `READY`, ela manda você voltar.

O comando `.claude/commands/create-pr.md` torna a dependência explícita antes de qualquer coisa:

Pré-requisitos, nesta ordem: 1. `/pr-ready`; só prosseguir se o veredito foi `READY`. 2. `/pre-review`; grava o selo `.agents/tmp/pre-review-<HEAD>.ok`. **Sem ele o hook `pre-pr-review-gate.ts` bloqueia a abertura do PR**, e `commit` novo invalida o selo.

A fase final da skill é exatamente o comando que o hook intercepta:

Fase de abertura do PR

```
TYPE=feat
KEY=ABC-123
REVIEWER=seu-usuario
gh pr create \
  --title "$TYPE/$KEY - título da issue" \
  --body "$(cat description.md)" \
  --base main \
  --reviewer "$REVIEWER"
```

Entre os sinais de alerta “PARE” da skill estão “vai abrir PR sem `pr-ready` ter retornado `READY`” e “vai forçar push (proibido pelo hook)”; este último tem lastro em código: `tooling/agent-hooks/guard-shell.ts` bloqueia `git push --force` sem `--force-with-lease` ou `--force-if-includes`, com a mensagem “Comando bloqueado: `git push --force`. Use `--force-with-lease`.”

REGRA | Só um elo da cadeia não depende de boa vontade

`/pr-ready` e `/create-pr` são *skills*: um agente pode simplesmente não invocá-las. `/pre-review` também é uma skill, mas é a única com um *hook de host* exigindo o resultado dela no momento exato do `gh pr create`.

É por isso que o selo existe. Ele converte a etapa mais fácil de pular (a que exige julgamento e tempo) na única que não dá para pular. Se você for cortar algo deste fluxo ao adaptar o boilerplate, corte na ponta mecânica; ela é reproduzível. O portão 2 é o que não se recompra.

5.8 Antes do PR, localmente: /review

Há uma peça vizinha, opcional e mais barata que o ciclo completo: [.claude/commands/review.md](#) aceita uma chave de issue ou um path (/review src/features/auth) e executa as 6 fases da skill feature-audit, de “Phase 0 Impact Pre-Flight” a “Phase 5 Code Quality”. Saída: relatório com veredito READY / NEEDS FIXES e issues ordenadas por severidade. Ela ativa feature-audit, api-contract, layer-boundaries, no-barrel-policy, nextjs-react19-architecture, naming-conventions, core-principles, typescript-strict e, opcionalmente, gitnexus-impact-analysis.

Use durante a implementação, quando ainda dá para mudar de rumo barato. A pre-review é o portão do PR; a /review é o espelho no meio do caminho.

5.9 Depois do PR: pr-review-response

O método não termina no `gh pr create`. A skill `pr-review-response` trata os comentários de review; de bot ou de humano; em 4 fases, com **três paradas obrigatórias para autorização humana**.

As 4 fases do pr-review-response

Fase	O que acontece
0	Contexto: <code>gh pr checkout <number></code> e <code>gh pr view <number> --json ...</code>
1	Coleta via <code>gh api .../pulls/{number}/comments --paginate</code> e <code>.../reviews --paginate</code>
2	Classificação em FIX / JUSTIFY / PARTIAL. Aguardar aprovação do usuário antes de executar
3	Aplicar fixes, rodar <code>bun run lint && bun run typecheck && bun test</code> . Aguardar usuário commitar e pushar
4	Responder cada thread via <code>gh api .../comments/{comment_id}/replies</code> , com autorização explícita

A classificação é o núcleo: **FIX** é problema real, corrige. **JUSTIFY** é comentário válido, mas a implementação atual está certa por motivo específico. **PARTIAL** é parte aplica, parte justifica.

Entre as regras obrigatórias: “Nunca responder sem aprovação do usuário”; “Nunca push automático: usuário controla”; “Sempre ler o código antes de classificar (nunca só pelo texto do comment)”; e a mais reveladora, “Se comment menciona pattern do projeto, verificar se o pattern de fato existe”.

A tabela de referência da skill traz um caso que fecha o círculo com a arquitetura deste boilerplate: o comentário “Por que buscar dados no Server Component?” é classificado como **JUSTIFY**; “RSC-puro: sem React Query/cache”. A skill já antecipa que um reviewer

externo vai sugerir exatamente o que o repositório proíbe. E responde à racionalização mais comum de todas: “CodeRabbit pediu, eu já vi, posso aplicar direto” → “Não. Plan + aprovação.”

5.10 O mesmo princípio, aplicado ao reviewer de IA

A integração opcional com CodeRabbit (documentada em [integracoes/coderabbit/](#)) repete a doutrina da pré-review, invertida para input externo:

Review por IA acelera o loop, mas **feedback de IA é claim, não verdade**: o CodeRabbit erra, sugere o que as nossas regras proíbem (check defensivo, fallback silencioso, abstração especulativa) e comenta sobre código que já mudou. O método não trata o bot como autoridade: ele entra como uma primeira passada barata, e o julgamento fica com você.

Compare com a linha da pré-review: “Feedback de reviewer é claim, não veredito.” É a mesma regra aplicada a subagente interno e a bot externo; ou seja, “claim, não veredito” é um princípio do método, não uma frase solta.

A disciplina operacional é um filtro de 3 eixos, aplicado a cada comentário, lendo o código real no HEAD atual:

1. **Ainda se aplica?** Se o ponto já não existe no código, descarte.
2. **Bate com as nossas regras?** Cruze com [AGENTS.md](#) e [CODE-PATTERN.md](#). “O CodeRabbit às vezes sugere o que a gente **proíbe**: check defensivo, fallback ?? ' ', abstração especulativa, wrapper que só repassa, validação de regra de negócio (isso é do backend).” Se contradiz uma regra nossa, o veredito é pular, citando a regra.
3. **Está correto?** “O CodeRabbit alucina.” Verifique o mérito técnico contra o código, não contra a autoridade do bot; quando der, prove com um teste rápido antes de aplicar.

E o método não depende dele: “a skill pré-review-response e o comando de review funcionam com qualquer reviewer (inclusive humano), e o feature-audit cobre a validação local antes do PR.”

5.11 Os limites do portão, ditos com honestidade

Um doc que só elogia o próprio mecanismo é propaganda. Estes são os limites reais, verificados no código:

CUIDADO | O que o gate não faz

- **Não valida o conteúdo do selo.** O hook faz apenas `existsSync`. Escrever o arquivo à mão passa. O guardrail é contra esquecimento e automatismo do agente, não contra fraude deliberada do dono do repositório.
- **Não roda no CI.** O selo está no [.gitignore](#), então não existe verificação em [.github/workflows](#): o portão é local, na estação do agente, no instante do `gh pr create`.

- **Não expira por tempo.** A única invalidação é a mudança de HEAD. Não há comando para revogar um selo.
- **Não tem bypass.** Não existe flag, variável de ambiente ou allowlist no `pre-pr-review-gate.ts`. O único “sim, siga” é o ack de tamanho, e ele cobre exclusivamente o teto de 3000 LOC.
- **Não tem escala de severidade formal.** A skill fala em “severidade” e em “violação alta”, mas não define um enum P0/P1. O julgamento da gravidade continua sendo julgamento.
- **Não existe regra de lint que cubra A-G.** As regras `boilerplate/no-passthrough-mapper`, `no-defensive-parse`, `no-catalog-literal-compare` e `no-schema-output-convert` arranham partes de A e B. O resto é, nas palavras do próprio [CODE-PATTERN.md](#), “julgamento”.

Nada disso é acidente de implementação: é o desenho. O gate assume que você é honesto e que o agente é distraído, e resolve o segundo problema, que é o que acontece cem vezes por semana. Se o seu time precisa de prova auditável no servidor, o lugar de adicionar é o CI, não este hook.

REGRA | O resumo em uma frase

Lint prova que o código é legal. **Code review prova que o código é bom.** Este boilerplate automatizou o primeiro até o osso justamente para que o segundo tenha espaço, e depois colocou um hook na frente do `gh pr create` para que o segundo não vire opcional. Porque regra que dá para pular não é guardrail.

PARTE 6

Arquitetura: o DAG de camadas

O QUE VOCÊ LEVA DESTE CAPÍTULO

Você vai saber onde cada tipo de código deve morar, quais dependências são permitidas e como decidir quando algo sobe para [shared/](#). A tabela de decisão no final serve como consulta rápida.

Dentro de [src/](#), o boilerplate organiza o código em quatro camadas e deixa a direção das dependências explícita:

A regra em uma linha (ARCHITECTURE.md)

```
app → features → lib → shared → (externo)
```

Isso é um DAG, um grafo dirigido acíclico: nenhuma seta volta. O desenho não é só uma convenção de estilo. Import invertido, import entre features irmãs e barrel de re-export quebram o bun run lint; duas regras custom do repo (boilerplate/enforce-boundaries e boilerplate/no-index-barrel) trabalham junto com import/no-relative-parent-imports para fechar o desvio mais óbvio.

Esta seção mostra cada camada, o que o lint realmente consegue proteger e onde ainda há uma lacuna declarada. No fim, uma tabela ajuda a decidir onde colocar uma mudança concreta.

6.1 As quatro camadas

Camadas, o que cada uma pode importar e seu papel (ARCHITECTURE.md)

Camada	Pode importar	Papel
app/	tudo	Rotas, layouts, composição
features/<dom>/	shared , lib , a própria feature	Slices de domínio. Não importa sibling feature
lib/	shared , externo	Infra: api/ (ky + apiClient), auth/ , env , proxy
shared/	só externo	Folha pura: ui/ , utils/ , schemas/ , types/

Guarde as duas últimas linhas: `lib/auth/` e `shared/` são justamente as entradas dessa tabela que **não existem no repo**. As duas ausências são deliberadas e estão explicadas adiante, e voltam na tabela de divergências no fim da seção.

REGRA | Os três invariantes do DAG

- `shared` é **folha**: não importa nada interno. Nem `lib`, nem `features`, nem `app`.
- O que é compartilhado entre features **sobe para `shared`**: feature nunca importa feature.
- **Sem barrel** (`index.*` de re-export).

Imposto por: `boilerplate/enforce-boundaries`, `boilerplate/no-index-barrel`, `import/no-relative-parent-imports`.

A decisão está registrada como ADR-008 em [MEMORY.md](#), com as alternativas que foram descartadas e o motivo:

POR QUE | ADR-008: por que não existe uma camada `entities`

As duas alternativas avaliadas e recusadas foram: (1) uma camada `entities` entre `features` e `lib`; descartada porque *o domínio do front é fino: vive no backend*; e (2) import lateral livre entre features; descartada porque *vira spaghetti*.

O primeiro ponto é o que dá coerência ao resto do boilerplate: se a regra de negócio mora no backend, o front não precisa de uma camada de entidades ricas. Ele precisa de um gateway HTTP (`lib/api`), de slices de tela (`features`) e de primitivos sem dono (`shared`). Status do ADR: ativo.

6.2 Como o lint decide: rank numérico, não heurística

A regra não adivinha. Ela atribui um número a cada camada e compara:

`tooling/oxlint-rules/rules/enforce-boundaries.ts`

```
type Layer = "app" | "features" | "lib" | "shared";

const LAYER_RANK: Record<Layer, number> = {
  app: 4,
  features: 3,
  lib: 2,
  shared: 1,
};

// Matches .../src/<layer>/... in a file path (forward or backward slashes).
const FILE_LAYER_RE = /[\/\\]src[\/\\](app|features|lib|shared)[\/\\]/;
// Matches @/<layer> no início do specifier, com ou sem barra (ex.: @/shared
// e @/shared/x: sem o (\/|$/) um import direto da raiz da camada burlava).
const IMPORT_LAYER_RE = /^@\/(app|features|lib|shared)(\/|$/);
```

O corpo do visitante de `ImportDeclaration` tem quatro passos, nesta ordem exata:

enforce-boundaries.ts – os quatro passos

```
// 1. Only act on internal imports that use the @/<layer> alias.
const toLayer = importLayer(spec);
if (!toLayer) return; // external or non-aliased: ignore

// 2. Determine which layer the current file belongs to.
const filename: string = getFilename(context);
const fromLayer = layerOf(filename);
if (!fromLayer) return; // file not under a recognised layer: ignore

const fromRank = LAYER_RANK[fromLayer];
const toRank = LAYER_RANK[toLayer];

// 3. Reversed-direction check: lower rank importing from higher rank.
if (toRank > fromRank) {
  context.report({ node, messageId: "reversedDirection", data: { from:
    fromLayer, to: toLayer } });
  return; // report once per import declaration
}

// 4. Cross-feature check: features/<A> importing from @/features/<B>
if (fromLayer === "features" && toLayer === "features") {
  const fromFeature = featureOf(filename);
  const toFeature = featureOfSpec(spec);
  if (fromFeature !== undefined && toFeature !== undefined &&
    fromFeature !== toFeature) {
    context.report({ node, messageId: "crossFeature" });
  }
}
```

São só duas mensagens de erro no repo inteiro para boundaries. Vale reconhecê-las de cara, porque é o que você vai ver no terminal:

As duas mensagens (transcritas com -> no lugar da seta)

```
reversedDirection:
"Import direction violation: '{{from}}' cannot import from '{{to}}'.
Layers must flow app -> features -> lib -> shared."

crossFeature:
"Cross-feature import: a feature must not import from a sibling feature.

Move shared code to shared/ instead."
```

POR QUE | Por que rank numérico e não uma lista de pares permitidos

Uma tabela de pares (app pode importar features, app pode importar lib, etc.) tem doze entradas e precisa ser reescrita quando alguém acrescenta uma camada. O rank tem quatro entradas e a checagem é uma linha: `if (toRank > fromRank)`.

Camada nova = uma linha no `LAYER_RANK` e uma alternativa no regex. É o tipo de decisão que só aparece quando você precisa mexer, e aí você vai agradecer.
 Nota de leitura: a regra reporta **uma vez por `ImportDeclaration`**. Se o `import` já violou a direção, ela dá `return` antes da checagem `cross-feature`. Um `import` errado nunca gera dois erros.

O alias `@/` que a regra reconhece vem do `tsconfig.json`:

```
tsconfig.json

"paths": {
  "@/*": ["/src/*"],
  "@test/*": ["/test/*"]
}
```

E o registro em `.oxlintrc.json`, nível `error` (linhas 272-273 e 190):

```
.oxlintrc.json

"boilerplate/enforce-boundaries": "error",
"boilerplate/no-index-barrel": "error",
...
"import/no-relative-parent-imports": "error",
```

6.3 O que a regra NÃO vê

Este é o ponto que separa quem entendeu o guardrail de quem confia nele cegamente. A regra tem dois `return` de escape logo no começo, e ambos são deliberados.

CUIDADO | Escopo real do `enforce-boundaries`

- Só age em imports com o alias `@/`.** `if (!toLayer) return; // external or non-aliased: ignore.` Import de pacote externo (`clsx`, `ky`, `react`) nunca dispara, e, mais importante, um import **relativo** (`../../features/perfil/x`) também não seria visto por esta regra. É por isso que `import/no-relative-parent-imports` está em `error` ao lado: sem ela, `../` seria um `bypass` completo do DAG. As duas regras funcionam em par; desligar uma neutraliza a outra.
- Só age em arquivos sob `src/`.** `if (!fromLayer) return; // file not under a recognised layer: ignore.` Consequência direta e verificável: `src/proxy.ts` está na raiz de `src`, não sob uma camada, então a regra o ignora por completo (ver a subseção sobre `proxy.ts`).

Havia um terceiro furo, e ele foi fechado de propósito. É o grupo final do `IMPORT_LAYER_RE`, visível no primeiro bloco de código desta subseção: sem ele, um import da *raiz* da camada (`@/app`, sem barra) não casava com o padrão e passava batido. O comentário no arquivo registra o caso, e há um teste dedicado a ele, chamado literalmente `fires: import de @/app sem barra final (bypass corrigido)`.

6.4 O DAG em ação: imports reais do repo

A implementação de referência é pequena (24 arquivos em `src/`, contando o `favicon.ico` e os testes), e isso é bom: dá para ler o DAG inteiro em cinco minutos. Os imports abaixo foram copiados dos arquivos, não parafraseados.

`src/app/perfil/page.tsx`; app (rank 4) importa features (3) e lib (2). Direção descendo, OK:

`src/app/perfil/page.tsx`

```
import { Suspense } from "react";

import { PerfilForm } from "@features/perfil/perfil-form";
import { perfilSchema } from "@features/perfil/perfil.schema";
import { apiClient } from "@lib/api/client";
import { API_ENDPOINTS } from "@lib/api/endpoints";
import { kyServer } from "@lib/api/ky.server";
```

`src/features/perfil/perfil.actions.ts`; feature importa a si mesma e desce para lib. Nunca uma feature irmã:

`src/features/perfil/perfil.actions.ts`

```
"use server";

import { revalidatePath } from "next/cache";

import { parsePerfilForm, type PerfilFormInput } from "@features/perfil/perfil.schema";
import { apiClient } from "@lib/api/client";
import { API_ENDPOINTS } from "@lib/api/endpoints";
import { kyServer } from "@lib/api/ky.server";
```

`src/lib/api/client.ts`; lib no seu limite inferior: só externo:

`src/lib/api/client.ts`

```
import type { StandardSchemaV1 } from "@standard-schema/spec";
import { HTTPError, type KyInstance, type Options } from "ky";
```

Repare em `src/features/perfil/perfil-form.tsx`: mesmo importando um arquivo da própria feature, na mesma pasta, o caminho é o alias completo:

```
src/features/perfil/perfil-form.tsx
```

```
import { usePerfilForm } from "@features/perfil/use-perfil-form";
import type { Perfil } from "@features/perfil/perfil.schema";
```

Não é `./use-perfil-form`. É consequência da regra à regra nativa que bloqueia imports relativos para pastas ascendentes, somada à convenção. O efeito colateral é bom: todo import do repo diz em que camada e em que feature o símbolo mora, sem você abrir nada. Uma busca com `grep -rn` pela string da feature devolve a superfície inteira dela.

Um fato final que fecha o desenho: um `grep` por `@/app` em `src/` não retorna nada. Nenhum arquivo do repo importa da camada `app`. É o que se espera de um topo de DAG; `app` é composição, não biblioteca.

6.5 Cross-feature: a proibição que mais dói

REGRA | Feature não importa feature

`features/auth` não pode importar de `features/perfil`, nem o contrário. Quando duas features precisam do mesmo código, esse código **sobe para `shared`**. Não existe "só desta vez", não existe import lateral "temporário".

POR QUE | Por que import lateral vira spaghetti

O import lateral é sempre razoável no momento em que é escrito: `features/perfil` precisa de um formatador que já existe em `features/auth`, e copiar parece pior que importar. O problema aparece seis meses depois, quando `auth` não pode mais ser alterada sem revisar `perfil`, e o grafo de features deixou de ser um conjunto de slices para virar um bloco único; só que sem nenhum arquivo que declare essa dependência.

Subir para `shared` custa um arquivo a mais e devolve a propriedade: `shared` não tem dono, é folha, e ninguém quebra ninguém ao mexer nela dentro do contrato.

CUIDADO | O erro que você vai cometer primeiro

Precisar de um tipo da feature vizinha. Se `perfil` precisa do tipo `Usuario` que `auth` definiu, a resposta não é importar de `auth`; é decidir se esse tipo pertence ao contrato de API (e aí mora no `*.schema.ts` de quem o consome, espelhando o backend) ou se é primitivo compartilhado (e aí sobe para `shared`). `enforce-boundaries` dispara igual para `import type`: é um `ImportDeclaration` como qualquer outro.

6.6 shared é folha: e hoje nem existe no disco

Esta é a parte em que o repo é honesto e você precisa ler o diretório antes de acreditar no diagrama. A tabela de camadas descreve `shared/` com quatro subpastas. Confira:

Estado real de `src/`

```
$ ls -a src/  
.  ..  app  features  lib  proxy.ts
```

Não existe `src/shared/`. Não é que esteja vazio: o diretório não foi criado, e portanto `ui/`, `utils/`, `schemas/` e `types/` também não existem. E isso é intencional; o próprio `ARCHITECTURE.md` anota na árvore de pastas: `shared/{ui,utils,schemas,types}/` # folha pura; vazia até o 1º primitivo cross-feature.

POR QUE | Um diretório ausente é melhor que quatro pastas com um util cada

`shared` não é onde você começa: é onde as coisas chegam quando provam que pertencem a mais de uma feature. Criar `shared/utils/format-date.ts` antes de existir um segundo caller é exatamente a abstração especulativa que o resto do boilerplate combate. O caminho existe no documento para marcar o lugar; a pasta e o conteúdo aparecem por promoção, não por planejamento.

O mesmo raciocínio explica outras duas ausências que confundem quem lê só a documentação:

- **As features de referência são flat.** Não existem `components/`, `hooks/`, `types/`, `utils/`, `constants/` dentro de `features/auth` ou `features/perfil`. São três arquivos soltos na raiz de `auth` (`login.schema.ts`, `use-login-form.ts`, `login-form.tsx`) e quatro na raiz de `perfil` (`perfil.schema.ts`, `perfil.actions.ts`, `use-perfil-form.ts`, `perfil-form.tsx`). Essas pastas são o *destino* quando a feature cresce, não um esqueleto a preencher.
- **Não existe `lib/auth/`,** e a ausência é uma decisão de arquitetura, não uma pendência.

POR QUE | Por que não há `lib/auth/` (ARCHITECTURE.md)

Texto do repo: *"a auth deste desenho não tem estado no front. O proxy.ts checa a presença do cookie para rotear, e o kyServer encaminha o cookie ao backend; quem valida sessão e autorização é sempre o backend. Um módulo de auth no front seria justamente a camada que o ADR-007 proíbe."*

6.7 Sem barrel

Barrel é o arquivo `index.ts` que só re-exporta os vizinhos para permitir `import { a, b, c } from '@features/auth'`. Aqui isso é erro de lint.

A regra `custom boilerplate/no-index-barrel` age apenas em arquivos cujo base-name é `index.ts`, `index.tsx`, `index.js` ou `index.jsx`; qualquer outro nome é ignorado. Dentro deles:

tooling/oxlint-rules/rules/no-index-barrel.ts (docstring)

```
export { foo } from "./foo"; // ExportNamedDeclaration with source →
  ERROR
export * from "./bar";      // ExportAllDeclaration →
  ERROR

export const x = 1;          // ExportNamedDeclaration without source
  → OK
export { y };                // ExportNamedDeclaration without source
  → OK
```

A mensagem, literal:

Mensagem de boilerplate/no-index-barrel

```
"index.* nao pode ser barrel de re-export; importe direto do modulo
(cobre o gap do oxc/no-barrel-file que so pega export *)"
```

Ela trabalha junto com duas regras nativas em `.oxlintrc.json`:

`.oxlintrc.json`, linhas 50-51

```
"oxc/no-barrel-file": ["error", { "threshold": 0 }], // bundle-barrel-
  imports: qualquer export * é erro
"import/no-namespace": "error", // import * as arrasta o módulo inteiro
```

POR QUE | Os quatro motivos (skill **no-barrel-policy**)

- **Importação opaca:** lendo `import { useLogin } from '@/features/auth'` ninguém sabe onde mora `useLogin` sem abrir o `index.ts`.
- **Mascara dead code:** re-export marca tudo como "usado", código morto escapa do lint.
- **Cria ciclos invisíveis:** A re-exporta B que re-exporta A, e o erro de runtime só aparece na produção.
- **Atrapalha tree-shaking:** o bundler conserva mais código do que precisa.

Import: certo e errado (skill no-barrel-policy)

Certo	Errado
<code>@/features/auth/hooks/use-login</code>	<code>@/features/auth</code>
<code>@/features/auth/schemas/login.schema</code>	<code>@/features/auth/schemas</code>
<code>@/features/auth/types/api.types</code>	<code>@/features/auth/types</code>
<code>@/shared/ui/button</code>	<code>@/shared/ui</code>
Não re-exportar	<code>export * from './use-login'</code>

Não há exceção nem para o design system: a skill é explícita em dizer que `shared/ui/` também não terá `index.ts` quando existir. Importe `@/shared/ui/button`.

CUIDADO | O GAP declarado do lint

A própria skill admite o buraco: *"named re-export (export { x } from) em arquivo que não é index é GAP do lint: fica por disciplina humana."* O lint cobre `index.*` (regra custom) e `export *` em qualquer arquivo (oxc/no-barrel-file). Um `export { foo } from './foo'` dentro de `foo.ts` passa, e existe teste que documenta isso como comportamento esperado.

Caça manual, quando desconfiar:

Varredura de barrel fora de index (saída esperada: vazia)

```
rg "^export \{.*\} from" src/ -g '!**/index.*'
```

Hoje o repo passa limpo: não há nenhum `index.ts` em `src/`, nem nenhum `export { ... } from`. O único `index.ts` do projeto é `tooling/oxlint-rules/index.ts`, entrypoint do plugin oxlint; não é código de aplicação.

6.8 proxy.ts: dentro do repo, fora do DAG

REGRA | O arquivo é `src/proxy.ts`, irmão de `app/`, não dentro dele

Next 16 renomeou `middleware.ts` para `proxy.ts`. O arquivo mora na raiz de `src`, no mesmo nível de `app/`. `src/app/proxy.ts` o Next simplesmente não carrega.

O ADR-010 em `MEMORY.md` registra isso como uma correção. O erro já foi cometido e resolvido aqui, e a documentação do Next foi conferida no arquivo `node_modules/next/dist/docs/.../proxy.md`:

node_modules/next/dist/docs/.../proxy.md

Note: The ``middleware`` file convention is deprecated and has been renamed to ``proxy``.

Create a ``proxy.ts`` (or ``js``) file **in** the project root, or inside ``src`` **if** applicable, so that it is located at the same level **as** ``pages`` or ``app``.

O conteúdo é curto e o JSDoc explica o papel dele melhor do que qualquer paráfrase:

src/proxy.ts

```
export function proxy(request: NextRequest): NextResponse {
  if (request.cookies.has(SESSION_COOKIE)) {
    return NextResponse.next();
  }
  return NextResponse.redirect(new URL("/login", request.url));
}

/**
 * Áreas autenticadas. Entre por prefixo, não rota a rota: listar rota
 * a rota
 * deixa toda rota nova sem gate por padrão, que é o erro caro.
 */
export const config = {
  matcher: ["/perfil/:path*"],
};
```

O JSDoc da função deixa explícito o limite da responsabilidade: *"É gate de roteamento, não autorização. A autorização real é sempre do backend, que rejeita as chamadas de dado independentemente do que o cookie diga; aqui o objetivo é não largar o usuário numa tela que responderia 403."* O nome do cookie é uma constante única (`const SESSION_COOKIE = "session_token";`), com a nota: trocar o nome ali é o único ajuste se o seu backend usar outro.

CUIDADO | `proxy.ts` não é coberto pelo `enforce-boundaries`

Ponto arquitetural verificável: como `FILE_LAYER_RE` exige `/src/<layer>/` no caminho, `src/proxy.ts` não pertence a camada nenhuma para efeito da regra; ela faz `return` e não checa *nenhum* import dele. É um arquivo de convenção do framework, fora do DAG imposto.

Na prática: um import errado ali não vai ser pego pelo lint de boundaries. Mantenha `proxy.ts` do tamanho que ele tem hoje (43 linhas, importando só `next/server`) e o problema não existe.

E o hábito contrário; criar `middleware.ts`, que é o que todo agente de IA tende a fazer por memória de treino; tem um guardrail próprio, que **não** é lint:

```
tooling/oxlint-rules/rules/__tests__/no-middleware-file.test.ts
```

```
/**
 * Guardrail: Next 16 renomeou `middleware.ts` → `proxy.ts`.
 *
 * Um agente, por hábito de treino, tende a criar `middleware.ts`. Este
 * gate
 * falha se esse arquivo existir, apontando para `proxy.ts` (convenção
 * atual do
 * Next 16). oxlint não checa "arquivo que não deveria existir", por
 * isso é um
 * bun:test.
 */
```

O teste procura variações `.ts`, `.tsx`, `.js` e `.jsx` dentro de `src/` e na raiz. Se alguém criar o arquivo, o `bun test` quebra; não o lint.

6.9 Os testes que provam o DAG

O arquivo `tooling/oxlint-rules/rules/__tests__/custom-rules.test.ts` não testa a regra por unidade: ele escreve fixtures em disco e roda o oxlint de verdade. Ler essa lista é a forma mais rápida de calibrar o que dispara e o que não dispara.

Fixtures de enforce-boundaries

Arquivo	Import	Dispara?
<code>src/shared/a.ts</code>	<code>import { x } from "@app/home";</code>	sim
<code>src/features/cart/b.ts</code>	<code>import { x } from "@features/checkout/api";</code>	sim
<code>src/lib/f.ts</code>	<code>import { a } from "@app";</code>	sim
<code>src/app/c.ts</code>	<code>import { useCart } from "@features/cart/hooks";</code>	não
<code>src/features/cart/d.ts</code>	<code>import { x } from "@features/cart/util";</code>	não
<code>src/shared/e.ts</code>	<code>import clsx from "clsx";</code>	não

Fixtures de no-index-barrel

Arquivo	Código	Dispara?
<code>index.ts</code>	<code>export { foo } from './foo';</code>	sim
<code>index.ts</code>	<code>export * from './foo';</code>	sim
<code>index.ts</code>	<code>export const answer = 42;</code>	não
<code>foo.ts</code>	<code>export { foo } from './foo';</code>	não (o GAP)

O último caso é o GAP declarado da subseção anterior, e está no teste como comportamento esperado. Isso é uma escolha: o repo prefere um teste que documenta o limite a um teste que finge cobertura.

POR QUE | Por que esses testes existem (ADR-009)

jsPlugins do oxlint é alpha, sem semver. O repo pina o oxlint (hoje 1.77.0) e mantém grep-gate de CI como backstop. O ADR diz o essencial: *"Subir a versão exige rodar `bun test` das regras custom: elas são o canário da API do plugin."* Se você atualizar o oxlint e as regras custom pararem de disparar silenciosamente, o boilerplate inteiro vira documentação sem enforcement; os testes existem para que isso apareça como falha, não como ausência de erro.

Detalhe honesto: os dois grep-gates do [.github/workflows/ci.yml](https://github.com/workflows/ci.yml) cobrem cache ('use cache'/unstable_cache) e URL de backend fora de [src/lib/api/](https://src.lib/api/). **Não existe grep-gate de CI para boundaries nem para barrel**; esses dependem do oxlint funcionando.

6.10 Quando subir para shared: a regra 1/2/3

Não há critério numérico no repo para "quando criar uma feature nova"; uma feature é um slice de domínio, e ponto. O que existe, e é preciso, é o critério de *extração e colocação*, na seção 9 do [CODE-PATTERN.md](#):

REGRA | Extração: 1/2/3

- **Um caller** → inline.
- **Segundo caller real** → extração local (na feature ou em [lib](#)).
- **Abstração genérica** (factory, provider, hook genérico) → exige **3 callers** concretos.
- **Primitivo concreto de produto** (DateRangePicker, calendar-month, formatters de data/moeda) → pode ir direto para [shared](#) quando o destino é reutilização entre features; isso é **colocação**, não abstração especulativa.

Três consequências dessa seção que impactam diretamente a leitura do DAG:

- **Normalizar só na borda externa** (request/response do backend, submit de form). Normalizer sobre dado que o próprio app produziu; prop de RSC, estado interno, entrada já tipada de catálogo const; é proibido.
- **Regra de negócio, limiar e agregação vão para o backend.** O front envia parâmetros opacos e renderiza a resposta. Não espelhe campo de domínio que a API já tipou. É o mesmo raciocínio que matou a camada `entities` no ADR-008.
- **Wrapper que só repassa** (inclusive para o `ApiClient`) → chame direto. Helper cerimonial (`isRecord`, `ensureArray`) sobre valor que o schema já garantiu → apague.

O enforcement aqui é declaradamente **parcial**. O plugin encontra mapper pass-through, parse defensivo, comparação literal e conversor de output de schema. O lint type-aware encontra condição desnecessária. O resto é julgamento.

CUIDADO | Duas regras do plugin são `warn`, e por um bom motivo

Dessa lista, `no-passthrough-mapper` não está em "error": está em "warn", e `no-literal-dispatch`, que nem aparece na lista da seção 9, também. As outras três regras `boilerplate/*` citadas acima e o `typescript/no-unnecessary-condition` são error.

O comentário no `.oxlinttrc.json` explica o par de exceções: *"Ambas veem só a AST do arquivo, sem saber quantos callers a função tem, e como supressão de lint é proibida no repo, error aqui seria um bloqueio sem saída pra um padrão às vezes legítimo."*

Traduzindo: `warn` aqui não é "regra fraca", é "candidato para review". Você não pode desligar a regra com um comentário de supressão (há um `guard` para isso no `bun run verify`), então uma regra que erra teria que ser `warn` ou não existir.

6.11 Tabela de decisão: "quero fazer X no arquivo Y, está certo?"

Consolidado da skill `layer-boundaries` e das mensagens reais do `.oxlinttrc.json`. Todo "não" abaixo é erro de lint, exceto onde marcado.

O que a camada NÃO pode (skill `layer-boundaries`)

Camada	NÃO pode
<code>app/</code>	Conter regra de negócio reutilizável. Ser importada por <code>features/lib/shared.*.screen.tsx</code> .
<code>features/<dom>/</code>	Importar sibling feature . Importar <code>app</code> . <code>Barrel</code> . <code>ky/fetch cru</code> .
<code>lib/</code>	Importar <code>features/app</code> . Usar <code>ky</code> fora de <code>lib/api</code> .
<code>shared/</code>	Importar qualquer coisa interna (<code>app/features/lib</code>).

Decisão rápida com a fonte do veredito

Quero fazer X no arquivo Y	Veredito	Quem barra
<code>import { x } from "@app" em src/lib/</code>	não	enforce-boundaries (caso testado: bypass corrigido)
<code>import de feature irmã</code>	não	enforce-boundaries, crossFeature
<code>import { x } from "../lib/api/client"</code>	não	import/no-relative-parent-imports
<code>export * from "./foo" em qualquer arquivo</code>	não	oxc/no-barrel-file (threshold: 0)
<code>export { foo } from "./foo" em index.ts</code>	não	boilerplate/no-index-barrel
<code>export { foo } from "./foo" em foo.ts</code>	passa no lint, proibido	disciplina (GAP declarado)
<code>import ky fora de src/lib/api/</code>	não	"ky só pode ser usado em src/lib/api/ (gateway único de HTTP)."
<code>import * as v from "valibot" fora de *.schema.ts</code>	não	"valibot só pode ser usado em arquivos *.schema.ts (validação confinada às bordas)."
<code>import axios</code>	não	"axios proibido: use ky em src/lib/api/ ."
<code>jose/jsonwebtoken</code>	não	"Não decodifique/assine JWT no front: o backend é a fonte da verdade."
<code>@tanstack/react-query/swr</code>	não	"React Query proibido: leitura é RSC, sem data-fetching/cache no client." / "SWR proibido: leitura é RSC, sem data-fetching no client."
<code>export default fora de src/app/**</code>	não	import/no-default-export
<code>criar src/middleware.ts</code>	não	<code>no-middleware-file.test.ts</code> quebra o bun test
<code>useQuery em componente de feature</code>	não	leitura é RSC; ver <code>nextjs-react19-architecture</code>
<code>formatPhoneBR(phone) dentro de lib/api</code>	não	<code>apiClient</code> entrega dado validado; util/componente formata
<code>try {...} catch { Toast.show() } em lib/api/action</code>	não	propaga; a camada de cima trata

Os *overrides* do `.oxlintrc.json` são cirúrgicos e vale conhecê-los, porque explicam por que o código de referência parece violar regras:

- `src/app/**` libera `import/no-default-export`; *"Next.js App Router requer default export nas rotas/layouts/pages."* Só isso.
- `**/*.schema.ts` libera **apenas** `valibot`. `ky`, `jose`, `jsonwebtoken`, `zod`, `react-query`, `swr` e `axios` seguem banidos ali.
- `src/lib/api/**` libera **apenas** `ky`. `Valibot` segue banido nessa pasta.
- `tooling/**` libera `default export` e `import relativo pai`; *"oxlint jsPlugins exige default export no arquivo de plugin e nas rules"*; além de `no-console` e das regras de `type assertion`, porque `tooling/` não é código de aplicação.

6.12 O gate

O DAG só vale porque roda. O comando canônico está no `package.json`:

`package.json (scripts)`

```
"lint": "oxlint --disable-nested-config --type-aware",
"lint:fix": "oxlint --fix --disable-nested-config --type-aware",
"verify:fast": "bun run lint && bun run fmt:check && bun run knip &&
  bun run test",
"verify": "bun run lint && bun run fmt:check && bun run typecheck &&
  bun run typecheck:tooling && bun run knip && bun run test && bun
  tooling/no-lint-suppression.ts && bun tooling/no-heavy-public-assets.
  ts"
```

REGRA | Um gate só: `bun run verify`

`bun run verify` cobre `lint type-aware`, `oxfmt`, `typecheck` do `app/testes/tooling`, `knip`, testes, guard de supressão e budget de assets. No loop de iteração, `bun run verify:fast` basta. **Se uma regra te bloquear, siga o padrão: não suprima a regra**; há um script (`tooling/no-lint-suppression.ts`) dentro do próprio `verify` caçando supressões.

Divisão de trabalho: violação de barrel e de boundaries é o `bun run lint` que pega (são regras `oxlint`). Já os testes das regras custom e o guardrail do `middleware.ts` só rodam no `bun test`. Rodar só `bun run lint` deixa passar o arquivo proibido, por isso o gate é `verify` (ou `verify:fast`), que roda os dois.

6.13 Onde a documentação e o código divergem

O repo é honesto o bastante para deixar as divergências visíveis, e você vai encontrá-las. Regra de sobrevivência: **o código e o `package.json` vencem a prosa**.

Divergências verificadas

O doc diz	O repo tem
ARCHITECTURE.md : versão pode ficar defasada após customização	package.json : baseline atual next 16.3.0, typescript 7.0.2, oxlint 1.77.0; confirme sempre no arquivo
Tabela de camadas lista auth/ e proxy como conteúdo de lib/	lib/auth/ não existe (deliberado); proxy.ts está na raiz de src , fora de qualquer camada
Skill layer-boundaries: artefatos em components/ , hooks/ , types/ , utils/ , constants/	As features de referência são flat (3 arquivos em auth , 4 em perfil); essas pastas são o destino quando a feature crescer
CLAUDE.md e a skill layer-boundaries dizem que o gate é <code>bun run lint + bun run typecheck</code>	package.json e CODE-PATTERN.md definem <code>bun run verify</code> , que é o gate real
shared/{ui,utils,schemas,types}/ na árvore de pastas	src/shared/ não existe: src/ tem só app/ , features/ , lib/ e proxy.ts

CUIDADO | Para agentes: leia o arquivo, não a memória

Este boilerplate pode ser trabalhado com apoio frequente de agentes, e as duas fontes de erro mais caras são exatamente as duas armadilhas desta seção: criar [middleware.ts](#) porque a referência antiga diz que é assim, e criar um [index.ts](#) de barrel porque "é o padrão da comunidade". Ambas têm guardrail; uma no `bun test`, outra no `lint`; justamente porque são hábito, não descuido. A instrução que abre o [AGENTS.md](#) vale aqui inteira: esta não é a versão do Next que você conhece.

PARTE 7

Dados e bordas: RSC puro, sempre fresco

O QUE VOCÊ LEVA DESTE CAPÍTULO

Você vai entender o caminho completo de leitura e mutação, onde a validação acontece e por que esta base evita cache e busca de dados no cliente. Leia antes de criar seu primeiro endpoint.

Esta é a decisão que mais muda a rotina de quem chega de uma aplicação React tradicional. A leitura acontece no servidor, a mutação passa por Server Actions e os dados voltam sempre frescos. O efeito é não depender de React Query, SWR, hooks de fetch ou cache para montar a tela. A regra cabe em três linhas; o restante da seção mostra o código por trás dela e o que o lint reprova quando alguém tenta voltar ao caminho antigo.

REGRA | A doutrina, em três linhas

Leitura de dados = Server Component. Só isso. Sem lib de data-fetching, sem fetch em useEffect.
Mutação de dado = Server Action, seguida de revalidatePath ou router.refresh.
Cache é proibido: 'use cache', unstable_cache, cache: "force-cache", revalidate = false ou > 0, dynamic: 'force-static'. React.cache() (de-dup por request) é permitido.

7.1 Onde a doutrina está escrita (e por que em quatro lugares)

A mesma regra aparece repetida de propósito. Documento canônico serve para o humano; o inject de agente serve para o Claude Code / Codex, que não lê o repositório inteiro a cada sessão.

As quatro fontes da mesma regra

Fonte	Onde
Regra de código	CODE-PATTERN.md §1 (Dados) e §2 (Bordas)
Visão arquitetural	ARCHITECTURE.md , seções "Fluxo de dados: RSC-puro, sempre fresco" e "Bordas"
Decisão e porquê	MEMORY.md : ADR-003, ADR-004, ADR-005, ADR-006, ADR-022, ADR-025
Injetado no agente	tooling/agent-hooks/inject-patterns.ts

O texto que o agente recebe em toda sessão é o mesmo, condensado:

tooling/agent-hooks/inject-patterns.ts (trecho)

Dados: RSC, sempre fresco:

- Leitura de dados = Server Component. Só isso. Não buscar dados no client. Sem lib de data-fetching (sem SWR, sem React Query). Proibido fetch/watch em `useEffect`.
- Proibido cache: `'use cache'`, `'unstable_cache'`, `'cache: "force-cache"'`, `'revalidate = false | >0'`, `'dynamic: "force-static"'`. `'React.cache()'` (dedup por request) é permitido. Fetches paralelos com `Promise.all`.
- Mutação de dado = Server Action; depois `'revalidatePath'/'router.refresh'` pra repuxar fresco.
- Login/logout/refresh = chamada direta do browser ao backend (ele seta o cookie).

POR QUE | Por que sem cache (ADR-003 e ADR-004)

O ADR-003 é direto: *"é um CRM: frescor > performance de cache. É também o default do Next 16."* A alternativa descartada foi ISR, pelo motivo óbvio: introduz staleness.

O ADR-004 fecha a porta do client: *"SWR é cache → contradiz o 'sempre fresco' (ADR-003). RSC-puro é mais coerente."* O status registrado tem uma nota honesta: *"ativo (revertido de uma escolha inicial por SWR)."* Ou seja, a escolha inicial do projeto era SWR e foi desfeita. Isso não é dogma de quem nunca usou a alternativa.

7.2 Leitura = Server Component (o exemplo real)

O boilerplate tem exatamente uma feature de leitura, e ela é o exemplo canônico do padrão inteiro: `src/app/perfil/page.tsx`.

src/app/perfil/page.tsx

```
import { Suspense } from "react";

import { PerfilForm } from "@/features/perfil/perfil-form";
import { perfilSchema } from "@/features/perfil/perfil.schema";
import { apiClient } from "@/lib/api/client";
import { API_ENDPOINTS } from "@/lib/api/endpoints";
import { kyServer } from "@/lib/api/ky.server";

/**
 * Carrega o perfil e monta o formulário.
 *
 * Fica separado da página porque ler o cookie (dentro do `kyServer`)
 * torna esta
 * subárvore dinâmica. Com `cacheComponents` ligado, o Next exige que a
 * parte
```

```
src/app/perfil/page.tsx (continuação)

* dinâmica esteja sob um `<Suspense>`: o shell estático vai no primeiro byte e o
* dado chega em streaming. Isso não é cache — cada request busca fresco.
*/
async function PerfilCarregado(): Promise<React.ReactNode> {
  // O apiClient valida a resposta contra o schema antes de qualquer coisa chegar
  // na UI, então o componente recebe dado tipado, não `unknown`. Se a chamada
  // falhar, o erro sobe pro `error.tsx` mais próximo: nada de `catch` que
  // fabrica um perfil vazio.
  const perfil = await apiClient(kyServer, API_ENDPOINTS.perfil.me, perfilSchema);

  return <PerfilForm perfil={perfil} />;
}

/**
 * Perfil do usuário: a feature de exemplo do boilerplate, ponta a ponta.
 *
 * * Leitura é RSC — nenhum dado é buscado no client. Mutação é Server Action com
* `revalidatePath` (ver `perfil.actions.ts`).
*/
export default function PerfilPage(): React.ReactNode {
  return (
    <main className="mx-auto flex w-full max-w-3xl flex-col gap-8 px-6 py-16">
      <header className="flex flex-col gap-2">
        <h1 className="text-2xl font-semibold tracking-tight">Perfil</h1>
        <p className="text-sm text-foreground/60">
          Leitura em Server Component, mutação em Server Action. Edite os campos e salve.
        </p>
      </header>

      <Suspense fallback={<p className="text-sm text-foreground/60">Carregando perfil...</p>}>
        <PerfilCarregado />
      </Suspense>
    </main>
  );
}
```

Três detalhes estruturais que não são estéticos:

1. O `async` não está na page. `PerfilPage` é síncrono. Só `PerfilCarregado` é `async`, e ele vive sob `<Suspense>`. É esse split que permite ao Next emitir o shell estático imediatamente e streamar o dado depois. Se você colocar o `await` na própria page, a rota inteira vira dinâmica bloqueante e (como veremos) o build reprova.

2. O componente recebe dado tipado, não unknown. O `apiClient` valida a resposta contra `perfilSchema` antes de qualquer coisa chegar na UI. O tipo de `perfil` é inferido do schema, não declarado à mão.

3. Não há try/catch. Se a chamada falhar, o erro sobe. O comentário no arquivo é explícito: *"nada de catch que fabrica um perfil vazio"*. Fallback silencioso é a forma mais cara de esconder um bug de integração.

CUIDADO | O exemplo de `CODE-PATTERN.md` usa uma feature que não existe

O §1 do `CODE-PATTERN.md` ilustra o padrão com uma página `clientes` e um `clientesSchema`. Isso é ilustração: **não existe feature `clientes` no repositório**. O código real, o único, é `perfil`. Se você (ou um agente) for procurar `src/features/clientes/`, não vai achar.

7.3 `cacheComponents` está ligado: e não é cache

Este é o ponto que mais confunde quem abre o `next.config.ts` pela primeira vez e vê uma flag com "cache" no nome num repositório que proíbe cache.

next.config.ts (trecho)

```
const nextConfig: NextConfig = {
  reactCompiler: true,
  typedRoutes: true,
  // Shell estático + buraco dinâmico em streaming. NÃO é cache: cada request
  // busca fresco; o que fica pré-renderizado é só a casca sem dado. A
  // parte que
  // lê cookie/dado precisa ficar sob um `` (ver src/app/perfil/page.tsx).
  cacheComponents: true,
  productionBrowserSourceMaps: false,
  // ... (o rewrite de dev vem aqui, comentado: ver adiante)
};
```

POR QUE | ADR-025: streaming, não cache

"Dá shell estático no primeiro byte + dado em streaming (a rota sai como Partial Prerender no build). Isso não contradiz o ADR-003: nada fica em cache, o que é pré-renderizado é só a casca sem dado, e cada request busca fresco."

O ADR-003 foi emendado para registrar isso explicitamente, em vez de deixar a aparente contradição para o leitor resolver sozinho.

A cadeia causal completa, que explica por que `src/app/perfil/page.tsx` tem aquele formato e não outro:

1. `kyServer` chama `await cookies()` no seu hook `beforeRequest`.
2. Ler `cookies()` marca a subárvore como dinâmica.

3. Com `cacheComponents: true`, parte dinâmica fora de `<Suspense>` **reprova o build** com `blocking-prerender-dynamic`.
4. Por isso `PerfilCarregado` é um componente separado, sob `<Suspense>`.

O ADR-025 registra que a rota sai como `Partial Prerender` no build, e o [README.md](#) descreve o arquivo com a mesma etiqueta: `src/app/perfil/page.tsx` leitura RSC sob `<Suspense>` (`Partial Prerender`).

CUIDADO | O aviso do Bun no build não é um erro seu

O ADR-025 registra o caveat: *"sob Bun o build imprime um aviso de que o Next não garante o comportamento de Cache Components por causa da implementação de `setTimeout` do runtime. O output sai correto; se um dia morder, remover a flag custa uma linha (a rota volta a ser dinâmica pura)."*

Você vai ver esse aviso no primeiro `bun run build`. Não é configuração errada nem regressão do seu código. A saída de emergência está documentada e custa uma linha.

7.4 Mutação = Server Action, depois `revalidatePath`

O par da leitura é `src/features/perfil/perfil.actions.ts`:

`src/features/perfil/perfil.actions.ts` (trecho)

```
"use server";

import { revalidatePath } from "next/cache";
// ...

/**
 * Resultado da mutação, no formato que o `useActionState` do React 19
 * consome.
 * `erro` carrega a mensagem que o backend devolveu – texto já seguro
 * pra UI.
 */
export type SalvarPerfilResult = { ok: true } | { ok: false; erro:
  string };

export async function salvarPerfil(entrada: PerfilFormInput): Promise<
  SalvarPerfilResult> {
  try {
    const validado = parsePerfilForm(entrada);
    // Sem schema: o contrato deste endpoint não devolve `data`, só o
    envelope.
    await apiClient(kyServer, API_ENDPOINTS.perfil.me, undefined, {
      method: "put",
      json: validado,
    });
  } catch (error: unknown) {
    // O `apiClient` já converteu a falha na mensagem segura do backend
    (ou no
    // texto de conexão). Repassar aqui é entregar feedback, não
    engolir erro.
  }
}
```

```
src/features/perfil/perfil.actions.ts (trecho) (continuação)
```

```
    return { ok: false, erro: error instanceof Error ? error.message :  
      ERRO_INESPERADO };  
  }  
  
  revalidatePath("/perfil");  
  return { ok: true };  
}
```

Duas coisas merecem parágrafo próprio.

A validação roda de novo no servidor. O formulário já validou no client via `valibotResolver`, e mesmo assim a action chama `parsePerfilForm`. O JSDoc do arquivo diz o porquê em uma frase: *“validação de client é UX, não fronteira de segurança”*. Qualquer um pode chamar a Server Action fora do seu formulário.

O `revalidatePath` aqui não invalida cache; não existe cache para invalidar. Ele força o re-render da árvore de servidor. É a única razão de a tela mostrar o valor novo depois do salvar.

CUIDADO | Esquecer o `revalidatePath` não quebra nada visivelmente

Sem a chamada, o PUT vai para o backend, o backend grava, a action retorna `ok: true`, e a tela continua exibindo o valor antigo. Nenhum erro, nenhum log, nenhuma regra de lint. É o bug mais fácil de introduzir nesta arquitetura, porque o caminho feliz parece ter funcionado.

O tipo de retorno, `SalvarPerfilResult`, é uma união discriminada por `ok`: ou deu certo, ou tem mensagem; não há `erro: string | null` com `ok` sempre presente. O JSDoc descreve esse formato como “o que o `useActionState` do React 19 consome”, mas repare que o consumidor real no repositório **não** é `useActionState`: é o hook `src/features/perfil/use-perfil-form.ts`, que chama `salvarPerfil` dentro do `handleSubmit` do `react-hook-form` e reflete o resultado em `useState`. O ADR-015 descarta `useActionState` explicitamente como default (*“melhor pra forms públicos/-simples e progressive enhancement, mas pobre em UX rica e incompatível com o login browser-direct”*). A união discriminada continua sendo o formato certo; só não pelo motivo que o JSDoc sugere.

7.5 Login é a única exceção: browser-direct, por causa do cookie

Toda leitura sai do servidor. Login não. E o motivo é mecânico, não estilístico.

REGRA | A regra do cookie

O `Set-Cookie` e `HttpOnly` emitido pelo backend só pousa no browser se a requisição **partiu** do browser. Login feito por Server Action perderia o cookie no caminho: quem receberia o `Set-Cookie` seria o processo Node/Bun, não o navegador do usuário.

Daí as duas instâncias `ky`, e a divisão exata entre elas:

As duas instâncias ky

	kyServer	kyClient
Arquivo	src/lib/api/ky.server.ts	src/lib/api/ky.client.ts
Base URL	serverEnv.INTERNAL_API_URL	getClientApiBaseUrl()
Usada em	RSC e Server Actions	Login (e, quando existirem, logout e refresh)
Cookie	encaminha o cookie da request atual via cookies()	credentials: "include": o browser envia e recebe sozinho
Leitura de dado	sim, sempre	nunca

O JSDoc do `useLoginForm` não deixa margem: *“Esta é a ÚNICA família de chamada que sai do browser direto pro backend (kyClient), e o motivo é o cookie: o Set-Cookie HttpOnly da resposta só pousa se a requisição partir do browser. Login feito por Server Action perderia o cookie no caminho. Leitura de dado, ao contrário, é sempre RSC.”*

`src/features/auth/use-login-form.ts` (submit)

```
const entrar = form.handleSubmit(async (credenciais) => {
  setErroServidor(null);
  try {
    // Sem schema: a identidade vem do cookie, o contrato não devolve `data`.
    await apiClient(kyClient, API_ENDPOINTS.auth.login, undefined, {
      method: "post",
      json: credenciais,
    });
  } catch (error: unknown) {
    setErroServidor(error instanceof Error ? error.message :
    ERRO_INESPERADO);
    return;
  }
  router.replace(destino);
  router.refresh();
});
```

O `router.refresh()` depois do `replace` não é redundância: ele repuxa a árvore de servidor já com o cookie, e é o que faz o [src/proxy.ts](#) parar de redirecionar para o login.

7.5.1 A consequência: onde o cookie pousa

Escolher `browser-direct` tem preço, e ele está documentado em [src/lib/api/base-url.ts](#). `NEXT_PUBLIC_API_URL` aceita dois formatos, e a escolha é inteiramente sobre o cookie:

Os dois formatos de NEXT_PUBLIC_API_URL

Valor	O que implica
"/" (same-origin)	Backend responde no mesmo origin. <i>"É o único jeito de um cookie HttpOnly de login pousar sem depender de SameSite=None."</i> Em dev vem do rewrite /api/* do next.config.ts ; em produção, de um domínio same-site.
URL absoluta	Backend em outro origin. Exige CORS com Access-Control-Allow-Credentials no backend e cookie SameSite=None; Secure.

O rewrite de dev existe, mas vem **comentado** no [next.config.ts](#), com a instrução de quando ligar:

next.config.ts (trecho comentado)

```
// Proxy same-origin só em dev. Login/logout/refresh são chamadas
// browser-direct
// (o backend é quem emite o cookie), e o `Set-Cookie` HttpOnly só
// pousa se a
// requisição sair do próprio origin. Em produção o domínio já é same-
// site, então
// não há rewrite. Descomente quando o backend de dev subir em outro
// host.
//
// async rewrites() {
//   const apiBaseUrl = process.env.NEXT_PUBLIC_API_URL;
//   if (process.env.NODE_ENV === "production" || !apiBaseUrl) {
//     return [];
//   }
//   return [{ source: "/api/:path*", destination: `${apiBaseUrl}/api
// :path*` }];
// },
```

CUIDADO | "O login retorna 200 mas continuo deslogado"

É o sintoma clássico de cookie que não pousou. Ordem de checagem: (1) o backend está em outro origin e NEXT_PUBLIC_API_URL é absoluta sem CORS com credentials do lado do backend; (2) o cookie não é SameSite=None; Secure; (3) você está em dev com backend em outro host e o rewrite do [next.config.ts](#) continua comentado. O 200 é real; o Set-Cookie é que foi descartado pelo navegador.

Detalhe que evita uma dor de cabeça no build: getClientApiBaseUrl() devolve "/" quando typeof window === "undefined". O comentário explica: *"No servidor a instância é criada mas nunca dispara requisição, então sem window devolve "/" e não exige env: build e prerender ficam livres de configuração."* Sem isso, um build sem NEXT_PUBLIC_API_URL quebraria por uma env que aquele ambiente nem usa.

7.6 apiClient: o gateway que torna "buscar sem validar" impossível

Todo acesso ao backend passa por uma função só, em `src/lib/api/client.ts`. O JSDoc dela resume a tese do produto inteiro: *"É por causa desta função que 'buscar sem validar' é impossível no repo: o lint proíbe fetch cru pro backend e ky fora de lib/api, então todo dado que entra no app passou por um schema."*

7.6.1 O envelope

O boilerplate assume que o backend responde sempre no mesmo envelope:

`src/lib/api/client.ts (envelope)`

```
type ApiEnvelope = {
  success: boolean;
  message: string;
  data?: Json;
  pagination?: Json;
};
```

`success` e `message` são fixos do contrato, então nenhum schema de chamada precisa redeclará-los. O schema que você passa descreve **só o data** daquele endpoint. E há uma nota de portabilidade que importa se o seu backend responde outro formato: *"este type e o requestEnvelope são o ÚNICO lugar a mudar; o resto do app não conhece o envelope."*

7.6.2 A validação, e por que ela não usa a API do valibot

`src/lib/api/client.ts (núcleo)`

```
const envelope = await requestEnvelope(ky, path, options);

// `success`/`message` são fixos do contrato, então não são
// revalidados aqui
// (sem narrowing defensivo). Só o `data` específico do endpoint
// passa pelo
// schema da chamada; endpoint sem `data` não recebe schema.
if (!dataSchema) {
  return undefined;
}
const result = await dataSchema["~standard"].validate(envelope.data);
if (result.issues) {
  throw new Error(result.issues[0]?.message);
}
return result.value;
```

POR QUE | Por que Standard Schema, e não parse do valibot

O lint confina `import "valibot"` aos arquivos `*.schema.ts`. `src/lib/api/client.ts` não é um deles; logo, não pode importar valibot. A saída foi validar pela interface Standard Schema (a propriedade `"~standard"` do schema), que qualquer validador moderno implementa.

O ADR-022 registra a consequência prática: *"é isso que permite `lib/api` validar sem violar o confinamento de `valibot` aos `*.schema.ts`. Por isso `@standard-schema/spec` é dependência de runtime."* E é mesmo: no `package.json`, `@standard-schema/spec` está em `dependencies`, não em `devDependencies`.

Efeito colateral bem-vindo: trocar valibot por outro validador não toca em `lib/api`.

7.6.3 As duas overloads

`src/lib/api/client.ts` (assinaturas)

```
export async function apiClient<Schema extends StandardSchemaV1>(
  ky: KyInstance,
  path: string,
  dataSchema: Schema,
  options?: Options,
): Promise<StandardSchemaV1.InferOutput<Schema>>;

/**
 * Sobrecarga para endpoints sem `data` no contrato (ex.: login, cuja
 * identidade
 * vem do cookie HttpOnly): dispara a requisição, propaga o `message`
 * do backend
 * em erro e não devolve dado.
 */
export async function apiClient(
  ky: KyInstance,
  path: string,
  dataSchema?: undefined,
  options?: Options,
): Promise<undefined>;
```

É por isso que o `login` e o `PUT /perfil` chamam `apiClient(..., undefined, ...)`: o terceiro argumento é `undefined` explícito, e o tipo de retorno vira `Promise<undefined>`. Não é uma brecha para pular validação; é a codificação, no tipo, de "este endpoint não devolve data".

7.6.4 Erro na borda

O tratamento é em duas faixas. Erro HTTP **com** envelope do backend: propaga a `message` do contrato, que já é texto seguro para a UI. Erro **sem** envelope (rede, timeout, DNS, CORS, ou um proxy devolvendo não-JSON num 5xx): troca por um texto fixo, com o erro original preservado no `cause`.

```
src/lib/api/client.ts (fallback de conexão)
```

```
const CONNECTION_ERROR_MESSAGE =  
  "Não foi possível concluir a solicitação. Tente novamente em  
  instantes.";
```

O comentário justifica: o `error.message` cru nesse caso é o `TypeError: Failed to fetch` do browser; técnico e em inglês. A borda troca por texto de produto; o log mantém o contexto via `cause`.

7.6.5 O que não está lá, de propósito

```
src/lib/api/client.ts (rodapé do arquivo)
```

```
// Sem variante paginada aqui de propósito: o repo proíbe abstração sem  
// caller  
// real (YAGNI, ver CODE-PATTERN §9). Quando o seu primeiro endpoint  
// devolver  
// `pagination` como irmã de `data` no envelope, extraia uma `  
//   apiClientPaginated`  
// reaproveitando o `requestEnvelope` acima — a borda já está pronta  
// pra isso.
```

Vale ler isso como declaração de método, não como omissão. O ADR-022 diz o mesmo com outras palavras: *"Escopo consciente: sem variante paginada e sem lib/auth. Zero caller = YAGNI (CODE-PATTERN §9), e o knip reprovaria."* O knip roda no gate; código morto não passa.

7.7 Zonas de import: a permissão é exceção local a uma proibição global

O mecanismo que sustenta tudo acima é simples e vale entender por inteiro, porque é ele que transforma "convenção" em "não compila".

Camada 1; proibição global. O `no-restricted-imports` built-in, em [.oxlintrc.json](#), bane globalmente: `valibot`, `ky`, `jsonwebtoken`, `jose`, `zod`, `@tanstack/react-query`, `react-query`, `swr`, `axios`, `assert` e `node:assert`. Cada um com mensagem própria, em português:

```
.oxlintrc.json (mensagens, trecho)
```

```
"SWR proibido: leitura é RSC, sem data-fetching no client."
"React Query proibido: leitura é RSC, sem data-fetching/cache no client."
"valibot só pode ser usado em arquivos *.schema.ts (validação confinada às bordas)."
"ky só pode ser usado em src/lib/api/ (gateway único de HTTP)."
```

Camada 2; exceção por zona. Dois overrides devolvem *um* pacote de volta, e só um:

As duas zonas, mutuamente exclusivas

Zona (glob)	Libera	Continua banido
**/*.schema.ts	apenas valibot	ky, jose, jsonwebtoken, zod, React Query, SWR, axios
src/lib/api/**	apenas ky	valibot, jose, jsonwebtoken, zod, React Query, SWR, axios

Camada 3; o gap de subpath. O `no-restricted-imports` built-in casa **specifier exato**, então `swr/mutation` escaparia. A regra `custom` `boilerplate/no-forbidden-imports` fecha isso com `source === pkg || source.startsWith(pkg + "/")`. O cabeçalho da regra dá os exemplos reais: `swr/mutation`, `@tanstack/react-query/devtools`, `zod/v4-mini`, `jose/jwt/verify`. Ela também reimpõe as duas zonas, com mensagens dedicadas (`valibotZone`, `kyZone`).

POR QUE | Por que as zonas são mutuamente exclusivas

Se `lib/api` pudesse importar `valibot`, o `apiClient` teria chamado `parse()` direto e o gateway ficaria acoplado a um validador. A proibição cruzada; `/*.schema.ts` pode `valibot` e não pode `ky`; `lib/api` pode `ky` e não pode `valibot`; é o que *forçou* a solução via `Standard Schema`. A restrição produziu o desenho melhor, não o contrário.

7.8 Os schemas: forma do contrato, não regra de negócio

Existem três `/*.schema.ts` no repositório: `src/features/perfil/perfil.schema.ts`, `src/features/auth/login.schema.ts` e `src/lib/env.schema.ts`.

REGRA | O schema descreve a FORMA do contrato

Do `src/features/perfil/perfil.schema.ts`: *"minLength(1) está aqui porque o backend rejeita nome vazio; não é um limiar inventado no front. Qualquer regra que o backend não imponha (limite de caracteres 'porque a UI fica feia', faixa de valores, elegibilidade) NÃO entra: authz e regra de negócio são do backend."*

src/features/perfil/perfil.schema.ts (trecho)

```
export const perfilSchema = object({
  id: string(),
  nome: pipe(string(), minLength(1, "Informe o nome.")),
  email: pipe(string(), email("E-mail inválido.")),
  /** `nullable` porque o spec marca o campo como opcional no cadastro.
   */
  telefone: nullable(string()),
});

/** Perfil como o backend devolve. Fonte única do tipo – não redeclare
  à mão. */
export type Perfil = InferOutput<typeof perfilSchema>;
```

Note que o tipo sai de InferOutput, nunca de uma type escrita à mão em paralelo. Duas fontes da verdade divergem; é questão de tempo.

O mesmo arquivo hospeda parsePerfilForm, e a justificativa de localização fecha o círculo com as zonas de import: *"Mora aqui, e não na Server Action, porque valibot é confinado aos *.schema.ts: a borda de validação é este arquivo, e quem chama recebe o valor já tipado."*

Detalhe de estilo com dois motivos, de [CODE-PATTERN.md](#):

CODE-PATTERN.md §2

```
// Named imports (NUNCA `import * as v`: import/no-namespace proíbe; e
  named
// import tree-shakeia, que é o ponto do valibot).
```

7.8.1 Env é borda como qualquer outra

src/lib/env.schema.ts é o exemplo que costuma surpreender: variável de ambiente também passa por schema, e o arquivo é um *.schema.ts justamente por causa do confinamento.

src/lib/env.schema.ts (trecho)

```
const serverEnvSchema = object({
  /** Base URL do backend para chamadas server-to-server (RSC e Server
    Actions). */
  INTERNAL_API_URL: pipe(string(), url("INTERNAL_API_URL precisa ser
    uma URL absoluta.")),
});
```

O JSDoc de parseServerEnv declara a política: *"Falha fechado: env ausente ou malformada é erro de configuração, e estourar no boot é melhor do que descobrir com um fetch"*

para *undefined/rota em produção*.” Env `NEXT_PUBLIC_*` não passa por aqui; ela é inlinada pelo Next no bundle do browser, e quem cuida dela é [src/lib/api/base-url.ts](#).

7.9 Endpoints centralizados, e as duas base URLs

Todos os paths REST vivem num arquivo só. O boilerplate inteiro tem exatamente dois:

`src/lib/api/endpoints.ts`

```
export const API_ENDPOINTS = {
  /** Sessão do usuário. Chamado browser-direct: o backend emite o cookie. */
  auth: {
    login: "api/v1/login",
  },

  /** Perfil do usuário autenticado – a feature de exemplo do boilerplate. */
  perfil: {
    me: "api/v1/perfil",
  },
} as const;
```

Regra do arquivo: paths relativos, **sem barra inicial**; a base URL vive nas instâncias `ky`. Endpoint novo entra aqui no domínio correspondente, não como `const` local dentro da feature; centralizar dá um lugar só para revisar os paths contra o spec do backend (skill `api-contract`).

E as duas base URLs, de `.env.example`:

`.env.example (trecho)`

```
# Base URL do backend para o BROWSER (login/logout: fluxos onde o Set-Cookie
# HttpOnly precisa pousar). Duas formas:
# "/" same-origin (dev: ligue o rewrite
# comentado no next.config.ts; prod: domínio same-site).
# https://api.example.com outro origin: exige CORS com credentials
# no backend.
NEXT_PUBLIC_API_URL=https://api.example.com

# Base URL do backend para o SERVIDOR (RSC e Server Actions, via
# kyServer).
INTERNAL_API_URL=https://api.example.com

# Fonte do contrato OpenAPI/Swagger (usada pela skill api-contract)
API_CONTRACT_PATH=/caminho/para/openapi
```

`INTERNAL_API_URL` é sempre absoluta: “no servidor não existe o proxy `same-origin`. Validada por [src/lib/env.schema.ts](#), então ausente ou malformada estoura no boot.”

7.10 O que é proibido, e qual regra pega cada coisa

Nada aqui é "boa prática recomendada". Cada linha abaixo reprova o gate.

Proibições e a regra que as impõe

Proibido	Regra	Escopo
"use cache" (inclusive variantes)	no-use-cache	todo o repo
unstable_cache importado de next/cache	no-use-cache	todo o repo
cache: "force-cache"	no-use-cache	todo o repo
revalidate diferente de 0 no fetch	no-use-cache	fetch inline
dynamic = "force-static"	no-force-static	src/app/**
revalidate falso ou maior que 0	no-force-static	src/app/**
fetch, apiClient ou ky.* em effect	no-fetch-in-effect	todo o repo
watch() do RHF em effect	no-watch-in-effect	todo o repo
swr, React Query, axios, zod ou libs JWT	no-forbidden-imports	todo o repo
os mesmos pacotes, specifier exato	no-restricted-imports	todo o repo
fetch cru apontando para o backend	api-through-lib	fora de lib/ api/
export const fetchCache	no-restricted-exports	todo o repo
await dentro de loop (waterfall)	no-await-in-loop	todo o repo
await dentro de Promise.all(...)	unicorn/no-await-in-promise-methods	todo o repo

As mensagens são em português e explicam a alternativa, não só o veto:

Mensagens reais das regras

```
// boilerplate/no-fetch-in-effect
"Não busque dados em useEffect. Leitura de dados é em Server Component
(RSC); mutação é Server Action."

// boilerplate/no-force-static
"`revalidate` diferente de 0 força cache/staleness (número > 0 = ISR;
false =
never-revalidate/estático permanente). Use 0 (sempre fresco)."
```

```
// boilerplate/no-forbidden-imports
"Import proibido neste repo. Leitura é RSC/Server Actions/apiClient,
validação é
valibot em *.schema.ts, e authz é sempre no backend."
```

7.10.1 Nuances de escopo que evitam falso-positivo

As regras foram escritas para não atrapalhar. Vale conhecer os limites, porque eles também dizem o que *não* é bloqueado:

- `no-force-static` só roda em `src/app/**`. `dynamic` e `revalidate` são route segment config; fora de rota, um `export const revalidate` de domínio seria falso-positivo. O filtro é um regex ancorado em início-de-string ou barra, porque o `oxlint` reporta path relativo em runs normais; um `.includes("/src/app/")` daria falso-negativo (trecho abaixo).
- `revalidate = 0` é **permitido**. `dynamic = "force-dynamic"` também. O que a regra ataca é `staleness`, não `dinamismo`.
- O caso 4 de `no-use-cache` só dispara em `fetch inline`, via `isInlineFetchNextRevalidateProperty`. Um objeto de domínio com campo `revalidate` não é reportado.
- `api-through-lib` isenta qualquer arquivo cujo path contenha `src/lib/api/`; ele é a implementação. E dispara pelo texto-fonte do primeiro argumento de `fetch` (cobrindo string literal, template literal e concatenação) quando esse texto contém `example.com`, `NEXT_PUBLIC_API_URL` ou `INTERNAL_API_URL`. Ou seja: é um gate por domínio e env var, não um "proibido todo fetch".

O filtro de escopo do `no-force-static`, literal:

tooling/oxlint-rules/rules/no-force-static.ts (escopo)

```
if (!/(^|\/)src\/app\/\./.test(getFilename(context).replace(/\\/g,
"/")) {
  return {};
}
```

CUIDADO | Adapte o gate de `api-through-lib` ao seu domínio

A lista de domínios da regra é `["example.com"]`, herdada do `.env.example`. Se o seu backend responde em outro host e você escrever `fetch("https://api.suaempresa.com/...")` numa feature, a regra **não** vai pegar. Edite `BACKEND_DOMAINS` em `tooling/oxlint-rules/rules/api-through-lib.ts` logo depois de configurar seu ambiente. É a diferença entre um lint que impõe e um lint que parece impor.

7.11 O que é permitido (e por quê)

React.cache(). Explicitamente não reportado. O cabeçalho de `tooling/oxlint-rules/rules/no-use-cache.ts` lista as isenções:

`tooling/oxlint-rules/rules/no-use-cache.ts` (cabeçalho)

```
* NOT reported (React request-dedup, which is allowed):
* - `import { cache } from "react"`
* - `cache()` or `React.cache()` call expressions
```

O argumento cabe numa linha, e é assim que o `VERCEL-OXLINT-MAPPING.md` o registra: **dedup não é staleness**. `React.cache()` deduplica chamadas idênticas *dentro da mesma request*; o dado continua sendo buscado fresco a cada request. Há teste travando o comportamento em `tooling/oxlint-rules/rules/__tests__/custom-rules.test.ts`, e ele passa por **duas** regras diferentes (`no-use-cache` e `no-manual-memo`), liberado nas duas.

Promise.all. Permitido e prescrito pelo `CODE-PATTERN.md` §1: *"Fetches paralelos com Promise.all (sem waterfall)"*. O lint não consegue *exigir* paralelismo; o próprio `.oxlintrc.json` admite no cabeçalho que tudo que é arquitetural (`Promise.all`, `Suspense streaming`, `next/dynamic`, `React.cache`, `LRU`, serialização de props RSC) *"NÃO tem equivalente de linter e fica documentado em VERCEL-OXLINT-MAPPING.md"*. O que o lint faz é atacar o oposto: `no-await-in-loop` e `unicorn/no-await-in-promise-methods`, com o comentário inline *"await dentro de Promise.all() = waterfall"*.

CUIDADO | Permissões, não exemplos

Não existe uso de `React.cache()` nem de `Promise.all` em `src/`. Eles aparecem só nos testes das regras e nos docs. São permissões que você vai exercer quando tiver duas leituras independentes na mesma página; não código de referência para copiar.

7.12 O que NÃO vem no pacote

Esta lista existe para você não perder tempo procurando. Todos os itens abaixo são ausências verificadas, e a maioria é deliberada.

Ausências verificadas

Item	Situação
logout	Não existe. Os JSDoc de <code>kyClient</code> , <code>kyServer</code> e <code>src/lib/api/base-url.ts</code> mencionam "login, logout e refresh", mas <code>API_ENDPOINTS</code> só tem <code>auth.login</code> .
Refresh de token	Não implementado. <code>src/proxy.ts</code> só checka presença do cookie e redireciona; o JSDoc diz que é ali que o refresh proativo entra, relayando o <code>Set-Cookie</code> . Lugar reservado, não código pronto.
<code>error.tsx</code> , <code>loading.tsx</code> , <code>not-found.tsx</code>	Não existem. <code>src/app/</code> tem apenas <code>favicon.ico</code> , <code>globals.css</code> , <code>layout.tsx</code> , <code>page.tsx</code> , <code>login/page.tsx</code> e <code>perfil/page.tsx</code> . O <code>ARCHITECTURE.md</code> lista <code>error.tsx</code> , <code>global-error.tsx</code> e <code>not-found.tsx</code> em "Fora de escopo (fases futuras)"; <code>loading.tsx</code> aparece só como parte do padrão de leitura prescrito, sem arquivo correspondente.
Feature clientes	Não existe. É ilustração do <code>CODE-PATTERN.md</code> .
<code>apiClientPaginated</code>	Não existe, por decisão (YAGNI + knip reprovaria).
<code>src/lib/auth/</code>	Não existe, por decisão: "Um módulo de <code>auth</code> no <i>front</i> seria justamente a camada que o ADR-007 proíbe".
<code>middleware.ts</code>	Não existe. É <code>src/proxy.ts</code> , na raiz de <code>src</code> , irmão de <code>app/</code> (ADR-010).

CUIDADO | O comentário do `perfil/page.tsx` promete um arquivo que não está lá

O JSDoc de `PerfilCarregado` diz "o erro sobe pro `error.tsx` mais próximo". Hoje não há `error.tsx` em lugar nenhum: o erro sobe até o boundary padrão do Next. Criar um `error.tsx` é provavelmente a primeira coisa que você vai adicionar depois de plugar seu backend.

7.13 Como verificar que você não quebrou a doutrina

O gate canônico é um só, e roda tudo:

`package.json`

```
bun run verify
# = lint && fmt:check && typecheck && typecheck:tooling && knip && test
#   && bun tooling/no-lint-suppression.ts && bun tooling/no-heavy-
#     public-assets.ts

bun run lint    # oxlint --disable-nested-config --type-aware
```

Repare no `--type-aware`: o lint tem acesso a tipos, não só à AST. E note o `tooling/no-lint-suppression.ts` no fim da cadeia; suprimir uma regra com comentário não é uma saída, é outra reprovação. As regras desta seção não são negociáveis por `oxlint-disable`.

REGRA | Fluxograma de decisão, para você e para o agente

Preciso **ler** dado? → Server Component `async` sob `<Suspense>`, chamando `apiClient(kyServer, ...)` com `schema`.

Preciso **escrever** dado? → Server Action com `"use server"`, revalidando o payload via `*.schema.ts`, chamando `apiClient(kyServer, ...)`, terminando em `revalidatePath`.

Preciso que o backend **emita um cookie**? → `apiClient(kyClient, ...)` a partir do browser, terminando em `router.refresh()`.

Nenhum dos três? → então provavelmente não é acesso a dado. Se você está pensando em `useEffect`, pare.

PARTE 8

A camada de referência que já vem escrita

O QUE VOCÊ LEVA DESTE CAPÍTULO

Você vai percorrer um fluxo real, do login à rota protegida, e aprender qual padrão copiar na sua primeira feature. O foco é reconhecer as bordas, não decorar cada linha.

Antes de abrir qualquer arquivo, vale entender o tamanho da amostra: o `src/` inteiro tem **19 arquivos** de aplicação (`.ts/.tsx`), mais 3 de teste e 1 de mock. É uma linha vertical completa: login, rota protegida, leitura em Server Component e mutação em Server Action. Não há framework interno nem camadas inventadas só para parecerem completas. O que existe foi escrito para servir de molde à sua primeira feature.

Árvore real de `src/` e `test/` (comando: `find src test -type f`)

```
src/app/{layout.tsx,page.tsx,globals.css,favicon.ico}
src/app/login/page.tsx
src/app/perfil/page.tsx
src/features/auth/{login.schema.ts,use-login-form.ts,login-form.tsx}
src/features/perfil/{perfil.schema.ts,perfil.actions.ts,use-perfil-form.
  ts,perfil-form.tsx}
src/lib/api/{client.ts,ky.server.ts,ky.client.ts,base-url.ts,endpoints.
  ts}
src/lib/api/__tests__/{client.test.ts,base-url.test.ts}
src/lib/{env.ts,env.schema.ts}
src/lib/__tests__/env.schema.test.ts
src/proxy.ts
test/fetch-mock.ts
```

POR QUE | Por que tão pouco código

Este boilerplate parte do princípio de que a maior parte do código vai ser escrita por agentes de IA. Um agente não precisa de scaffolding: precisa de *um exemplo correto por padrão* e de portões que reprovem o desvio. Cada arquivo acima existe porque estabelece um padrão que não dá para inferir do lint sozinho. Nada foi escrito “para depois”. A regra que sustenta isso é o CODE-PATTERN §9: um caller → inline; segundo caller real → extração local; abstração genérica → três callers concretos.

O que cada arquivo da camada de referência estabelece

Arquivo	Padrão que ele fixa
src/lib/api/client.ts	O gateway único: envelope, tratamento de erro na borda, validação por Standard Schema
src/lib/api/ky.server.ts	Leitura server-side com encaminhamento de cookie
src/lib/api/ky.client.ts	A única saída browser-direct (login/logout/refresh)
src/lib/api/base-url.ts	Resolução de base URL do browser e a decisão same-origin vs. CORS
src/lib/api/endpoints.ts	Catálogo central de paths, agrupado por domínio
src/lib/env.schema.ts	Env validada com valibot, falhando fechado no boot
src/lib/env.ts	server-only como barreira de bundle
src/proxy.ts	Gate de roteamento das áreas autenticadas (Next 16)
src/features/auth/*	Formulário + hook + schema; a exceção do cookie
src/features/perfil/*	Leitura RSC + mutação por Server Action + revalidatePath
src/app/perfil/page.tsx	Subárvore dinâmica sob <Suspense> com cacheComponents
test/fetch-mock.ts	Onde o mock de IO entra: na borda, não em cada teste

8.1 O gateway: [src/lib/api/client.ts](#)

São 134 linhas e é a peça mais importante do repo. Todo dado que entra no aplicativo passa por aqui.

8.1.1 O envelope que ele assume

O boilerplate assume que o backend responde sempre no mesmo envelope, em sucesso e em erro:

src/lib/api/client.ts: o contrato assumido

```
/** Valor JSON arbitrário: o `data` do envelope vem de `JSON.parse`,  
    então é JSON, não `unknown`. */  
type Json = string | number | boolean | null | { [key: string]: Json }  
  | Json[];  
  
type ApiEnvelope = {  
  success: boolean;  
  message: string;  
  data?: Json;  
  pagination?: Json;  
};
```

O JSDoc desse type já responde a primeira pergunta que um dev sênior faz ao abrir o arquivo; “e se meu backend responde diferente?”:

“success e message são fixos do contrato: sempre presentes e iguais em todo endpoint. ... Se o seu backend responde outro formato, este type e o requestEnvelope são o ÚNICO lugar a mudar; o resto do app não conhece o envelope.”

Consequência prática: nenhum *.schema.ts de feature redeclara success/message. O schema de cada chamada descreve **só o data**. Isso está explícito no código:

src/lib/api/client.ts, linhas 118-123

```
// `success`/`message` são fixos do contrato, então não são  
revalidados aqui  
// (sem narrowing defensivo). So o `data` específico do endpoint  
passa pelo  
// schema da chamada; endpoint sem `data` não recebe schema.  
if (!dataSchema) {  
  return undefined;  
}
```

8.1.2 Duas classes de falha, dois textos

O tratamento de erro fica isolado em requestEnvelope, que roda antes de qualquer validação:

src/lib/api/client.ts: requestEnvelope

```

async function requestEnvelope(ky, path, options): Promise<ApiEnvelope>
{
  try {
    return await ky(path, options).json<ApiEnvelope>();
  } catch (error: unknown) {
    if (error instanceof HTTPError) {
      const failure: unknown = error.data;
      if (
        failure !== null &&
        typeof failure === "object" &&
        "message" in failure &&
        typeof failure.message === "string"
      ) {
        throw new Error(failure.message, { cause: error });
      }
    }
    throw new Error(CONNECTION_ERROR_MESSAGE, { cause: error });
  }
}

```

As duas classes de falha e o que a UI recebe

Situação	O que o apiClient lança
Erro HTTP com envelope do backend (401, 422, 409...)	A message do próprio backend, texto já pensado para humano
Transporte (rede, DNS, timeout, CORS) ou infra fora do backend (proxy devolvendo HTML num 502)	CONNECTION_ERROR_MESSAGE, com o erro original preservado em { cause: error }
data presente mas fora do schema	A primeira mensagem de issue do schema

O texto de conexão é literal, em português, e mora numa const:

src/lib/api/client.ts, linha 33

```

const CONNECTION_ERROR_MESSAGE =
  "Nao foi possivel concluir a solicitacao. Tente novamente em instantes.";

```

POR QUE | Por que trocar a mensagem em vez de repassar o erro cru

O JSDoc explica: nesses casos “não existe a message do contrato, e o `error.message` cru (o `TypeError: Failed to fetch` do browser) é técnico e em inglês”. O erro original não é descartado; vai no `cause`, “pra manter contexto no log sem vaziar pra tela”. Você ganha diagnóstico completo e uma UI que nunca mostra jargão de transporte.

CUIDADO | `error.response.json()` não funciona aqui: e isso é do ky 2

O código lê `error.data`, não `error.response.json()`. O comentário no fonte é explícito: “ky já consome o corpo do response e entrega o envelope parseado em `error.data`”. A tipagem do ky (^{2.0.2}, em [node_modules/ky/distribution/errors/HTTPError.d.ts](#)) confirma: `data: T | string | undefined`, populado antes dos hooks `beforeError`, e “the response body is automatically consumed when populating `error.data`”. Se você copiar um trecho de tratamento de erro de um projeto com ky 1.x, ele vai silenciosamente quebrar.

Vale notar o que o narrowing de failure *não* é: não é desconfiança do backend. O comentário deixa claro que a defesa é “contra resposta de infra fora do backend (um proxy/gateway devolvendo não-JSON num 5xx)”. Existe um teste dedicado exatamente a esse cenário.

8.1.3 Standard Schema, não valibot: e por que isso não é preciosismo

A assinatura tem duas sobrecargas, e a validação não chama nenhuma API do valibot:

src/lib/api/client.ts – sobrecargas e validação

```
export async function apiClient<Schema extends StandardSchemaV1>(
  ky: KyInstance,
  path: string,
  dataSchema: Schema,
  options?: Options,
): Promise<StandardSchemaV1.InferOutput<Schema>>;
export async function apiClient(
  ky: KyInstance,
  path: string,
  dataSchema?: undefined,
  options?: Options,
): Promise<undefined>;

// ...
const result = await dataSchema["~standard"].validate(envelope.data);
if (result.issues) {
  throw new Error(result.issues[0]?.message);
}
return result.value;
```

O JSDoc traz o argumento inteiro em duas frases:

“É por causa desta função que ‘buscar sem validar’ é impossível no repo: o lint proíbe fetch cru pro backend e ky fora de `lib/api`, então todo dado que entra no app passou por um schema.”

“A validação roda pela interface Standard Schema (`schema["~standard"]`), não pela API do valibot: o lint confina `import "valibot"` aos `*.schema.ts`, então `lib/api` valida sem acoplar a um validador específico.”

Isso não é uma preferência de estilo: é uma **consequência mecânica** do lint. A regra `custom boilerplate/no-forbidden-imports` implementa a zona por caminho de arquivo, literalmente:

`tooling/oxlint-rules/rules/no-forbidden-imports.ts`

```
if (matchesPackage(source, "valibot") && !filename.endsWith(".schema.ts")) {
  context.report({ node, messageId: "valibotZone" });
  return;
}
if (matchesPackage(source, "ky") && !filename.includes("src/lib/api/")) {
  context.report({ node, messageId: "kyZone" });
}
```

REGRA | Se o `apiClient` usasse `safeParse`, ele não passaria no lint

`src/lib/api/client.ts` não termina em `.schema.ts`. Um `import { safeParse } from "valibot"` ali dispara `valibotZone` e reprova `bun run lint`, que é o primeiro passo do `bun run verify`. Por isso `@standard-schema/spec` está em `dependencies (runtime)`, não em `devDependencies`: `"@standard-schema/spec": "^1.1.0"`.

O `.oxlintrc.json` fecha o cerco pelo outro lado: `no-restricted-imports` global bane `valibot` com a mensagem “`valibot` só pode ser usado em arquivos `*.schema.ts` (validação confinada às bordas)”. O override para `"files": ["src/lib/api/**"]` libera `ky` e **mantém `valibot` banido**; o comentário no próprio arquivo diz: “Camada de API interna libera APENAS `ky`; `valibot/jose/jsonwebtoken` seguem banidos”.

POR QUE | Por que `@hookform/resolvers/valibot` não cai na regra

`matchesPackage` compara `source === pkg || source.startsWith(pkg + "/")`. `@hookform/resolvers/valibot` não começa com `valibot`, então passa. É por isso que `use-login-form.ts` e `use-perfil-form.ts` podem usar `valibotResolver` fora de um `.schema.ts` e o lint continua verde. A regra bloqueia o acesso à API do validador, não o uso do resolver de formulário.

A outra metade do cerco é `boilerplate/api-through-lib`: dispara em `fetch(...)` cujo **texto-fonte do primeiro argumento** contenha `example.com`, `NEXT_PUBLIC_API_URL` ou `INTERNAL_API_URL`, exceto em arquivos sob `src/lib/api/`. A mensagem é “`nao chame o backend com fetch cru; use apiClient/ky de lib/api`”.

CUIDADO | `api-through-lib` é heurística de texto, não análise de tipo

A regra roda `context.getSourceCode().getText(args[0])` e procura substrings numa lista fixa (`BACKEND_DOMAINS = ["example.com"]`). Se o seu backend é `api.suaempresa.com.br` e alguém escreve `fetch("https://api.suaempresa.com.br/v1/x")` dentro de um componente, **a regra não dispara**. Ao trocar o backend, edite `BACKEND_DOMAINS` em `tooling/oxlint-rules/rules/`

api-through-lib.ts. Esse é um dos poucos pontos do repo onde o enforcement depende de você configurar.

8.1.4 O rodapé do arquivo: o que não está lá

src/lib/api/client.ts, linhas 131-134

```
// Sem variante paginada aqui de proposito: o repo proibe abstracao sem caller
// real (YAGNI, ver CODE-PATTERN §9). Quando o seu primeiro endpoint devolver
// `pagination` como irma de `data` no envelope, extraia uma `apiClientPaginated`
// reaproveitando o `requestEnvelope` acima - a borda ja esta pronta pra isso.
```

Repare no desenho: pagination?: Json já está no type do envelope e requestEnvelope já é uma função separada. O ponto de extração existe; a abstração, não. Isso é o CODE-PATTERN §9 aplicado ao próprio boilerplate.

8.2 As duas instâncias ky

kyServer vs. kyClient

	ky.server.ts	ky.client.ts
Onde roda	RSC e Server Actions	Browser
Base URL	serverEnv.INTERNAL_API_URL	getClientApiBaseUrl()
Autenticação	hook beforeRequest copia o cookie da request	credentials: "include"
Para que serve	toda leitura de dado e as mutações	só login/logout/refresh

`src/lib/api/ky.server.ts`

```
export const kyServer = ky.create({
  baseUrl: serverEnv.INTERNAL_API_URL,
  throwHttpErrors: true,
  headers: { Accept: "application/json" },
  hooks: {
    beforeRequest: [
      async ({ request }) => {
        const cookieHeader = (await cookies()).toString();
        if (cookieHeader) {
          request.headers.set("cookie", cookieHeader);
        }
      },
    ],
  },
});
```

Três efeitos que o JSDoc nomeia e que valem entender antes de mexer:

1. A instância é *module-level* (única por processo), mas o hook roda *a cada chamada*. Logo o cookie é sempre o da requisição corrente. Não há vazamento de sessão entre usuários.
2. Ler `cookies()` marca a rota como dinâmica. Isso não é efeito colateral indesejado: é o mecanismo que garante dado sempre fresco, coerente com a doutrina sem cache (ADR-003).
3. `next/headers` só existe no servidor, o que “impede este client de vazar pro bundle do browser”. A barreira é do bundler, não da disciplina do time.

E a fronteira explícita, no próprio JSDoc: “Fluxos que precisam do Set-Cookie da resposta pousando no browser (login, logout, refresh) NÃO passam por aqui: usam o `kyClient browser-direct`, porque cookie re-emitado numa chamada server-side se perde.”

CUIDADO | É ky 2: a opção é `baseUrl`, não `prefixUrl`

O ky v1 usava `prefixUrl`. Aqui é ^2.0.2, e a opção correta é `baseUrl` (confirmado em node_modules/ky/distribution/types/options.d.ts). Snippets de ky que você achar por aí provavelmente estão na versão antiga.

`src/lib/api/ky.client.ts`

```
export const kyClient = ky.create({
  baseUrl: getClientApiBaseUrl(),
  credentials: "include",
  throwHttpErrors: true,
  headers: { Accept: "application/json" },
});
```

CUIDADO | O JSDoc cita três fluxos; o repo entrega um

`ky.client.ts` diz existir “para login, logout e refresh”. Na prática, `API_ENDPOINTS.auth` tem apenas login. Logout e refresh **não estão implementados**; os comentários marcam onde eles entram, não o que já existe. Mesma coisa no `proxy.ts`, que diz “quando você precisar de refresh proativo, é aqui que ele entra, relayando o Set-Cookie da resposta”. É indicação de lugar, não de código pronto.

8.3 `src/lib/api/base-url.ts`: onde o cookie pausa`src/lib/api/base-url.ts`

```
export function getClientApiBaseUrl(): string {
  if (typeof window === "undefined") {
    return "/";
  }

  const baseUrl = process.env.NEXT_PUBLIC_API_URL;
  if (!baseUrl) {
    throw new Error(
      "NEXT_PUBLIC_API_URL nao configurada. Copie o .env.example para .env.local e preencha."
    );
  }
  return baseUrl;
}
```

O curto-circuito do `typeof window === "undefined"` tem motivo declarado, não é paranoia: “No servidor a instância é criada mas nunca dispara requisição, então sem `window` devolve `"/` e não exige `env`: build e prerender ficam livres de configuração.”

O JSDoc documenta os dois formatos aceitos de `NEXT_PUBLIC_API_URL`, e enquadra a escolha como uma decisão sobre cookie, não sobre conveniência:

Os dois formatos de `NEXT_PUBLIC_API_URL`

Valor	O que ele implica
<code>"/</code> (same-origin)	Em dev vem do <code>rewrite /api/*</code> do <code>next.config.ts</code> (comentado por padrão); em prod, de domínio same-site. “É o único jeito de um cookie <code>HttpOnly</code> de login pousar sem depender de <code>SameSite=None</code> .”
URL absoluta	Backend em outro origin. “Exige CORS com <code>Access-Control-Allow-Credentials</code> no backend e cookie <code>SameSite=None; Secure</code> .”

O `rewrite` existe, comentado, exatamente para o caso de dev com backend em outro host:

next.config.ts, linhas 17-23 (comentadas no repo)

```
// async rewrites() {
//   const apiBaseUrl = process.env.NEXT_PUBLIC_API_URL;
//   if (process.env.NODE_ENV === "production" || !apiBaseUrl) {
//     return [];
//   }
//   return [{ source: "/api/:path*", destination: `${apiBaseUrl}/api/:path*` }];
// },
```

8.4 src/lib/api/endpoints.ts: 23 linhas, o arquivo inteiro

src/lib/api/endpoints.ts

```
export const API_ENDPOINTS = {
  /** Sessão do usuário. Chamado browser-direct: o backend emite o cookie. */
  auth: {
    login: "api/v1/login",
  },

  /** Perfil do usuário autenticado - a feature de exemplo do boilerplate. */
  perfil: {
    me: "api/v1/perfil",
  },
} as const;
```

Quatro fatos que valem para todo endpoint que você adicionar:

- Paths são **relativos e sem barra inicial**. A base URL vive nas instâncias ky, nunca aqui.
- O `as const` é a única exceção permitida ao banimento de type assertion no repo (CODE-PATTERN §5).
- Regra escrita no JSDoc: "Cada novo endpoint entra aqui no domínio correspondente, não como const local."
- O mesmo path (`perfil.me`) serve GET e PUT. O verbo vai nas options do `apiClient`, não no catálogo.

O que muda no seu projeto: *todos* os paths. `api/v1/...` é convenção do backend de exemplo.

8.5 Env que falha fechado

src/lib/env.schema.ts

```
const serverEnvSchema = object({
  INTERNAL_API_URL: pipe(string(), url("INTERNAL_API_URL precisa ser
    uma URL absoluta.")),
});

export type ServerEnv = InferOutput<typeof serverEnvSchema>;

export function parseServerEnv(source: Record<string, string |
  undefined>): ServerEnv {
  const result = safeParse(serverEnvSchema, source);

  if (!result.success) {
    const problems = result.issues
      .map((issue) => `${getDotPath(issue)}: ${issue.message}`)
      .join("\n");
    throw new Error(`Configuracao de ambiente invalida:\n${problems}`);
  }

  return result.output;
}
```

POR QUE | Falhar no boot, não no primeiro request

O JSDoc: “Falha fechado: env ausente ou malformada é erro de configuração, e estourar no boot é melhor do que descobrir com um fetch para undefined/rota em produção.” O `getDotPath` existe para que a mensagem *nomeie a variável*; há um teste dedicado só a isso, cuja justificativa é: “sem o nome, o erro manda o dev caçar qual das envs quebrou”.

O consumo é um arquivo de 11 linhas:

src/lib/env.ts (arquivo completo)

```
import "server-only";

import { parseServerEnv, type ServerEnv } from "@lib/env.schema";

export const serverEnv: ServerEnv = parseServerEnv(process.env);
```

`server-only` “garante que importar isto de um Client Component quebra o build, não o runtime”.

REGRA | A assimetria server/client é intencional

Só a env do servidor passa por schema. O JSDoc explica: “Env `NEXT_PUBLIC_*` é inlinada pelo Next no bundle do browser e não passa por este schema; ver `lib/api/base-url`”.

Ou seja: `env.schema.ts` cuida do servidor, `base-url.ts` cuida do cliente, cada um falhando fechado no seu lado. Não é esquecimento.

O `.env.example` documenta três variáveis: `NEXT_PUBLIC_API_URL`, `INTERNAL_API_URL` e `API_CONTRACT_PATH` (esta última consumida pela skill `api-contract`, não pelo código). Adicionar uma variável nova é uma linha no `serverEnvSchema` e uma no `.env.example`.

8.6 `src/proxy.ts`: o gate de roteamento

`src/proxy.ts` (parte relevante)

```
const SESSION_COOKIE = "session_token";

export function proxy(request: NextRequest): NextResponse {
  if (request.cookies.has(SESSION_COOKIE)) {
    return NextResponse.next();
  }
  return NextResponse.redirect(new URL("/login", request.url));
}

export const config = {
  matcher: ["/perfil/:path*"],
};
```

REGRA | No Next 16 é `proxy.ts`, não `middleware.ts`: e mora em `src/`, não em `src/app/`

O JSDoc: “É o `proxy.ts` do Next 16, que substitui o `middleware.ts`, e mora na raiz de `src`, irmão de `app/`, não dentro dele.” (ADR-010). Se você (ou um agente com dados de treino antigos) criar um `middleware.ts`, ele simplesmente não roda.

Dois pontos de doutrina explícitos no arquivo:

- **É gate de roteamento, não autorização.** “A autorização real é sempre do backend, que rejeita as chamadas de dado independentemente do que o cookie diga; aqui o objetivo é não largar o usuário numa tela que responderia 403.”
- **Matcher por prefixo, nunca rota a rota.** “Listar rota a rota deixa toda rota nova sem gate por padrão, que é o erro caro.”

O front nunca decodifica o JWT (ADR-007), e isso é imposto: `jose` e `jsonwebtoken` estão banidos tanto no `no-restricted-imports` do `.oxlintrc.json` quanto na lista `ALWAYS_FORBIDDEN` do `no-forbidden-imports.ts`; esta última cobrindo subpaths, então nem `jose/jwt/verify` escapa. Há zero linhas de código de JWT no repo.

O único ajuste esperado no seu projeto é o nome do cookie: “Trocar o nome aqui é o único ajuste se o seu backend usar outro.”

8.7 A feature auth: a única saída browser-direct

8.7.1 login.schema.ts: 17 linhas que ensinam a regra da borda

src/features/auth/login.schema.ts

```
export const loginSchema = object({
  email: pipe(string(), email("E-mail inválido.")),
  senha: pipe(string(), minLength(1, "Informe a senha.")),
});

export type LoginInput = InferOutput<typeof loginSchema>;
```

O JSDoc desse arquivo ensina o critério melhor do que qualquer parágrafo de guia:

“Note o que NÃO está aqui: regra de força de senha, tamanho mínimo ‘de segurança’, bloqueio por tentativas. Isso é política do backend; o schema do front descreve só a forma do payload. `minLength(1)` existe porque enviar campo vazio é erro de formulário, não regra de negócio.”

REGRA | O schema descreve a FORMA do contrato, não a regra de negócio

O mesmo critério reaparece em `perfil.schema.ts`: “`minLength(1)` está aqui porque o backend rejeita nome vazio; não é um limiar inventado no front. Qualquer regra que o backend não imponha (limite de caracteres ‘porque a UI fica feia’, faixa de valores, elegibilidade) NÃO entra: authz e regra de negócio são do backend.”

8.7.2 use-login-form.ts: a exceção do cookie

src/features/auth/use-login-form.ts: o submit

```
const entrar = form.handleSubmit(async (credenciais) => {
  setErroServidor(null);
  try {
    // Sem schema: a identidade vem do cookie, o contrato não devolve `data`.
    await apiClient(kyClient, API_ENDPOINTS.auth.login, undefined, {
      method: "post",
      json: credenciais,
    });
  } catch (error: unknown) {
    setErroServidor(error instanceof Error ? error.message :
      ERRO_INESPERADO);
    return;
  }
  router.replace(destino);
  router.refresh();
});
```

Quatro decisões embutidas em dez linhas:

- É a sobrecarga sem schema (`dataSchema: undefined → Promise<undefined>`). O contrato de login não devolve data: a identidade vem no cookie.
- `kyClient`, não `kyServer`. O JSDoc: “Esta é a ÚNICA família de chamada que sai do browser direto pro backend, e o motivo é o cookie: o `Set-Cookie HttpOnly` da resposta só pousa se a requisição partir do browser. Login feito por Server Action perderia o cookie no caminho.”
- `router.replace(destino)` seguido de `router.refresh()`: “repuxa a árvore de servidor já com o cookie: é o que faz o `proxy.ts` parar de redirecionar pro login”.
- `destino: Route`; tipo do próprio Next, com `typedRoutes: true` no `next.config.ts`. Rota inexistente reprova no build, não em produção.

O fallback `ERRO_INESPERADO` existe só para o caso de a falha não ser um `Error`: “Texto de último recurso: falha sem `Error` não tem mensagem própria pra mostrar”. Não é o caminho normal; o caminho normal é a mensagem que o `apiClient` já preparou.

8.7.3 login-form.tsx: UI fina

O componente só compõe: `register/formState` do React Hook Form, `noValidate` no `<form>`, `aria-invalid` nos inputs, erros em `<p role="alert">`, botão com `disabled={formState.isSubmitting}` alternando “Entrando...”/“Entrar”. Zero estado local, zero fetch.

8.8 A feature perfil: leitura RSC + mutação por Server Action

8.8.1 perfil.schema.ts: três coisas num arquivo

`src/features/perfil/perfil.schema.ts`

```
export const perfilSchema = object({
  id: string(),
  nome: pipe(string(), minLength(1, "Informe o nome.")),
  email: pipe(string(), email("E-mail invalido.")),
  /** `nullable` porque o spec marca o campo como opcional no cadastro.
   */
  telefone: nullable(string()),
});

export type Perfil = InferOutput<typeof perfilSchema>;

export const perfilFormSchema = object({
  nome: pipe(string(), minLength(1, "Informe o nome.")),
  email: pipe(string(), email("E-mail invalido.")),
});

export type PerfilFormInput = InferOutput<typeof perfilFormSchema>;

export function parsePerfilForm(entrada: unknown): PerfilFormInput {
  const resultado = safeParse(perfilFormSchema, entrada);
  if (!resultado.success) {
```

src/features/perfil/perfil.schema.ts (continuação)

```
// `issues` do valibot e tupla não-vazia: o primeiro item sempre
// existe.
throw new Error(resultado.issues[0].message);
}
return resultado.output;
}
```

O detalhe arquitetural mais fácil de errar aqui: `parsePerfilForm` mora no `*.schema.ts`, **não** na Server Action. O JSDoc dá o motivo: “Mora aqui, e não na Server Action, porque `valibot` é confinado aos `*.schema.ts`: a borda de validação é este arquivo, e quem chama recebe o valor já tipado.”

POR QUE | O mesmo constraint, resolvido pelos dois lados

O confinamento do `valibot` empurrou o `apiClient` para o Standard Schema (validador agnóstico dentro de `lib/api`) e empurrou a revalidação da action para dentro do schema (a função de `parse` vai até o `valibot`, em vez do `valibot` vir até a action). Duas soluções diferentes para a mesma regra de lint, e ambas resultam em código mais desacoplado do que se a regra não existisse.

Repare também em `resultado.issues[0].message`, sem optional chaining: o `valibot` tipa `issues` como tupla não-vazia, então o primeiro item sempre existe, e o lint `type-aware` (`typescript/no-unnecessary-condition`) acusaria um `?.` morto ali. Contraste com `client.ts`, que usa `result.issues[0]?.message` porque o Standard Schema tipa como array comum. A diferença vem do contrato das duas APIs, não de uma flag global de acesso indexado.

8.8.2 `perfil.actions.ts`: a mutação

src/features/perfil/perfil.actions.ts

```
"use server";

export type SalvarPerfilResult = { ok: true } | { ok: false; erro:
  string };

export async function salvarPerfil(entrada: PerfilFormInput): Promise<
  SalvarPerfilResult> {
  try {
    const validado = parsePerfilForm(entrada);
    // Sem schema: o contrato deste endpoint não devolve `data`, só o
    // envelope.
    await apiClient(kyServer, API_ENDPOINTS.perfil.me, undefined, {
      method: "put",
      json: validado,
    });
  } catch (error: unknown) {
    return { ok: false, erro: error instanceof Error ? error.message :
      ERRO_INESPERADO };
  }
}
```

```
src/features/perfil/perfil.actions.ts (continuação)
```

```

    }

    revalidatePath("/perfil");
    return { ok: true };
  }

```

Dois racionais citáveis do JSDoc:

“Revalida no servidor mesmo o formulário já tendo validado no client: validação de client é UX, não fronteira de segurança. Depois do sucesso, revalidatePath repuxa o RSC; é assim que o dado volta fresco, já que o repo não tem cache.”

“O apiClient já converteu a falha na mensagem segura do backend (ou no texto de conexão). Repassar aqui é entregar feedback, não engolir erro.”

O retorno é uma union discriminada, não um objeto com campos opcionais. Copie esse formato para toda action nova.

8.8.3 use-perfil-form.ts: RHF, não useActionState

O hook usa useForm com valibotResolver(perfilFormSchema), recebe defaultValues do perfil que veio do RSC, chama a Server Action direto no handleSubmit e traduz SalvarPerfilResult em erroServidor/salvo com useState. O rationale de não usar useActionState está em ADR-015 e CODE-PATTERN §8. O JSDoc resume o desenho: “A validação de client usa o MESMO schema que a Server Action revalida no servidor; um contrato, dois pontos de checagem.”

8.9 As páginas: por que <Suspense> não é opcional

```
src/app/perfil/page.tsx
```

```

async function PerfilCarregado(): Promise<React.ReactNode> {
  const perfil = await apiClient(kyServer, API_ENDPOINTS.perfil.me,
    perfilSchema);
  return <PerfilForm perfil={perfil} />;
}

export default function PerfilPage(): React.ReactNode {
  return (
    <main className="mx-auto flex w-full max-w-3xl flex-col gap-8 px-6
py-16">
      {/* header ... */}
      <Suspense fallback={<p className="text-sm text-foreground/60">
Carregando perfil...</p>}>
        <PerfilCarregado />
      </Suspense>
    </main>
  )
}

```

```
src/app/perfil/page.tsx (continuação)
```

```
    );  
  }
```

O JSDoc de `PerfilCarregado` explica a separação:

“Fica separado da página porque ler o cookie (dentro do `kyServer`) torna esta subárvore dinâmica. Com `cacheComponents` ligado, o Next exige que a parte dinâmica esteja sob um `<Suspense>`: o shell estático vai no primeiro byte e o dado chega em streaming. Isso não é cache; cada request busca fresco.”

CUIDADO | Sem `<Suspense>` o build reprova

`next.config.ts` tem `cacheComponents: true`. Uma leitura dinâmica fora de um boundary de `Suspense` faz o build falhar com `blocking-prerender-dynamic` (ADR-025, CODE-PATTERN §1). Não é um aviso que dá para ignorar: é build vermelho. O padrão a copiar é sempre este; subcomponente `async` isolado, chamada dentro dele, `<Suspense>` na página.

O comentário sobre erro na mesma função diz: “Se a chamada falhar, o erro sobe pro `error.tsx` mais próximo: nada de catch que fabrica um perfil vazio.” A doutrina está certa; só saiba que `error.tsx` **não existe no repo** (nem `not-found.tsx`, nem `loading.tsx`). Hoje o erro sobe até o error boundary padrão do Next. O `ARCHITECTURE.md` lista esses arquivos como “fora de escopo (fases futuras)”.

`src/app/login/page.tsx` é a contraparte: Server Component fino que só compõe, com a rota de destino tipada.

```
src/app/login/page.tsx
```

```
const DESTINO_POS_LOGIN: Route = "/perfil";
```

8.10 Os testes: só a borda, e de propósito

Três arquivos, 13 testes, todos em `src/lib`. Confirmado por execução:

```
bun test src/lib
```

```
13 pass  
0 fail  
13 expect() calls  
Ran 13 tests across 3 files. [28.00ms]
```

Cobertura entregue

Arquivo	O que exercita
<code>src/lib/api/____tests____/client.test.ts</code>	6 testes: data validado; sobrecarga sem data; message do backend em 422; texto de conexão em falha de transporte; texto de conexão em 502 não-JSON; rejeição por schema
<code>src/lib/api/____tests____/base-url.test.ts</code>	3 testes: servidor devolve "/" sem env; browser devolve a env; browser falha fechado sem env
<code>src/lib/____tests____/env.schema.test.ts</code>	4 testes: env válida; ausente; não-absoluta ("/api"); mensagem nomeando a variável

O mock de IO fica num arquivo só, com o motivo escrito:

```
test/fetch-mock.ts
```

```
export function fetchReturning(body: unknown, status: number): typeof
  fetch {
    return () =>
      Promise.resolve(
        new Response(JSON.stringify(body), {
          status,
          headers: { "content-type": "application/json" },
        }),
      );
  }

export function fetchFailing(reason: string): typeof fetch {
  return () => Promise.reject(new TypeError(reason));
}
```

“Ficam aqui, e não em cada teste, porque a borda de IO é uma só; é nela que o mock entra.” O alias `@test/*` está no `tsconfig.json` (`"@test/*": [". /test/*"]`), e os testes compilam sob `tsconfig.test.json` (`types: ["bun"], composite: true, outDir: . / .tsbuild-test`); separado “para os globais do Bun não vazarem pro código de produção”.

O detalhe mais instrutivo do `client.test.ts` é que ele **escreve um Standard Schema à mão**, em vez de importar um `*.schema.ts` de feature:

```
src/lib/api/__tests__/client.test.ts: schema fake
```

```
/**
 * Standard Schema minimo, escrito a mao.
 *
 * Nao importa um `*.schema.ts` de feature de proposito: `lib` nao pode
 * importar
 * `features` (`boilerplate/enforce-boundaries`), e o que este teste
 * exercita e o
 * contrato Standard Schema que o `apiClient` consome - nao o valibot.
 */
const perfilSchema: StandardSchemaV1<unknown, Perfil> = {
  "~standard": {
    version: 1,
    vendor: "test",
    validate: (value) => { /* ... */ return { issues: [{ message: "
    Perfil invalido." }] } }; },
  },
};
```

E a injeção do fake é por opção do ky, sem monkey-patch global: `ky.create({ baseUrl: "https://api.example.com", fetch: fetchImpl, retry: 0 })`.

CAUIDADO | Não há nenhum teste em `src/features/` nem `src/app/`

Não existe um único `*.test.tsx` no repo. A cobertura entregue é só da borda de IO e da env, coerente com o [AGENTS.md](#): “mock de IO na borda de `lib/api`”. Se o seu time quer teste de componente, a infraestrutura para isso (renderer, jsdom) **não vem**; é decisão sua e adição sua.

8.11 Passo a passo: seu primeiro endpoint

Cada passo abaixo é modelado num arquivo que existe no repo.

Passo 0: configurar

Copie `.env.example` para `.env.local`. Preencha `INTERNAL_API_URL` e `NEXT_PUBLIC_API_URL`. Se o backend de dev está em outro host e você quer same-origin, descomente o `rewrites()` do `next.config.ts`.

Passo 1: path no catálogo

Em `src/lib/api/endpoints.ts`, dentro do domínio, sem barra inicial:

```
src/lib/api/endpoints.ts
```

```
clientes: { lista: "api/v1/clientes" },
```

Passo 2: schema na feature

Crie `src/features/clientes/clientes.schema.ts`. O nome precisa terminar em `.schema.ts`. Use named imports e exporte o schema junto com o tipo inferido:

```
src/features/clientes/clientes.schema.ts
```

```
import { array, object, string, type InferOutput } from "valibot";

export const clientesSchema = array(object({ id: string(), nome: string() }));
export type Cliente = InferOutput<typeof clientesSchema>[number];
```

Passo 3: leitura em RSC

Em `src/app/clientes/page.tsx`, siga o molde de `perfil/page.tsx`: subcomponente async isolado, chamada ao `apiClient` e tudo sob `<Suspense>`. Função nomeada mantém return type explícito.

Passo 4: mutação, se houver

Crie `clientes.actions.ts` com "use server". O parse continua no `*.schema.ts`; a action chama o `apiClient`, revalida a rota no sucesso e devolve uma union discriminada como `SalvarPerfilResult`.

Passo 5: formulário, se houver

Use `use-clientes-form.ts` para estado e submit. Deixe `clientes-form.tsx` focado em UI. Nunca faça fetch em `useEffect`; o lint reprova.

Passo 6: proteger a rota

Acrescente o prefixo no matcher do `src/proxy.ts`:

```
src/proxy.ts
```

```
export const config = {
  matcher: ["/perfil/:path*", "/clientes/:path*"],
};
```

Passo 7: teste, se você mexeu no gateway

Teste em `src/lib/api/__tests__`, usando o mock de borda e um Standard Schema local. A camada `lib` não pode importar features.

Passo 8: os portões

Os dois modos de verificação (scripts reais do package.json)

```
bun run verify:fast    # lint → fmt:check → knip → test
bun run verify        # lint → fmt:check → typecheck → typecheck:
  tooling →            # knip → test → tooling/no-lint-suppression.ts
  →                    # tooling/no-heavy-public-assets.ts
```

Detalhes que valem saber: `lint` é `oxlint --disable-nested-config --type-aware`; `typecheck` roda `next typegen`, depois `tsc --noEmit` do `app` e `tsc --noEmit -p tsconfig.test.json` dos testes; `test` é `bun test --pass-with-no-tests --parallel`.

8.12 O que não vem: e o motivo verificado

Ausências deliberadas

Ausente	Onde está registrado	Motivo declarado
<code>apiClientPaginated</code>	comentário de <code>client.ts</code> + <code>MEMORY.md</code>	Zero caller = YAGNI. <code>requestEnvelope</code> já é o ponto de extração
<code>src/lib/auth/</code>	<code>ARCHITECTURE.md</code> + <code>MEMORY.md</code>	"A auth deste desenho não tem estado no front." Um módulo de auth no front seria justamente a camada que o ADR-007 proíbe
<code>src/shared/</code>	<code>ARCHITECTURE.md</code>	"Vazia até o 1º primitivo cross-feature." O diretório sequer existe no disco
<code>error.tsx</code> , <code>not-found.tsx</code> , <code>loading.tsx</code>	<code>ARCHITECTURE.md</code>	"Fora de escopo (fases futuras)"
logout e refresh de token	JSDoc de <code>ky.client.ts</code> e <code>proxy.ts</code>	Mencionados como o lugar onde entram; não implementados

REGRA | O knip é o que torna "zero caller = YAGNI" verdadeiro, não retórico

`knip.json` (ADR-017) define entry em `src/app/**` + `tooling`, e project cobrindo `src/**`, `tooling/**`, `test/**`. Uma `apiClientPaginated` exportada e não chamada seria reportada como export não usado e **reprovaria** `bun run verify`. Ou seja: você não consegue commitar abstração especulativa neste repo mesmo que queira. Rodado na versão atual, o knip sai limpo.

Duas duplicações que parecem descuido e não são: a const CAMPO_CLASS tem o mesmo literal Tailwind em [login-form.tsx](#) e [perfil-form.tsx](#). Extrair para [src/shared/](#) exigiria um terceiro caller real (CODE-PATTERN §9). Dois callers ainda é duplicação aceita.

8.13 Divergências entre a documentação e o código

Este boilerplate é honesto sobre a própria doutrina, mas alguns pontos dos docs e dos comentários estão adiantados em relação ao código. Vale saber quais, para não caçar arquivo que não existe.

CUIDADO | Números da stack: confie no [package.json](#)

A tabela de stack deve ser conferida no arquivo-fonte. O baseline atual do [package.json](#) traz: next 16.3.0, oxlint 1.77.0, typescript 7.0.2, @typescript/native-preview 7.0.0-dev.20260707.2, react 19.2.8, ky ^2.0.2, valibot ^1.4.2, packageManager: bun@1.4.0-canary.1. Onde divergir, o [package.json](#) manda.

Referências a coisas que ainda não existem

Onde	O que referencia
JSDoc de requestEnvelope (client.ts)	Diz que o tratamento é “comum ao apiClient e ao apiClientPaginated ”; o segundo não existe
Comentário em src/app/perfil/page.tsx	Diz que o erro sobe pro error.tsx mais próximo; não há error.tsx no repo
JSDoc de ky.client.ts	Cita login, logout e refresh; só login está implementado
ARCHITECTURE.md	Lista auth/ como conteúdo de lib/ na tabela de camadas, mas explica adiante que lib/auth/ não existe de propósito
CLAUDE.md	Descreve <code>bun test</code> como “testes Bun das regras custom”; na prática roda também os 13 testes de src/lib

Nada disso quebra código. Mas se um agente ler o JSDoc de requestEnvelope e sair procurando [apiClientPaginated](#) para reusar, ele vai gastar contexto à toa; ou, pior, vai criar a função “para consertar a inconsistência” e reprovar no knip. **A camada de referência é para copiar o padrão, não para completar as lacunas que os comentários apontam.** As lacunas são o roteiro do seu primeiro dia, não bugs.

PARTE 9

Guardrails: as regras que o lint impõe

O QUE VOCÊ LEVA DESTE CAPÍTULO

Você vai entender quais desvios o projeto reprova, onde cada regra vive e quais limites ainda dependem de review. Consulte este capítulo quando um erro de lint parecer arbitrário.

O catálogo de julgamento mora em [.claude/docs/guardrails-catalog.md](#). Ele é a fonte das categorias A–G usadas pela [.agents/skills/pre-review/SKILL.md](#). Aqui, o foco é separar o que o lint consegue impor do que ainda precisa de contexto humano.

Documentação explica o motivo; o gate aparece no momento do desvio. O Stack Sênior usa os dois e manda para a automação tudo o que pode ser verificado de forma objetiva.

Isso não significa transformar preferência pessoal em erro. O projeto separa três tipos de decisão:

- o que está comprovadamente errado reprova;
- o que merece atenção, mas pode ser legítimo, gera aviso;
- o que exige contexto continua sendo responsabilidade do code review.

REGRA | O princípio por trás dos guardrails

Uma regra precisa ser específica o bastante para impedir um desvio real sem obrigar o agente a reescrever código correto apenas para satisfazer o linter.

9.1 O mapa antes dos detalhes

O enforcement vive em camadas porque uma única ferramenta não enxerga tudo.

Onde cada proteção vive

Camada	Responsabilidade
Oxlint nativo	Erros gerais de TypeScript, React, imports, performance e consistência
Plugin boilerplate/	Decisões específicas desta arquitetura
Lint type-aware	Condições mortas, assertions inseguras e incoerências que dependem de tipos
Scripts em Bun	Supressão de lint e peso dos arquivos em <code>public/</code>
Testes de guardrail	Situações que dependem da existência de arquivos ou de scripts do <code>package.json</code>
Backstop do CI	Redundância para as regras mais caras de perder caso a API do plugin mude

No dia a dia, você não precisa memorizar essa divisão. A mensagem deve dizer o que aconteceu e qual é a saída esperada. O mapa importa quando você quer mudar uma regra com segurança.

9.2 As 17 regras do projeto

Quinze regras reprovam o gate. Duas geram aviso porque podem produzir falsos positivos sem conhecer os callers.

Arquitetura e dependências

Regra	O que protege
<code>enforce-boundaries</code>	Mantém a direção <code>app > features > lib > shared</code> e impede uma feature de importar outra feature
<code>no-index-barrel</code>	Evita <code>index.ts</code> usado apenas como barrel de re-export
<code>no-forbidden-imports</code>	Fecha subpaths de pacotes proibidos e cobre <code>export ... from</code>

Dados, cache e bordas

Regra	O que protege
no-use-cache	Barra diretivas e opções que introduzem cache ou staleness
no-force-static	Impede configuração estática incompatível com dados sempre frescos
api-through-lib	Mantém chamadas ao backend dentro de <code>src/lib/api/</code>
no-fetch-in-effect	Impede leitura de dados em effects no cliente
no-watch-in-effect	Evita <code>watch()</code> do react-hook-form dentro de effect

React e estado

Regra	O que protege
no-derived-state-in-effect	Sinaliza estado derivado por effect quando o valor pode ser calculado diretamente
no-manual-memo	Evita <code>useMemo</code> , <code>useCallback</code> e memo redundantes com React Compiler

Tipos, schemas e forma do código

Regra	O que protege
no-assert	Barra <code>assert</code> como substituto de tipo ou validação de borda
no-defensive-parse	Evita parse envolvido por <code>try/catch</code> que engole erro e fabrica default
no-schema-output-convert	Leva a transformação para o próprio schema quando existe apenas para remodelar seu output
no-catalog-literal-compare	Mantém IDs de catálogo no registry, não espalhados em comparações
no-focused-tests	Impede <code>test.only</code> , <code>it.only</code> e <code>describe.only</code> no gate
no-passthrough-mapper	Aviso: levanta mapper que apenas copia ou renomeia campos
no-literal-dispatch	Aviso: levanta função que só despacha literais

POR QUE | Por que duas regras apenas avisam

Um mapper pode ser uma abstração inútil ou uma tradução legítima entre o contrato do backend e o domínio. Uma função de literais pode ter um caller e merecer ficar inline, ou três callers e evitar duplicação. A AST do arquivo não conhece essa história. Por isso o linter levanta o candidato e a review decide.

9.3 Zonas de import

Algumas dependências são permitidas apenas onde exercem uma responsabilidade clara:

- `valibot` entra em arquivos `*.schema.ts`;
- `ky` entra em `src/lib/api/`;
- `zod`, `React Query`, `SWR`, `axios`, `jose` e `jsonwebtoken` continuam proibidos;
- `assert` e `node:assert` continuam cobertos pela regra `custom global`.

Essa divisão impede que uma feature faça HTTP por conta própria ou que validação de borda se espalhe pela aplicação. Ela também explica por que alguns imports funcionam em um arquivo e falham no arquivo ao lado.

CUIDADO | Override substitui a configuração anterior

Os overrides de `no-restricted-imports` reescrevem a lista inteira. Ao mudar uma zona, confira o que foi liberado e o que deixou de aparecer no array. A regra `no-assert` funciona como proteção global justamente porque essas listas locais podem variar.

9.4 O lint que conhece os tipos

O comando `bun run lint` usa `--type-aware`. Seis regras dependem do programa TypeScript completo:

- `typescript/no-unnecessary-condition`;
- `no-unnecessary-template-expression`;
- `no-unnecessary-boolean-literal-compare`;
- `no-unsafe-type-assertion`;
- `consistent-return`;
- `no-redundant-type-constituents`.

Elas encontram fallback morto, optional chaining sobre valor garantido, narrowing desnecessário e assertion que promete mais do que o tipo provou. É aqui que a orientação “confie no tipo” deixa de ser conselho e passa a reprovar.

O baseline do `tsconfig.json` acompanha o Avante e não ativa `noUncheckedIndexedAccess`. Quando o código acessa uma posição que pode estar vazia, use

uma API que preserve o estreitamento (por exemplo, `array.at(0)`) e trate o caso ausente. A flag não muda o runtime: ela só muda o quanto o TypeScript infere de um acesso por índice, por isso o código deve escolher uma API que deixe o caso ausente visível quando ele for possível.

CUIDADO | O pre-commit é mais barato

O pre-commit olha apenas os arquivos staged e roda sem type-aware. Um erro que depende dos tipos pode aparecer somente no `bun run lint`, no pre-push ou no CI. Rode o lint completo durante uma mudança estrutural para receber esse retorno mais cedo.

9.5 O que fica fora do linter

9.5.1 Supressão local

O `tooling/no-lint-suppression.ts` procura diretivas de `eslint-disable` e `oxlint-disable`. Ele vive fora do plugin porque uma regra de lint poderia ser desligada pela mesma diretiva que tenta proibir.

REGRA | Corrija a causa ou mude a regra de forma explícita

Não existe supressão local. Se a regra estiver errada para o seu produto, altere a configuração, o teste e a decisão registrada. Essa mudança fica visível no diff e passa por review.

9.5.2 Peso de assets

O `tooling/no-heavy-public-assets.ts` reprova arquivos acima de 200 KB em `public/`. O limite vale para qualquer formato porque o custo chega ao cliente do mesmo jeito.

Se um asset maior for legítimo, a saída é editar `BUDGET_BYTES` com justificativa. Uma allowlist por extensão criaria um buraco fácil de contornar e difícil de manter.

9.5.3 Arquivos e scripts que não deveriam existir

Algumas condições viram teste porque o linter não consegue enxergá-las:

Guardrails implementados como teste

Teste	O que impede
<code>no-middleware-file.test.ts</code>	A volta de <code>middleware.ts</code> em um projeto Next 16 que usa <code>proxy.ts</code>
<code>no-node-runtime.test.ts</code>	Node em scripts do <code>package.json</code> e artefatos de runtime fora do padrão Bun
<code>no-heavy-public-assets.test.ts</code>	Regressão no budget de arquivos públicos

9.6 Redundância onde a perda seria cara

O plugin JS do oxlint usa uma API alpha. Por isso o CI mantém dois grep-gates simples:

- um procura diretivas de cache proibidas;
- outro procura URL do backend fora de `src/lib/api/`.

Eles não substituem a análise do lint. Funcionam como rede de segurança se o plugin deixar de carregar depois de uma atualização.

CUIDADO | Troque o domínio de exemplo em dois lugares

Antes do primeiro deploy, substitua `example.com` em `BACKEND_DOMAINS`, dentro de `api-through-lib.ts`, e no grep-gate do workflow de CI. Enquanto o placeholder continuar, essa proteção não conhece o seu backend.

9.7 Como responder a um erro

Quando um guardrail interromper o trabalho:

1. leia a mensagem completa e identifique o arquivo que a emitiu;
2. descubra qual decisão a regra protege;
3. aplique o caminho recomendado e rode o menor gate relevante;
4. finalize com `bun run verify`;
5. se o contexto do seu produto contrariar a regra, trate isso como mudança de arquitetura.

Resposta rápida

Situação	Próxima ação
Regra com severidade <code>error</code>	Corrigir o padrão ou propor mudança explícita da regra
Regra com severidade <code>warn</code>	Verificar callers e julgar no contexto
Erro type-aware	Confiar no tipo ou corrigir a borda que produz um tipo mentiroso
Asset acima de 200 KB	Otimizar na origem ou justificar novo budget
Falha depois de atualizar oxlint	Rodar os testes das regras custom e conferir a API do plugin

9.8 Limites conhecidos

O guia não esconde os pontos que ainda pedem atenção:

- seis regras custom não têm fixture dedicada no teste de integração;
- o app e o teste de integração usam oxlint 1.77.0;
- `enforce-boundaries` olha imports, enquanto alguns formatos de re-export dependem de outra regra;
- as duas regras advisory não conhecem quantidade de callers.

Esses limites não anulam o gate. Eles mostram onde uma atualização merece validação manual e onde o code review continua indispensável.

PRÓXIMO PASSO: use a mensagem como documentação

Na próxima falha de lint, não procure primeiro como silenciar. Abra o arquivo da regra, leia o motivo e compare com a decisão do seu produto. O objetivo do guardrail é encurtar esse caminho.

PARTE 10

Os portões: verify, lefthook e CI

O QUE VOCÊ LEVA DESTE CAPÍTULO

Você vai saber qual verificação roda em cada etapa, por que os portões não são duplicação e onde procurar quando um comando falha no commit, no push ou no CI.

O boilerplate transforma a arquitetura em uma sequência de verificações. Cada uma tem nome, arquivo e momento de execução. Aqui você acompanha os quatro pontos em que uma mudança pode ser barrada, do mais barato, durante a escrita, ao mais completo, no CI.

O centro do fluxo é um único script do `package.json`: `verify`. Hooks de agente, `pre-commit`, `pre-push` e CI invocam esse script, ou um subconjunto dele, em momentos diferentes. Entender `verify` é entender o contrato de qualidade da base.

10.1 As quatro camadas

Onde o código é barrado, em ordem cronológica

Camada	O que roda	Custo
1. Agente (durante a escrita)	Hooks em <code>tooling/agent-hooks/</code> : formata a cada Write, bloqueia shell destrutivo, roda <code>verify:fast</code> antes de encerrar o turno	ms a segundos
2. <code>git commit</code>	<code>lefthook pre-commit: oxcint + oxfmt --check</code> só nos arquivos staged	sub-segundo
3. <code>git push</code>	<code>lefthook pre-push: bun run verify</code> no projeto inteiro	segundos
4. CI (push e PR)	<code>dois grep-gates + bun run verify + bun run build</code>	minutos

POR QUE | Por que quatro e não uma

Cada camada existe porque a anterior não consegue ver o que ela vê. O hook de agente é o único que roda *enquanto* o código é escrito, quando corrigir é trivial. O `pre-commit` só olha o staged, então é rápido o bastante pra não desestimular commits pequenos. O `pre-push` é a última barreira antes de o código sair da sua máquina, então checka tudo. O CI é o único que roda numa máquina limpa, com `--frozen-lockfile`, e o único que roda o `next build` de verdade. Uma camada só ou seria lenta demais pra escrever código, ou frouxa demais pra segurar.

10.2 bun run verify: a cadeia canônica

Duas linhas do `package.json` definem os dois gates:

package.json, linhas 18-19 (literal)

```
"verify:fast": "bun run lint && bun run fmt:check && bun run knip && bun run test",
"verify": "bun run lint && bun run fmt:check && bun run typecheck && bun run typecheck:tooling && bun run knip && bun run test && bun tooling/no-lint-suppression.ts && bun tooling/no-heavy-public-assets.ts"
```

São comandos encadeados com `&&`: a cadeia **para na primeira falha**. Isso é intencional e muda como você lê a saída; se o lint reprovou, você não sabe nada sobre os testes ainda. Corrija de cima pra baixo.

As oito etapas do `verify`, na ordem exata

#	Comando real	Reprova quando
1	<code>oxlint --disable-nested-config --type-aware</code>	qualquer regra built-in (categorias <code>correctness</code> / <code>perf</code> / <code>suspicious</code> como erro) ou qualquer uma das 17 regras custom do plugin <code>boilerplate/*</code>
2	<code>oxfmt --check</code>	algum arquivo de código ou JSON está fora do formato
3	<code>next typegen</code> , depois <code>tsc --noEmit</code> , depois <code>tsc --noEmit -p tsconfig.test.json</code>	erro de tipo no app (já com as rotas tipadas geradas) ou nos testes
4	<code>tsc --noEmit -p tooling/tsconfig.json</code>	erro de tipo nas regras custom de lint e nos hooks de agente
5	<code>knip --no-progress</code>	existe arquivo, export ou dependência sem consumidor
6	<code>bun test --pass-with-no-tests --parallel</code>	qualquer teste vermelho
7	<code>bun tooling/no-lint-suppression.ts</code>	existe diretiva de supressão de lint no código-fonte
8	<code>bun tooling/no-heavy-public-assets.ts</code>	algum arquivo em <code>public/</code> passa de 200 KB

O `verify:fast` é o mesmo gate sem as etapas 3, 4, 7 e 8; ou seja, lint, formato, knip e testes. É deliberadamente o subconjunto que não paga o preço do `tsc`, e é exatamente o que o hook de Stop do agente executa.

Rodando o gate completo (medições reais, cache quente, Bun 1.4.0-canary.1)

```
$ bun run verify
...
121 pass
 0 fail
272 expect() calls
Ran 121 tests across 12 files.

# tempos observados nesta máquina:
#   bun run lint      0,59 s   (saída vazia em sucesso)
#   bun run knip      0,41 s   (exit 0, saída vazia)
#   bun run verify:fast 15,4 s
#   bun run verify    15,9 s   (13,8 s são o `bun test`)
```

Duas coisas úteis de saber sobre essa saída. Primeiro: sucesso é silêncio. O `oxlint` e o `knip` não imprimem nada quando passam; se você viu texto, leia. Segundo: o `typecheck` imprime um aviso do `Next.js` que **não** é falha:

Log esperado, não é erro

```
Detected @typescript/native-preview as TypeScript compiler. Some Next.js
TypeScript features (like type checking during build) require the
standard
typescript package.
```

É consequência de usar o compilador TypeScript nativo (o `tsgo`), decisão registrada no [MEMORY.md](#) como ADR-011. O `typecheck` de verdade acontece nas etapas 3 e 4, explicitamente, e no `bun run build`.

REGRA | O gate é o contrato

`bun run verify` verde é a definição de “pronto” neste repositório. Não existe “passa no CI mas falha local” nem o contrário: o CI roda o mesmo comando. Se você precisa desligar uma etapa pra seguir, o problema é o código, não o gate.

10.3 Camada 1: os hooks de agente

O boilerplate assume que a maior parte do código será escrita por agente. Por isso o primeiro portão roda dentro da sessão do agente, não no git. Os scripts vivem em [tooling/agent-hooks/](#) (TypeScript rodando em Bun) e são apontados por três JSONs finos, um por host: [.claude/settings.json](#), [.codex/hooks.json](#) e [.cursor/hooks.json](#). Todos resolvem o caminho do script da mesma forma, com `git rev-parse --show-toplevel` e `pwd` como fallback.

Wiring verificado em `.claude/settings.json`

Evento	Script	Timeout
SessionStart	<code>inject-patterns.ts</code>	;
UserPromptSubmit	<code>reinforce-patterns.ts</code>	;
PreToolUse (Bash)	<code>pre-pr-review-gate.ts, guard-shell.ts</code>	15000 cada
PostToolUse (Edit/Write)	<code>reinforce-after-write.ts, format-after-write.ts</code>	30000
Stop	<code>verify-before-stop.ts</code>	300000

Formatação automática, sem bloqueio. O `format-after-write.ts` roda o `oxfmt` no arquivo recém-escrito e sai com zero sempre. Não é um gate, é conveniência: o agente não perde turno com diff de formatação.

`tooling/agent-hooks/format-after-write.ts (trecho)`

```
const FORMATTABLE = /\.(ts|tsx|js|jsx|mjs|cjs)$/;
// ...
Bun.spawnSync(["bunx", "oxfmt", filePath], { stdout: "ignore", stderr:
  "ignore" });
process.exit(0);
```

Repare que `.md` não está na regex; Markdown não é formatado aqui, coerente com a decisão do `.oxfmt.rc.json` descrita adiante.

O agente não encerra o turno com o gate vermelho. O `verify-before-stop.ts` é o hook mais importante da camada 1:

`tooling/agent-hooks/verify-before-stop.ts (trecho)`

```
const verification = Bun.spawnSync(["bun", "run", "verify:fast"], {
  cwd: process.cwd(),
  stdout: "inherit",
  stderr: "inherit",
});

if (verification.exitCode !== 0) {
  emitStopContinue(
    host,
    "Stop bloqueado: `bun run verify:fast` falhou. Corrija os erros do
    gate e só então encerre o turno.",
  );
}
```

Ou seja: o agente é obrigado a continuar trabalhando até lint, formato, knip e testes passarem. O timeout de 300000 ms (cinco minutos) existe porque esse hook pode acabar rodando a suíte inteira.

Guard de shell. O `guard-shell.ts` intercepta cada Bash antes de executar e nega: `rm` apontando pra raiz ou home, `git reset --hard`, `git clean -f`, `sudo`, `chmod 777`, leitura do `.env` real (o `.env.example` passa), `curl` canalizado pra bash, e `git push --force` sem `--force-with-lease` ou `--force-if-includes`. Também barra commit e push direto em branch protegida:

tooling/agent-hooks/guard-shell.ts, linha 12

```
const PROTECTED = ["main", "release", "staging", "develop"];
```

Portão de PR. O `pre-pr-review-gate.ts` bloqueia `gh pr create` enquanto não existir o selo do HEAD gravado em `.agents/tmp/`, e recusa PR acima do teto:

tooling/agent-hooks/pre-pr-review-gate.ts (trechos)

```
const MAX_PR_LOC = 3000;
const BASE_BRANCH = "main";
// ...
if (!existsSync(`.agents/tmp/pre-review-${head}.ok`))
```

O wiring dos três hosts tem cobertura de teste própria (`tooling/agent-hooks/hook-configs.test.ts` e os testes por script). O ADR-016 do `MEMORY.md` explica o motivo: o teste reprova se algum host apontar para um script que não existe ou que estoura. Um hook quebrado é um gate silenciosamente desligado, e o único jeito de detectar isso é testar o JSON de configuração.

10.4 Camada 2: lefthook pre-commit

O `lefthook.yml` tem 23 linhas. O pre-commit inteiro é isto:

lefthook.yml, bloco pre-commit

```

pre-commit:
  parallel: true
  commands:
    oxlint:
      glob: "*.ts,tsx,js,jsx"
      # --no-error-on-unmatched-pattern: não falhar quando os staged
      files
      # forem todos ignorados (ex.: só .claude/** ou docs).
      run: bunx oxlint --disable-nested-config --no-error-on-unmatched-
        pattern {staged_files}
    oxfmt:
      glob: "*.ts,tsx,js,jsx,mjs,cjs,json"
      run: bunx oxfmt --check {staged_files}

```

Três detalhes que valem atenção:

- São **duas** checagens e só duas: lint e formato. Não há typecheck nem teste no pre-commit, de propósito; o comentário no topo do arquivo diz "barato e focado nos arquivos staged, pra não travar o loop de commit".
- O oxlint aqui roda **sem** `--type-aware`, diferente do `bun run lint` do verify. É a versão rápida, que não precisa do serviço de tipos.
- `--no-error-on-unmatched-pattern` evita o falso vermelho de um commit que só toca docs ou `.claude/`, quando a lista de staged files é filtrada até sobrar nada.

O `--disable-nested-config` aparece nos dois lugares e é o ADR-012: worktrees aninhados (`.claude/worktrees/`) carregavam um `.oxlintrc.json` próprio e o oxlint passava a lintar com a config errada. Com a flag, só a config da raiz vale.

CUIDADO | Os docs canônicos estão desatualizados neste ponto

O `ARCHITECTURE.md` e o `VERCEL-OXLINT-MAPPING.md` descrevem o pre-commit como "oxlint + typecheck + test". O `lefthook.yml` real não roda typecheck nem test. O mesmo vale para o hook de Stop: os docs dizem "typecheck + oxlint", o script roda `verify:fast`. Quando doc e arquivo divergem, o arquivo é a verdade; ele é o que executa.

10.5 Camada 3: lefthook pre-push

lefthook.yml, bloco pre-push

```

pre-push:
  commands:
    verify:
      run: bun run verify

```

Uma linha. O comentário acima dela no arquivo explica a diferença de filosofia em relação ao pre-commit: "É a última barreira pré-remoto, então tudo é checado no projeto inteiro; não só o staged."

CUIDADO | Os git hooks não estão instalados no pacote que você recebeu

O diretório entregue não é um repositório git: não existe `.git` na raiz. O lefthook se instala sozinho via `postinstall` do próprio pacote (habilitado pelo `trustedDependencies` do `package.json`, que lista `sharp`, `unrs-resolver` e `lefthook`), mas essa instalação precisa de um repositório pra escrever os hooks. Sequência correta: `git init` e depois `bun install` de novo, ou `bunx lefthook install`. Enquanto você não fizer isso, as camadas 2 e 3 simplesmente não existem e nada te avisa.

POR QUE | Por que o gate funciona mesmo sem git

Os scripts foram escritos assumindo esse cenário. O `tooling/no-lint-suppression.ts` usa `git grep` quando há repositório e cai pro `grep` recursivo quando não há; o comentário do próprio arquivo diz que "o comprador extrai o zip e roda `bun run verify` antes do `git init`". O `guard-shell.ts` deixa passar commit e push quando não consegue descobrir a branch. Você pode rodar `bun run verify` no minuto zero, antes de qualquer coisa.

10.6 As duas etapas do `verify` que não são lint

Etapa 7: proibição de supressão. Esta é a peça mais conceitualmente interessante do gate. Ela existe fora do linter por um motivo declarado no cabeçalho do arquivo:

`tooling/no-lint-suppression.ts` (doc do arquivo)

```
* Não é uma regra do plugin oxlint de propósito: uma regra de lint
  pode ser
* desligada pela própria diretiva que ela proíbe. Um grep fora do
  linter, não.
```

É o ADR-020. Uma regra `boilerplate/no-lint-suppression` seria autodestrutiva: bastaria uma diretiva de supressão apontando pra ela mesma. Um grep externo não tem esse problema. Detalhe fino de implementação; as agulhas da busca são montadas em runtime pra o guard não se autodetectar:

`tooling/no-lint-suppression.ts` (trecho)

```
// Montados em runtime pra este próprio arquivo não dar auto-match na
  busca.
const NEEDLES = [["eslint", "disable"].join("-"), ["oxlint", "disable"].
  join("-")];
```

O guard cobre `.ts`, `.tsx`, `.js`, `.jsx`, `.mjs`, `.cjs`, `.mts` e `.cts`, e trata exit code direito: exit 1 das duas ferramentas de busca significa apenas "nenhuma ocorrência", enquanto exit maior que 1 é falha real da busca, e também reprova, porque um gate que não conseguiu procurar não é um gate verde.

REGRA | Não existe escape hatch de lint

Você não pode suprimir uma regra pra fazer o gate passar. Nem localmente, nem "só desta vez". Se uma regra está atrapalhando, ou o código está errado, ou a regra está errada, e a regra é um arquivo TypeScript em [tooling/oxlint-rules/rules/](https://github.com/oxlint/oxlint-rules) que você pode editar, com teste ao lado. Editar a regra é honesto e aparece no diff; suprimir a regra é invisível.

Etapas 8: budget de assets. Tudo em [public/](#) é baixado pelo cliente, e asset pesado na origem é o gargalo número um de LCP. O guard é curto:

tooling/no-heavy-public-assets.ts (trecho)

```
const BUDGET_BYTES = 200 * 1024;

const glob = new Bun.Glob("public/**/*");
```

Não há allowlist, e o cabeçalho do arquivo justifica: "exceção por formato ou diretório vira buraco no gate (foto disfarçada de PNG é o caso clássico que fura qualquer allowlist de extensão)". O guard também não lê os bytes do arquivo nem julga formato; só peso. Se um asset legítimo precisa passar de 200 KB, a instrução é ajustar o BUDGET_BYTES com justificativa, o que aparece no diff, em vez de subir binário pesado calado.

10.7 knip: código e dependência mortos

O [knip.json](#) é curto e cada linha tem consequência:

knip.json (integral, com os comentários reais)

```
{
  "$schema": "./node_modules/knip/schema.json",
  "entry": [
    "src/app/**/*.ts,tsx",
    "tooling/agent-hooks/*.ts",
    "tooling/*.ts",
    "integracoes/coderabbit/cr-comments.ts"
  ],
  "project": ["src/**/*.ts,tsx", "src/**/*.css", "tooling/**/*.ts", "test/**/*.ts", "*.ts,mjs"],
  "ignoreDependencies": [
    // Fornece o `tsgo`; invocado pelo toolchain, não importado por nenhum módulo.
    "@typescript/native-preview",
```

```
knip.json (integral, com os comentários reais) (continuação)

    // Instala os git hooks; roda pelo lefthook.yml, não por import.
    "lefthook"
  ]
}
```

Os entypoints são as rotas ([src/app/](#)), os scripts de tooling e de hooks, e o script do CodeRabbit. Todo o resto do projeto precisa ser alcançável a partir daí; se não é, é código morto e o gate reprova. As duas únicas dependências isentas são justamente as que existem sem nenhum `import` no código, e cada uma carrega a justificativa inline.

POR QUE | Por que knip importa mais num boilerplate (ADR-017)

Citação literal do [MEMORY.md](#): "o no-unused-vars do oxlint só enxerga dentro do módulo. Código morto cross-module é o que realmente acumula, e num boilerplate é pior: o comprador herda arquivos que ninguém chama e presume que são necessários."

O mesmo ADR registra o efeito concreto: o knip "pegou o `server-only` órfão no [package.json](#) e forçou o [lib/api/ky.client.ts](#) a ter caller real; foi o knip que exigiu a tela de login, fechando o ciclo de auth que o [proxy.ts](#) já prometia ao redirecionar para `/login`". O gate não só limpou código: ele obrigou uma feature a existir.

10.8 oxfmt, e por que Markdown fica de fora

A configuração de formatação inteira é uma linha:

```
.oxfmttrc.json (integral)

{
  "$schema": "./node_modules/oxfmt/configuration_schema.json",

  // Defaults do oxfmt já batem com o estilo do repo (printWidth 100,
  // tabWidth 2,
  // semi, aspas duplas, trailingComma "all", endOfLine "lf"). Não
  // restateamos
  // defaults — só o que difere fica aqui.
  //
  // Markdown fica de fora: os docs canônicos (AGENTS/ARCHITECTURE/CODE
  // -PATTERN/
  // MEMORY/VERCEL-OXLINT-MAPPING) e as skills são prosa mantida à mão;
  // reformatá-los é churn sem valor. Código e config JSON, sim.
  "ignorePatterns": ["**/*.md"]
}
```

O ADR-018 resume a escolha da ferramenta: "mesmo ecossistema do oxlint (um binário, zero config duplicada). Os docs canônicos são prosa mantida à mão; reformatá-los é churn sem valor e polui o diff da review."

O oxfmt roda em três lugares: no `PostToolUse` dos agentes (formatando), no `pre-commit` sobre os `staged` (checando) e no `verify` sobre o projeto (checando,

via `fmt:check`). Na prática, se você usa agente, o formato nunca reprova; ele já foi corrigido no momento da escrita. Escrevendo à mão, `bun run fmt` conserta tudo de uma vez.

10.9 Três contextos de TypeScript, e por que

O `verify` chama o `tsc` três vezes, com três configs. Não é redundância:

Os três contextos de tipos

Config	Cobre	Por que separado
<code>tsconfig.json</code>	o app (<code>src/</code> , exceto testes)	é o código de produção; exclui <code>tooling</code> , <code>test</code> e os <code>*.test.ts(x)</code>
<code>tsconfig.test.json</code>	<code>*.test.ts(x)</code> e <code>test/</code>	liga os tipos do Bun sem deixá-los vaziar para produção
<code>tooling/</code> <code>tsconfig.json</code>	regras custom de lint e hooks	também tipos do Bun, e <code>noImplicitAny</code> relaxado

O `tsconfig.test.json` tem duas opções que parecem erradas fora de contexto, e o próprio arquivo explica:

`tsconfig.test.json` (integral)

```
{
  "extends": "./tsconfig.json",
  "compilerOptions": {
    // `composite` é exigido para o tsconfig raiz referenciar este
    projeto (a IDE
    // resolve os testes por aqui). Projeto referenciado não pode fixar
    `noEmit`
    // no config, então o emit aponta para um diretório descartável e o
    typecheck
    // roda com `--noEmit` na linha de comando (nada é emitido na prá
    tica).
    "composite": true,
    "noEmit": false,
    "outDir": "./tsbuild-test",
    "types": ["bun"]
  },
  "include": ["src/**/*.test.ts", "src/**/*.test.tsx", "test/**/*.ts",
    "next-env.d.ts"]
}
```

O `noEmit: false` não emite nada porque o `--noEmit` da linha de comando vence. O diretório `.tsbuild-test/` na raiz é subproduto disso e pode ser ignorado. O relaxamento do `noImplicitAny` no `tooling` tem justificativa própria no arquivo: "A API de plugin do

oxlint é alpha e entrega context/node sem tipos estáveis"; é uma concessão localizada, não uma política.

10.10 CI: um workflow, um job, quatro passos

O `.github/workflows/ci.yml` tem 59 linhas e é o único workflow do repositório (o `.github/` contém só ele e o `CODEOWNERS`). Dispara em push e `pull_request`, sem filtro de branch, com permissão mínima:

`.github/workflows/ci.yml` (trechos)

```
permissions:
  contents: read

jobs:
  gate:
    runs-on: ubuntu-latest
    steps:
      # Actions pinadas por SHA (supply-chain)+ sem persistir
      # credencial no checkout.
      - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5
        # v4.3.1
        with:
          persist-credentials: false

      - uses: oven-sh/setup-
        bun@0c5077e51419868618aeaa5fe8019c62421857d6 # v2.2.0
        with:
          # Alinhado ao `packageManager` do package.json (quem gerou o
          # bun.lock).
          bun-version-file: package.json

      - run: bun install --frozen-lockfile
```

Três decisões de supply-chain num bloco só: actions pinadas por SHA (não por tag, que é mutável), `persist-credentials: false` no checkout (o token do GitHub não fica escrito no `.git/config` pros steps seguintes), e a versão do Bun lida do `package.json`; o campo `packageManager` vale `bun@1.4.0-canary.1`, que é a mesma versão que gerou o `bun.lock`.

A ordem dos steps depois disso é: **grep-gate 1** → **grep-gate 2** → **bun run verify** → **bun run build**. Os greps rodam *antes* do gate canônico, de propósito: são baratos e independem do plugin de lint carregar.

O build recebe env placeholder:

.github/workflows/ci.yml (step de build)

```
- name: Build (typegen + typecheck + next build sob Bun)
  run: bun run build
  env:
    # A borda falha fechada sem a base URL; no build nenhuma
    request sai de
    # verdade (o prerender vira dinâmico no `cookies()`), então
    um
    # placeholder local basta.
    NEXT_PUBLIC_API_URL: http://localhost:3333
    INTERNAL_API_URL: http://localhost:3333
```

Não existe step de cache de dependências, matriz de versões, nem job de deploy. É um job só, e ele é um portão; nada mais.

10.10.1 Grep-gate 1: cache proibido

.github/workflows/ci.yml, primeiro grep-gate

```
# Backstop em grep dos guardrails que dependem do jsPlugins (
alpha): se o
# plugin um dia falhar em carregar, estes gates ainda barram as
violações.
- name: Grep-gate, sempre fresco
  run: |
    if grep -rEn "['\"]use cache[ :a-z]*['\"]|unstable_cache" src
    --include='*.ts' --include='*.tsx' --include='*.js' --include='*.jsx'
    ' 2>/dev/null; then
      echo "::error::cache proibido ('use cache' / unstable_cache
    ) em src/"; exit 1
    fi
```

POR QUE | Por que duplicar em grep uma regra que já existe em lint

O comentário do próprio workflow é a resposta: as regras custom rodam sobre jsPlugins, que é uma API alpha do oxlint. Se o plugin falhar em carregar; por bug, por upgrade, por config quebrada; o oxlint pode sair verde sem ter avaliado *nenhuma* das 17 regras boilerplate/*. O grep não depende de nada: é um if num shell.

O ADR-009 fecha o raciocínio: "jsPlugins é alpha (sem semver) → oxlint pinado (hoje 1.77.0) + grep-gate de CI como backstop. Subir a versão exige rodar bun test das regras custom: elas são o canário da API do plugin." O pin é real e visível no [package.json](#): "oxlint": "1.77.0" e "oxfmt": "0.62.0", sem caret, ao contrário de quase todas as outras devDeps.

CUIDADO | O backstop é parcial, não equivalente

A regra que o grep cobre é boilerplate/no-use-cache, em [tooling/oxlint-rules/rules/no-use-cache.ts](#), e ela detecta quatro padrões: a diretiva

"use cache" (inclusive as variantes remote e private), o import de unstable_cache, cache: "force-cache" e fetch com next.revalidate. O grep pega só os dois primeiros. Se o plugin cair, os outros dois passam. O backstop compra tempo, não substitui a regra, por isso os testes das regras custom são parte do verify. (A regra, corretamente, **não** reporta cache() nem React.cache(): memoização por request é permitida, cache entre requests não é.)

10.10.2 Grep-gate 2: backend só em src/lib/api

.github/workflows/ci.yml, segundo grep-gate

```
- name: Grep-gate, backend só em src/lib/api
  run: |
    # Backend = URL http(s) do domínio do backend fora de src/lib
    /api.
    # `-o` extrai uma URL por linha (não a linha inteira): sem
    # isso, uma linha
    # com uma URL pública E uma de backend juntas seria
    # descartada pelo
    # allowlist, escondendo a violação. Troque `example\.com`
    # pelo seu domínio.
    if grep -rEon "https?://[^\"]*" *example\.com" src --include='
    *.ts' --include='*.tsx' --include='*.js' --include='*.jsx' 2>/dev/
    null | grep -vE '^src/lib/api/'; then
      echo "::error::URL do backend fora de src/lib/api"; exit 1
    fi
```

Esse é o gate que **você precisa configurar**. O literal example\.com é placeholder e precisa virar o domínio real do seu backend. E ele não está sozinho: o mesmo domínio está hardcoded na regra de lint irmã, [tooling/oxlint-rules/rules/api-through-lib.ts](#):

tooling/oxlint-rules/rules/api-through-lib.ts (trecho)

```
const BACKEND_DOMAINS = ["example.com"];
const API_ENV_VARS = ["NEXT_PUBLIC_API_URL", "INTERNAL_API_URL"];
```

Essa regra dispara em fetch() cru cujo primeiro argumento contenha o domínio ou uma das env vars, e isenta qualquer arquivo cujo caminho inclua src/lib/api/; a implementação em si. Grep e regra formam o par: a regra pega no editor, o grep pega mesmo se o plugin morrer.

REGRA | Trocar o domínio exige mexer em dois arquivos

.github/workflows/ci.yml (linha 41) e [tooling/oxlint-rules/rules/api-through-lib.ts](#) (BACKEND_DOMAINS). Trocar só um deixa metade do enforcement apontando pro domínio errado, e um gate que aponta pro domínio errado é um gate

que sempre passa. Os outros lugares onde `example.com` aparece são `.env.example` e fixtures de teste, e não fazem parte do enforcement.

CUIDADO | O grep-gate 2 reprova no repositório como entregue

Rodando o comando exato do CI neste pacote, antes de qualquer customização:

```
src/lib/__tests__/env.schema.test.ts:7:https://api.example.com
src/lib/__tests__/env.schema.test.ts:8:https://api.example.com
::error::URL do backend fora de src/lib/api
```

O teste de schema de env usa `https://api.example.com` como fixture e mora em `src/lib/__tests__/`, que não casa com o `allowlist ^src/lib/api/`; os testes que vivem dentro de `src/lib/api/__tests__/` passam justamente porque o caminho começa certo. Como o pacote é entregue sem `.git`, esse CI nunca rodou aqui.

Na prática isso se resolve sozinho no momento em que você troca `example.com` pelo seu domínio: o fixture do teste deixa de casar com o padrão. Mas se você der `git init` e `push` antes de trocar, o primeiro CI vem vermelho, e vem vermelho por um motivo que não é um bug seu. Troque o domínio primeiro.

10.11 Sequência recomendada no dia zero

Do zip ao primeiro push com o CI verde

```
$ bun install
$ bun run verify          # funciona sem git; deve passar

# troque o domínio placeholder nos DOIS lugares:
# .github/workflows/ci.yml (linha 41, `example.com`)
# tooling/oxlint-rules/rules/api-through-lib.ts (BACKEND_DOMAINS)

$ git init
$ bun install          # agora o lefthook instala os git hooks
$ bun run verify        # confirma que nada quebrou na troca
$ git add -A && git commit -m "chore: bootstrap" # dispara o pre-commit
$ git push              # dispara o pre-push (verify)
```

REGRA | O que você não deve fazer

Não adicione `--no-verify` ao seu `git push`. Não comente uma etapa do `verify` pra "destravar". Não adicione um diretório ao `ignorePatterns` do `oxfmt` pra evitar reformatar. Cada uma dessas ações desliga um portão inteiro e não deixa rastro óbvio, e o valor deste boilerplate é exatamente que a arquitetura não depende de disciplina humana. Um gate desligado transforma o produto num template comum com bastante documentação.

PARTE 11

Hooks de agente: o guardrail que roda sozinho

O QUE VOCÊ LEVA DESTE CAPÍTULO

Você vai entender quando cada hook entra em ação, o que ele pode bloquear e como os três editores compartilham o mesmo comportamento. Leia antes de desligar um hook que parece inconveniente.

Documentação depende de alguém consultá-la na hora certa. Num repositório trabalhado também com agentes, isso é frágil: o [AGENTS.md](#) pode ter ficado fora da conversa algumas mudanças atrás, e uma contribuição bem-intencionada pode chegar à `main` fora do padrão. Por isso, sete scripts entram nos eventos do ciclo de desenvolvimento e repetem as proteções no momento em que elas fazem diferença.

Eles vivem em [tooling/agent-hooks/](#): são 12 arquivos, 1088 linhas, das quais 432 são testes. Há sete hooks e quatro suítes. Os três hooks com teste dedicado ([guard-shell.ts](#), [pre-pr-review-gate.ts](#) e [verify-before-stop.ts](#)) são justamente os que bloqueiam alguma coisa; um hook que nega um comando precisa de evidência mais forte do que um hook que apenas acrescenta contexto.

REGRA | Guardrail não é privilégio de um editor

Os mesmos sete scripts rodam em Claude Code, Codex e Cursor. Cada host só aponta para eles pelo seu JSON ([.claude/settings.json](#), [.codex/hooks.json](#), [.cursor/hooks.json](#)). Nenhuma lógica de guardrail vive dentro de config de host.

11.1 O mapa: sete hooks, três hosts

Os sete hooks e o momento em que entram

Script	Momento do ciclo	Bloqueia
inject-patterns.ts	Início da sessão	Não
reinforce-patterns.ts	Envio de cada prompt	Não
pre-pr-review-gate.ts	Antes de um comando de shell	Sim
guard-shell.ts	Antes de um comando de shell	Sim
reinforce-after-write.ts	Depois de escrever ou editar	Não
format-after-write.ts	Depois de escrever ou editar	Não
verify-before-stop.ts	Antes de encerrar o turno	Sim

Os nomes exatos dos eventos mudam entre os hosts, mas o momento é o mesmo. Nos hooks de escrita, Claude usa `Edit|Write`; Codex acrescenta `apply_patch`; Cursor separa `afterFileEdit` para formatação de `postToolUse` para reforço.

O resto das diferenças é de configuração, não de comportamento:

Ajustes que existem só em um host

Host	O que é específico dele
Claude Code	Único com timeout: 15000 ms nos dois <code>PreToolUse</code> , 30000 ms no <code>format</code> , 300000 ms (5 min) no <code>Stop</code> . Único com bloco <code>permissions</code> .
Codex	Único com <code>statusMessage</code> em todo hook, em PT-BR ("Rodando <code>bun run verify:fast</code> ", "Guarda de shell e branch protegida", ...). Único que casa <code>apply_patch</code> .
Cursor	"version": 1 no topo, <code>failClosed: true</code> nos dois hooks de shell e <code>loop_limit: 3</code> no <code>stop</code> .

11.2 Um script, três contratos: [host.ts](#)

[tooling/agent-hooks/host.ts](#) (192 linhas) é a única peça do repositório que sabe que existem hosts diferentes. Todo o resto escreve código de guardrail e nada de protocolo.

tooling/agent-hooks/host.ts: cabeçalho e o tipo que revela o desenho

```
/**
 * I/O multi-host para agent hooks (Claude Code · Cursor · Codex).
 * Scripts vivem em tooling/agent-hooks/; cada host só aponta via seu
 * JSON.
 */
export type AgentHost = "claude" | "cursor";
```

Três hosts, dois valores no tipo. O comentário dentro de `detectHost` explica: *"Claude Code e Codex compartilham o contrato `hookSpecificOutput`".* São dois contratos de I/O, não três, e o tipo não finge o contrário.

A detecção do host tem três degraus, nessa ordem:

1. `CURSOR_VERSION` ou `CURSOR_PROJECT_DIR` no ambiente;
2. `AGENT_HOOK_SURFACE === "cursor"`; escape manual, usado pelos testes;
3. heurística de *casing*: se `hook_event_name` começa com letra minúscula, é Cursor.

O terceiro degrau é o detalhe elegante: a diferença de naming dos eventos (`sessionStart` contra `SessionStart`) é o detector. Nenhuma tabela de eventos precisa ser mantida em dois lugares.

Emissores do `host.ts`: um por veredito

Função	O que sai em cada contrato
<code>emitAdditionalContext</code>	Cursor: <code>additional_context</code> ; Claude/Codex: <code>hookSpecificOutput.additionalContext</code>
<code>emitDeny</code>	cursor: <code>permission: "deny" + user_message/agent_message</code> ; claude/codex: <code>permissionDecision: "deny"</code>
<code>emitAllow</code>	cursor: <code>permission: "allow"</code> ; claude/codex: exit 0 silencioso
<code>emitStopOk</code>	<code>"{}"</code> nos dois
<code>emitStopContinue</code>	Cursor: <code>followup_message</code> ; Claude/Codex: exit 2 + <code>stderr</code>

Todos os emissores têm tipo de retorno `never`; terminam em `process.exit`. É por isso que cada hook é uma cadeia linear de *guard clauses* sem uma única variável de estado: não existe "continuar depois de decidir".

O `host.ts` também esconde as normalizações chatas. `readShellCommand` sabe que *"Cursor manda command no topo; Claude/Codex em `tool_input.command`".* `readEditedFilePath` tenta `raw.file_path`, `tool_input.file_path`, `tool_input.path` e, em último caso, faz parse do próprio patch:

tooling/agent-hooks/host.ts: os dois formatos de patch reconhecidos

```
const updateMatch = command.match(/.*\s+(?:Update|Add)\s+File:\s+
  +(.+)/);
...
const plusMatch = command.match(/^.*\s+[ab]\/(.+)\$/m);
```

O primeiro é o formato `apply_patch` do Codex; o segundo, `diff` unificado.

POR QUE | Por que hooks compartilhados, e não um `.sh` por host (ADR-016)

"Antes os hooks eram `.sh` dentro de `.claude/`, então Codex e Cursor rodavam sem guardrail nenhum; a regra existia só onde o `.md` era lido. Script único + JSON fino elimina a divergência entre hosts por construção, e `hook-configs.test.ts` reprova o gate se um host apontar para um script que não existe ou que estoura."

Alternativas descartadas, registradas no mesmo ADR: caminho relativo quebra quando o agente roda a partir de um subdiretório; duplicar o script por host garante drift.

11.3 O prefixo que todos os 21 comandos compartilham

Sete hooks vezes três configs dá 21 invocações. Todas as 21 têm exatamente a mesma forma:

A resolução de caminho usada em todo hook, nos três hosts

```
SCRIPT=guard-shell.ts
bun "$(git rev-parse --show-toplevel 2>/dev/null || pwd)/tooling/agent-
  hooks/$SCRIPT"
```

São duas propriedades embutidas numa linha. O `--show-toplevel` faz o hook funcionar quando o agente é aberto num **subdiretório** do repositório. O `2>/dev/null || pwd` faz funcionar **sem repositório git nenhum**.

POR QUE | O fallback || `pwd` não é paranoia

O comentário do teste diz o cenário exato: "o comprador extrai o zip e abre o agente ANTES de `git init`, e sem o fallback todo hook morreria no `git rev-parse`".

A mesma hipótese aparece em outros dois lugares do tooling: o ADR-020 registra que `tooling/no-lint-suppression.ts` "usa `git grep` quando há repositório e cai para `grep -r` quando ainda não há", e o `guard-shell.ts` deixa `git commit/push` passarem fora de repositório git pela mesma razão. Não é um workaround solto: é uma premissa de produto aplicada de forma consistente.

11.4 Os quatro hooks que só falam

11.4.1 inject-patterns.ts: SessionStart

Injeta um bloco de convenções no contexto assim que a sessão abre. O corpo executável do script tem duas linhas (`readHookInput + emitAdditionalContext`); o resto das 97 linhas é o `import` e a constante `PATTERNS`.

O cabeçalho declara a doutrina do arquivo:

tooling/agent-hooks/inject-patterns.ts

```
/**
 * SessionStart / sessionStart – injeta as convenções do repo no
 * contexto do agente
 * já no início da sessão, pra ele escrever conforme os patterns ANTES
 * de qualquer
 * write (reforço proativo, sem bloquear).
 *
 * REGRAS, NÃO OPÇÕES: cada linha é uma diretiva única. Não oferecer
 * alternativas
 * (menu vira alucinação): sempre o caminho prescrito.
 *
 * Fonte canônica dos guardrails: AGENTS.md. Este texto é o resumo
 * carregado por
 * sessão; quando divergir, o AGENTS.md vence.
 */
```

O bloco injetado se chama # boilerplate: convenções (REGRAS, não opções) e cobre: stack fixa, runtime 100% Bun, dados (RSC, proibição de cache), auth (cookie JWT, [src/proxy.ts](#)), arquitetura (app → features → lib → shared → externo), bordas (valibot/ky/apiClient), extração e abstração, qualidade e commit/push/PR. A última linha termina em As regras acima são impostas por oxlint., e é literalmente verdade, o que é o ponto do produto.

"Menu vira alucinação" merece parágrafo próprio: o texto nunca oferece duas formas de fazer a mesma coisa. Um agente que recebe alternativas escolhe, e escolher é onde ele erra.

11.4.2 reinforce-patterns.ts: UserPromptSubmit

22 linhas. Reinjeta um bloco menor a **cada turno**. O título do bloco define o recorte com precisão cirúrgica:

tooling/agent-hooks/reinforce-patterns.ts: o título é a especificação

```
# Reforço por turno – REGRA, não opção (julgamento que o lint NÃO pega)
```

Ou seja: o que já é imposto por oxlint não entra aqui. O que entra é julgamento; "faça a menor mudança de boundary que resolve o requisito", "um caller fica inline; extração local comum começa no 2º caller real", "abstração genérica reutilizável exige pelo menos 3 callers/usos concretos", "nunca suprima lint por comentário/diretiva nem desative regra

pra passar o gate: corrija a causa". O hook passa `{ continue: true }` como extra: nunca bloqueia o prompt do usuário.

11.4.3 `reinforce-after-write.ts`: `PostToolUse`

42 linhas, e a única lógica de verdade é a filtragem: o lembrete muda conforme a extensão do arquivo que acabou de ser escrito.

tooling/agent-hooks/reinforce-after-write.ts: o filtro

```
const path = readEditedFilePath(raw);
const isSource = /^(^|[/\])src[/\]/.test(path) && !/\.test\.tsx?$/.test(path);
const isTsx = path.endsWith(".tsx");
const isTs = path.endsWith(".ts");

if (!isSource || (!isTs && !isTsx)) {
  process.exit(0);
}
```

Só dispara em `src/`, e ignora `*.test.ts(x)`. Emite um bloco CORE (comum aos dois) mais TS_ONLY ou TSX_ONLY:

O que muda entre lógica e componente

Bloco	Conteúdo
CORE	Sem fallback defensivo para caso que o tipo já impede; erro do backend não vira workaround no front; YAGNI (nada de wrapper/helper com 1 uso); nunca suprimir lint; em dúvida, PERGUNTE.
TS_ONLY	Schema sem regra de negócio inventada (<code>minValue(5)</code> citado como exemplo de limiar mágico); sem wrapper que só repassa para o <code>apiClient</code> ; sem helper cerimonial (<code>isRecord</code> , <code>ensureArray</code>); nome concreto.
TSX_ONLY	<code>.tsx</code> é UI fina (SRP); sem valor hardcoded (<code>cor/px/raio/sombra/fonte</code> sai de token do DESIGN.md); <code>watch()/form.watch()</code> proibido no corpo (use <code>useWatch</code>); acessibilidade é "done"; mídia raster via <code>next/image</code> .

A mensagem montada abre com `# Você acabou de escrever um componente (.tsx); revise antes de seguir ou a variante lógica (.ts)`. É reforço no momento exato em que o erro acabou de ser cometido, não no início da sessão.

11.4.4 `format-after-write.ts`: `PostToolUse / afterFileEdit`

22 linhas, empatado com o `reinforce-patterns.ts` como o hook mais curto do repositório. O núcleo é

`tooling/agent-hooks/format-after-write.ts`

```
const FORMATTABLE = /\.?(ts|tsx|js|jsx|mjs|cjs)$/;
...
Bun.spawnSync(["bunx", "oxfmt", filePath], { stdout: "ignore", stderr:
  "ignore" });
process.exit(0);
```

Roda `oxfmt` no arquivo específico, com `stdout` e `stderr` silenciados, e nunca bloqueia (o cabeçalho declara "Nunca bloqueia (`exit 0`)"). Markdown fica de fora **de propósito**.

POR QUE | Por que o formatter ignora `**/*.md` (ADR-018)

"Os docs canônicos são prosa mantida à mão; reformatá-los é churn sem valor e polui o diff da review." O mesmo ADR confirma os três pontos onde o `oxfmt` roda: no `PostToolUse` dos agentes, no pré-commit e no `verify` (via `fmt:check`). O hook é a primeira das três redes, não a única.

11.5 Os três hooks que negam

11.5.1 `verify-before-stop.ts`: o agente não diz "pronto" com o gate vermelho

26 linhas. Essa é a lógica inteira:

`tooling/agent-hooks/verify-before-stop.ts`

```
const { host, raw } = await readHookInput();

if (isStopHookActive(raw) || raw.status === "aborted") {
  emitStopOk();
}

const verification = Bun.spawnSync(["bun", "run", "verify:fast"], {
  cwd: process.cwd(),
  stdout: "inherit",
  stderr: "inherit",
});

if (verification.exitCode !== 0) {
  emitStopContinue(
    host,
    "Stop bloqueado: `bun run verify:fast` falhou. Corrija os erros do
    gate e só então encerre o turno.",
  );
}

emitStopOk();
```

O que ele roda está no `package.json`:

```
package.json: os dois gates
```

```
"verify:fast": "bun run lint && bun run fmt:check && bun run knip &&
  bun run test",
"verify": "bun run lint && bun run fmt:check && bun run typecheck &&
  bun run typecheck:tooling && bun run knip && bun run test && bun
  tooling/no-lint-suppression.ts && bun tooling/no-heavy-public-assets.
  ts"
```

O Stop roda a versão *fast*: sem tsc, sem os dois scripts extras. É a escolha certa; typecheck é a parte cara, e ela ainda é cobrada no pre-push do lefthook, que roda bun run verify. O timeout de 5 minutos do Claude existe justamente porque o hook roda um gate real, não um check simbólico.

A guarda anti-loop é dupla e cada host usa a sua: stop_hook_active (Claude/Codex) e status === "aborted" saem cedo com {}; no Cursor, o loop_limit: 3 do JSON faz o trabalho.

O bloqueio em si é a única parte que diverge por host:

```
tooling/agent-hooks/host.ts: emitStopContinue
```

```
if (host === "cursor") {
  process.stdout.write(JSON.stringify({ followup_message: reason }) + "
  \n");
  process.exit(0);
}
// Claude: exit 2 bloqueia. Codex: exit 2 + stderr continua o turno.
process.stderr.write("Stop blocked: bun run verify:fast failed.\n");
process.exit(2);
```

O `verify-before-stop.test.ts` cobre os quatro casos: pula quando stop_hook_active: true (e verifica, por arquivo sentinela, que o verify **não** rodou), permite quando o exit é 0, bloqueia com exit 2 + stderr quando é 1, e re-roda numa continuação do Cursor ({ hook_event_name: "stop", loop_count: 1 }).

11.5.2 guard-shell.ts: destrutivo e branch protegida no mesmo hook

137 linhas, duas famílias. O cabeçalho justifica a junção antes que você pergunte:

tooling/agent-hooks/guard-shell.ts

```
/**
 * PreToolUse Bash / beforeShellExecution – guarda única de shell.
 *
 * Duas famílias, na ordem: comando destrutivo/perigoso (irreversível
 * ou vazando
 * segredo) e commit/push em branch protegida. Um hook só porque é o
 * mesmo evento
 * e o mesmo parsing; separar em dois seria duplicar leitura de stdin
 * por comando.
 */
```

Família 1; destrutivo e vazamento. A lista é literal e curta:

tooling/agent-hooks/guard-shell.ts: DESTRUCTIVE

```
const DESTRUCTIVE: ReadonlyArray<{ pattern: RegExp; label: string }> =
[
  {
    pattern: /\brm\s+(?:-[a-zA-Z]+\s+)*(?:\./|~|\\*)\\/(?:(?:\s|$)/,
    label: "rm no diretório raiz/home",
  },
  { pattern: /\bgit\s+reset\s+--hard\b/, label: "git reset --hard" },
  { pattern: /\bgit\s+clean\s+--[a-zA-Z]*f/, label: "git clean -f" },
  { pattern: /(?:^|[\|;&]\s*)sudo\s/, label: "sudo" },
  { pattern: /\bchmod\s+(?:-[a-zA-Z]+\s+)*777\b/, label: "chmod 777" },
];
```

Mais três checagens que ficam fora da lista porque têm exceção:

As três checagens com exceção deliberada

Checagem	Regra
READS_REAL_ENV	Bloqueia cat/less/more/head/tail/bat sobre .env; .env.example é liberado por lookahead negativo ("template versionado").
UNSAFE_FORCE_PUSH	Bloqueia git push --force e -f, mas SAFE_FORCE_FLAGS libera --force-with-lease e --force-if-includes. A mensagem de deny já diz qual usar.
PIPE_TO_SHELL	Bloqueia curl/wget canalizado para bash/sh/zsh. "Baixe, leia e só então execute."

O comentário do arquivo é honesto sobre o alcance: "O objetivo é barrar o caminho óbvio, não construir um sandbox; quem quer burlar consegue, mas o agente não erra por descuido." Isso é o escopo correto. Um hook não é um jail.

Família 2; branch protegida. `const PROTECTED = ["main", "release", "staging", "develop"]`; O hook bloqueia em dois casos: a branch atual está na lista, **ou** o git push tem um nome protegido nos argumentos; git push origin HEAD:main é pego, e existe teste explícito para isso.

Dois detalhes de implementação que valem leitura:

```
tooling/agent-hooks/guard-shell.ts: currentBranch()

/**
 * Fora de um repositório git não existe branch a proteger – `git
 * commit`/`push`
 * já falhariam sozinhos. Deixa passar em vez de estourar (o comprador
 * roda o
 * agente antes do `git init`).
 */
function currentBranch(): string {
  try {
    // `branch --show-current` (não `rev-parse --abbrev-ref HEAD`)
    porque funciona
    // em branch órfã: no repo recém-inicializado, o PRIMEIRO commit
    ainda cai na
    // `main` e precisa ser barrado – e ali o `rev-parse` falha.
  }
}
```

O caso da branch órfã é exatamente o que uma implementação apressada perde: o primeiro commit do comprador cai na main, e é o commit que mais interessa barrar.

REGRA | JSON inválido nega, não libera

Se a entrada do hook não parseia, os dois guards de shell **negam**: "Comando bloqueado: o shell guard recebeu JSON inválido..." no `guard-shell.ts` e "PR travada: o hook recebeu JSON inválido..." no `pre-pr-review-gate.ts`. Existe teste com esse nome; denies malformed JSON instead of allowing it. Os hooks informativos, no mesmo caso, saem com exit 0. Fail-closed onde bloquear importa, fail-open onde só se perde um lembrete.

11.5.3 pre-pr-review-gate.ts: o portão do gh pr create

118 linhas. Só age quando o comando casa `gh pr create`; qualquer outro vai direto para `emitAllow`. Duas constantes governam o hook:

```
tooling/agent-hooks/pre-pr-review-gate.ts

const MAX_PR_LOC = 3000;

/** Base default de PR. Alinhada à seção "Branch base" do CLAUDE.md. */
const BASE_BRANCH = "main";
```

São quatro portas, em ordem:

1. **Base válida.** Aceita `--base main`, `-B main` ou qualquer `feature/*` (PR empilhado). Qualquer outra base nega: PR travada: a base informada foi ``develop``, mas PR deste repositório entra em ``main`` ou empilha numa ``feature/*``. Se há flag de base mas o parser não extrai valor, também nega.
2. **HEAD identificável.** `git rev-parse HEAD`; se falha, nega. Fora de repositório git, `gh pr create` é bloqueado com segurança, e sem escrever em `stderr`, o que o teste assevera.
3. **Selo de pré-review.** `existsSync('.agents/tmp/pre-review-<HEAD>.ok')`.
4. **Teto de tamanho.** Se não existe `.agents/tmp/pr-size-ack-<HEAD>.ok`, roda `git diff --numstat origin/<base>...HEAD`, soma adições + deleções (linha binária - conta 0) e nega acima de 3000.

As duas mensagens de deny são instruções acionáveis, não reclamações:

Deny da porta 3: selo ausente

```
PR travada: a pré-review de guardrails não rodou pro commit atual (${
  head.slice(0, 8)}).
Rode a skill `pre-review` (`/pre-review`) primeiro. Ela grava o selo
que libera o gh pr create.
Qualquer commit novo depois disso exige nova pré-review.
```

Deny da porta 4: teto de LOC, com a saída explícita

```
`PR grande: ${loc} linhas (add+del) vs \`${base}\`, acima do teto de ${
  MAX_PR_LOC}. ` +
`Antes de abrir, pergunte ao usuário: dividir em PRs menores (
  recomendado) vs seguir. ` +
`Se mandar seguir, grave o ack e reexecute:\n` +
`  git rev-parse HEAD > ${sizeAck}\n` +
`0 ack vale só pra este commit.`
```

Quem grava os selos é a skill `.agents/skills/pre-review/SKILL.md`:

Os dois selos gravados pela skill pré-review

```
git rev-parse HEAD > .agents/tmp/pre-review-$(git rev-parse HEAD).ok
git rev-parse HEAD > .agents/tmp/pr-size-ack-$(git rev-parse HEAD).ok
```

`.agents/tmp/` está no `.gitignore`: o selo é local e não viaja no repositório. E como o nome do arquivo carrega o SHA do HEAD, **qualquer commit novo invalida o selo**. Não existe "revisei ontem".

POR QUE | Por que a pré-review precisa de portão (ADR-021)

"pr-ready é mecânico (lint/type/test/diff/secrets) e não pega o que custa caro na review humana: abstração especulativa, fallback defensivo, regra de negócio inventada no front, slop. Sem gate, a etapa vira opcional, e regra que dá para pular não é guardrail."

A cadeia oficial é /pr-ready (mecânico) → /pre-review (julgamento por subagentes, grava o selo) → /create-pr. O ADR fecha com "não se sobrepõem".

11.6 Falso-positivo é bug de guardrail

Um guardrail que bloqueia comando legítimo é abandonado, e um guardrail abandonado não protege nada. Duas defesas no repositório existem só por causa disso, e as duas estão documentadas em teste.

A primeira é a âncora do `rm -rf`. O comentário do arquivo:

tooling/agent-hooks/guard-shell.ts: por que a regex é ancorada no alvo

```
* `rm -rf` é ancorado no ALVO, não em substring: `rm -rf /` casaria dentro de
* `rm -rf /home/user/projeto/dist`, bloqueando um comando legítimo. Só alvo raiz
* (`/`), home (`~`) e o coringa (`*`) – sozinhos ou com barra final – entram.
```

A segunda é o `stripQuotedHeredocBodies`, aplicado em *todo* `readShellCommand`:

tooling/agent-hooks/host.ts: heredoc literal é dado, não comando

```
/**
 * Remove o corpo dos heredocs com delimitador entre aspas (`<<'EOF' ... EOF`),
 * que o shell trata como dado literal, não como comando.
 *
 * Sem isso, escrever um arquivo cujo conteúdo MENCIONA um comando faz os guards
 * dispararem: um doc que explica `gh pr create` travaria a própria escrita do
 * doc. Só o delimitador entre aspas é removido – heredoc sem aspas expande
 * variáveis e substituição de comando, então ali ainda pode haver execução real.
 */
function stripQuotedHeredocBodies(command: string): string {
  return command.replace(
    /<<-?\s*(['"])([A-Za-z_][A-Za-z0-9_]*)\1[\s\S]*?^\s*\2\s*$/gm,
    "<<HEREDOC",
  );
}
```

A distinção final é o que separa isso de um `replace` ingênuo: só o heredoc com delimitador **entre aspas** é neutralizado. Sem aspas o shell expande variáveis e substituição de comando, então ali ainda pode haver execução real, e o guard continua olhando.

O `guard-shell.test.ts` transforma os dois casos em teste de *allow*, com o comentário preservado:

tooling/agent-hooks/guard-shell.test.ts: os allows que documentam os falsos-positivos

```
test.each([
  // O bug que o `rm -rf /` como substring criava: qualquer caminho
  // absoluto casava.
  "rm -rf /home/user/projeto/dist",
  "rm -rf ./node_modules",
  // Heredoc literal é dado: documentar `sudo rm -rf /` não é executá-
  // lo.
  "cat > doc.md <<'EOF'\nNunca rode sudo rm -rf / no servidor.\nEOF",
  "git push --force-with-lease origin feature/x",
  "cat .env.example",
  "bun run verify",
])("allows %s", (command) => {
```

O `pre-pr-review-gate.test.ts` repete a defesa do heredoc no seu próprio domínio, com o teste ignora o comando citado dentro de um heredoc literal: escrever um doc que *explica* `gh pr create` não pode travar na escrita do documento.

CUIDADO | Se você endurecer um padrão, escreva o allow junto

Cada linha desse `test.each` de *allow* é um bug que já existiu. Ao adicionar um padrão novo em **DESTRUCTIVE**, adicione no mesmo commit o caso legítimo que ele *não* pode pegar. Sem isso, o próximo endurecimento reintroduz o falso-positivo e alguém vai desligar o hook em vez de consertá-lo.

11.7 O bloco `permissions`: a camada grosseira do Claude

O `.claude/settings.json` tem um bloco `permissions` que não existe nos configs do Codex e do Cursor; é recurso nativo do host, não do boilerplate.

O *allow* tem 23 entradas e serve para não gerar prompt de permissão no que o agente faz o tempo todo: as tools de leitura (Read, Glob, Grep), git **só de leitura/navegação** (`git status*`, `git diff*`, `git log*`, `git branch*`, `git checkout*`), os scripts do *gate* (`bun run lint*`, `bun run fmt*`, `bun run knip*`, `bun run test*`, `bun test*`, `bun run typecheck*`, `bun run verify*`, `bun run build*`, `bun install*`, `bun run dev*`, `bun run start*`) e utilitários (`ls`, `wc`, `gh`, `bunx`).

Repare no que **não** está lá: nenhum `git commit`, `git push` ou `git merge`. Esses caem no fluxo normal de permissão e ainda passam pelo `guard-shell.ts`.

O *deny* tem oito entradas:

```
.claude/settings.json: permissions.deny

"Bash(rm -rf /)", "Bash(rm -rf ~)", "Bash(git push -f *)", "Bash(git
reset --hard*)",
"Bash(git clean -f*)", "Bash(sudo *)", "Bash(chmod 777*)", "Bash(cat .
env*)"
```

POR QUE | Por que isso não é duplicação inútil do `guard-shell.ts`

É uma sobreposição **parcial e mais grosseira**; a camada que existe mesmo se o hook não rodar. E a comparação mostra exatamente o que o hook agrega:

- o deny nativo bloqueia `git push -f *`; o hook é quem sabe distinguir `--force-with-lease`;
- o deny bloqueia `cat .env*`, incluindo `.env.example`; o hook libera `.env.example` de propósito;
- `Bash(rm -rf /)` como glob é literalmente a forma ingênua que a regex ancorada do hook teve de refinar.

O mesmo arquivo tem um bloco env com duas opções: `ENABLE_TOOL_SEARCH: "true"` e `CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING: "1"`.

11.8 `hook-configs.test.ts`: o teste que amarra os três configs

84 linhas, quatro testes, varrendo `[".claude/settings.json", ".codex/hooks.json", ".cursor/hooks.json"]` com `test.each`. O `collectCommands` percorre o JSON **recursivamente** coletando toda chave `command` que seja string, por isso funciona igual nos três formatos, apesar de o aninhamento ser diferente em cada um.

Os quatro testes e o que cada um impede

Teste	Impede
aponta para os scripts compartilhados	Comando sem o prefixo canônico, ou na forma relativa (<code>bun tooling/agent-hooks/</code>) que quebraria em subdiretório.
só referencia hooks que existem e rodam	Config apontando para script inexistente, ou script que estoura com entrada mínima.
a skill pre-review é descobrível	<code>.agents/skills/pre-review/SKILL.md</code> sumir; o hook manda rodar <code>/pre-review</code> , o teste garante que ela existe onde os três hosts a encontram.
toda superfície de enforcement tem dono no CODEOWNERS	<code>/.agents/**</code> , <code>/.claude/**</code> , <code>/.codex/**</code> , <code>/.cursor/**</code> e <code>/tooling/**</code> sem owner.

O segundo teste é o mais interessante porque **executa cada hook de verdade**:

tooling/agent-hooks/hook-configs.test.ts: o smoke test

```
// Entrada mínima é o pior caso do host: nenhum hook pode estourar por
// isso.
// `stop_hook_active` faz o verify-before-stop sair cedo – rodar o gate
// inteiro aqui seria testar o `verify`, não o wiring (ele tem teste pr
// óprio).
const result = Bun.spawnSync(["bun", scriptPath], {
  cwd: join(ROOT, "src"),
  stdin: Buffer.from(JSON.stringify({ stop_hook_active: true })),
  ...
  env: {
    ...process.env,
    ...(configPath === ".cursor/hooks.json" ? { AGENT_HOOK_SURFACE: "
    cursor" } : {}),
  },
});
expect(result.exitCode).toBe(0);
```

Dois detalhes que não são acidente: o cwd é ROOT/src; **de propósito, um sub-diretório**, que é justamente o cenário em que o caminho relativo quebraria; e o AGENT_HOOK_SURFACE=cursor é forçado quando o config é o do Cursor, exercitando o outro ramo do [host.ts](#). O teste não verifica só que o arquivo existe: verifica que ele roda, nos dois contratos, a partir de onde o agente realmente costuma estar.

Os quatro hooks informativos ([inject-patterns.ts](#), [reinforce-patterns.ts](#), [reinforce-after-write.ts](#), [format-after-write.ts](#)) não têm suíte dedicada; são cobertos por esse smoke test e mais nada. É uma escolha proporcional ao risco: o pior que um deles faz ao errar é deixar de lembrar o agente de algo.

11.9 Desligar um hook: o mecanismo e o preço

REGRA | Não existe kill switch

Nenhum dos sete scripts tem flag de bypass, variável de ambiente de desligamento ou modo "skip". As únicas variáveis de ambiente que os hooks leem são CURSOR_VERSION, CURSOR_PROJECT_DIR e AGENT_HOOK_SURFACE, e as três são de **detecção de host**, não de bypass. Nenhum documento do repositório ensina a desligar hook, porque a alavanca é só uma.

A alavanca é o config do host: remover (ou comentar) a entrada do evento no JSON. Quer desligar só num host? Edita só aquele JSON. Quer nos três? Edita os três; é o preço simétrico de o desenho ser multi-host por construção.

E existe um custo mecânico, não retórico. O [hook-configs.test.ts](#) exige, por config:

tooling/agent-hooks/hook-configs.test.ts: o piso

```
expect(commands.length).toBeGreaterThan(0);
...
expect(scripts.size).toBeGreaterThan(0);
```

Ou seja: **esvaziar um dos três configs reprova bun test**; que está dentro do `verify:fast` (rodado pelo `Stop`) e dentro do `verify` (rodado no `pre-push` do `lefthook`). Desligar *um* hook individual não quebra o teste; desligar *todos* de um host, sim.

O que você perde em cada caso, deduzido do que cada hook faz:

Custo de desligar cada hook

Hook desligado	O que passa a acontecer
<code>verify-before-stop.ts</code>	O agente pode encerrar o turno com <code>lint/fmt/knip/test</code> vermelhos. O <code>verify</code> completo ainda pega no <code>pre-push</code> ; mais tarde e mais caro.
<code>guard-shell.ts</code>	Sobra a <code>deny list</code> do Claude, e nada em Codex/Cursor. Some a proteção de branch (<code>main/release/staging/develop</code>), incluindo o caso da branch órfã no primeiro commit.
<code>pre-pr-review-gate.ts</code>	<code>/pre-review</code> vira opcional; somem o teto de 3000 LOC e a validação da base do PR. O ADR-021 antecipa isso: <i>"regra que dá para pular não é guardrail"</i> .
<code>inject-patterns.ts</code> e os dois <code>reinforce</code>	As regras continuam nos <code>.md</code> , mas dependem de o agente ter lido e mantido em contexto. É exatamente o cenário pré-ADR-016: <i>"a regra existia só onde o .md era lido"</i> .
<code>format-after-write.ts</code>	Diff sujo até o <code>fmt:check</code> do <code>verify</code> ou o <code>oxfmt --check</code> do pré-commit reprovar.

CUIDADO | Desligar o hook não desliga a regra

Todos os cinco casos acima têm em comum: a regra continua existindo e continua sendo cobrada, só que **mais tarde no ciclo**. Você não removeu o gate; moveu o custo do turno atual para o `pre-push`, ou para a review humana, ou para produção. Se um hook está atrapalhando, o conserto quase sempre é refinar o padrão (como a âncora do `rm -rf` e o `stripQuotedHeredocBodies` fizeram), não remover a entrada do JSON.

11.10 Onde os hooks aparecem no resto da documentação

Referências cruzadas

Arquivo	O que diz
README.md	Resumo em uma linha: <code>SessionStart</code> injeta as regras; <code>UserPromptSubmit</code> reforça a cada turno; <code>PostToolUse</code> formata com <code>oxfmt</code> e relembra o que o lint não pega; <code>PreToolUse</code> bloqueia comando destrutivo, <code>commit/push</code> em branch protegida e abertura de PR sem pré-review; <code>Stop</code> impede "concluído" com o gate vermelho.
CLAUDE.md	A seção Branch base diz que as skills <code>pr-ready/pre-review/pr-creation</code> e os hooks guard-shell.ts / pre-pr-review-gate.ts leem dali. A seção Multi-host encerra com <i>"Guardrail não é privilégio de um editor."</i>
AGENTS.md	<i>"Nunca commite nem pushe direto na main; o hook guard-shell bloqueia."</i> E, nos comandos: <code>verify:fast</code> "é o que o hook de Stop roda".
ARCHITECTURE.md	tooling/agent-hooks/ listado como hooks de agente compartilhados (Claude Code, Codex, Cursor).
MEMORY.md	ADR-016 (hooks compartilhados e fallback de caminho), ADR-018 (<code>oxfmt</code> no <code>PostToolUse</code>), ADR-021 (selo por HEAD como portão do PR).

Se você mudar a branch base na seção **Branch base** do [CLAUDE.md](#), lembre que `BASE_BRANCH` no [pre-pr-review-gate.ts](#) e `PROTECTED` no [guard-shell.ts](#) são constantes independentes; o comentário do código aponta para o [CLAUDE.md](#), mas nada sincroniza os dois automaticamente.

PARTE 12

Integrações de terceiros

O QUE VOCÊ LEVA DESTE CAPÍTULO

Você vai decidir se CodeRabbit, GitNexus ou impeccable resolvem uma necessidade real no seu fluxo. As três integrações são opcionais e entram sem virar dependência do gate principal.

O boilerplate é auto-suficiente: `bun run lint`, `bun run typecheck`, `bun test` e as skills do kit não dependem de nenhum serviço externo. As três ferramentas desta seção são opções para ampliar o fluxo, não peças escondidas do produto. Cada uma tem sua própria documentação em [integracoes/](#).

Ferramentas de terceiros (opcionais, ensinadas em [integracoes/](#)): **gitnexus** (grafo do código), **impeccable** (qualidade de UI a partir de [DESIGN.md](#)) e **CodeRabbit** (review de PR por IA).

O diretório tem exatamente cinco arquivos:

Conteúdo real de `integracoes/`

```
integracoes/gitnexus.md
integracoes/impeccable.md
integracoes/coderabbit/README.md
integracoes/coderabbit/pr-review.md
integracoes/coderabbit/cr-comments.ts
```

Não há um [integracoes/README.md](#) de índice: o único README é o do CodeRabbit. Os arquivos `.md` explicam instalação e disciplina; não são instaladores. Nenhuma das três ferramentas vem instalada ou participa do gate principal. Você adota apenas a que resolver um problema real no seu fluxo.

REGRA | As três são opcionais de verdade

Nenhum gate do repo depende delas. Você pode nunca instalar nenhuma e o método continua fechado: [pr-ready](#) cobre o gate mecânico, [pre-review](#) o julgamento, [feature-audit](#) a auditoria semântica, e [design-system](#) os princípios visuais. As integrações **potencializam**, não sustentam.

12.1 Mapa: o que já vem pronto e o que é opt-in

Estado real de cada peça de integração no repo

Item	Estado verificado
integracoes/*.md (3 docs)	Presentes. São documentação; nenhuma ferramenta vem instalada
integracoes/coderabbit/cr-comments.ts	Presente e dentro dos gates do repo (tsconfig, oxlint, knip)
/pr-review (CodeRabbit, phase-aware)	Opt-in: mora em integracoes/coderabbit/pr-review.md , precisa ser copiado
/review-pr (genérico, 4 fases)	Default: .claude/commands/review-pr.md já instalado
Skill pr-review-response	Default: já instalada em .claude/skills/
Fase 0 de impact pre-flight (GitNexus)	Opcional , marcada como tal em feature-audit, workflow-refactor, /audit, /review, /refactor
Skill gitnexus-impact-analysis	Não está em .claude/skills/ : vem com o produto de terceiros
Bloco GitNexus no CLAUDE.md	Slot comentado e vazio (linha 65)
.gitnexus/ no .gitignore	Não existe: o doc manda adicionar à mão
.impeccable/ no .gitignore	Presente
PRODUCT.md / DESIGN.md	Presentes como template , para o /impeccable init preencher
ADR sobre integrações no MEMORY.md	Não existe
Hook / lefthook / CI citando as três	Não existe

12.2 CodeRabbit: review de PR por IA

12.2.1 O que é e por que entra no método

CodeRabbit (<https://coderabbit.ai>) é um reviewer de pull request por IA. Instalado como GitHub App, ele analisa o diff a cada abertura ou atualização de PR e posta comentários inline (threads) mais um review-resumo com nitpicks, cada um com severidade e sugestão de fix.

O [integracoes/coderabbit/README.md](#) não vende a ferramenta: ele define o *postura* com que ela deve ser usada. A tese central, verbatim:

Review por IA acelera o loop, mas **feedback de IA é claim, não verdade**: o CodeRabbit erra, sugere o que as nossas regras proíbem (check defensivo, fallback silencioso, abstração especulativa) e comenta sobre código que já mudou. O método não trata o bot como autoridade: ele entra como uma primeira passada barata, e o julgamento fica com você.

POR QUE | Por que um bot de review precisa de um filtro escrito

O boilerplate proíbe coisas que a maioria dos linters e reviewers genéricos *recomenda*: check defensivo, fallback silencioso, abstração especulativa, wrapper que só repassa, validação de regra de negócio no front. Sem um filtro explícito, aplicar as sugestões do bot em massa **desfaz** as regras que o resto do kit impõe. O documento existe para que a integração não vire um vetor de erosão da arquitetura.

12.2.2 O filtro de 3 eixos

É a peça mais importante da integração. Está resumida no README e escrita de forma normativa no comando [integracoes/coderabbit/pr-review.md](#). Antes de tudo vem uma pré-condição: ler o **código real no HEAD atual**, sem confiar na linha citada no comentário (ela envelhece). Depois, para cada comentário ou nitpick, nesta ordem:

1. **Ainda se aplica?** Se o ponto já não existe no código, descarte.
2. **Bate com as nossas regras?** Cruze com [AGENTS.md](#) / [CODE-PATTERN.md](#). O CodeRabbit às vezes sugere o que a gente **proíbe**: check defensivo, fallback ?? ' ', abstração especulativa, wrapper que só repassa, validação de regra de negócio (isso é do backend). Se contradiz uma regra nossa, o veredito é pular, citando a regra.
3. **Está correto?** O CodeRabbit alucina. Verifique o mérito técnico contra o código, não contra a autoridade do bot. Quando der, **prove** (teste rápido) antes de aplicar.

REGRA | Um “pular” bem justificado vale mais que aplicar tudo

A frase aparece nos dois documentos (README e comando). Ela existe para desarmar a métrica errada: taxa de aplicação de sugestões não é sinal de qualidade. O veredito “pular, porque contradiz a regra X” é uma decisão de engenharia; “aplicar porque o bot pediu” não é.

12.2.3 Como plugar

O README descreve quatro passos:

1. **Instalar o GitHub App** do CodeRabbit no repo, pela fonte oficial. A partir daí ele roda automático em todo PR.
2. Quando os comentários chegarem, rodar `/pr-review <número-do-PR>`. Esse comando é a cópia de-brandada em [integracoes/coderabbit/pr-review.md](#): instale-o como [.claude/commands/pr-review.md](#).
3. Os dados vêm do script `integracoes/coderabbit/cr-comments.ts <pr>`, não de scraping do GitHub pelo LLM.

4. O julgamento é conduzido pela skill `pr-review-response`, que já vem no kit. Nas palavras do README: “O comando `/pr-review` é a porta de entrada; a skill é o método.”

Instalação do comando `opt-in`

```
cp integracoes/coderabbit/pr-review.md .claude/commands/pr-review.md
```

CUIDADO | `/pr-review` e `/review-pr` são comandos diferentes

O repo já traz `.claude/commands/review-pr.md` instalado: treze linhas, genérico, apenas ativa a skill `pr-review-response` e delega as quatro fases dela. Ele **não** tem as fases TRIAR/RESOLVER/AGUARDANDO nem usa o `cr-comments.ts`.

O `/pr-review` (com hífen no meio invertido) é o específico de CodeRabbit e **não existe** em `.claude/commands/` até você copiar. Os nomes são quase idênticos e é fácil digitar um achando que está chamando o outro. Se você espera as fases e não as vê, provavelmente chamou o default.

12.2.4 As três fases do `/pr-review`

O comando é *phase-aware* porque o CodeRabbit leva minutos para reanalisar um commit: fechar o loop é uma re-execução, não um passo bloqueante. O frontmatter do arquivo declara exatamente as ferramentas que ele pode usar:

Frontmatter de `integracoes/coderabbit/pr-review.md`

```
---
description: Processa o review do CodeRabbit num PR (triar, responder,
  resolver) guiado e phase-aware
argument-hint: "[número-do-PR]"
allowed-tools: Bash(bun integracoes/coderabbit/cr-comments.ts:*), Bash(
  gh:*), Bash(git:*), Bash(bun run:*), Bash(bun test:*), Read, Edit,
  Write, Grep, Glob, AskUserQuestion
---
```

Na fase 0 o comando roda o script, lê o “bilhete” `.claude/tmp/cr-<PR>.json` (se existir), cruza com as threads, **anuncia a fase detectada e confirma com o usuário antes de agir**:

Detecção de fase em `/pr-review`

Fase	Condição e ação
TRIAR	Há <code>threads.isResolved</code> : <code>false</code> ainda sem decisão no bilhete. É o caso inicial: julgar, aplicar, commitar, responder
RESOLVER	Todas as decididas estão <code>replied</code> : <code>true</code> e a última fala de <code>replies</code> é do <code>CodeRabbit</code> . Lê a resposta nova e resolve o que convergiu
AGUARDANDO	A última fala é nossa e ele ainda não rebateu. “Diga isso e pare; não invente resposta dele”

Threads já `isResolved` são ignoradas; `isOutdated` é tratada com cuidado (o comentário pode ser sobre código que já mudou); e os `reviewSummaries` (nitpicks colapsados no corpo do review) entram na triagem mesmo sem thread própria.

Fase TRIAR. Cinco passos. (1) Julgar nos 3 eixos. (2) Apresentar por item o `arquivo:linha`, o ponto do `CodeRabbit` em uma linha, **prós**, **contras**, **edge cases** e o **veredito** recomendado; colher a decisão com `AskUserQuestion`, uma pergunta por comentário (aplicar / pular / parcial), em lotes se houver mais de uns quatro. (3) Aplicar os aprovados seguindo os patterns do repo (“a fix do `CodeRabbit` é ponto de partida, não evangelho”), rodar o gate local completo e commitar dividido por domínio. (4) Responder as threads. (5) Gravar o bilhete.

Gate local exigido antes de commitar na fase TRIAR

```
bun run lint && bun run typecheck && bun run typecheck:tooling && bun test
```

O template de resposta é fixo em três pontos: começa com `@coderabbitai`; se aplicado, diz o que foi feito mais o sha curto; se pulado, justifica citando a regra nossa, ou que o código já cobre, ou que contradiz a arquitetura. E sempre fecha pedindo verificação cética:

“não confie cego na solução; reanalisa o que foi commitado e me diz se introduziu bug ou edge case novo.”

Tom PT-BR, direto, sem travessão, *human-like*. Os nitpicks do corpo não são thread respondível: viram um reply-resumo único, se valer a pena. Postar é irreversível, então o comando mostra todos os drafts e espera o OK explícito antes de:

Postagem da resposta na própria thread

```
OWNER=sua-organizacao
REPO=seu-repositorio
PR=123
COMMENT_ID=456
TEXT="texto da resposta"
gh api "repos/$OWNER/$REPO/pulls/$PR/comments" \
  -f body="$TEXT" -F "in_reply_to=$COMMENT_ID"
```

O bilhete `.claude/tmp/cr-<PR>.json` guarda, por item: `commentId`, `threadId`, `path`, `line`, `verdict`, `replied`, `summary`, `commit`. É o que permite a re-execução saber onde parou.

POR QUE | Por que o bilhete vive em `.claude/tmp/` (gitignored)

O `.gitignore` explica a família inteira de selos e bilhetes locais: “São por-HEAD e por-máquina: versionar libreria o gate pra todo mundo.” O estado de triagem de um PR é seu, do seu HEAD; commitá-lo transformaria um controle de processo em artefato compartilhado e destravaria o gate na máquina alheia.

Fase RESOLVER. Classifica cada thread respondida em **Convergiu** (confirmou a fix ou aceitou a justificativa), **Achou algo real** (pegou bug ou edge case verdadeiro na fix commitada) ou **Insistindo errado / alucinando de novo**. O comando é explícito: “a re-análise dele também é claim”. Se ele achou algo real, o conserto **não** acontece ali: os itens voltam para TRIAR, passando de novo pelo filtro de 3 eixos.

REGRA | Guarda de convergência (anti ping-pong)

Se uma thread **já passou por um ciclo** (respondemos, ele rebateu) e ele **não traz fato técnico novo**, o veredito padrão é **encerrar** com justificativa final, nunca reabrir. Só “achou algo real” justifica nova rodada.

Resolver também é *outward-facing*: o comando lista as threads que vão ser resolvidas, espera o OK, e só então executa:

Resolução de thread via GraphQL

```
THREAD_ID=PRRT_kwD0123456
gh api graphql \
  -f query='mutation($id:ID!){ resolveReviewThread(input:{threadId:$id}){ thread { isResolved } } }' \
  -F "id=$THREAD_ID"
```

12.2.5 `cr-comments.ts`: por que existe um script

O JSDoc do topo do arquivo é a melhor explicação do desenho, e vale ler inteiro:

integracoes/coderabbit/cr-comments.ts, linhas 1-19 (trecho)

```
/**
 * Fetch + normaliza os comentários do CodeRabbit num PR, pra o comando
 * `/pr-review` (skill `pr-review-response`) consumir dados determiní-
 * sticos: o
 * LLM gasta token julgando, não paginando o GitHub nem limpando
 * markdown.
 *
 * Fonte da verdade = GraphQL `reviewThreads`. É o único lugar que dá,
 * por thread,
 * `isResolved`/`isOutdated` + o node id (pra resolver depois via
 * `resolveReviewThread`) + o `databaseId` do comentário (pra responder
 * via REST
 * `in_reply_to`). As REST `/pulls/{n}/comments` não trazem o estado de
 * resolução
 * da thread, então não servem como fonte única.
 */
```

Duas decisões de arquitetura estão condensadas aí. A primeira é econômica: paginar API e limpar markdown é trabalho determinístico, e trabalho determinístico não deve consumir contexto de LLM. A segunda é técnica: só o GraphQL entrega, na mesma resposta, o estado de resolução da thread, o *node id* para resolver e o *databaseId* para responder via REST. Uma única query cobre os três usos.

Uso do script

```
bun integracoes/coderabbit/cr-comments.ts 42
bun integracoes/coderabbit/cr-comments.ts # sem argumento: PR da
branch atual
```

Sem argumento, o script resolve o PR da branch via `gh pr view --json number -q .number`. Detalhe que o próprio código comenta: a chamada usa `.nothrow()` porque o `$` do Bun lançaria `ShellError` antes da mensagem amigável de erro aparecer. A saída é JSON no stdout, no formato `{ pr, threads, reviewSummaries }`, e o JSDoc registra que “não imprime nada sensível; só dados do PR”.

Pontos concretos da normalização:

- `stripNoise` remove os blocos `<details>...</details>` (os colapsáveis de “Prompt for AI Agents”, “Tools”, “Committable suggestion”), os comentários HTML de controle (cr-comment, fingerprinting) e colapsa três ou mais quebras de linha. Mantém título, descrição e o diff sugerido em texto.
- `extractTitle` pega o primeiro trecho em negrito do corpo, que é como o CodeRabbit titula a observação.
- `extractSeverity` lê a linha que vem antes do título, no formato de três campos separados por barra vertical (categoria, severidade, esforço). O JSDoc é honesto sobre o valor disso: “Dica de priorização; o agente lê o corpo de qualquer forma.”

- Só entram threads cujo **primeiro** comentário é do CodeRabbit. O login usado é `coderabbitai`, com nota no código: “sem o sufixo `[bot]` que a API REST usa”.
- Cada thread emite `threadId` (alvo do `resolveReviewThread`), `commentId` (alvo do `in_reply_to`), `lastAuthor` e `replies`. São `lastAuthor` e `replies` que alimentam a detecção **RESOLVER versus AGUARDANDO**.
- `reviewSummaries` traz os reviews do bot com corpo não vazio. Comentário no código: “Contexto, não thread.”

POR QUE | O script obedece o lint do próprio boilerplate

Ele não é código de fora com passe livre. `tsconfig.json` inclui `**/*.ts` e exclui apenas `node_modules`, `tooling`, `.claude`, `test` e os `*.test.ts(x)`: o script entra no `tsc --noEmit`. O `ignorePatterns` do `.oxlintrc.json` não lista `integracoes/`: ele entra no `bun run lint`. E o `knip.json` o declara como *entry point* explícito, senão seria marcado como código morto (ninguém o importa):

```
"entry": ["src/app/**/*.{ts,tsx}", "tooling/agent-hooks/*.ts",
"tooling/*.ts", "integracoes/coderabbit/cr-comments.ts"]
```

O efeito aparece no próprio código, num comentário que documenta a restrição de lint que moldou a implementação: “Só threads abertas pelo CodeRabbit. `flatMap + []/[obj]` evita o `filter+non-null assertion` (banido pelo lint) e mantém o tipo estreitado.”

12.2.6 A skill `pr-review-response` (essa vem instalada)

A skill em `.claude/skills/pr-review-response/SKILL.md` é a peça genérica: funciona com CodeRabbit **ou** com reviewer humano. Estrutura em quatro fases: `contexto/checkout` (`gh pr checkout`, `gh pr view --json title,body,headRefName,baseRefName`); `coleta` (`gh api .../comments --paginate` e `.../reviews --paginate`); **análise e classificação em `FIX / JUSTIFY / PARTIAL`** com tabela obrigatória e aprovação do usuário antes de executar; aplicação dos fixes com `bun run lint && bun run typecheck && bun test` e espera pelo `commit/push` do usuário; e resposta via API com autorização explícita.

Endpoint de resposta usado pela skill

```
OWNER=sua-organizacao
REPO=seu-repositorio
PR=123
COMMENT_ID=456
gh api "repos/$OWNER/$REPO/pulls/$PR/comments/$COMMENT_ID/replies" \
  -f body="resposta para o reviewer"
```

As regras obrigatórias da skill: nunca responder sem aprovação; nunca push automático; sempre ler o código antes de classificar, nunca só o texto do comentário; “se comment menciona pattern do projeto, verificar se o pattern de fato existe”; respostas concisas, técnicas, não-defensivas; em dúvida, perguntar antes de classificar. Se o reviewer for humano, tira-se o prefixo `@coderabbitai` e ajusta-se o tom.

A tabela de racionalizações é curta e útil:

Racionalizações comuns, segundo a skill `pr-review-response`

Desculpa	Realidade
"CodeRabbit pediu, eu já vi, posso aplicar direto"	Não. Plan + aprovação.
"Pequeno fix, pulo o lint"	Sempre lint + typecheck.
"Vou só responder, sem aplicar"	Reviewer leu o código. Resposta sem fix = JUSTIFY explícito.

CUIDADO | Os dois documentos divergem em vocabulário e em endpoint

A skill usa `/comments/{comment_id}/replies` e o vocabulário **FIX / JUSTIFY / PARTIAL**. O comando `/pr-review` usa `-F in_reply_to=<commentId>` e o vocabulário **aplicar / pular / parcial**. A skill não menciona `cr-comments.ts` nem as fases TRIAR/RESOLVER/AGUARDANDO.

Não é contradição fatal (os dois caminhos postam reply em thread inline), mas se você ler os dois esperando um texto único, vai tropeçar. Trate o comando como a implementação CodeRabbit-específica e a skill como o método geral, exatamente como o README posiciona.

12.2.7 Limites e o cenário sem CodeRabbit

Limites verificáveis do script: o cap é de 100 threads e 50 comentários por thread (se o número de threads bater exatamente no cap, ele escreve um aviso no stderr); só enxerga threads **abertas pelo bot**, então uma thread iniciada por humano não aparece no JSON; e `reviewSummaries` não é respondível. O script também exige gh autenticado e o GitHub App instalado para haver qualquer dado.

O README fecha com a saída de emergência:

O método não depende dele: a skill `pr-review-response` e o comando `review` funcionam com qualquer reviewer (inclusive humano), e o `feature-audit` cobre a validação local antes do PR. O CodeRabbit só adiciona uma primeira passada automática; o filtro de 3 eixos continua sendo o que protege o codebase.

12.3 GitNexus: grafo do código e blast radius

12.3.1 O que é e qual problema ataca

GitNexus indexa o repositório num grafo de código (símbolos, relações, fluxos de execução) e expõe ferramentas via MCP para entender código, avaliar impacto e navegar com segurança. É de terceiros; instale pela fonte oficial.

O `integracoes/gitnexus.md` nomeia o problema com precisão:

O maior risco ao editar com IA é o **blast radius invisível**: mudar um símbolo e quebrar callers que ninguém mapeou. O GitNexus dá a disciplina de **impact**

analysis antes de editar e detect_changes antes de commitar: vira parte do `feature-audit` e do `workflow-refactor` (como fase opcional).

12.3.2 Setup

Instalar o CLI e registrar o MCP server

```
npm install -g gitnexus      # ou: bun add -g gitnexus
gitnexus --version

claude mcp add gitnexus -- gitnexus mcp
```

Como alternativa ao `claude mcp add`, o doc mostra o registro direto no `~/ .claude.json`, dentro de `mcpServers`:

Registro manual do MCP server

```
"gitnexus": { "command": "gitnexus", "args": ["mcp"] }
```

Depois: indexar o repo na raiz com `gitnexus analyze`, o que gera `.gitnexus/` (reindexe quando o índice ficar *stale*); reiniciar o Claude Code; e confirmar que as ferramentas `mcp__gitnexus__*` e os recursos `gitnexus://` aparecem.

CUIDADO | `.gitnexus/` não está no `.gitignore` do boilerplate

O passo 3 do doc diz “adicione ao `.gitignore`” e isso é instrução manual, não algo pré-aplicado. O `.gitignore` do repo tem `.impeccable/`, `.claude/tmp/`, `.agents/tmp/` e `.claude/worktrees/`, mas **não** tem `.gitnexus/`. Se você rodar `gitnexus analyze` e commitar sem olhar, o índice inteiro entra no repositório. Adicione a linha antes de indexar.

CUIDADO | Uma inconsistência menor no próprio doc

O setup instala o CLI global e usa `gitnexus analyze`, mas o bloco pronto para colar no `CLAUDE.md` manda reindexar com `npx gitnexus analyze`. Os dois funcionam; só saiba que a diferença existe para não achar que instalou errado.

12.3.3 A disciplina

Ferramentas MCP e quando o método exige cada uma

Momento	Ferramenta
Antes de editar função/classe/método	<code>mcp__gitnexus__impact({ target, direction: "upstream" })</code> e reportar callers diretos, processos afetados e nível de risco
Antes de commitar	<code>mcp__gitnexus__detect_changes()</code> , para confirmar que a mudança só afeta o escopo esperado
Explorar código desconhecido	<code>mcp__gitnexus__query({ query: "conceito" })</code> , em vez de <code>grep</code>
Contexto de um símbolo	<code>mcp__gitnexus__context({ name })</code>
Renomear	<code>mcp__gitnexus__rename</code> , que entende o call graph

REGRA | Nunca renomeie símbolo com find-and-replace

É a regra mais dura do bloco, e a que o grafo existe para resolver. Find-and-replace acerta strings, não referências: pega comentário, string literal e símbolo homônimo de outro módulo, e erra o re-export. Com GitNexus disponível, o rename passa pelo call graph.

12.3.4 O bloco para o CLAUDE.md

O doc entrega um bloco pronto, delimitado por `<!-- gitnexus:start -->` e `<!-- gitnexus:end -->`, que reescreve a disciplina acima em tom normativo (“DEVE rodar impact analysis antes de editar qualquer símbolo”, “NUNCA renomeie símbolos com find-and-replace”). Os delimitadores não são decoração: eles tornam o bloco atualizável por ferramenta sem tocar no resto do arquivo.

O `CLAUDE.md` da raiz já reserva o lugar exato, na linha 65:

Slot vazio no CLAUDE.md

```
<!-- Cole aqui o bloco de integração do GitNexus, se usar: ver
integracoes/gitnexus.md -->
```

12.3.5 Onde o kit já se acopla ao GitNexus

O acoplamento é real e está escrito nas skills, sempre marcado como opcional:

- `.claude/skills/feature-audit/SKILL.md` tem uma **Fase 0 (opcional): impact pre-flight**: rodar `detect_changes` no diff e `impact` (upstream) em cada símbolo crítico *antes* das demais fases, produzir uma tabela de blast radius no topo do relatório, e aplicar uma regra dura: símbolos com risco CRITICAL/HIGH exigem comentário explícito sobre teste, cobertura ou rollback; sem isso, o veredito é **NEEDS FIXES**.

- [.claude/skills/workflow-refactor/SKILL.md](#), seção **0.2 (Opcional) Pre-flight: blast radius**: mesma mecânica para cada símbolo público da feature no escopo, com “símbolo com risco CRITICAL → gate de aprovação explícito do usuário antes de tocar”.
- Os comandos [.claude/commands/audit.md](#), [review.md](#) e [refactor.md](#) listam na linha “Ativa:” a skill `gitnexus-impact-analysis` com o sufixo (opcional: ver [integracoes/gitnexus.md](#)).
- O [knowledge.template.md](#) registra que a base de conhecimento do código “potencializa com GitNexus”.

CUIDADO | A skill `gitnexus-impact-analysis` não está em [.claude/skills/](#)

Ela é citada por três comandos do kit, mas não é um arquivo do boilerplate: vem junto com o produto de terceiros. Se você chamar `/audit` sem GitNexus instalado, a menção simplesmente não resolve para nada, e é por isso que ela está marcada como opcional em todos os lugares. Não é arquivo faltando.

12.3.6 Sem GitNexus

REGRA | Nunca infira o blast radius sem evidência

O doc é explícito sobre o plano B: “Sem a ferramenta, faça o mapeamento manual (grep dos símbolos exportados) e registre o risco.” O mesmo texto ecoa no `workflow-refactor` (“NÃO infira o blast radius sem evidência”) e no `feature-audit`. O que é opcional é a *ferramenta*, não a evidência: sem grafo, você ainda deve o mapeamento, feito à mão e escrito no relatório.

12.4 impeccable: intenção visual

12.4.1 O que é

`impeccable`, de Paul Bakaus (github.com/pbakaus/impeccable, licença Apache-2.0), é uma skill de design para agentes de código. Segundo o [integracoes/impeccable.md](#): **1 skill com 23 comandos** sob `/impeccable` (`polish`, `audit`, `critique`, `distill`, `animate`, `bolder`, `quieter` e outros), mais um **CLI detector** com 41 regras determinísticas de “AI slop”, sem LLM e sem API key.

12.4.2 Por que entra no método

A IA, por padrão, produz UI “genérica/templated” (Inter em tudo, gradiente roxo→azul, card dentro de card, texto cinza sobre cor). O `impeccable` empurra a interface para algo **intencional e distinto** e é o motor por trás do artefato [DESIGN.md](#): o sistema visual (tokens) vive no [DESIGN.md](#), e o `impeccable` o cria, mantém e aplica aos componentes. É o complemento humano do `oxlint`: onde o lint garante *correção*, o `impeccable` garante *intenção visual*.

POR QUE | Lint não tem opinião sobre beleza

A tese arquitetural do boilerplate é “arquitetura imposta por lint”. Mas nenhuma regra de oxcint sabe dizer que um card dentro de um card é ruim, ou que a fonte padrão faz a tela parecer template. Essa metade do problema fica sem gate automático, e o [DESIGN.md](#) mais o `impeccable` são a resposta: tokens versionados para a parte objetiva, comandos de crítica para a parte de julgamento.

12.4.3 Setup

Instalar a skill na raiz do projeto

```
npx impeccable skills install
```

O instalador auto-detecta o harness e escreve em `.claude/skills/`. Como alternativa no Claude Code, via plugin: `/plugin marketplace add pbakaus/impeccable` seguido de `/plugin install impeccable`, e recarregar o Claude Code depois.

O segundo passo é rodar `/impeccable init` uma vez: ele pergunta se a superfície é **brand** (marketing/landing) ou **product** (app/dashboard) e escreve `PRODUCT.md` e `DESIGN.md`. O doc trata o caso deste repo explicitamente: o boilerplate já traz os dois como **template** (o `DESIGN.md` se autodeclara “Template” logo abaixo do frontmatter), então deixe o `init` preenchê-los, ou use `/impeccable document` para gerar o `DESIGN.md` a partir do código existente.

O terceiro passo é registrar no `CLAUDE.md` que o sistema visual normativo está no `DESIGN.md` e que trabalho de design passa pelo `/impeccable`.

12.4.4 A disciplina

- **DESIGN.md é a fonte da verdade visual:** tokens de cor, tipografia e espaçamento no frontmatter. Código consome os tokens, nunca valores mágicos, o que ecoa a skill `design-system` do kit.
- **Construiu ou ajustou UI?** Rode `/impeccable critique <alvo>` (review de UX: hierarquia, clareza) e `/impeccable polish <alvo>` (passada final e alinhamento ao design system) antes de considerar pronto.
- **Decisões visuais novas voltam para o DESIGN.md:** ele co-evolui com a UI.

Comando certo para a intenção (mapa do doc)

Intenção	Comando
Intensidade	<code>bolder/quieter</code>
Essência	<code>distill</code>
Motion	<code>animate</code>
Overflow, i18n, edge cases	<code>harden</code>
Empty states	<code>onboard</code>
Copy	<code>clarify</code>
Cirúrgicos	<code>layout/typeset/colorize</code>

Rodar `/impeccable` sozinho lista todos os comandos. E `/impeccable pin audit` cria um atalho `/audit`.

CUIDADO | `/impeccable pin audit` colide com o `/audit` do boilerplate

O kit já tem `.claude/commands/audit.md`, que é auditoria **arquitetural** via `feature-audit` (alinhamento com acceptance criteria, contrato, camadas, severidade). Fixar o `audit` do `impeccable` cria um segundo `/audit` com significado completamente diferente, o de auditoria visual.

O doc de integração não avisa disso: é observação de leitura dos dois arquivos. Se você usa as duas ferramentas, não fixe esse atalho, ou fixe com outro nome.

12.4.5 Gate determinístico, opcional

Detector de slop, sem LLM e sem API key

```
npx impeccable detect src/  
npx impeccable detect src/ --fast --json
```

São 41 issues de slop e qualidade pegos sem LLM. O doc classifica a coisa corretamente: “É ferramenta de dev (como o binário do `oxlint`): pode rodar ad hoc ou no CI ao lado do `bun run lint`.” A flag `--fast --json` serve para saída programática.

12.4.6 O bloco para o `CLAUDE.md` e a lista anti-slop

Como no `GitNexus`, há um bloco delimitado, aqui por `<!-- impeccable:start -->` e `<!-- impeccable:end -->`. A parte que mais importa é a lista de proibições:

Trecho do bloco impeccable para o CLAUDE.md

- Anti-slop: nada de Inter/Arial por padrão, gradiente roxo→azul, card-dentro-de-card, texto cinza sobre cor, preto/cinza puro (sempre tinja), easing bounce/elastic.

Diferente do GitNexus, o [CLAUDE.md](#) da raiz **não** tem um slot de cole-aqui para o impeccable: ele já referencia a integração direto, na seção de artefatos de memória: “[DESIGN.md](#): sistema visual normativo (tokens). Design via impeccable (ver [integracoes/impeccable.md](#)).”

12.4.7 Acoplamento com o kit

Das três integrações, o impeccable é a mais entranhada no boilerplate, e é a única com pegada pré-configurada:

- O [DESIGN.md](#) aponta para a integração: “Trabalho de design (criar/auditar/polir) via impeccable (ver [integracoes/impeccable.md](#)).”
- A skill [.claude/skills/design-system/SKILL.md](#) declara: “Stack de estilo **fixa**: **Tailwind v4** (nunca MUI/Emotion/NativeWind/RN). Pareia com o **impeccable** (ver [integracoes/impeccable.md](#)), que herda os SEUS tokens e bloqueia *slop*.”
- O [.gitignore](#) já prevê a ferramenta, com comentário: “config/journal local do impeccable live mode (tooling, não versionar)”, seguido de `.impeccable/`.

POR QUE | “Herda os SEUS tokens” é a direção certa da dependência

A skill `design-system` não delega o sistema visual ao impeccable: ela mantém os tokens no [DESIGN.md](#) e faz a ferramenta consumi-los. Se fosse ao contrário, tirar o impeccable levaria embora o design system junto. Do jeito que está, remover a ferramenta custa apenas os comandos de crítica e polimento.

12.4.8 Sem impeccable

O [DESIGN.md](#) sozinho já dá disciplina visual (tokens centrais, sem valores mágicos) e a skill `design-system` cobre os princípios. O impeccable apenas potencializa criação e auditoria: o método não depende dele para funcionar.

12.5 Como decidir o que instalar

Nenhuma das três tem custo de manutenção dentro do repo (não há dependência em [package.json](#), hook, nem step de CI apontando para elas), então a decisão é sobre onde está a sua dor:

Escolha por sintoma

Sintoma	Integração que ataca
Review humano vira gargalo e PRs param dias	CodeRabbit + <code>/pr-review</code>
Refactor quebra caller que ninguém tinha mapeado	GitNexus (impact pre-flight)
A UI gerada por agente sai correta mas com cara de template	impeccable + DESIGN.md

REGRA | A disciplina precede a ferramenta

Os três documentos terminam com a mesma seção (“Sem CodeRabbit”, “Sem GitNexus”, “Sem impeccable”) e a mesma mensagem: o método continua exigindo o comportamento mesmo sem a ferramenta. Filtrar feedback de IA em 3 eixos, mapear blast radius com evidência e consumir tokens em vez de valores mágicos são obrigações do repo. As integrações apenas tornam cada uma mais barata de cumprir.

PARTE 13

Decisões e porquês (ADRs)

O QUE VOCÊ LEVA DESTE CAPÍTULO

Você vai aprender a usar o histórico de decisões antes de alterar uma regra central e a registrar novos rumos sem apagar o contexto antigo. Este capítulo evita que arquitetura vire folclore.

O repo guarda suas decisões em [MEMORY.md](#): um log de 28 ADRs numerados (ADR-001 a ADR-028), com o **porquê** de cada escolha, as alternativas consideradas e o motivo do descarte.

Esse arquivo é consultado pelas outras peças do produto. O [CODE-PATTERN.md](#), por exemplo, manda o leitor buscar ali o rationale de forms; o [ARCHITECTURE.md](#) aponta para o mesmo registro; e o próprio código cita ADR por número. Em [src/proxy.ts](#):

`src/proxy.ts`: o código referencia o ADR por número

```
/**
 * Cookie HttpOnly de sessão emitido pelo backend no login. É a fonte
 * de verdade
 * de "tem sessão"; o front nunca decodifica, valida ou assina o JWT
 * que ele
 * carrega (ADR-007). Trocar o nome aqui é o único ajuste se o seu
 * backend usar
 * outro.
 */
```

Quem lê essa linha e quer entender *por que* o front não valida JWT tem um destino exato. É esse o serviço que o log presta: ele evita que a regra vire folclore.

13.1 Como o log é escrito

O cabeçalho do arquivo declara o contrato em quatro linhas:

MEMORY.md, linhas 3-7

```
Registro **append-only** das decisões de arquitetura/toolchain e o **porquê**, incl.
alternativas descartadas. Doc canônico (humano e agente). Não
reescrever histórico :
decisões revertidas viram uma **nova** entrada marcada como tal.

Formato por entrada: **Decisão · Contexto · Por quê · Alternativas
descartadas · Status.**
```

REGRA | Append-only, sem exceção

Decisão revertida não vira `git rm` da entrada antiga. Vira **nova** entrada, ou uma emenda apontando para ela. O valor do arquivo está justamente em conter as decisões que *não* sobreviveram: sem elas, o próximo agente (ou o próximo dev) refaz a mesma avaliação do zero e chega na mesma alternativa já descartada.

Os cinco campos do formato declarado (**Decisão, Contexto, Por quê, Alternativas descartadas, Status**) não são todos obrigatórios na prática, e as entradas reais usam campos ad-hoc quando precisam: Precisão importante: (ADR-001), Risco registrado: e Nota: (ADR-002), Emenda (ADR-025): (ADR-003), Localização (correção): (ADR-010), Detalhe técnico: e Escopo consciente: (ADR-022), Caveat conhecido: (ADR-025), Revisitar quando: (ADR-024 e ADR-027).

As três mecânicas do append-only, com exemplo real

O log não teoriza sobre append-only: ele exercita três formas diferentes de registrar mudança sem apagar nada.

Como cada tipo de mudança é registrado sem reescrever histórico

Mecânica	Exemplo no arquivo
Emenda cruzada	O ADR-003 (sem cache) não foi reescrito quando <code>cacheComponents</code> entrou. Ganhou uma linha Emenda (ADR-025): explicando por que a flag nova não contradiz a decisão velha. As duas entradas continuam legíveis lado a lado.
Reversão no Status	O ADR-004 termina com Status: ativo (revertido de uma escolha inicial por SWR). A escolha antiga por SWR não existe mais como entrada própria; o registro dela é essa parentética.
Correção marcada	O ADR-010 traz Localização (correção): documentando que os docs internos diziam <code>src/app/proxy.ts</code> e o caminho certo é <code>src/proxy.ts</code> . A correção é parte do texto, com o erro nomeado.

Há ainda uma quarta situação, e é a que mais diz sobre a régua do projeto: o ADR-027 é o único com Status: não adotado (candidato). O log registra também o que foi **avaliado e recusado**, não só o que entrou. Os statuses efetivamente usados no arquivo

são três: ativo (26 entradas, algumas com parentética), ativo (deploy ainda não validado) no ADR-002, e não adotado (candidato) no ADR-027.

POR QUE | Por que isso importa num repo escrito por agentes

O hook de `SessionStart` injeta um resumo curto das regras e fecha apontando as fontes: *"Para detalhe/exemplo/porquê: ARCHITECTURE.md · CODE-PATTERN.md · MEMORY.md (log de decisões)."* Ou seja, o `MEMORY.md` é literalmente o lugar onde o agente vai buscar justificativa quando a regra resumida não bastar. Um log que apaga o histórico entrega ao agente uma foto sem contexto. Sem o porquê, o agente reintroduz a alternativa descartada com toda a confiança do mundo.

13.2 Índice dos 28 ADRs

Numeração contínua de 001 a 028, sem buracos. Quando uma decisão muda, a entrada antiga permanece como histórico e a nova entrada aponta explicitamente o que foi substituído.

Os ADRs 001 a 014 do `MEMORY.md`

ADR	Status	Decisão
001	ativo	TypeScript 7 nativo (tsc em Go) + <code>@typescript/native-preview</code> como ponte do Next
002	ativo*	Bun ponta a ponta; node proibido como runtime da app
003	ativo	Dados sempre frescos: sem cache do Next
004	ativo	Sem SWR, sem React Query, sem lib de data-fetching no client
005	ativo	valibot (não zod), importável só em <code>*.schema.ts</code>
006	ativo	ky como client HTTP, confinado a <code>src/lib/api</code>
007	ativo	Auth por cookie JWT; o backend é a fonte de authz
008	ativo	Feature-based; shared é folha; sem import cross-feature
009	ativo	Guardrails impostos por oxlint, não por <code>.md</code>
010	ativo	<code>src/proxy.ts</code> (Next 16), não <code>middleware.ts</code>
011	ativo	Patch do Next removido
012	ativo	oxlint com <code>--disable-nested-config</code>
013	ativo	Não prescrever modelo de branch; default trunk-based em main
014	ativo	Docs canônicos + inject de sessão como resumo

ADRs 015 a 028

ADR	Status	Decisão
015	ativo	Forms com react-hook-form + valibotResolver
016	ativo	AGENTS.md como contrato comum; hooks compartilhados multi-host
017	ativo	Knip como gate de código morto
018	ativo	oxfmt como formatter, com Markdown de fora
019	ativo	Lint type-aware (--type-aware + oxlint-tsgolint)
020	ativo	no-lint-suppression é script, não regra de lint
021	ativo	Pré-review com selo por HEAD como portão do PR
022	ativo	lib/api entregue como reference implementation
023	ativo	bun.lock versionado e .gitignore no pacote
024	histórico	Next 16.2 stable, não o canal preview
025	ativo	cacheComponents ligado: streaming, não cache
026	ativo	react-perf removido em favor do React Compiler
027	não adotado	oxlint-tailwindcss fora, por enquanto
028	ativo	Baseline Next 16.3, Bun 1.4 e toolchain alinhado ao Avante

(*) ADR-002: ativo (deploy ainda não validado).

13.3 Runtime e toolchain

ADR-001: TypeScript 7 nativo

A decisão é usar `typescript@7` (o port nativo em Go) como o `tsc` de verdade, mais `@typescript/native-preview` (o `tsgo`) como ponte para o Next 16.

O contexto explica a esquisitice: o `typescript@7` expõe o binário `tsc`, mas não expõe o [lib/typescript.js](#), a API JS clássica que o Next importa. Sem a ponte, o `next build` tentava reinstalar o TypeScript clássico. Com o `native-preview` presente, o Next 16 detecta e pula o type-check próprio, e o `tsc` nativo faz a checagem real.

```
package.json: os dois pacotes convivem de propósito
```

```
"@typescript/native-preview": "7.0.0-dev.20260707.2",
"typescript": "7.0.2"
```

CUIDADO | Ter `typescript` no `package.json` não é violação da decisão

O ADR-001 antecipa exatamente essa leitura errada, com um campo próprio (Precisão importante): *"não instalar o TypeScript clássico' significa não voltar para o TS 5/6. O pacote `typescript` na versão 7 é o compilador nativo; tê-lo no `package.json` é a decisão, não uma violação dela."* Não remova nenhum dos dois achando que está limpando duplicata.

O `tsgo` não é importado por nenhum módulo. Ele é invocado pelo `toolchain`, então o `knip` precisou de exceção explícita, com o motivo escrito no `knip.json`: *"Fornece o `tsgo`; invocado pelo `toolchain`, não importado por nenhum módulo."* Alternativas descartadas: `typescript` v5 (não é TS7) e usar só o `native-preview` (queriam o `tsc` versionado).

ADR-002: Bun ponta a ponta

REGRA | `node` não é runtime deste projeto

Bun em `dev`, `build`, `start` e `test` (via `--bun`), inclusive no container de produção. Os scripts carregam a flag explicitamente.

```
package.json: a flag --bun em todo lugar
```

```
"dev": "bun --bun next dev",
"build": "bun --bun next typegen && tsc --noEmit && bun --bun next build",
"start": "bun --bun next start",
```

O `"packageManager": "bun@1.4.0-canary.1"` fixa a versão, e o CI lê essa mesma versão do `package.json` (`bun-version-file: package.json`), com o comentário no YAML: *"Alinhado ao `packageManager` do `package.json` (quem gerou o `bun.lock`)"*. É uma peça só, do `lockfile` ao CI.

Este é o ADR com a justificativa mais fraca do arquivo, e ele reconhece isso. O campo **Porquê** diz apenas: *"stack rápida/coerente; o time decidiu Bun e2e."* O que compensa é o campo seguinte, que registra o preço:

CUIDADO | Risco registrado no próprio ADR, e Status honesto

"Next em produção sob Bun é caminho não-oficial → exige smoke test do output: 'standalone' sob Bun antes do deploy." E o Status é o único do arquivo com ressalva: `ativo` (`deploy` ainda não validado).

Verificado no baseline atual: output: "standalone" está no [next.config.ts](#) e o smoke test do build foi executado sob Bun. O deploy ainda exige copiar [public/](#) e [.next/static/](#) quando a imagem standalone for montada, conforme a documentação do Next. O [ARCHITECTURE.md](#) repete o aviso ("*Em produção (Dokploy) o container roda Bun: ponto de risco registrado*"). Se você vai para produção, esse smoke test é seu primeiro trabalho. O ADR já te disse isso.

Uma nota de coerência que evita discussão boba: o `oxlint` é um CLI Node (`#!/usr/bin/env node`). O ADR classifica isso como ferramenta de dev, igual ao git, e não como runtime da aplicação. Não conta como violação.

ADR-024 (histórico): Next 16.2 stable, não o canal preview

Este ADR é o exemplo mais claro de decisão tomada com *matriz de teste* e não com preferência. Ele descreve a decisão que valeu antes da adoção do Bun 1.4 canary; o [baseline atual está no ADR-028](#). A decisão é fixar `next@16.2.11 stable`, abrindo mão de `partialPrefetching` e de `experimental.useTypeScriptCli`, que só existem no 16.3.

A matriz que produziu a decisão (ADR-024)

Combinação testada	Resultado
Next 16.3 preview + bun 1.3.4	Estoura em Expected CommonJS module to have a function wrapper
Next 16.3 preview + bun 1.3.14 (latest)	O loader do React Compiler recusa a flag do Bun em <code>NODE_OPTIONS</code>
Next 16.3 preview + bun 1.4 canary	Passa, mas esse Bun não está publicado no registry
Next 16.2.11 stable + bun 1.3.4	Build verde. Adotado.

POR QUE | O argumento é de produto, não de engenharia

Texto literal do ADR: *"Bun ponta a ponta' (ADR-002) é inegociável e vendida como tal. Entre perder duas flags de preview e entregar um build que não roda na máquina do comprador, perder as flags é barato."*

Entregar um zip cujo `bun run build` falha na primeira tentativa do comprador custa mais que duas flags experimentais. O ADR ainda registra o gatilho de revisão: *"Revisitar quando: o bun 1.4 for publicado como estável no npm. Aí o 16.3 volta à mesa."*

Verificável no histórico: `"next": "16.2.11"` foi o pin anterior. No checkout atual, o schema do Next 16.3 valida as duas flags e o build verde confirmou o caminho.

ADR-019: Lint type-aware

O `bun run lint` roda com `--type-aware` (via `oxlint-tsgolint`), ligando `no-unnecessary-condition`, `no-unsafe-type-assertion`, `consistent-return` e `no-redundant-type-constituents`. A decisão complementar de usar ou não `noUncheckedIndexedAccess` foi revisitada no ADR-028; o baseline atual acompanha o `tsconfig.json` do Avante e não ativa essa opção.

POR QUE | Sem type-aware, metade da doutrina era só texto

Literal do ADR: *"metade da doutrina do repo ('confie no tipo, sem fallback para caso que o tipo já impede') era só texto. Nenhuma regra sintática pega ?? 0 sobre valor garantido. Type-aware é o que transforma essa regra em gate."*

E o par com `noUncheckedIndexedAccess` tem porquê próprio: *"sem ele, lista[0] mente (o tipo diz T, o runtime pode dar undefined) e o no-unnecessary-condition acusa como morto um guard que na prática é necessário. Com ! e as banidos, estreitar de verdade é a única saída honesta."*

O rationale foi replicado dentro do próprio `tsconfig.json`, quase palavra por palavra. padrão que se repete no repo inteiro: a explicação mora onde a configuração mora.

`tsconfig.json` (configuração histórica do ADR-019)

```
// Indexar array/record devolve `T | undefined`. Sem isto, `lista[0]` mente
// (tipo diz T, runtime pode ser undefined) e o `no-unnecessary-condition`
// do lint type-aware acusa como morto o guard que na prática é necessário.
// Com `!`/`as` banidos, a única saída honesta é estreitar de verdade.
"noUncheckedIndexedAccess": true,
```

Achado registrado na adoção: ligar o type-aware *"reprovou dois pontos de código morto que estavam no repo desde sempre"*.

ADR-018: oxfmt, com Markdown de fora

Formatter é `oxfmt` (pinado em `0.62.0`), mesmo ecossistema do `oxlint`: um binário, zero config duplicada. O `.oxfmttrc.json` é praticamente só o `ignore`, e o comentário dele é a decisão inteira:

`.oxfmttrc.json`

```
// Defaults do oxfmt já batem com o estilo do repo (printWidth 100,
// tabWidth 2,
// semi, aspas duplas, trailingComma "all", endOfLine "lf"). Não
// restateamos
// defaults — só o que difere fica aqui.
//
// Markdown fica de fora: os docs canônicos (AGENTS/ARCHITECTURE/CODE
// -PATTERN/
// MEMORY/VERCEL-OXLINT-MAPPING) e as skills são prosa mantida à mão;
// reformatá-los é churn sem valor. Código e config JSON, sim.
"ignorePatterns": ["**/*.md"]
```

Roda em três lugares: no `PostToolUse` dos agentes (via [format-after-write.ts](#)), no pré-commit do `lefthook` (`bunx oxfmt --check {staged_files}`) e no `verify` (`bun run fmt:check`).

ADR-012: `--disable-nested-config`

Motivo bem específico: `worktrees` aninhados (por exemplo `.claude/worktrees/*`, ignorados no `git`) têm `.oxlinttrc.json` próprio e acabavam sendo lintados via `nested-config`. Com a flag, só o config raiz vale. Ela aparece em três lugares: nos dois scripts do `package.json` e no pré-commit do `lefthook.yml`.

ADR-011: Patch do Next removido

Havia um `bun patch` no Next só para silenciar um log do `native-preview`. Foi removido: o custo (refazer o patch a cada upgrade do Next) era maior que o benefício (um log cosmético). O Status assume a consequência: ativo (log do `native-preview` reaparece: aceito). Se você ver esse log no build, ele é esperado.

13.4 Dados e cache: o grupo que mais gera dúvida

ADR-003: Dados sempre frescos, sem cache

REGRA | Cache do Next é proibido

'use cache', `unstable_cache`, `revalidate>0` e `force-static` são proibidos. Leitura é `RSC + Suspense`. `React.cache()` (`dedup` por request) é permitido.

O ADR dá dois argumentos, e o segundo é o que desarma a objeção mais comum: “é um CRM: frescor > performance de cache. É também o default do Next 16.” O projeto não está brigando com o framework; está aceitando o default dele. E o domínio decide o resto: num CRM, dado stale é dado errado na tela de quem está no telefone com o cliente. Alternativa descartada: ISR, porque introduz staleness por construção.

A regra `custom boilerplate/no-use-cache` cobre quatro formas de reintroduzir cache:

```
tooling/oxlint-rules/rules/no-use-cache.ts
```

```
* 1. The "use cache" directive at the top of a module or function
  body.
* 2. `import { unstable_cache } from "next/cache"`.
* 3. An object property `cache: "force-cache"`.
* 4. `fetch(url, { next: { revalidate: <number> 0 | false } })`.
```

Ela pega inclusive as variantes com sufixo ("use cache: remote", "use cache: private"), porque casa por startsWith. Há uma segunda regra separada, boilerplate/no-force-static, e as duas estão como "error".

Há também um backstop em grep no CI. Esse detalhe mostra o quanto a decisão é levada a sério:

```
.github/workflows/ci.yml
```

```
# Backstop em grep dos guardrails que dependem do jsPlugins (
alpha): se o
# plugin um dia falhar em carregar, estes gates ainda barram as
violações.
- name: Grep-gate, sempre fresco
  run: |
    if grep -rEn "['\"]use cache[ :a-z]*['\"]|unstable_cache" src
    --include='*.ts' ... ; then
      echo "::error::cache proibido ('use cache' / unstable_cache
) em src/"; exit 1
    fi
```

ADR-004: Sem SWR, sem React Query

POR QUE | Não é preferência de lib, é consequência do ADR-003

O S de SWR é *stale*. Adotar SWR (ou React Query) seria reintroduzir no client exatamente o cache que foi banido no server. O ADR é curto porque não precisa ser longo: "SWR é cache → contradiz o 'sempre fresco' (ADR-003). RSC-puro é mais coerente."

O Status registra que a decisão **inicial era SWR**: ativo (revertido de uma escolha inicial por SWR). É o exemplo mais limpo do append-only funcionando: a alternativa não foi apagada, virou parentética permanente.

O enforcement existe e é mais fino do que no-restricted-imports consegue ser. A regra custom no-forbidden-imports nasceu justamente para fechar o buraco de subpath:

```
tooling/oxlint-rules/rules/no-forbidden-imports.ts
```

```
* Closes the subpath gap left by exact `no-restricted-imports` entries.
* `no-restricted-imports` only matches the exact specifier, so a
  subpath
* import slips through. Examples:
* - `swr/mutation` is still SWR.
* - `@tanstack/react-query/devtools` is still React Query.
* - `zod/v4-mini` is still Zod (this repo uses valibot).
* - `jose/jwt/verify` is still a JWT library (authz belongs to the
  backend).
```

Lista banida por completo: `swr`, `@tanstack/react-query`, `@tanstack/react-query-devtools`, `@tanstack/query-core`, `react-query`, `axios`, `zod`, `jsonwebtoken`, `jose`.

CUIDADO | O ADR-004 cita `useActionState`/`useOptimistic`, mas o repo não usa nenhum dos dois

O ADR-004 diz "interatividade via React 19 (`useActionState`, `useOptimistic`)". Isso foi depois superado pelo ADR-015, que descartou `useActionState` puro para forms em favor de RHF. No `src/` entregue, os dois forms (`use-login-form.ts` e `use-perfil-form.ts`) usam `useForm` do `react-hook-form`, e `useOptimistic` não aparece.

Leia o ADR-004 pelo que ele decide de fato: **o que não usar** (SWR, React Query e qualquer lib de data-fetching no client). Não o leia como descrição do código atual.

ADR-025: `cacheComponents` ligado (streaming, não cache)

A objeção óbvia: "o ADR-003 proíbe cache, e o `next.config.ts` liga uma flag chamada `cacheComponents`. Qual dos dois vale?"

POR QUE | Os dois valem, porque o que é pré-renderizado não contém dado

Texto do ADR: *"dá shell estático no primeiro byte + dado em streaming (a rota sai como `Partial Prerender` no build). Isso **não** contradiz o ADR-003: nada fica em cache, o que é pré-renderizado é só a casca sem dado, e cada request busca fresco."*

O dado nunca é pré-renderizado nem armazenado. Ele chega por streaming, fresco, a cada request. O que sai no primeiro byte é HTML sem dado nenhum.

O mesmo argumento está escrito dentro da config:

`next.config.ts`

```
// Shell estático + buraco dinâmico em streaming. NÃO é cache: cada request
// busca fresco; o que fica pré-renderizado é só a casca sem dado. A
// parte que
// lê cookie/dado precisa ficar sob um `<Suspense>` (ver src/app/
// perfil/page.tsx).
cacheComponents: true,
```

O ponto mais interessante: a flag vira **aliada do enforcement**. Com ela ligada, ler cookie ou dado fora de um `<Suspense>` reprova o build com `blocking-prerender-dynamic`. Nas palavras do ADR: “é o Next forçando o padrão que a *skill vercel-react-best-practices* já prescrevia”. conselho de skill virou erro de build.

O exemplo canônico está entregue, e o JSDoc explica por que o componente foi separado da página:

`src/app/perfil/page.tsx`

```
/**
 * Carrega o perfil e monta o formulário.
 *
 * Fica separado da página porque ler o cookie (dentro do `kyServer`)
 * torna esta
 * subárvore dinâmica. Com `cacheComponents` ligado, o Next exige que a
 * parte
 * dinâmica esteja sob um `<Suspense>`: o shell estático vai no
 * primeiro byte e o
 * dado chega em streaming. Isso não é cache — cada request busca
 * fresco.
 */
async function PerfilCarregado(): Promise<React.ReactNode> {
  const perfil = await apiClient(kyServer, API_ENDPOINTS.perfil.me,
    perfilSchema);
  return <PerfilForm perfil={perfil} />;
}
```

A skill pre-review lista `cacheComponents` na seção “**Não flagrar**”, junto com `React.cache()`, `Promise.all` e `useOptimistic`: o reviewer automático foi ensinado a não confundir streaming com cache.

CUIDADO | Caveat sob Bun, registrado no ADR

“sob Bun o build imprime um aviso de que o Next não garante o comportamento de `Cache Components` por causa da implementação de `setTimeout` do runtime. O output sai correto; se um dia morder, remover a flag custa uma linha (a rota volta a ser dinâmica pura).” Saída de emergência documentada antes de ser necessária.

13.5 Bordas e validação

ADR-005: valibot, não zod

São duas decisões independentes empacotadas numa: *qual* validador e *onde* ele pode viver.

O *qual*: valibot é mais leve que zod, e isso importa porque este repo **tem** validação no client (os forms RHF usam `valibotResolver`, então o schema entra no bundle). O *onde*: importável só em `*.schema.ts`, o que centraliza a validação num tipo de arquivo com nome previsível. Alternativas descartadas: zod (bundle maior) e zod/mini ("meio-termo não necessário"). O fato de o meio-termo ter sido considerado e recusado mostra que a escolha foi deliberada.

O confinamento é imposto, com mensagem literal: *"valibot só pode ser importado em arquivos *.schema.ts."*

ADR-006: ky confinado a lib/api

Todo acesso ao backend passa por `apiClient`, que **exige** um validator. O objetivo é gateway único, tipado e validado, impedindo fetch cru espalhado pelo código. Mensagem da regra: *"ky só pode ser importado dentro de src/lib/api."* Complementam: axios banido e a regra boilerplate/api-through-lib como "error".

Há um segundo grep-gate no CI contra URL do backend fora da pasta, e o comentário dele mostra o nível de cuidado do repo com falso-negativo:

```
.github/workflows/ci.yml: por que e não a linha inteira
```

```
# `-o` extrai uma URL por linha (não a linha inteira): sem
isso, uma linha
# com uma URL pública E uma de backend juntas seria
descartada pelo
# allowlist, escondendo a violação. Troque `example\.com`
pelo seu domínio.
```

ADR-007: Auth por cookie JWT; authz é do backend

REGRA | O front nunca decodifica, valida ou assina JWT

O cookie JWT é emitido pelo backend. O front encaminha o cookie e mais nada. Cookies públicos existem só para UI. `jsonwebtoken` e `jose` estão banidos por import, inclusive por subpath (`jose/jwt/verify`).

O `src/proxy.ts` inteiro tem 43 linhas e só checa presença de cookie:

src/proxy.ts

```
export function proxy(request: NextRequest): NextResponse {
  if (request.cookies.has(SESSION_COOKIE)) {
    return NextResponse.next();
  }
  return NextResponse.redirect(new URL("/login", request.url));
}
```

O JSDoc separa as duas coisas que costumam ser confundidas: “É **gate de roteamento**, não **autorização**. A **autorização real** é sempre do backend, que **rejeita as chamadas de dado independentemente do que o cookie diga**; aqui o objetivo é não largar o usuário numa tela que responderia 403.” O matcher entra por prefixo, com porquê próprio: “*listar rota a rota deixa toda rota nova sem gate por padrão, que é o erro caro.*”

CUIDADO | Duas coisas para conferir antes de adotar

- 1. A premissa é mesmo eTLD+1.** O Contexto do ADR-007 diz: front e backend no mesmo eTLD+1 (.example.com) → cookie first-party SameSite=Lax, sem BFF e sem CSRF token. Se você rodar front e backend em domínios diferentes, a base do ADR cai e as alternativas descartadas (BFF proxy dedicado) voltam à mesa.
- 2. O refresh de token não está implementado.** O ADR-007 e o ADR-010 descrevem “refresh no proxy.ts”, mas o arquivo entregue só faz o gate por presença de cookie. O próprio JSDoc é honesto: “*quando você precisar de refresh proativo, é aqui que ele entra, relayando o Set-Cookie da resposta.*” É ponto de extensão documentado, não código entregue.

ADR-015: Forms com react-hook-form + valibotResolver

Este é o ADR que melhor mostra por que o arquivo existe: ele **contraria o default do framework**, admite o custo e diz em que situação a alternativa descartada seria a certa.

Decisão: forms usam RHF + valibotResolver, com schema em `*.schema.ts`, e o **mesmo schema revalida no server/backend** (o client é só UX). Mutações de dado vai por Server Action; auth que seta cookie (login/refresh) é browser-direct.

POR QUE | Dois motivos, e o segundo é um constraint técnico duro

“app autenticado (CRM) com forms ricos (validação por-campo, máscaras, campos dinâmicos) e o constraint do cookie de auth (Server Action não entrega Set-Cookie ao browser). RHF minimiza re-render.”

Ou seja: o login *não pode* ser Server Action, porque o Set-Cookie HttpOnly precisa pousar no browser. Isso não é gosto, é como cookie funciona.

A alternativa descartada vem com o mapa de quando ela seria melhor: “*useActionState + Server Action puro (default do Next, melhor pra forms públicos/simples e progressive enhancement, mas pobre em UX rica e incompatível com o login browser-direct). É preferível em páginas públicas: não é o caso aqui.*” Se o seu produto tiver landing pages com formulário público, o ADR já te diz que ali a escolha é outra.

O custo é aceito e nomeado: RHF + schema vão para o bundle do client (aceitável em área logada, e a validação no server continua). A regra boilerplate/no-watch-in-effect guarda o anti-pattern mais comum de RHF.

Status: ativo (corrige a contradição anterior do CODE-PATTERN que sugeria useActionState). O ADR existe, entre outras coisas, para consertar uma divergência entre documentos. O [CODE-PATTERN.md](#) fecha o loop apontando de volta para ele.

ADR-022: lib/api entregue como reference implementation

Decisão de produto: o boilerplate entrega lib/api (apiClient, kyServer, kyClient, base-url, endpoints), lib/env, [src/proxy.ts](#) e duas features de exemplo (auth, perfil), não só as regras.

POR QUE | Regra que aponta para código inexistente é "regra que mente"

"as regras de lint, as skills e os docs sempre falaram de apiClient(kyServer, rota, schema), mas o src/ só tinha layout.tsx e page.tsx. Regra que aponta para código inexistente é a 'regra que mente' que o CLAUDE.md proíbe, e o comprador teria de reconstruir a peça central antes da primeira feature."

O detalhe técnico é o mais elegante do repo, e responde uma pergunta que surge sozinha: se valibot só pode ser importado em [*.schema.ts](#), como o apiClient valida?

src/lib/api/client.ts: validação por Standard Schema

```
* A validação roda pela interface Standard Schema (`schema["~standard
"]`), não
* pela API do valibot: o lint confina `import "valibot"` aos `*.schema.
ts`,
* então `lib/api` valida sem acoplar a um validador específico.
const result = await dataSchema["~standard"].validate(envelope.data);
```

Essa chamada ao método validate acontece em runtime. Por isso, @standard-schema/spec é dependência da aplicação, não apenas de desenvolvimento.

O **Escopo consciente** do ADR também é verificável: não há variante paginada nem lib/auth, porque "zero caller = YAGNI (CODE-PATTERN §9), e o knip reprovaria". O arquivo termina justificando a própria ausência:

src/lib/api/client.ts: as últimas linhas do arquivo

```
// Sem variante paginada aqui de propósito: o repo proíbe abstração sem
// caller
// real (YAGNI, ver CODE-PATTERN §9). Quando o seu primeiro endpoint
// devolver
// `pagination` como irmã de `data` no envelope, extraia uma `
// apiClientPaginated`
// reaproveitando o `requestEnvelope` acima — a borda já está pronta
// pra isso.
```

Se você procurar `src/lib/auth` e não achar: é ausência deliberada, com ADR.

13.6 Arquitetura

ADR-008: Feature-based, shared folha, sem cross-feature

Camadas `app` → `features` → `lib` → `shared`; `shared` não importa nada interno; `feature` não importa sibling (o que for compartilhado sobe para `shared`). Sem barrel. Imposto por `boilerplate/enforce-boundaries` e `boilerplate/no-index-barrel`, ambas "error".

O que vale ler é o descarte. A alternativa mais popular hoje, uma camada `entities` à la Feature-Sliced Design, é recusada em uma linha: "domínio do front é fino: vive no backend". E o import lateral livre entre `features`: "vira spaghetti". Duas frases, discussão encerrada.

ADR-010: `src/proxy.ts`, não `middleware.ts`

O Next 16 renomeou `middleware.ts` para `proxy.ts`. O guardrail proíbe o nome antigo, e há teste dedicado para a regra (`no-middleware-file.test.ts`).

CUIDADO | A localização é `src/proxy.ts`, irmão de `app/`: não dentro de `app/`

Campo Localização (correção): do ADR: "o arquivo é `src/proxy.ts`, na raiz de `src`, irmão de `app/`, não dentro de `app/`. Ver `node_modules/next/dist/docs/01-app/03-api-reference/03-file-conventions/proxy.md`. Os docs diziam `src/app/proxy.ts`, que o Next simplesmente não carrega."

Colocado no lugar errado, o arquivo não dá erro: ele apenas não roda. O gate de rota some em silêncio.

Este é o melhor exemplo concreto do que o `AGENTS.md` avisa na primeira linha ("This is NOT the Next.js you know"): uma convenção nova, onde a memória do modelo e os docs internos do projeto estavam errados, e o único árbitro confiável foi o doc dentro do `node_modules`. O caminho citado existe; não há `middleware.md` nessa pasta.

ADR-014: Docs canônicos + inject como resumo

[ARCHITECTURE.md](#), [CODE-PATTERN.md](#) e [MEMORY.md](#) são fonte canônica para humano e agente. O inject de SessionStart é um resumo curto que aponta para eles, com precedência declarada em dois lugares: no JSDoc do script (*"Fonte canônica dos guardrails: AGENTS.md. Este texto é o resumo carregado por sessão; quando divergir, o AGENTS.md vence."*) e no fim do próprio texto injetado.

O mesmo JSDoc traz a doutrina de escrita do inject, que explica por que o texto é imperativo:

```
tooling/agent-hooks/inject-patterns.ts

* REGRAS, NÃO OPÇÕES: cada linha é uma diretiva única. Não oferecer alternativas
* (menu vira alucinação): sempre o caminho prescrito.
```

13.7 Enforcement: a tese do produto

ADR-009: Guardrails por oxlint, não por .md

REGRA | .md é referência; o gate é o lint

Estrutura, arquitetura e padrão são impostos por oxlint (built-in + overrides por zona + plugin custom boilerplate via jsPlugins). Contexto do ADR: *"código escrito por agentes → regra determinística > convenção textual."*

O risco está registrado junto com a mitigação: jsPlugins é alpha e não segue semver, então o oxlint é **pinado** ("oxlint": "1.77.0", sem ^, em contraste com quase todo o resto do devDeps) e há grep-gates de CI como backstop. E há um procedimento: *"Subir a versão exige rodar bun test das regras custom: elas são o canário da API do plugin."*

Números do repo: 20 arquivos de regra em [tooling/oxlint-rules/rules/](#), sendo 15 registradas como "error" e 2 como "warn".

As regras custom do plugin boilerplate

Severidade	Regras
error	enforce-boundaries, no-index-barrel, no-use-cache, no-force-static, api-through-lib, no-fetch-in-effect, no-watch-in-effect, no-derived-state-in-effect, no-manual-memo, no-forbidden-imports, no-focused-tests, no-assert, no-defensive-parse, no-schema-output-converter, no-catalog-literal-compare
warn	no-passthrough-mapper, no-literal-dispatch

As duas em warn têm justificativa escrita no `.oxlintc.json`: elas veem só a AST do arquivo, sem saber quantos callers a função tem. Como supressão de lint é proibida (ADR-020), error ali seria bloqueio sem saída para um padrão às vezes legítimo.

ADR-020: no-lint-suppression é script, não regra de lint

O ADR mais bonito do arquivo, e o raciocínio cabe em uma frase.

POR QUE | Uma regra de lint pode ser desligada pela diretiva que ela proíbe

"uma regra de lint pode ser desligada pela própria diretiva que ela proíbe; o guard se anulava. Um `git grep` fora do linter, não. Usa `git grep` quando há repositório e cai para `grep -r` quando ainda não há (o comprador roda `verify` antes do `git init`)."

Metanível diferente = imunidade. A solução é a ferramenta mais burra possível, e é exatamente por isso que funciona.

A ausência da regra está documentada no próprio plugin ([tooling/oxlint-rules/index.ts](#)), para que ninguém "conserte" o buraco reimplementando-a como regra. O script vive em [tooling/no-lint-suppression.ts](#) (63 linhas), tem teste próprio, e roda dentro do `verify`.

Dois detalhes de implementação que valem a leitura. O primeiro: o script se esconde da própria busca.

`tooling/no-lint-suppression.ts`

```
// Montados em runtime pra este próprio arquivo não dar auto-match na busca.  
const NEEDLES = [["eslint", "disable"].join("-"), ["oxlint", "disable"].join("-)];
```

O segundo: ele falha fechado. `exit 1` nas duas ferramentas de busca significa apenas "nenhuma ocorrência", então o script só trata como erro real quando `result.exitCode > 1`. Nesse caso, aborta, em vez de passar verde por não ter conseguido procurar.

A consequência declarada no ADR também é real: a mensagem do `no-manual-memo` deixou de oferecer supressão como escape hatch, porque isso era "a própria doutrina se contradizendo". Hoje ela é só "React Compiler memoiza sozinho. `useMemo/useCallback/-memo manual` é redundante. Remova a memoização manual."

ADR-017: Knip como gate de código morto

POR QUE | Num boilerplate, código morto é pior que em app normal

"o `no-unused-vars` do `oxlint` só enxerga dentro do módulo. Código morto cross-module é o que realmente acumula, e num boilerplate é pior: o comprador herda arquivos que ninguém chama e presume que são necessários."

E aqui está a melhor história do arquivo: uma ferramenta de dead code **obrigou a existência de uma feature**. Registrado no campo Efeito imediato: o knip pegou o

server-only órfão no `package.json` e forçou o `lib/api/ky.client.ts` a ter caller real. Nas palavras do registro, *"foi o knip que exigiu a tela de login, fechando o ciclo de auth que o proxy.ts já prometia ao redirecionar para /login"*.

O triângulo estava aberto: o `proxy.ts` redirecionava para `/login`, `/login` não existia, e o `ky.client.ts` não tinha quem o chamasse. Hoje `src/app/login/page.tsx` existe, `src/features/auth/` existe, e o server-only tem caller único e real em `src/lib/env.ts`, com o porquê no JSDoc (*"server-only garante que importar isto de um Client Component quebra o build"*).

O knip roda tanto no `verify` quanto no `verify:fast`.

ADR-026: react-perf removido

Vale citar como prova de que a régua deste repo não é "mais regra é melhor". O plugin `react-perf` e as regras `jsx-no-new-*` saíram do `.oxlintrc.json`, porque o React Compiler estabiliza props automaticamente (mesma razão do `boilerplate/no-manual-memo`). Na prática elas *"proíbiam qualquer handler de evento inline em client component, gerando atrito sem ganho"*.

O comentário sobreviveu no lugar de onde as regras saíram, para que ninguém as adicione de volta:

`.oxlintrc.json`

```
// react-perf removido: o React Compiler estabiliza props automaticamente
// (mesma razão do boilerplate/no-manual-memo). As regras jsx-no-new-* eram
// redundantes e impediam qualquer handler de evento em client component.
"react/react-compiler": "error", // projeto usa React Compiler
```

ADR-027: oxlint-tailwindcss fora (o único "não adotado")

O plugin barraria paleta default e valor arbitrary do Tailwind. Foi avaliado e recusado *por enquanto*, com um porquê específico do estágio do produto: *"ele só tem valor com um @theme real preenchido em globals.css, e aqui o design system é template. O DESIGN.md é para o adotante preencher. Ligar agora reprovaria qualquer classe do comprador antes de ele definir os tokens."*

Verificável: `src/app/globals.css` tem um `@theme` inline com apenas quatro tokens (`--color-background`, `--color-foreground`, `--font-sans`, `--font-mono`), que formam o template default. E o gatilho de revisão está escrito: *"Revisitar quando: o DESIGN.md do projeto adotante estiver preenchido e os tokens vivos no @theme."* Se você preencher o `DESIGN.md`, este ADR é a sua próxima tarefa de toolchain.

13.8 Processo de PR

ADR-013: a decisão de não decidir

REGRA | Modelo de branch é escolha sua, e o lugar de gravar é o CLAUDE.md

O boilerplate não prescreve gitflow nem trunk-based. Default: trunk-based em `main`, usado pelas skills `pr-ready` e `pr-creation`. Porquê: *"cada time tem seu fluxo; impor um cria atrito. `main` é o default git moderno e o menor denominador comum."*

Na prática o default está em três camadas, e a de cima é editável por você. O `CLAUDE.md` tem uma seção dedicada: *"`main` (default trunk-based)". Edite esta linha se o time usa outro fluxo... se estiver no default e ainda não confirmado, o agente pergunta **uma vez** antes de abrir PR e grava a escolha aqui: depois não pergunta mais."*

Abaixo dela, o hook aponta explicitamente para o doc:

```
tooling/agent-hooks/pre-pr-review-gate.ts
```

```
/** Base default de PR. Alinhada à seção "Branch base" do CLAUDE.md. */
const BASE_BRANCH = "main";
```

O hook rejeita base fora de `main` ou `feature/*`. O segundo caso permite PR empilhado sobre outra `feature`.

ADR-016: AGENTS.md como contrato comum, hooks multi-host

POR QUE | Guardrail que só existe num host não é guardrail

"antes os hooks eram .sh dentro de .claude/, então Codex e Cursor rodavam sem guardrail nenhum. A regra existia só onde o .md era lido. Script único + JSON fino elimina a divergência entre hosts por construção, e hook-configs.test.ts reprova o gate se um host apontar para um script que não existe ou que estoura."

Os hooks vivem em `tooling/agent-hooks/*.ts` (TypeScript rodado pelo Bun) e cada host só aponta para eles pelo seu JSON. O `host.ts` normaliza a I/O dos três contratos. A tabela abaixo mostra por que essa camada precisa existir: os nomes de evento diferem host a host, mas o script é o mesmo.

O mesmo script, três contratos de host

Script	Momento	Variações entre hosts
<code>inject-patterns</code>	Início da sessão	<code>SessionStart</code> ou <code>sessionStart</code>
<code>reinforce-patterns</code>	Envio do prompt	<code>UserPromptSubmit</code> ou <code>beforeSubmitPrompt</code>
<code>pre-pr-review-gate + guard-shell</code>	Antes do shell	<code>PreToolUse</code> ou <code>beforeShellExecution</code>
<code>format-after-write</code>	Depois da edição	<code>PostToolUse</code> ou <code>afterFileEdit</code>
<code>verify-before-stop</code>	Fim do turno	<code>Stop</code> ou <code>stop</code>

O campo Detalhe do ADR guarda uma sutileza que salva o primeiro contato do comprador com o produto: todos os comandos, nos três JSONs, resolvem o caminho com `$(git rev-parse --show-toplevel 2>/dev/null || pwd)`. “O fallback importa: o comprador extrai o zip e abre o agente **antes** do `git init`; sem ele, todo hook morreria no `git rev-parse`.” Alternativas descartadas: caminho relativo (quebra quando o agente roda a partir de subdiretório) e duplicar o script por host (“garante drift”).

ADR-021: Pré-review com selo por HEAD

POR QUE | “Regra que dá para pular não é guardrail”

A tese do produto inteiro em oito palavras. O ADR: “*pr-ready é mecânico (lint/type/test/-diff/secrets) e não pega o que custa caro na review humana: abstração especulativa, fallback defensivo, regra de negócio inventada no front, slop. Sem gate, a etapa vira opcional, e regra que dá para pular não é guardrail.*”

A mecânica: a skill pre-review revisa a branch com subagentes por domínio e grava um selo carimbado com o commit atual.

Como a skill grava o selo (`.agents/skills/pre-review/SKILL.md`)

```
mkdir -p .agents/tmp
git rev-parse HEAD > .agents/tmp/pre-review-$(git rev-parse HEAD).ok
```

O hook `tooling/agent-hooks/pre-pr-review-gate.ts` só reage a `gh pr create` e barra sem o selo do HEAD atual:

```
tooling/agent-hooks/pre-pr-review-gate.ts
```

```
if (!existsSync(`.agents/tmp/pre-review-${head}.ok`)) {
  emitDeny(
    host,
    `PR travada: a pré-review de guardrails não rodou pro commit atual
    (${head.slice(0, 8)}).\n` +
    `Rode a skill \pre-review\` (\pre-review\`) primeiro. Ela
    grava o selo que libera o gh pr create. ` +
    `Qualquer commit novo depois disso exige nova pré-review.`
  );
}
```

Carimbar por HEAD é o que impede o truque óbvio: rodar a pré-review, commitar mais três coisas e abrir o PR assim mesmo. Commit novo, selo inválido.

O mesmo hook aplica um teto de `MAX_PR_LOC = 3000` (add + del contra origin/<base>), com ack **separado e também por HEAD** (`.agents/tmp/pr-size-ack-<HEAD>.ok`). A mensagem manda o agente perguntar antes: “Antes de abrir, pergunte ao usuário: dividir em PRs menores (recomendado) vs seguir.”

Cadeia declarada: `/pr-ready` → `/pre-review` → `/create-pr`, e o ADR afirma que elas “*não se sobrepõem*”. Isso é concreto: a skill `pre-review` tem uma seção “**Não duplicar gates automáticos**” listando o que ela deliberadamente *não* reexamina (cache proibido, barrels, camadas, `document.cookie`, JWT, `middleware.ts`, dead code, runtime Node), porque “*bun run verify cobre essas famílias*”. A skill gasta atenção só onde o gate mecânico é cego.

E há confirmação humana obrigatória antes de o selo liberar o PR: “Depois do relatório, peça confirmação ao usuário antes de liberar a PR.”

13.9 Entrega do produto

ADR-023: `bun.lock` versionado e `.gitignore` no pacote

Parece trivial, mas é o ADR que impede o produto de nascer quebrado.

POR QUE | Sem lockfile, o CI saía vermelho por construção

“o CI roda `bun install --frozen-lockfile`. Sem lockfile versionado ele falhava no primeiro push, ou seja, o produto saía com o CI vermelho por construção. Sem `.gitignore`, o primeiro `git add .` do comprador commitava `node_modules/`, `.next/` e `.env.local`.”

Verificado na raiz: `bun.lock`, `.gitignore` e `.gitattributes` presentes; o CI de fato usa `--frozen-lockfile`.

O padrão transversal: "o comprador extrai o zip e roda antes do git init"

Essa premissa aparece em **três** ADRs independentes (016, 020, 023) e é a assinatura de que o produto foi testado a partir do estado de *entrega*, não do estado de *desenvolvimento*. Ela tem consequência em código, em três lugares distintos:

- `tooling/no-lint-suppression.ts` detecta se há repositório e alterna entre `git grep` e `grep -rnI --exclude-dir=node_modules --exclude-dir=.next`.
- Os três JSONs de hooks usam `$(git rev-parse --show-toplevel 2>/dev/null || pwd)`.
- O próprio diretório entregue **não é um repositório git**. `git rev-parse --is-inside-work-tree` falha na raiz. É a prova viva do cenário.

A cadeia completa de gates

As decisões acima convergem numa pilha de seis camadas, todas com comando literal verificável.

package.json: os dois níveis de verificação

```
"verify:fast": "bun run lint && bun run fmt:check && bun run knip && bun run test",
"verify": "bun run lint && bun run fmt:check && bun run typecheck && bun run typecheck:tooling && bun run knip && bun run test && bun run tooling/no-lint-suppression.ts && bun run tooling/no-heavy-public-assets.ts"
```

Onde cada gate roda

Camada	Gatilho	O que roda
Agente, pós-escrita	PostToolUse / afterFileEdit	formata e reforça as regras
Agente, fim de turno	Stop / stop	<code>bun run verify:fast</code> (bloqueia o encerramento)
Agente, antes do PR	PreToolUse Bash	selo de pré-review + teto de 3000 LOC + base branch
Git pre-commit	arquivos staged	<code>oxlint + oxfmt --check</code> só nos staged
Git pre-push	push	<code>bun run verify</code> no projeto inteiro
CI	push / pull_request	<code>2 grep-gates + bun run verify + bun run build</code>

O `lefthook.yml` explica a divisão em dois níveis com uma frase: *"Pré-commit: barato e focado nos arquivos staged, pra não travar o loop de commit. O gate completo é o*

pré-push.” E o agente que tentar encerrar o turno com o gate vermelho recebe: *“Stop bloqueado: bun run verify:fast falhou. Corrija os erros do gate e só então encerre o turno.”*

13.10 Onde o log admite dívida

Um log de decisões vale pelo que ele *não* maquia. Estes são os pontos em que o [MEMORY.md](#) descreve algo que ainda não existe no repo. Todos estão declarados no próprio arquivo, nenhum escondido.

Dívidas e lacunas declaradas

ADR	O que ainda não está lá
002	O smoke test de output: 'standalone' sob Bun é exigido pelo ADR, mas não há Dockerfile nem output: 'standalone' no repo. Status literal: deploy ainda não validado.
007 / 010	O refresh de token no proxy.ts está descrito como decisão, mas não implementado. O arquivo só faz o gate por presença de cookie; o refresh é ponto de extensão documentado no JSDoc.
004	Cita useActionState/useOptimistic, nenhum dos dois presente em src/; o ADR-015 depois descartou useActionState para forms.
027	Decisão pendente por design, aguardando o DESIGN.md preenchido.

Há também coisas no repo **sem** ADR próprio, e vale saber que são escolhas não documentadas no log: [tooling/no-heavy-public-assets.ts](#) (o segundo script de gate, dentro do verify e com teste próprio), a escolha do lefthook como runner de git hooks, typedRoutes: true e productionBrowserSourceMaps: false no [next.config.ts](#), e o rewrite de dev que está comentado nesse mesmo arquivo. Nenhuma delas contradiz um ADR existente; simplesmente não passaram pelo log.

13.11 Continuar o log (recomendação, não regra do repo)

CUIDADO | Sendo exato: não existe regra no repo mandando você escrever novos ADRs

Nenhum lint, hook, script ou doc do boilerplate obriga a adicionar entradas ao [MEMORY.md](#). Todas as referências ao arquivo, nos [.md](#), nos [.json](#) e nos [.ts](#), são de *leitura*. O que vem a seguir é recomendação deste guia, apoiada em fatos do repo, não uma regra imposta.

Dito isso, três fatos verificáveis fazem o hábito valer a pena:

1. **O agente lê o [MEMORY.md](#) em toda sessão.** O inject de SessionStart fecha apontando: *“Para detalhe/exemplo/porquê: ARCHITECTURE.md · CODE-PATTERN.md · MEMORY.md (log de decisões).”* ADR que você escrever entra no caminho de contexto do

agente daí em diante. É a forma mais barata de ensinar uma decisão sua a um agente que você nem abriu ainda.

2. **Os outros docs já delegam rationale por número.** O [CODE-PATTERN.md](#) manda para o ADR-015, o ADR-003 emenda o ADR-025, o ADR-022 cita o ADR-007, e o [src/proxy.ts](#) cita (ADR-007) dentro do código. Parar de escrever quebra essa cadeia: a próxima decisão sua vira folclore de PR.
3. **O formato já tem tudo que você precisa. Decisão · Contexto · Por quê · Alternativas descartadas · Status**, mais campos ad-hoc quando fizer falta (`Risco` registrado, `Caveat` conhecido, `Revisitar` quando). Numeração contínua, `append-only`, e `Status` honesto, inclusive `não adotado` (`candidato`) quando você avaliar algo e recusar.

REGRA | Se for continuar o log, mantenha as três propriedades que o tornam útil

Append-only: reverteu? Nova entrada ou emenda cruzada, nunca `git rm`. **Alternativa descartada nomeada:** sem ela, a próxima pessoa reavalia a mesma opção do zero. **Status honesto:** o ADR-002 diz `deploy` ainda `não validado` e o ADR-027 diz `não adotado`. Um log em que tudo está ativo e tudo deu certo é marketing, não memória.

Os melhores ADRs deste arquivo têm uma característica em comum, e ela é copiável: eles registram o *custo*, não só o benefício. O ADR-015 admite que RHF engorda o bundle e diz onde a alternativa descartada seria melhor. O ADR-024 admite que perdeu duas flags. O ADR-002 admite que a justificativa é fraca e registra o risco em seguida. O ADR-025 admite um warning sob Bun e já deixa a saída de emergência escrita. Decisão sem custo declarado não foi decidida; foi só preferida.

PARTE 14

Quando algo dá errado

O QUE VOCÊ LEVA DESTE CAPÍTULO

Você vai identificar qual gate falou, interpretar a mensagem e chegar à correção sem procurar um atalho que desligue a proteção. Use este capítulo como diagnóstico, não como leitura linear.

Este boilerplate não tem um `TROUBLESHOOTING.md` separado. A regra vive perto do código que a impõe, para não virar uma página esquecida. O custo dessa escolha aparece na primeira semana: algumas mensagens de erro não contam sozinhas de onde vieram.

Esta seção faz a ponte. Cada sintoma abaixo foi reproduzido no repositório real: as mensagens vêm dos scripts que as emitem, os outputs de build vêm de execuções com o código quebrado e as regras de escape vêm dos testes que provam o comportamento. Use os blocos como pistas de diagnóstico, não como comandos para colar sem ler.

14.1 Primeiro passo: descobrir qual gate te barrou

Existem cinco lugares diferentes que podem reprovar você, com escopos diferentes. Saber qual falou é metade do diagnóstico.

Os gates do repo, do mais barato ao mais completo

Gate	O que roda
Pré-commit (lefthook)	<code>oxlint e oxfmt --check só nos arquivos staged, sem --type-aware</code>
Hook de Stop do agente	<code>bun run verify:fast</code> no projeto inteiro
Pré-push (lefthook)	<code>bun run verify</code>
CI (.github/workflows/ci.yml)	<code>dois grep-gates, bun run verify e bun run build</code>
Hooks PreToolUse	barram o comando antes de ele rodar (shell e <code>gh pr create</code>)

A composição exata dos dois scripts de verificação, lida do `package.json`:

```
package.json: os dois gates
```

```
"verify:fast": "bun run lint && bun run fmt:check && bun run knip &&
  bun run test",

"verify": "bun run lint && bun run fmt:check && bun run typecheck &&
  bun run typecheck:tooling && bun run knip && bun run test && bun
  tooling/no-lint-suppression.ts && bun tooling/no-heavy-public-assets.
  ts"
```

O encadeamento é com `&&`: a execução para na **primeira** etapa vermelha. Se o `lint` falhar, você não vê o resultado do `knip`, e isso é de propósito, o loop fica curto. Mas significa que “consertei o `lint` e agora apareceu erro de `knip`” não é uma regressão, é a segunda etapa finalmente rodando.

CUIDADO | `verify:fast` não roda `typecheck`

Compare as duas linhas acima: `typecheck` e `typecheck:tooling` existem só no `verify`. O [AGENTS.md](#) confirma em uma linha: “*bun run verify:fast:subconjunto rápido pro loop (sem typecheck). É o que o hook de Stop roda.*”

Consequência prática: **um erro de tipo atravessa o hook de Stop inteiro**. O agente encerra o turno com tudo verde e o erro só estoura no pré-push ou no CI. Some a isso o fato de o pré-commit rodar `oxlint sem --type-aware` (veja o [lefthook.yml](#)) e você tem duas classes de erro; tipo e `lint type-aware`; que só aparecem no `git push`.

Antes de commitar de verdade, rode `bun run verify` completo uma vez. Custa mais, mas é literalmente o mesmo comando que o pré-push e o CI vão rodar.

14.2 O hook de Stop bloqueia o encerramento do turno

14.2.1 O que é

[tooling/agent-hooks/verify-before-stop.ts](#) tem 26 linhas e faz uma coisa só:

```
tooling/agent-hooks/verify-before-stop.ts (núcleo)
```

```
const { host, raw } = await readHookInput();

if (isStopHookActive(raw) || raw.status === "aborted") {
  emitStopOk();
}

const verification = Bun.spawnSync(["bun", "run", "verify:fast"], {
  cwd: process.cwd(),
  stdout: "inherit",
  stderr: "inherit",
});

if (verification.exitCode !== 0) {
  emitStopContinue(
    host,
```

```
tooling/agent-hooks/verify-before-stop.ts (núcleo) (continuação)
```

```
    "Stop bloqueado: `bun run verify:fast` falhou. Corrija os erros do
    gate e só então encerre o turno.",
  );
}
```

O mesmo script está registrado nos três hosts: em [.claude/settings.json](#) no evento `Stop` com `timeout: 300000` (cinco minutos), em [.codex/hooks.json](#) com `statusMessage "Rodando bun run verify:fast"`, e em [.cursor/hooks.json](#) no evento `stop` com `loop_limit: 3`.

POR QUE | Por que o gate roda no Stop e não no commit

O código deste repo pode ser produzido com apoio de agentes. Quando um turno termina com o gate vermelho, o custo costuma cair no próximo contexto: você abre o editor e encontra o problema. Rodar no `Stop` devolve o erro para quem acabou de escrever, ainda dentro do contexto em que ele foi criado. É o único ponto do fluxo em que consertar é barato.

14.2.2 O sintoma

Como o `spawnSync` usa `stdout: "inherit"`, você vê o output **inteiro** do `verify:fast`; os erros de `oxlint`, `oxfmt`, `knip` ou `bun test` aparecem como se você tivesse rodado o comando na mão. Depois disso, no Claude Code, aparece uma frase fixa em inglês:

```
stderr do hook, no Claude Code e no Codex
```

```
Stop blocked: bun run verify:fast failed.
```

CUIDADO | A mensagem em PT-BR só aparece no Cursor

O `reason` em português que o hook monta é descartado nos hosts que usam `exit code`. Veja [tooling/agent-hooks/host.ts](#):

```
host.ts: emitStopContinue
```

```
export function emitStopContinue(host: AgentHost, reason: string):
  never {
    if (host === "cursor") {
      process.stdout.write(JSON.stringify({ followup_message: reason
    }) + "\n");
      process.exit(0);
    }
    // Claude: exit 2 bloqueia. Codex: exit 2 + stderr continua o
    // turno.
    process.stderr.write("Stop blocked: bun run verify:fast failed.\
n");
    process.exit(2);
  }
```

Não procure a frase "Stop bloqueado" no Claude Code: ela não vai aparecer. O sinal útil é o output do `verify:fast` logo acima.

14.2.3 O que fazer

Rode `bun run verify:fast` você mesmo e ataque a **primeira** etapa vermelha. Boa parte do ruído é mecânico e some com dois comandos:

```
Limpeza mecânica antes de investigar
```

```
bun run lint:fix
bun run fmt
bun run verify:fast
```

Se sobrou erro depois disso, é conteúdo, não formatação, e cada categoria tem sua subseção abaixo.

14.2.4 O escape hatch, e o que ele não é

O hook sai cedo quando `stop_hook_active` é verdadeiro ou quando `raw.status === "aborted"`. Ou seja, uma segunda tentativa de Stop encadeada não re-verifica; é uma proteção contra loop de hook, não um bypass que você aciona de propósito. O teste [tooling/agent-hooks/verify-before-stop.test.ts](#) cobre isso com o caso *"skips verification while a Stop hook is already active"*, verificando que o arquivo sentinela nem chega a ser criado.

REGRA | Nunca desligue o hook para "destravar"

Se você editar o `.claude/settings.json` pra remover o hook de Stop, você não resolveu nada: o pré-push roda o `verify` completo (superconjunto do `verify:fast`) e

o CI roda o `verify` **mais** o `build`. Você só trocou um erro barato, com contexto, por um erro caro, sem contexto.

14.3 O guard de shell bloqueou um comando

`tooling/agent-hooks/guard-shell.ts` roda em `PreToolUse` com matcher Bash nos três hosts (timeout: 15000 no Claude, failClosed: true no Cursor). Ele avalia duas famílias, nesta ordem.

14.3.1 Família 1: destrutivo ou vazamento de segredo

Os padrões são literais do arquivo:

guard-shell.ts: padrões barrados

```
const DESTRUCTIVE: ReadonlyArray<{ pattern: RegExp; label: string }> =
[
  { pattern: /\brm\s+(?:-[a-zA-Z]+\s+)*(?:\|~|\*)\|/(?:(?:\s|$)/, label:
    "rm no diretório raiz/home" },
  { pattern: /\bgit\s+reset\s+--hard\b/, label: "git reset --hard" },
  { pattern: /\bgit\s+clean\s+-[a-zA-Z]*f/, label: "git clean -f" },
  { pattern: /(?:^|[\|;&]\s*)sudo\s/, label: "sudo" },
  { pattern: /\bchmod\s+(?:-[a-zA-Z]+\s+)*777\b/, label: "chmod 777" },
];

const READS_REAL_ENV = /\b(?:cat|less|more|head|tail|bat)\s+[^|;&]*\.
  env(?:!\.example)\b/;
const UNSAFE_FORCE_PUSH = /\bgit\s+push\b[^|;&]*(?:--force(?:\s|$)|\s-f
  (?:\s|$))/;
const SAFE_FORCE_FLAGS = /--force-with-lease|--force-if-includes/;
const PIPE_TO_SHELL = /\b(?:curl|wget)\b[^|]*\|[^|]*\b(?:bash|sh|zsh)\b
  /;
```

As mensagens que você vai ver, e a saída de cada uma:

Mensagens do guard de shell e a correção

Mensagem	Como sair
Comando bloqueado: casa padrão destrutivo <i>label</i>	O <i>label</i> diz qual dos cinco padrões casou. Reescreva o comando ancorado no alvo real.
leitura de arquivo .env real (segredo)	Leia o <code>.env.example</code> : a mensagem já sugere isso, e é ele que lista as chaves esperadas.
<code>git push --force</code> . Use <code>--force-with-lease</code> .	Troque a flag. <code>--force-with-lease</code> e <code>--force-if-includes</code> passam.
download canalizado pra shell	Baixe o script, leia, e só então execute.
o shell guard recebeu JSON inválido	Reexecute; se persistir, a configuração de hooks do host está quebrada.

14.3.2 Família 2: branch protegida

```
guard-shell.ts: lista de branches
```

```
const PROTECTED = ["main", "release", "staging", "develop"];
```

O bloqueio dispara em **dois** cenários independentes, e o segundo pega gente desprevenida:

1. A branch **atual** está na lista; qualquer `git commit` ou `git push` morre ali.
2. O `git push` **menção**a uma branch protegida nos argumentos. `git push origin HEAD:main` é bloqueado mesmo você estando numa feature branch. Existe teste dedicado pra isso: *"denies a push that targets a protected branch from a feature branch"*.

A mensagem inclui a branch atual e aponta pra saída correta: criar uma feature branch e abrir PR com base na main. O `AGENTS.md` diz a mesma coisa em prosa: *"Nunca commite nem pushe direto na main; o hook guard-shell bloqueia."*

14.3.3 Como reconhecer um falso-positivo (o repo já resolveu três classes)

Antes de acusar o hook de estar errado, cheque se seu caso não é um dos três que já foram tratados. Cada um tem teste.

1. `rm -rf` é ancorado no alvo, não em substring. O comentário está no próprio arquivo:

guard-shell.ts: comentário do padrão de rm

```
* `rm -rf` é ancorado no ALVO, não em substring: `rm -rf /` casaria dentro de
* `rm -rf /home/user/projeto/dist`, bloqueando um comando legítimo. Só alvo raiz
* (`/`), home (`~`) e o coringa (`*`) – sozinhos ou com barra final – entram.
```

Comandos que **passam**, verificados em `guard-shell.test.ts`: `rm -rf /home/user/projeto/dist`, `rm -rf ./node_modules`, `git push --force-with-lease origin feature/x`, `cat .env.example`, `bun run verify`.

2. Heredoc com delimitador entre aspas é dado, não comando. O `host.ts` remove o corpo antes de avaliar:

host.ts: `stripQuotedHeredocBodies`

```
function stripQuotedHeredocBodies(command: string): string {
  return command.replace(
    /<<-?\s*(['"])([A-Za-z_][A-Za-z0-9_]*)\1[\s\S]*?\s*\2\s*$/gm,
    "<<HEREDOC",
  );
}
```

O motivo, na palavra do próprio arquivo: *"um doc que explica gh pr create travaria a própria escrita do doc"*. Sem isso, documentar o fluxo do repo era impossível de dentro do repo. O teste correspondente escreve um arquivo contendo a frase "Nunca rode `sudo rm -rf /` no servidor" e verifica que passa.

CUIDADO | O limite é explícito: só heredoc entre aspas

`<<'EOF'` e `<<"EOF"` têm o corpo removido. `<<EOF` sem aspas **não**, porque ali o shell expande variável e substituição de comando, então o conteúdo é código executável de verdade e o guard continua ativo.

Se o agente travou escrevendo documentação, a correção é usar `<<'EOF'`. Não é desligar o hook.

3. Fora de um repositório git, o guard não bloqueia branch. A função `currentBranch()` devolve string vazia num try/catch. O comentário explica o cenário exato:

guard-shell.ts: currentBranch

```
/**
 * Fora de um repositório git não existe branch a proteger – `git
 * commit`/`push`
 * já falhariam sozinhos. Deixa passar em vez de estourar (o comprador
 * roda o
 * agente antes do `git init`).
 */
```

Dentro dela, a escolha de comando também é deliberada: `git branch --show-current` em vez de `git rev-parse --abbrev-ref HEAD`, porque no repo recém-inicializado (branch órfã, sem nenhum commit) o `rev-parse` falha, e o **primeiro** commit, que cai na main, escaparia. Existe teste com esse nome: *"denies the very first commit on a protected branch (branch órfã)"*.

REGRA | O guard falha fechado

JSON malformatado na entrada resulta em deny, não em allow; teste *"denies malformed JSON instead of allowing it"*. Idem para o evento `beforeShellExecution` do Cursor sem campo `command`. Um hook quebrado bloqueia o comando; ele nunca vira um no-op silencioso.

14.4 O portão de PR travou o `gh pr create`

`tooling/agent-hooks/pre-pr-review-gate.ts` roda em `PreToolUse/Bash` **antes** do `guard-shell` (essa é a ordem no `.claude/settings.json`). Ele só age se o comando casar `/\bgh\s+pr\s+create\b/`; qualquer outra coisa recebe `allow` imediato. Duas constantes governam tudo:

pre-pr-review-gate.ts: constantes

```
const MAX_PR_LOC = 3000;

/** Base default de PR. Alinhada à seção "Branch base" do CLAUDE.md. */
const BASE_BRANCH = "main";
```

São quatro motivos de travamento, avaliados nesta ordem.

14.4.1 Base inválida

Checado antes de tocar em git. Aceita `main` ou qualquer coisa começando com `feature/` (empilhamento de PR). Aceita `--base` e `-B`, com espaço ou `=`, com aspas simples ou duplas.

Base recusada

```
PR travada: a base informada foi `develop`, mas PR deste repositório
entra em
`main` ou empilha numa `feature/*`. Reexecute com `--base main` ou
`--base feature/minha-tarefa`.
```

Se você passou `--base` sem valor parseável, a mensagem é outra; ela pede a forma correta em vez de reclamar do valor.

14.4.2 Selo de pré-review ausente

pre-pr-review-gate.ts: checagem do selo

```
if (!existsSync(`.agents/tmp/pre-review-${head}.ok`)) {
  emitDeny(host,
    `PR travada: a pré-review de guardrails não rodou pro commit atual
    (${head.slice(0, 8)}).\n` +
    `Rode a skill `pre-review` (\`/pre-review\`) primeiro. Ela grava
    o selo que libera o gh pr create.` +
    `Qualquer commit novo depois disso exige nova pré-review.`);
}
```

POR QUE | Por que o SHA está no nome do arquivo

Não existe rotina de invalidação de selo. O nome do arquivo **carrega** o SHA do HEAD: commitou de novo, o SHA muda, o `existsSync` procura outro nome e não acha. A invalidação é uma consequência mecânica de como o arquivo é nomeado; não tem estado a corromper, nem cache a limpar.

O diretório `.agents/tmp/` está no `.gitignore`. O selo é estritamente local: ele não viaja no PR, não vai pro CI, e não libera nada na máquina de outra pessoa.

A saída é rodar `/pre-review`. A skill (`.agents/skills/pre-review/SKILL.md`) grava o selo **só após confirmação sua**; não é um passo automático:

O selo que a pré-review grava no fim

```
mkdir -p .agents/tmp
git rev-parse HEAD > .agents/tmp/pre-review-$(git rev-parse HEAD).ok
```

14.4.3 Teto de 3000 linhas

Se não existir um ack para o HEAD atual, o hook roda `git diff --numstat origin/<base>...HEAD`, soma adições mais remoções (ignorando linhas com `-`, que são binários) e compara com `MAX_PR_LOC`.

PR acima do teto

```
PR grande: 4210 linhas (add+del) vs `main`, acima do teto de 3000.
Antes de
abrir, pergunte ao usuário: dividir em PRs menores (recomendado) vs
seguir.
Se mandar seguir, grave o ack e reexecute:
git rev-parse HEAD > ".agents/tmp/pr-size-ack-$(git rev-parse HEAD).
ok"
O ack vale só pra este commit.
```

Detalhe de implementação com efeito prático: a existência do ack faz o hook **pular o git diff inteiro**, não apenas ignorar o resultado. Ou seja, com o ack no lugar, nem a comparação com origin acontece; o que também contorna, de lado, o erro da próxima subseção.

Para conferir o tamanho antes de tentar, o comando equivalente (o mesmo que a skill pre-review usa):

Contar as linhas do PR antes de abrir

```
git diff --numstat origin/main...HEAD | awk -F\t ' $1!="-" {s+=$1+$2}
END {print s}'
```

14.4.4 Falhas de git: e o erro mais provável na vida real

Se o `git rev-parse HEAD` falhar, o hook nega com uma mensagem pedindo que você confirme estar dentro do repositório (teste *"denies safely outside a Git repository"*, que exige inclusive stderr vazio). Mas o caso que vai te morder de verdade é o outro:

CUIDADO | A comparação é contra origin/main, não contra a main local

Mensagem quando o diff falha

```
PR travada: não foi possível comparar o HEAD com `origin/main`.
Rode
`git fetch origin main` e tente novamente.
```

Clone raso, remote recém-adicionado, ou uma main local que nunca foi sincronizada: em todos esses casos a ref origin/main não existe e o hook trava sem que exista nada de errado com o seu PR. A mensagem já entrega a correção. Faça o `git fetch` e reexecute.

14.4.5 Falso-positivo já tratado, e a cadeia oficial

O mesmo tratamento de heredoc do `guard-shell` vale aqui. O teste é literal em `pre-pr-review-gate.test.ts`: *"ignora o comando citado dentro de um heredoc literal"; "Escrever um doc que EXPLICA gh pr create não é abrir uma PR. Sem isto, documentar o próprio fluxo travava na escrita do documento."*

REGRA | A cadeia é /pr-ready → /pre-review → /create-pr

Está no `AGENTS.md` e no `.claude/commands/create-pr.md`: `/pr-ready` é o gate mecânico, `/pre-review` é o julgamento (e o que grava o selo), `/create-pr` abre. Pular direto pro `gh pr create` é exatamente o que o hook existe pra impedir, e a ordem não é decorativa, cada etapa produz o insumo da seguinte.

14.5 O knip acusou um arquivo ou export órfão

14.5.1 Por que isso acontece o tempo todo

Olhe o `knip.json`. Dentro de `src/`, os **únicos** entry points são as rotas:

knip.json (trecho)

```
"entry": [
  "src/app/**/*.{ts,tsx}",
  "tooling/agent-hooks/*.ts",
  "tooling/*.ts",
  "integracoes/coderabbit/cr-comments.ts"
],
"project": ["src/**/*.{ts,tsx}", "src/**/*.css", "tooling/**/*.ts", "
  test/**/*.ts", "*.ts,mjs"]],
```

Qualquer arquivo em `src/features/`, `src/lib/` ou `src/shared/` que não seja alcançável por uma cadeia de imports a partir de uma rota é, por definição, *unused file*. É por isso que criar a feature **antes** da rota que a consome reprova o gate; sempre. Não é bug de configuração, é o desenho.

Os dois erros que você vai ver, capturados criando as violações de propósito:

Arquivo órfão

```
Unused files (1)
src/shared/orfao-temp.ts
error: script "knip" exited with code 1
```

Export órfão

```
Unused exports (1)
ENDPOINT_ORFAO  src/lib/api/endpoints.ts:25:14
```

Repare que o segundo caso é mais sutil: o **arquivo** é usado, o **export** não. Adicionar uma constante “que já vai ser útil depois” reprova o gate na hora.

14.5.2 O que fazer: conectar ou apagar

POR QUE | ADR-017: por que knip é gate e não sugestão

Do **MEMORY.md**: “o `no-unused-vars` do `oxlint` só enxerga dentro do módulo. Código morto `cross-module` é o que realmente acumula, e num boilerplate é pior: o comprador herda arquivos que ninguém chama e presume que são necessários.”

O ADR também registra o efeito histórico, que é o melhor argumento a favor da regra: o knip “pegou o `server-only` órfão no `package.json` e forçou o `lib/api/ky.client.ts` a ter `caller real`; foi o knip que exigiu a tela de login, fechando o ciclo de auth que o `proxy.ts` já prometia ao redirecionar para `/login`”. Uma feature inteira do boilerplate existe porque o gate recusou código sem consumidor.

REGRA | A saída é conectar ou apagar: nunca a ignore-list

Escreveu um componente ou schema que ainda não tem rota? Escreva a rota, ou apague o arquivo e escreva os dois juntos quando for a hora. YAGNI é regra do repo, e o knip é o que a torna executável.

A lista `ignoreDependencies` do **knip.json** tem exatamente dois itens, ambos com comentário explicando o critério: `@typescript/native-preview` (“fornece o `tsgo`; invocado pelo `toolchain`, não importado por nenhum módulo”) e `lefthook` (“instala os `git hooks`; roda pelo `lefthook.yml`, não por `import`”). Esse é o único precedente: binário de `toolchain` que nenhum módulo importa. Código-fonte seu nunca entra ali.

14.6 O `no-unnecessary-condition` acusou um `guard` que parece necessário

Esta é a reprovação mais confusa do repo, porque o lint parece estar errado. Às vezes está, e às vezes você é que está.

14.6.1 As regras type-aware que estão ligadas

.oxlintrc.json: bloco type-aware

```
"typescript/no-unnecessary-condition": "error",
"typescript/no-unnecessary-template-expression": "error",
"typescript/no-unnecessary-boolean-literal-compare": "error",
"typescript/no-unsafe-type-assertion": "error",
"typescript/consistent-return": "error",
"typescript/no-redundant-type-constituents": "error",
```

O comentário logo acima delas declara o alvo: *"fallback morto: ??/?./!= null/template sobre valor que o tipo já garante; assertion insegura"*.

14.6.2 Acesso indexado e narrowing

tsconfig.json: política atual

```
// O baseline acompanha o Avante; o caso ausente fica explícito na API usada.
// Para arrays comuns, prefira `at(0)` a um acesso que esconda o narrowing.
```

O baseline atual não ativa `noUncheckedIndexedAccess`, para acompanhar o `tsconfig.json` do Avante (ADR-028). Isso não autoriza ignorar valores ausentes: quando o acesso pode retornar nada, use uma API cujo tipo preserve esse fato e faça o narrowing explícito. Com esta função:

Guard sobre acesso indexado

```
type Item = { readonly id: number };

export function primeiro(lista: ReadonlyArray<Item>): number {
  const item = lista.at(0);
  if (!item) { return -1; }
  return item.id;
}
```

Indexar diretamente com `lista[0]` só é aceitável quando o contrato já garante a posição; para uma lista comum, `.at(0)` deixa o caso ausente visível ao TypeScript e ao leitor.

14.6.3 O caso em que a regra está certa

O mesmo lint, sobre um `return valor ?? 0;` onde `valor` é `number`:

Fallback morto, corretamente acusado

```
error typescript(no-unnecessary-condition): Unnecessary optional chain
on a non-nullish value.
```

14.6.4 Como decidir, em uma pergunta

Diagnóstico do no-unnecessary-condition

O que a regra acusou	Leitura correta
Guard sobre acesso indexado (<code>lista.at(0), record[chave]</code>)	Preserve o narrowing e trate o caso ausente; não use <code>!</code> nem <code>assertion</code> .
<code>?? 0, [], ?? ''</code> sobre valor que o tipo ou o schema já garantiu	A regra está certa. Apague o fallback.

O [CODE-PATTERN.md](#) nomeia o segundo caso: *"Estado impossível (tipo/schema/control flow já garantiu): confie no narrowing"*, com a lista do que não fazer; `??, || [], ?? 0, branch cerimonial, assert, !`.

CUIDADO | Nos dois casos, `!`, `as` e `assert` continuam proibidos

`typescript/no-non-null-assertion` e `typescript/no-unsafe-type-assertion` estão em error, e `boilerplate/no-assert` é regra do plugin custom. Não existe atalho: se o valor pode ser `undefined`, você estreita de verdade.

Existe um único ponto do repo onde `no-unsafe-type-assertion` está desligado; o override de `tooling/**`, com a justificativa registrada no [.oxlinttrc.json](#) de que *"a API alpha do oxlint não publica tipos"*. Isso vale para as regras de lint, que consomem essa API. Não é precedente para `src/`.

14.7 O build reprova por dado lido fora de `<Suspense>`

14.7.1 De onde vem

O `next.config.ts` liga `cacheComponents`: `true`, com o comentário que explica a natureza da flag:

next.config.ts (trecho)

```
// Shell estático + buraco dinâmico em streaming. NÃO é cache: cada request
// busca fresco; o que fica pré-renderizado é só a casca sem dado. A parte que
// lê cookie/dado precisa ficar sob um `<Suspense>` (ver src/app/perfil/page.tsx).
cacheComponents: true,
```

Com a flag ligada, ler cookie ou dado fora de um `<Suspense>` não é warning: reprova o build.

14.7.2 O output real

Removendo o `<Suspense>` de `src/app/perfil/page.tsx` e rodando o build:

next build com dado fora do Suspense

```
Error: Route "/perfil": Uncached data was accessed outside of <Suspense>. This
delays the entire page from rendering, resulting in a slow user
experience.
Learn more: https://nextjs.org/docs/messages/blocking-route
    at main (<anonymous>)
    at body (<anonymous>)
    at html (<anonymous>)
To get a more detailed stack trace and pinpoint the issue, try one of
the following:
- Start the app in development mode by running `next dev`, then open
"/perfil"
  in your browser to investigate the error.
- Rerun the production build with `next build --debug-prerender` to
  generate
  better stack traces.
Error occurred prerendering page "/perfil". Read more: https://nextjs.
org/docs/messages/prerender-error
Export encountered an error on /perfil/page: /perfil, exiting the build.
```

Existe uma variante irmã, no mesmo arquivo do Next, para quem lê a fonte dinâmica diretamente: *"Route <rota>: Runtime data such as cookies(), headers(), params, or searchParams was accessed outside of <Suspense>"*.

CUIDADO | Correção a um erro nos docs do próprio repo

O `MEMORY.md` (ADR-025) e o `CODE-PATTERN.md` dizem que o build reprova com `blocking-prerender-dynamic`. Esse slug **não existe** no Next 16.3.0; um `grep` recursivo em `node_modules/next/dist` não retorna nenhum arquivo com ele.

O slug real emitido por `node_modules/next/dist/server/app-render/dynamic-rendering.js` é `blocking-route`. Se você procurar pela string docu-

mentada, não vai achar nada e vai achar que está lendo outro erro. Procure pela frase `Uncached data was accessed outside of <Suspense>`.

14.7.3 O padrão canônico

`src/app/perfil/page.tsx` é a referência do repo: extraia a subárvore que faz IO num componente `async` separado e envolva só ela.

`src/app/perfil/page.tsx` (trecho)

```
async function PerfilCarregado(): Promise<React.ReactNode> {
  const perfil = await apiClient(kyServer, API_ENDPOINTS.perfil.me,
    perfilSchema);
  return <PerfilForm perfil={perfil} />;
}

// ... dentro do <main> da página:
<Suspense fallback={<p className="text-sm text-foreground/60">
  Carregando perfil...</p>}>
  <PerfilCarregado />
</Suspense>
```

O JSDoc do próprio arquivo explica o critério de corte: ler o cookie (dentro do `kyServer`) torna aquela subárvore dinâmica, então é ela, e só ela; que vai sob o `<Suspense>`. O shell sai no primeiro byte, o dado chega em streaming.

Duas ferramentas de diagnóstico que o próprio Next sugere e que valem o atalho: `next build --debug-prerender` para stack trace útil, ou abrir a rota em `next dev`.

Quando está certo, o build classifica `/perfil` assim:

Classificação das rotas num build verde

Rota	Tipo
<code>/, /_not-found, /login</code>	Static (pré-renderizada como conteúdo estático)
<code>/perfil</code>	Partial Prerender (HTML estático + conteúdo dinâmico em streaming)

POR QUE | Por que isso não contradiz a regra de "sem cache"

ADR-025: "Isso **não** contradiz o ADR-003: nada fica em cache, o que é pré-renderizado é só a casca sem dado, e cada request busca fresco." Partial Prerender é sobre **quando** os bytes saem, não sobre reaproveitar resposta. O ADR também registra o escape hatch: "se um dia morder, remover a flag custa uma linha (a rota volta a ser dinâmica pura)".

14.8 Avisos do build que não são problema

Dois avisos aparecem em todo build verde. Ambos têm ADR dedicado dizendo, em resumo, para não fazer nada.

14.8.1 setTimeout e Cache Components sob Bun

Sai no meio de "Generating static pages"

```
Next.js cannot guarantee that Cache Components will run as expected due
to the
current runtime's implementation of `setTimeout()`.
Please report a github issue here: https://github.com/vercel/next.js/
issues/new/
```

ADR-025, "Caveat conhecido": *"sob Bun o build imprime um aviso de que o Next não garante o comportamento de Cache Components por causa da implementação de setTimeout do runtime. O output sai correto".* O build termina com exit 0 e a rota `/perfil` sai corretamente como Partial Prerender.

O contexto histórico está no ADR-024. O baseline atual segue o ADR-028: Next 16.3.0 com Bun 1.4.0-canary.1, partialPrefetching e o compilador Rust do React. O build verde foi repetido com as variáveis do `.env.example`; o único aviso remanescente é o conhecido sobre setTimeout do Bun em Cache Components.

14.8.2 O log do @typescript/native-preview

Aparece **duas vezes** por build; uma no `next typegen`, outra no `next build`:

Log do compilador nativo

```
Detected @typescript/native-preview as TypeScript compiler. Some Next.
js
TypeScript features (like type checking during build) require the
standard
typescript package.
```

POR QUE | ADR-011: por que ninguém silencia isso

"Decisão: não patchar o Next para silenciar o log do native-preview. Por quê: o patch (bun patch) tinha custo de manutenção (refazer a cada upgrade do Next) maior que o benefício (um log cosmético)." Status: ativo, log aceito.

E a mensagem não indica perda real de checagem. O script de build é `next typegen && tsc --noEmit && next build`; o type check acontece no tsc nativo, explicitamente, antes do build. O Next 16, ao detectar o native-preview, pula a checagem própria de propósito; é por isso que a linha "Running TypeScript ... Finished TypeScript in 44ms"

aparece com um número absurdamente baixo. Esses 44ms são o Next **pulando**, não checando.

CUIDADO | Não instale `typescript@5` para "resolver" esse log

ADR-001 antecipa exatamente esse impulso: *"não instalar o TypeScript clássico' significa não voltar para o TS 5/6. O pacote `typescript` na versão 7 é o compilador nativo... Os dois pacotes convivem de propósito."* O `package.json` tem `typescript` em 7.0.2 e `@typescript/native-preview` em 7.0.0-dev.20260707.2, e isso está correto.

14.9 O plugin custom do oxlint quebrou num upgrade

14.9.1 O risco declarado

O `.oxlintrc.json` registra o plugin em uma linha: `"jsPlugins": [". / tooling/oxlint-rules/index.ts"]`. São 17 regras registradas em `tooling/oxlint-rules/index.ts`, 15 como error e 2 como warn (`no-passthrough-mapper` e `no-literal-dispatch`).

ADR-009 nomeia o risco sem rodeio: *"jsPlugins é alpha (sem semver) → oxlint **pinado** (hoje 1.77.0) + **grep-gate de CI** como backstop. Subir a versão exige rodar `bun test` das regras custom: elas são o canário da API do plugin."*

14.9.2 O canário

`tooling/oxlint-rules/rules/__tests__/custom-rules.test.ts` não usa o `RuleTester` do oxlint. O motivo está no cabeçalho: *"We deliberately do NOT use oxlint's RuleTester: it rejects Bun at runtime and forces Node. This integration approach tests the real oxlint behavior under Bun instead."* Ele escreve fixtures em `tmpdir()` e roda o oxlint de verdade com saída JSON.

Ele detecta três modos de falha, e falha **alto** em todos; de propósito:

`custom-rules.test.ts`: os dois guards que falham alto

```
// Sem saída = oxlint crashou (ex.: plugin não carregou). Falhar ALTO:
// senão os
// testes "allows" passariam em falso (hasRule retornaria false num
// crash).
if (!raw) {
  throw new Error(`oxlint não produziu saída JSON (crash?). stderr:\n${
    result.stderr.toString()
  }`);
}
// ...
// JSON válido mas em formato inesperado (schema do oxlint mudou?) →
// falhar alto,
// senão os testes "allows" passariam em falso.
throw new Error(`oxlint: formato de saída inesperado: ${raw.slice(0,
  200)}`);
```

O terceiro ponto de acoplamento é o formato do código de diagnóstico:

custom-rules.test.ts: hasRule

```
/** oxlint formats JS-plugin rule codes as "boilerplate(<rule-id>)". */
function hasRule(diags: OxlintDiag[], ruleId: string): boolean {
  const needle = `boilerplate(${ruleId})`;
  return diags.some((d) => typeof d.code === "string" && (d.code === needle || d.code === ruleId));
}
```

Lendo a falha do canário

Sintoma	O que significa
oxlint não produziu saída JSON (crash?)	O plugin não carregou. A API do jsPlugins mudou.
oxlint: formato de saída inesperado	O schema de saída do oxlint mudou.
Todos os testes fires: vermelhos de uma vez	O formato do código de diagnóstico mudou; não são 15 regras quebradas, é uma mudança de API.
Um teste fires: isolado vermelho	Aí sim, é a regra.

CUIDADO | Inconsistência real: o canário testa outra versão

A linha que invoca o oxlint no teste é:

custom-rules.test.ts, linha 55

```
["bunx", "oxlint@1.77.0", "-f", "json", "-c", OXLINTRC,
  fixturePath],
```

O `package.json` pisa "oxlint": "1.77.0", e é a mesma 1.77.0 que o `bun run lint` e o canário usam.

Efeito prático: uma quebra que só apareça numa versão diferente pode passar batida no `bun test` até que essa linha seja atualizada junto. Ao subir a versão do oxlint, suba **as duas**.

14.9.3 O backstop de CI

Se o plugin um dia não carregar, dois grep no CI ainda barram as violações mais críticas:

.github/workflows/ci.yml: os grep-gates

```
# Backstop em grep dos guardrails que dependem do jsPlugins (alpha): se
# o
# plugin um dia falhar em carregar, estes gates ainda barram as violaçõ
# es.
- name: Grep-gate, sempre fresco
  run: |
    if grep -rEn "['\"]use cache[ :a-z]*['\"]|unstable_cache" src --
    include='*.ts' --include='*.tsx' --include='*.js' --include='*.jsx'
    2>/dev/null; then
      echo "::error::cache proibido ('use cache' / unstable_cache) em
      src/"; exit 1
    fi

- name: Grep-gate, backend só em src/lib/api
  run: |
    if grep -rEon "https?://[^\"] ]*example\.com" src --include='*.ts'
    --include='*.tsx' --include='*.js' --include='*.jsx' 2>/dev/null |
    grep -vE '^src/lib/api/'; then
      echo "::error::URL do backend fora de src/lib/api"; exit 1
    fi
```

REGRA | Troque **example.com** pelo seu domínio, hoje

O comentário no próprio workflow já pede: *"Troque `example.com` pelo seu domínio."* Enquanto o domínio do seu backend não estiver ali, esse segundo backstop não protege nada; ele passa verde sobre qualquer violação. É uma das poucas edições obrigatórias do dia 1.

14.9.4 Procedimento de upgrade do oxlint

Subindo a versão do oxlint

```
# 1. suba a versão em package.json
# 2. suba a versão na linha 55 de tooling/oxlint-rules/rules/__tests__/
  custom-rules.test.ts
bun install
bun test
bun run typecheck:tooling
bun run lint
```

Se o canário estourar com "crash?" ou "formato de saída inesperado", a API do plugin mudou: volte o pin, e porte as regras com calma. Os grep-gates do CI seguram as duas famílias mais críticas (cache proibido e URL de backend fora de `lib/api`) enquanto isso.

14.10 A saída que não existe: suprimir o lint

Vai bater a tentação, especialmente com um agente no comando: colocar um comentário de supressão e seguir a vida. Não funciona neste repo.

[tooling/no-lint-suppression.ts](#) roda dentro do `verify` e faz um `grep` atrás de diretivas de supressão do ESLint e do `oxlint` no código-fonte. O cabeçalho explica os dois pontos que importam:

`tooling/no-lint-suppression.ts: cabeçalho`

```
/**
 * Guard do `verify`: proíbe diretivas de supressão de lint do ESLint e
 * do oxlint
 * no código-fonte. Toda ocorrência deve ser corrigida na origem; não h
 * á escape
 * hatch porque uma supressão permitiria burlar os guardrails com o
 * gate verde.
 *
 * Não é uma regra do plugin oxlint de propósito: uma regra de lint
 * pode ser
 * desligada pela própria diretiva que ela proíbe. Um grep fora do
 * linter, não.
 */
```

O detalhe de implementação que mostra o cuidado: as strings procuradas são **montadas em runtime** (`["eslint", "disable"].join("-")`) para o próprio arquivo não dar auto-match na busca. E o script usa `git grep` quando existe repositório, caindo para `grep` recursivo quando não existe, porque, nas palavras do comentário, “o comprador extrai o *zip* e roda *bun run verify* antes do *git init*”.

A mensagem, se você tentar:

`Supressão encontrada`

```
Proibido: diretiva de supressão de lint encontrada no código-fonte:
<arquivo>:<linha>:<trecho>
Corrija a causa do lint; supressões não são permitidas.
```

REGRA | Não existe escape hatch para o lint

ADR-020 registra a decisão de tirar esse guard do plugin `oxlint` e transformá-lo em script justamente porque uma regra de lint pode ser desligada pela diretiva que ela proíbe. Um `grep` fora do linter, não.

Se uma regra está errada no seu caso, a saída é discutir a regra; ajustar o `.oxlintrc.json` com justificativa registrada, como o repo já faz nos overrides por zona. Nunca silenciar a ocorrência.

14.11 Um cenário que só aparece no dia 1: ainda não há `git init`

Quando você extrai o produto e abre o agente antes de inicializar o repositório, uma parte do troubleshooting acima simplesmente não se aplica, e isso é intencional, não um acidente.

Comportamento antes do `git init`

Componente	O que acontece
Resolução de caminho dos hooks	Se o <code>git rev-parse</code> falha, os três hosts usam o diretório atual.
<code>guard-shell</code> , branch protegida	<code>currentBranch()</code> devolve string vazia; deixa passar (o <code>git commit</code> falharia sozinho de qualquer forma)
<code>pre-pr-review-gate</code>	<code>git rev-parse HEAD</code> falha; nega o <code>gh pr create</code> com mensagem explícita
<code>no-lint-suppression.ts</code>	Usa <code>grep</code> recursivo em vez de <code>git grep</code>

O ADR-016 explica o motivo do fallback: o comprador pode extrair o zip e abrir o agente **antes** do `git init`. O teste [tooling/agent-hooks/hook-configs.test.ts](#) também reprova o gate quando um host aponta para um script inexistente ou quebrado.

Assim que você rodar `git init` e criar o primeiro commit, o guard de branch protegida entra em ação imediatamente: o primeiro commit cai na `main` e é barrado de propósito. Crie uma branch de feature antes.